

Task 2: Group Report

Group Chenonceau Project Plan



Group Leader: John Hoarau

Group Members:

Martin Harambat, Pierre Bédrune, Clémence-Philomène Hinot

Prepared for November 21, 2025

Contents

Abstract	2
1 Introduction	3
2 Allocation of Roles and Responsibilities	4
3 Project Management Strategy	5
3.1 Methodology	5
3.2 Tools	5
4 Provisional Timeline	6
5 Conclusion	7

List of Tables

1 Group Chenonceau Role Allocation	4
2 Key Project Deadlines	6

List of Figures

1 Provisional GANTT Chart for Group Chenonceau, updated on 11/20/2025 .	6
---	---

Abstract

This report outlines the operational strategy for Group Chenonceau regarding the Digital Engineering Design Group Project. Following the first group meeting, this document defines the allocation of roles and responsibilities among members (John Hoarau, Martin Harambat, Pierre Bédrune, and Clémence-Philomène Hinot) based on the strengths identified in Task 1 [1, 2]. Furthermore, it establishes the project management methodology, selecting specific tools for communication and task tracking, and presents a provisional GANTT chart timeline to ensure all module deadlines are met [3].

1 Introduction

Group Chenonceau was formed to undertake the "Applications of Computational Engineering Design Optimisation" project. The group consists of four members from the MSc in Computational Software Techniques in Engineering.

An initial meeting was held to discuss the individual strengths and development opportunities identified in Task 1. This report synthesises those findings into a cohesive team structure, ensuring that the Belbin team roles [1] are distributed to maximise efficiency. Additionally, this report details the digital infrastructure the team will use to collaborate remotely between ESTIA (France) and Cranfield (UK).

2 Allocation of Roles and Responsibilities

Based on the analysis of individual personality profiles (ENTJ, ENTP, INTJ) and the Skills Matrices submitted in Task 1, the group has agreed on the following role allocation.

Table 1: Group Chenonceau Role Allocation

Member	Primary Roles	Justification & Responsibilities
John Hoarau	Group Leader (Chair), Shaper	Identified as a "leader" (ENTJ) [1]. Responsible for overall direction, maintaining the timeline, collating reports, and communicating with the module leader.
Martin Harambat	Innovator, Investigator	Identified as an "Innovator" (ENTP) [2]. Primary responsibility is generating technical solutions and researching optimisation methods. Will focus on creative problem-solving.
Pierre Bédrune	Investigator, Evaluator	Identified as a "Debater" (ENTP) [4]. Will work with Martin on research and technical implementation. Responsible for evaluating the feasibility of proposed ideas.
Clémence-Philomène Hinot	Organiser, Finisher	Identified as an "Architect" (INTJ) [5]. Responsible for structure, quality control, reviewing documentation, and ensuring that "details" (a weakness of the ENTPs) are handled correctly.

3 Project Management Strategy

3.1 Methodology

The group will adopt an Agile methodology as required by the module brief [3]. This will involve:

- **Weekly Scrums:** Short meetings to review progress on the current "Sprint" (Task).
- **Stand-ups:** Ad-hoc updates via messaging to address immediate blockers.
- **Iterative Development:** Focusing on delivering functional components for each Task deadline rather than attempting a "big bang" delivery at the end.

3.2 Tools

The group has selected the following free project management tools:

- **Communication:** *Cranfield Teams* for instant messaging, voice meetings, and screen sharing.
- **Task Management:** *Notion*. A Kanban board will be used to track tasks through the "To Do", "Doing", and "Done" stages, as well as a Gantt chart to meet our deadlines, and some issue attributions to divide tasks and track the evolution of each member. This addresses the organisational weaknesses identified by the ENTP members [2].
- **File Storage:** *Cranfield OneDrive*. A shared drive has been established to store all reports, research papers, and assets.
- **Version Control:** *GitHub*. All code related to the optimisation algorithms and software development will be hosted in a private repository.

4 Provisional Timeline

The group has established a provisional timeline (GANTT Chart) to track key deliverables. Figure 1 illustrates the high-level schedule. And was made using Notion App, as it is free and could fit our work style for each of us.

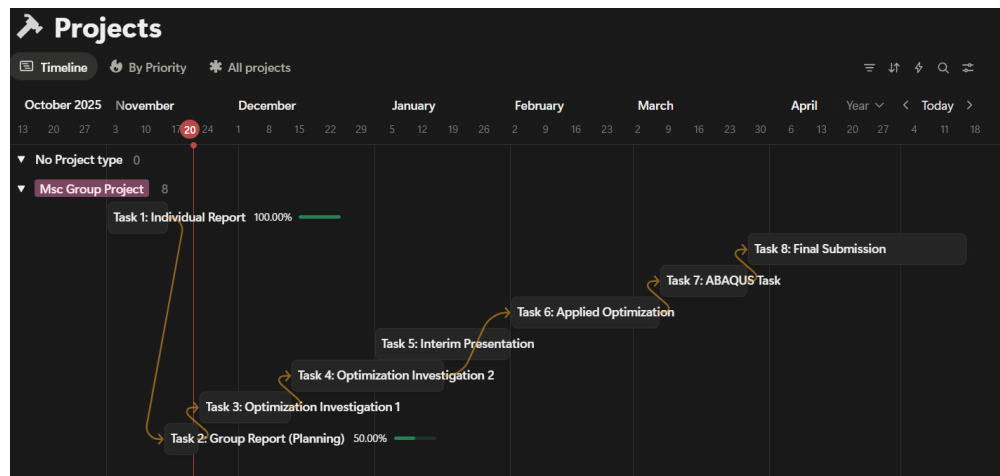


Figure 1: Provisional GANTT Chart for Group Chenonceau, updated on 11/20/2025

The key milestones derived from the module brief [3] are listed in Table 2.

Table 2: Key Project Deadlines

Task	Description	Deadline
Task 2	Group Report (Planning)	21/11/2025
Task 3	Optimisation Investigation 1	12/12/2025
Task 4	Optimisation Investigation 2	16/01/2026
Task 5	Interim Presentation	Jan 2026
Task 6	Applied Optimisation	06/03/2026
Task 7	ABAQUS Task	26/03/2026
Final	Final Submission (Parts 1, 2, 3)	15/05/2026

5 Conclusion

Group Chenonceau has successfully established its operational structure. By leveraging the leadership of John Hoarau, the creative drive of Martin Harambat and Pierre Bédrune, and the organisational rigour of Clémence-Philomène Hinot, the team is well-balanced to tackle the project's challenges. With the project management tools in place and a clear timeline defined, the group is ready to commence Task 3.

References

- [1] John Hoarau. *Task 1: Individual Report*. Submitted to Cranfield University. 2025.
- [2] Martin Harambat. *Task 1: Individual Report*. Submitted to Cranfield University. 2025.
- [3] Chris Phillips. *Group Project Spring 2026: MSc in Computational & Software Techniques in Engineering*. Module Brief. Oct. 2025.
- [4] Pierre Bédrune. *Task 1: Individual Report*. Submitted to Cranfield University. 2025.
- [5] Clémence-Philomène Hinot. *Task 1: Individual Report*. Submitted to Cranfield University. 2025.

Task 3: Optimisation Project

Group Chenonceau: Particle Swarm and Simulated Annealing Optimisation Project (constrained and unconstrained)



Group Leader: John Hoarau

Group Members:

Martin Harambat, Pierre Bédrune, Clémence-Philomène Hinot

Prepared for December 14, 2025

Contents

1	Introduction	2
2	Methodology and Algorithms	3
2.1	Method 1: Particle Swarm Optimisation (PSO)	3
2.1.1	Single-objective algorithm	3
2.1.2	Multi-objective algorithm	3
2.2	Method 2: Simulated Annealing (SA)	5
3	Problem 1: Two-Bar Plane Truss Design	8
3.1	Problem description	8
3.2	Methodology	10
3.2.1	General explanation	10
3.2.2	Penalty function	10
3.3	Results	10
3.3.1	PSO simulation	10
3.3.2	SA simulation	13
3.4	Discussion	17
4	Problem 2: Sensor Calibration	19
4.1	Problem description	19
4.2	Cost function	19
4.3	Methodology	19
4.4	Settings	20
4.5	Result	20
4.6	Work discussion	21
5	Problem 3: Gear Train Design	23
5.1	Problem description	23
5.2	Cost function	23
5.3	Settings	24
5.4	Results	24
5.4.1	PSO simulation	24

5.4.2	SA simulation	25
5.5	Discussion	26
6	General Conclusion	28
6.1	Performance Comparison	28
6.2	Hyperparameter Sensitivity and Robustness	29
7	References	31
8	Appendix	32
8.1	Multi-objective Particle Swarm Optimisation choice	32
8.2	Python code	32
8.2.1	Two-Bar Plane Truss Problem Definition	32
8.2.2	PSO Simulation Script (Truss)	39
8.2.3	Steinhart-Hart Problem Definition	58
8.2.4	Simulated Annealing Class	64
8.2.5	Gear Train Problem and Runner	73

1 Introduction

This report investigates unconstrained and constrained nonlinear optimisation techniques. Specifically, it compares the performance of **Particle Swarm Optimisation (PSO)** and **Simulated Annealing (SA)** [1]. These algorithms are applied to three distinct engineering problems:

1. **Two-Bar Plane Truss Design** (Constrained, Multi-objective)
2. **Sensor Calibration** (Unconstrained, Curve fitting)
3. **Gear Train Design** (Constrained, Discrete/Integer optimization)

Statement on the Use of Generative AI During the preparation of this work, we, as the authors, used ChatGPT (OpenAI) [2] and Claude AI (Anthropic) [3] to assist with code efficiency optimisation and debugging for the Particle Swarm Optimisation and Simulated Annealing scripts. After using this tool, we reviewed and edited the content as needed and took full responsibility for the final report's content and the code's functionality.

2 Methodology and Algorithms

2.1 Method 1: Particle Swarm Optimisation (PSO)

The Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. A population of candidate solutions moves through the domain. Through several time steps, each particle adjusts its position based on its personal best and on the global best. This allows the algorithm to converge to a solution that matches a criterion [4].

2.1.1 Single-objective algorithm

For single-objective problems (like the Gear Train Design), the standard PSO is used to minimise a single error function.

- **Continuous Movement:** Particles move in a continuous space according to standard velocity and position update rules.
- **Integer Rounding (Discrete Adaptation):** For the Gear Train Design problem, which requires an integer number of teeth, the particle positions are rounded to the nearest integer to satisfy the discrete constraint.
- **Convergence Tracking:** The best error (difference from the target ratio) is stored at each iteration to generate a convergence curve.

2.1.2 Multi-objective algorithm

Description

The multi-objective PSO finds Pareto-optimal solutions using a selection based on dominance and diversity [5]. It offers advantages such as ease of implementation, few parameters to

tune, and good applicability to nonlinear problems, but also limitations such as possible early convergence, sensitivity to parameter settings, and no guarantee of global optimality [4].

The MOPSO-CD (Multi-Objective Particle Swarm Optimisation with Crowding Distance) is an advanced extension of the PSO. This particular algorithm was chosen from a list of alternative solutions based on specific criteria (section 8.1). Its key features are:

- Crowding distance-based leader selection,
- External archive management,
- Tournament selection,
- Enhanced exploration.

Performance metrics

Metrics are used to measure the algorithm's performance. We have selected some of the most common criteria:

- The best value per objective, which tracks the minimum value achieved for each objective function at every generation,
- The Pareto front size evolution, which tracks how many solutions are on the Pareto front at each generation,
- The normalised convergence, which compares both objectives normalised to $[0, 1]$.

2.2 Method 2: Simulated Annealing (SA)

Problem description

Simulated Annealing is an optimisation method inspired by the annealing process in metallurgy [6]. The algorithm explores the search space and accepts neighbouring solutions. In this algorithm, ameliorations are always accepted, and degradations are accepted with a probability. The probability decreases stepwise (decreasing temperature). At the beginning, this acceptance mechanism helps to escape local minima. Afterwards, the algorithm converges to a global minimum [4].

Cost function

In this method, we use this function to decide if a worst solution can be accepted:

$$P(\text{acceptance}) = e^{-\frac{E_{new} - E_{actual}}{T}} \quad (1)$$

Where:

- E_{actual} : Actual energy
- E_{new} : The new worst energy
- $P(\text{acceptance})$: the probability to accept E_{new}
- T : temperature parameter

Every iteration the temperature T decreases by the cooling_rate:

$$T = T \times \text{cooling_rate} \quad (2)$$

Multi-Objective Strategy (Weight Scalarization)

Unlike MOPSO, the Simulated Annealing program is inherently a mono-objective algorithm. To solve the bi-objective Truss problem, a scalarization approach is used to transform the problem into a single objective function.

- **Scalarization:** A weighted sum combines the two objectives (Weight and Displacement) into a single scalar Energy function.
- **Normalisation :** A scaling factor (Scale = 10000) is applied to the displacement term to normalise both objectives to comparable orders of magnitude.
- **Weight Distribution:** To approximate a Pareto front, 200 independent runs are performed with the weight w_1 uniformly distributed between 0.005 and 0.995 (where $w_2 = 1 - w_1$). This allows the algorithm to explore different trade-offs, from minimising weight ($w_1 \approx 1$) to minimising displacement ($w_1 \approx 0$).

Performance metrics

To evaluate the performance of the SA algorithm, the following metrics were tracked:

- **Convergence profiles:** Analysis of the best energy and current energy evolution over iterations to assess stability and exploration.
- **Pareto Front Approximation:** The collection of non-dominated solutions found across the 200 independent runs is plotted to visualise the trade-off curve (the "Banana shape").
- **Decision Space Analysis:** Mapping the decision variables (x_1, x_2) of the final solutions to identify distinct regions corresponding to different trade-off preferences.

Application to Gear Train Design (Discrete Problem)

For the discrete gear train problem, the SA algorithm explores the search space by randomly perturbing the current solution (integer number of teeth).

- **Exploration:** High temperatures allow for wide exploration, accepting worse solutions with higher probability.
- **Exploitation:** As temperature decreases, the algorithm converges around a good solution, calculating the probability of acceptance based on the energy difference.
- **Constraint Handling:** The integer constraints are inherent to the neighbor generation process.

Settings

It is important to consider:

- *initial_temp*: the temperature at the beginning
- *cooling_rate*: The speed of the freezing
- *min_temp*: The temperature where we stop the process

3 Problem 1: Two-Bar Plane Truss Design

2.1.2 Multi-Objective algorithm

3.1 Problem description

The Two-Bar Truss problem is composed of two diagonal bars (of aluminium here) connected at a node and fixed at two upper points (Figure 1). On the central node, we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.

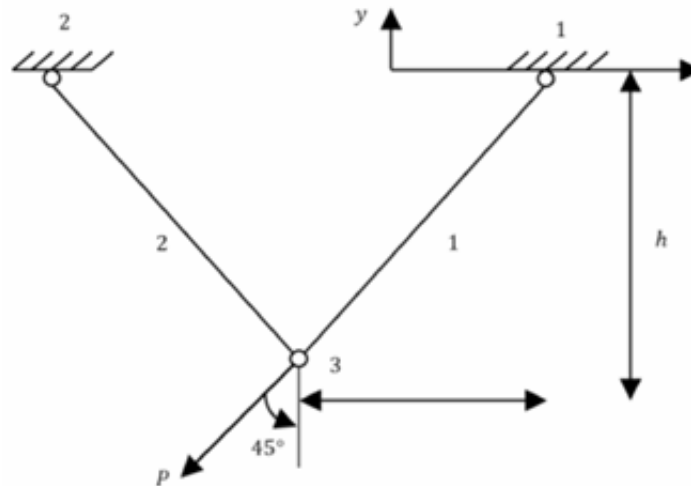


Figure 1: Representation of the two-bar plane truss problem [7]

There are two design variables: the positions of joints 1 and 2. They are defined by the two equations: f_1 , the structural weight truss, and f_2 , the displacement of joint 3. The conflict between minimising weight and minimising displacement is a classic example of multi-objective optimisation [5].

$$f_1(x) = 2\rho h x_2 \sqrt{(1 + x_1^2)} \quad (3)$$

$$f_2(x) = \frac{Ph(1 + x_1^2)^{1.5} \sqrt{(1 + x_1^4)}}{2\sqrt{2}Ex_1^2x_2} \quad (4)$$

These functions are constrained by two stress constraint functions g_1 (tension) and g_2 (compression).

$$g_1(x) = \frac{P(1 + x_1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (5)$$

$$g_2(x) = \frac{P(-x_1 + 1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (6)$$

The variables x_1 and x_2 are calculated from the following formulas: $x_1 = \frac{x}{h}$ and $x_2 = \frac{A}{A_{min}}$
 $0.1 \leq x_1 \leq 2.25$ and $0.5 \leq x_2 \leq 2.5$

If the x_1 variable is connected to the physical sense of the truss geometry, the position boundaries are: $x_{min} = 0.1 * h = 0.254m$ and $x_{max} = 2.25 * h = 5.72m$

The parameters are converted to International System units.

- $\rho = 0.283lb/in^3 = 7833kg/m^3$;
- $h = 100 \text{ in} = 2.54 \text{ m}$;
- $P = 10^4lb = 44482.2N$;
- $E = 3 \cdot 10^7lb/in^2 = 2.07e11Pa$;
- $\sigma_0 = 2 \cdot 10^4lb/in^2 = 1.38e8Pa$;
- $A_{min} = 1in^2 = 6.5416e - 4m^2$.

3.2 Methodology

3.2.1 General explanation

The problem is defined by a class containing four functions: two for the objectives and two for the constraints.

3.2.2 Penalty function

The Pymoo library already includes constraint handling and constraint-violation measures during selection. For the multi-objective problem, it uses the Pareto ranking. To study how the penalty strength affects convergence and feasibility, explicit penalty terms will be added [5].

$$F_{penalised} = f(x) + k * \sum (\max(0, g_i(x))^2) \quad (7)$$

With: k the penalty factor.

Considering this equation, a penalty will be applied when the maximum between 0 and g_i is g_i , so the constraint is violated. This is realised with the following loop, where the penalty factor is fixed to 10 billion. If the penalty factor is significant, the algorithm quickly avoids infeasible solutions. If the penalty factor is small, the algorithm may spend longer exploring infeasible regions.

3.3 Results

3.3.1 PSO simulation

Best solution

To obtain the best solution, the algorithm minimises the weight function and then the displacement function. Then it finds the best compromise that satisfies the objective functions

while satisfying the constraints.

Table 1: Summary of key solutions (PSO)

Solution type	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min weight	0.8245	0.5	16.6361	0.0084
Min displacement	1.8777	2.5	136.5345	0.0011
Compromise	1.2560	1.1667	48.0982	0.0029

All the possible values of the design variables are shown in the decision space diagram: each point represents a candidate solution. The colours indicate a gradient: dark colours for the lower objective values, which are better for minimisation, and lighter shades for the higher objective values, which are worse for minimisation.

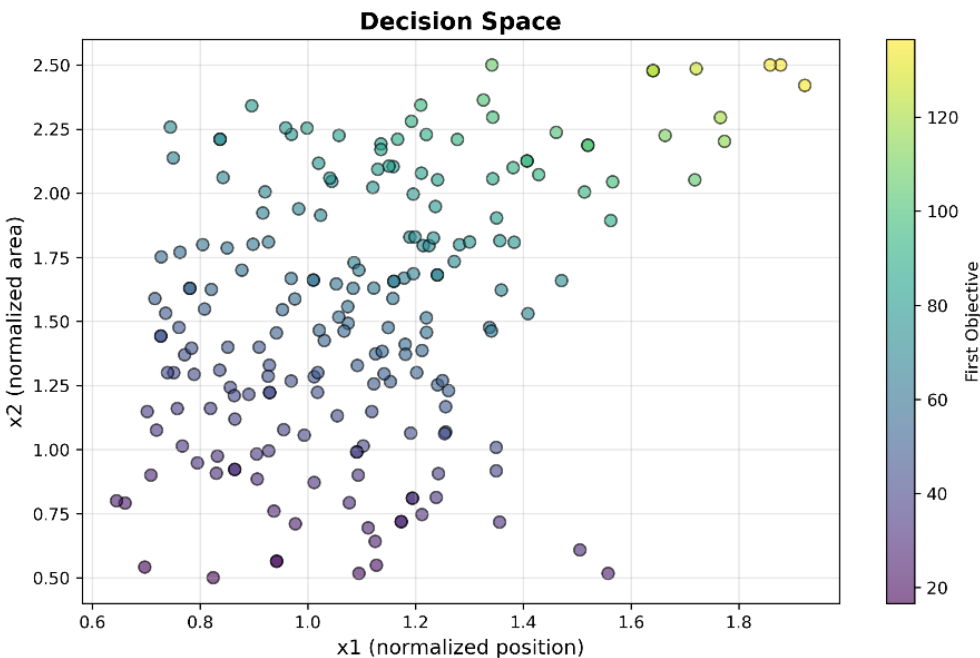


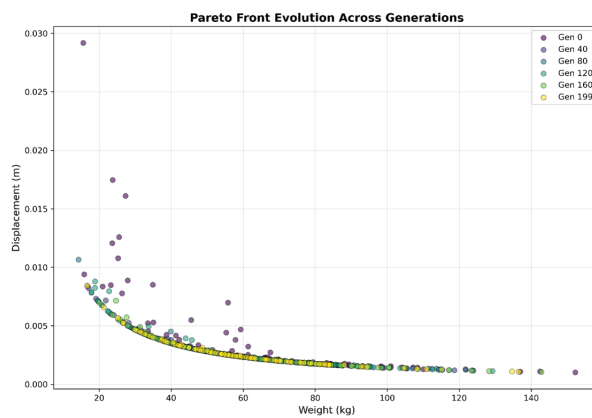
Figure 2: PSO Decision Space

Metrics

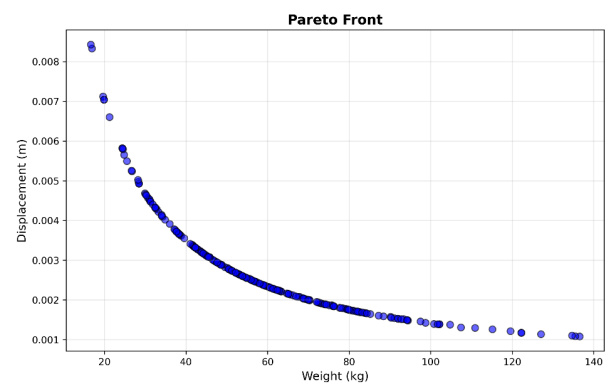
The Pareto front evolution across generations diagram shows that the first generations

are far away from the Pareto front, but through each time step, the solutions converge. This is possible thanks to the penalty function, which eliminates poor solutions. The visualisation helps identify regions of the decision space associated with the optimal solution.

The generation 199, or 200 time step, generates the Pareto front kept. The best solution is extracted from it.



(a) Pareto Front Evolution Across Generations



(b) PSO Pareto Front

Figure 3: Evolution of the Pareto Front and final result (PSO)

The convergence analyses the efficiency of the algorithm. The following diagrams describe the evolution of the best values at each generation.

- **Best value First objective:** The weight is minimised over time with an improvement of the solution. The curve decreases over the generations, so the minimum is stable.
- **Best value Second objective:** The algorithm finds solutions with smaller solutions through the generations. The curve converges around an optimal displacement.
- **Normalised convergence:** The objectives are normalised for comparison. The algorithm is simultaneously improving both objectives as the curves decrease.

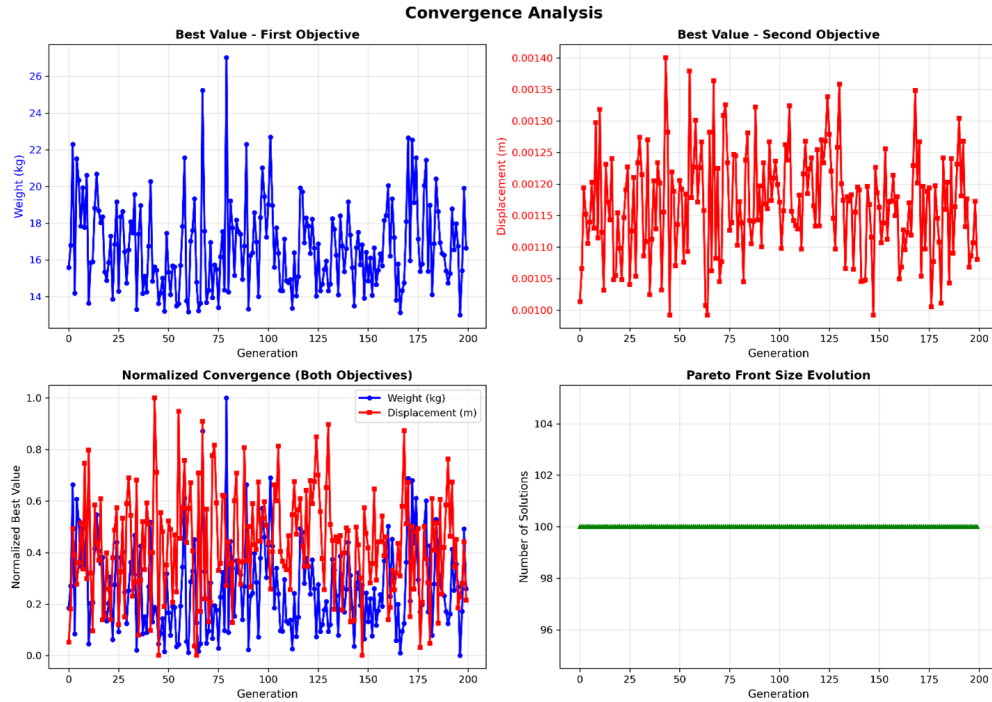


Figure 4: PSO Convergence Analysis

The Pareto front size evolution diagram shows the number of non-dominant solutions at each generation. Its constant size maintains diversity throughout optimisation. This is a good indicator to tell that the algorithm didnt lose diversity and didnt converge prematurely.

3.3.2 SA simulation

Weight Scalarization Strategy

The simulated annealing program is based on a mono-objective algorithm. We have a scalarization approach for transforming the problem (bi-objective) into a mono-objective function. To generate the Pareto front, 200 runs are made with the weight uniformly distributed between 0.005 and 0.995. A scale of 10000 is applied to the displacement for normalising both objectives to comparable orders of magnitude. The stress constraints are enforced in the

optimisation program via a penalty method, with the solution verified at the end. This double approach ensured that all Pareto fronts satisfied the structural constraints.

Scalar Energy function :

$$E(x) = w_1 \cdot f_1(x) + w_2 \cdot (scale_f2 \times f_2(x)) + \sum_{i=1,2} 10^6 \cdot \max(0, g_i(x)) \quad (8)$$

where :

- w_1, w_2 : scalar weight with $w_1 + w_2 = 1$ and $w_1 \in [0.005, 0.995]$
- $f_1(x)$: structural weight (kg)
- $f_2(x)$: displacement (m)
- $scale_f2 = 10000$: the objective normalize factor
- $g_i(x) \leq 0$: Stress constraints (tension g_1 and compression g_2)

Pareto Front

The Pareto front is the result of 200 runs of the Simulated Annealing program and exhibits an optimal distribution of solutions representing the trade-off between structural weight and node displacement. The valid solutions are comprised between: Weight: 14.39 kg to 213.64 kg Displacement: 0.848 mm to 8.382 mm The banana shape of the Pareto front saw the conflict between the structural optimisation: reduce the weight and increase the flexibility, and the displacement; otherwise, if we want rigidity, we need more material. We also found no single solution that simultaneously minimised both objectives.

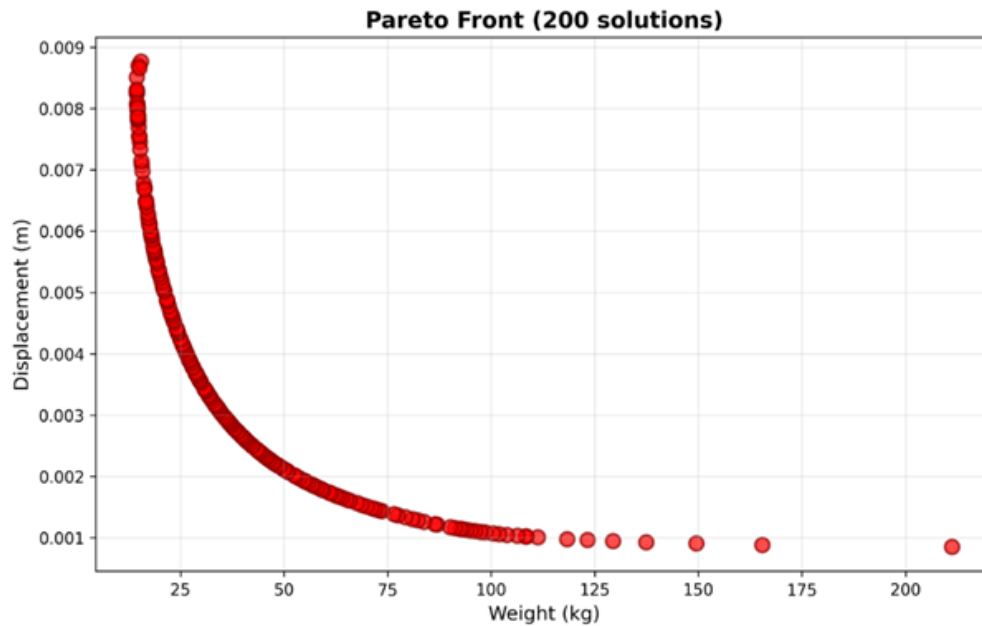


Figure 5: SA Pareto Front

Decision Space

The decision space distribution reveals a characteristic L shape. It is the result of the linear scalarization of the weight:

- Region 1 ($x_2 \in [0.5, 1.0]$, x_1 variable): light solution for the weight minimization ($w_1 \approx 1.0$)
- Region 2 ($x_2 \in [2.0, 2.5]$, x_1 variable): Minimize the node displacement ($w_1 \approx 0.0$)
- Space of transition ($x_2 \in [1.0, 2.0]$) : compromise solution

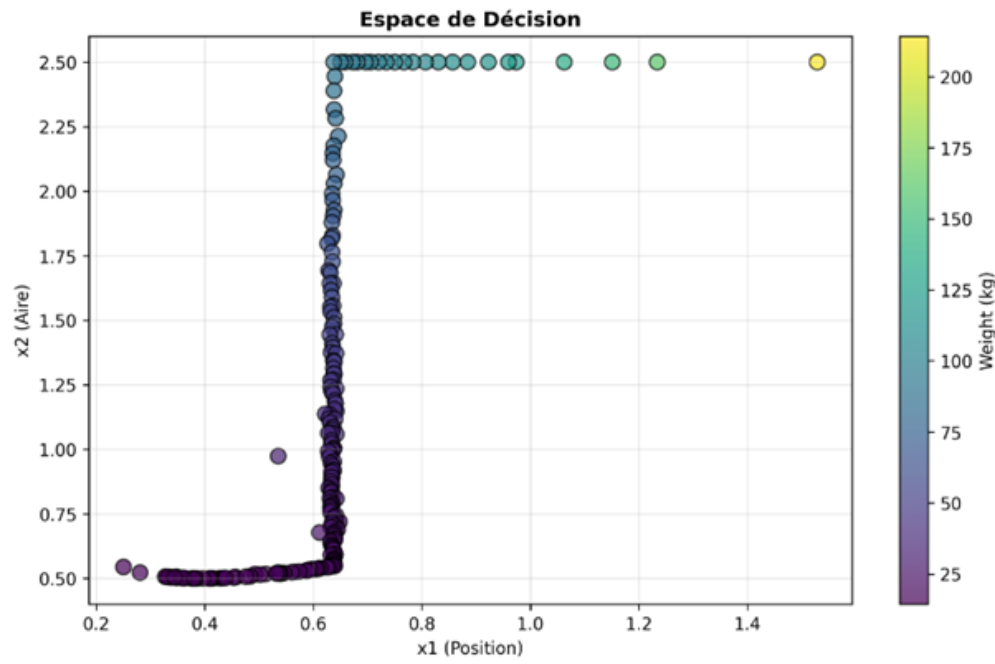


Figure 6: SA Decision Space

Convergence analysis

The convergence compartment of the 200 runs is represented on a distinct plot for each objective:

- Best weight evolution: Weight decreased to 200 kg by 14,39 kg. Significant improvement result after the first 50 runs. Stable execution after 150 runs that saw a complete exploration of the light region.
- Best displacement evolution: Speed convergence in 20 runs to the minimal value of 0,848 mm. We have a parabolic augmentation, and we start by optimising the displacement, so we have the best optimisation at the beginning and the worst after that.

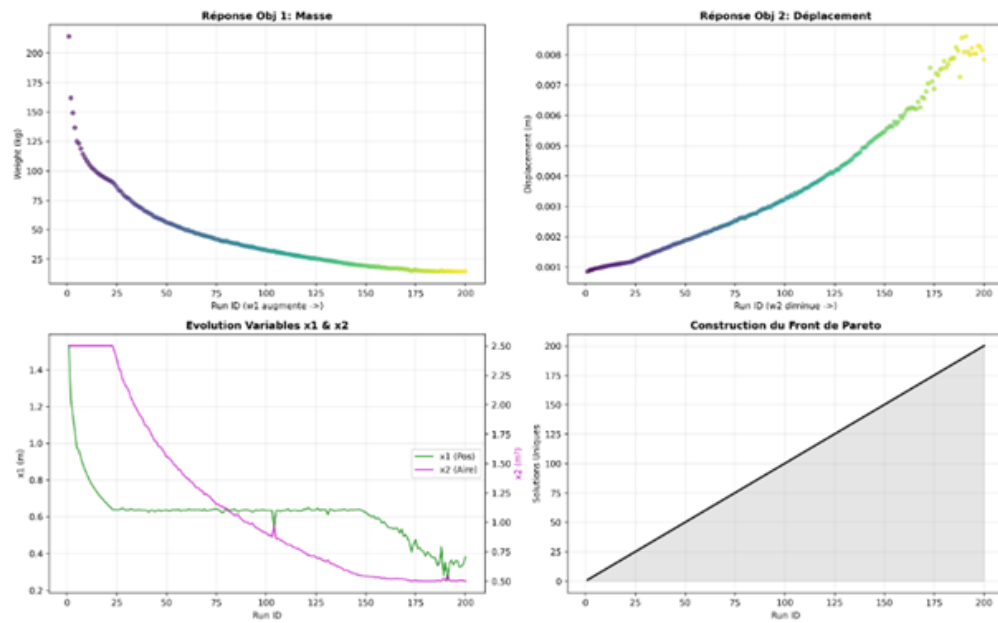


Figure 7: SA Convergence

3.4 Discussion

The best solutions found by the PSO and SA algorithms are almost similar. The differences between the two algorithms are explained by the proper operations they compute and the penalty treatment.

Table 2: Best solution comparison

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038

Table 3: Absolute differences between PSO and SA algorithms

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min weight	0.1822	0.037	0.2517	0.0003
Min displacement	0.37	0	21.4903	0.0001
Compromise	0.2131	0.1624	10.8446	0.0009

These two algorithms are efficient for treating the Two-Bar Plane Truss problem. The differences can be seen with the complexity (number of actions to obtain the solution), the shape of the convergence curve, and the calculation cost.

Table 4: Performance comparison of PSO and SA algorithms

Algorithm	Complexity	Convergence	Calculation cost
SA with 200 runs	106 400	Smooth and monotonous	Low
SA with 200 generations	40 000	Oscillating	High

The SA algorithm compensates for its large number of steps with its simplicity of calculation. In contrast, the PSO algorithm is more computationally expensive. Penalties play an essential role in both algorithms. To conclude, the choice of algorithm depends on the study's requirements: its a balance between computational costs and algorithm performance.

4 Problem 2: Sensor Calibration

4.1 Problem description

The objective is to determine the Steinhart-Hart coefficients (A, B, C) that model the relationship between Resistance (R) and Temperature (T) for a 10K thermistor. Model Equation:

$$T_{model} = \frac{1}{A + B \ln(R) + C(\ln(R))^3} \quad (9)$$

Note: T is in Kelvin, R in Ohms.

4.2 Cost function

The optimisation goal is to minimise the Sum of Squared Errors (SSE) between the measured temperature and the model-predicted temperature:

$$SSE = \sum (T_{measured,i} - T_{model,i})^2 \quad (10)$$

4.3 Methodology

Simulated Annealing

A probabilistic technique for approximating the global optimum of a given function.

Particle Swarm Optimisation (PSO)

A computational method that optimises a problem by iteratively trying to improve a candidate solution based on a given measure of quality. It solves a problem by maintaining a population of candidate solutions, here dubbed particles, and moving them in the search space according to a simple mathematical formula that depends on their positions and velocities.

4.4 Settings

Settings SA

- Initial Temperature: 50.0
- Cooling Rate: 0.995
- Minimum Temperature: 1e-6

Settings PSO

Implementation utilises the pymoo library (PSO algorithm).

- **Parameters:**
 - Population size: 50
 - Generations: 500
 - Inertia weight (w): 0.7
 - Cognitive parameter (c_1): 1.5
 - Social parameter (c_2): 1.5
- max_velocity_rate: Limits the maximum velocity of particles.

4.5 Result

The optimal Steinhart-Hart coefficients found for the provided dataset are:

Table 5: Optimisation Results Comparison (Steinhart-Hart)

Algorithm	A	B	C	Final SSE	Function Calls	Execution Time (s)
Simulated Annealing	1.114×10^{-3}	2.367×10^{-4}	7.761×10^{-8}	0.4323	3,537	0.099
PSO	1.127×10^{-3}	2.345×10^{-4}	8.625×10^{-8}	0.0089	25,000	0.678

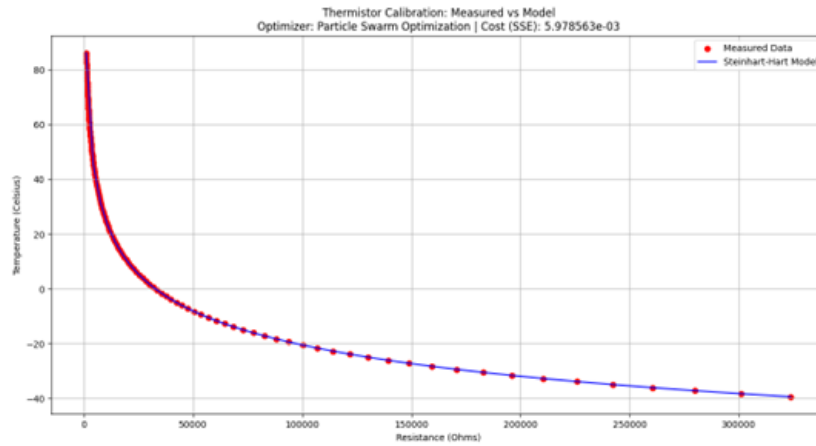
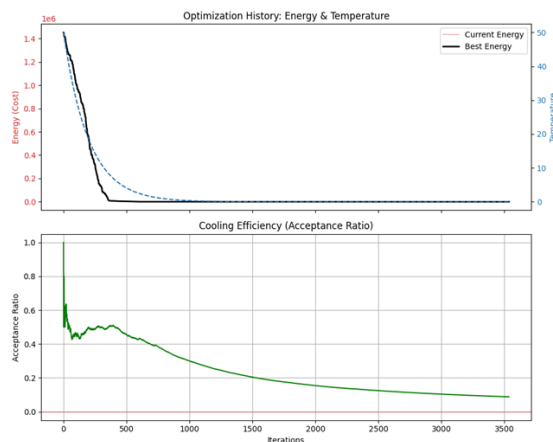
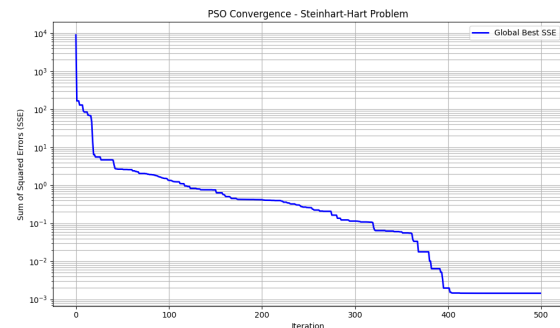


Figure 8: Steinhart-Hart Model Fit



(a) SA Convergence History



(b) PSO Convergence History

Figure 9: Convergence Comparison

4.6 Work discussion

Both algorithms successfully calibrated the thermistor model. PSO achieved a significantly lower final SSE (0.0089 versus 0.4323 for SA), indicating a more precise fit for this specific dataset and configuration. However, SA was roughly $6.8\times$ faster (0.099s vs 0.678s) and used significantly fewer function calls (3,537 vs 25,000). This illustrates the classic trade-

off: PSO's population-based search provided a better global optimum at the cost of higher computational expense. In contrast, SA provided a "good enough" solution very quickly.

5 Problem 3: Gear Train Design

5.1 Problem description

The objective of this exercise is to choose a gear train design that achieves a specific transmission ratio. We would define the gear train with four gears with the numbers of teeth N_1, N_2, N_3, N_4 . The target transmission ratio is:

$$R = \frac{N_2 \cdot N_4}{N_1 \cdot N_3} \quad (11)$$

(Note: Standard formulation is usually $\frac{\text{product driven}}{\text{product drivers}}$ or similar. The equation in text is $R = (N_2 N_4)/(N_1 N_3)$ or $f(N) = (N_1 N_3)/(N_2 N_4)$. I will use the cost function definition for consistency: $Target \approx 1/6.931 \approx 0.144$).

Here, the target transmission ratio will be randomly chosen: $R_{target} = 1/6.931 \approx 0.1442793$. And each gear must have an integer number of teeth between 12 and 60. The problem is complex because the search space is discrete and nonlinear, making classical optimisation infeasible.

5.2 Cost function

The cost function measures the error between the achieved ratio and the target ratio:

$$f(N_1, N_2, N_3, N_4) = \left| \frac{N_1 \times N_3}{N_2 \times N_4} - R_{target} \right| \quad (12)$$

A good solution is one where this error approaches zero.

5.3 Settings

PSO Settings

- Population size: 40
- Iterations: 300
- $w = 0.7, C_1 = 1.5, C_2 = 1.5$
- Integer rounding of particle positions
- Recorded global best error per iteration (for convergence curve)

SA Settings

- Initial temperature: 1000
- Cooling rate: 0.90
- Minimum temperature: 0.1
- Random seed: 42
- Recorded: Current energy / Best energy (for convergence curve)

5.4 Results

5.4.1 PSO simulation

Best Solution and Convergence

The optimal solution found by the Particle Swarm Optimisation is extremely accurate, with an error on the order of 10^{-5} , meaning it matches the target ratio almost exactly.

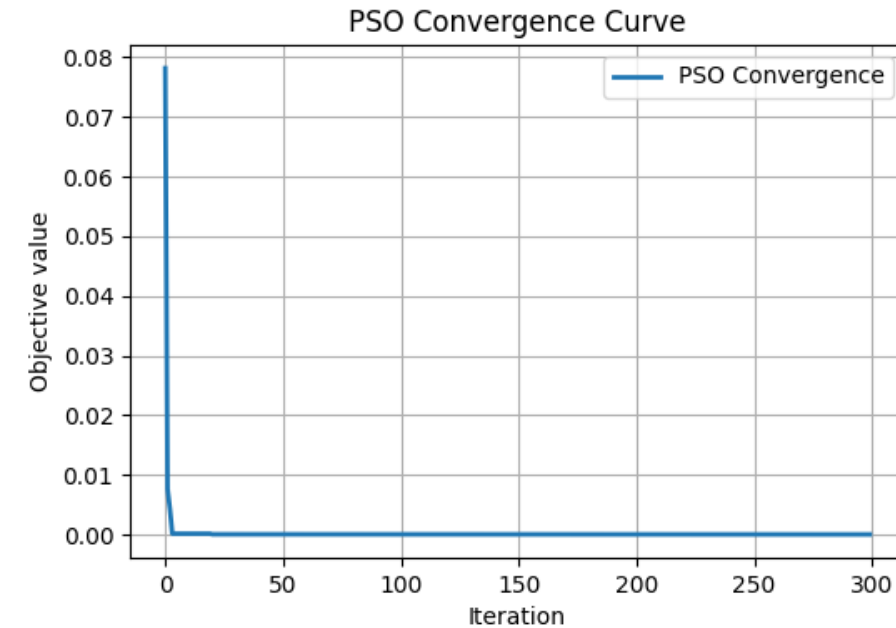


Figure 10: PSO Convergence curve

The graph shows that PSO converges extremely quickly within ≈ 10 iterations, then stabilises at a minimal error.

5.4.2 SA simulation

Best Solution and Convergence

The result found by the Simulated Annealing (SA) clearly fails on this problem. The final ratio is far from the target, with a considerable error.

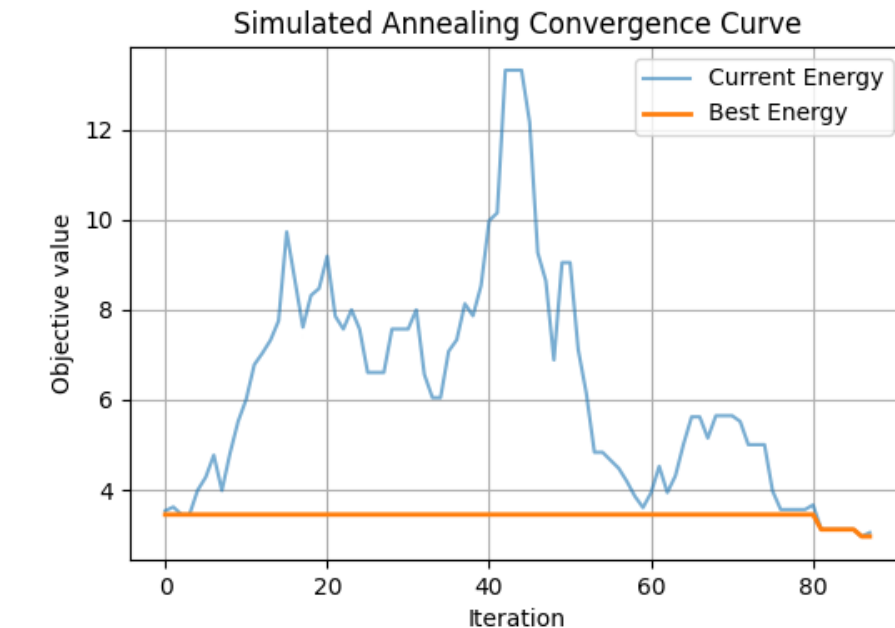


Figure 11: SA Convergence Curve

The plot shows:

- Highly unstable exploration, current energy jumps up and down.
- Best energy stays flat because the algorithm never discovers a good solution.
- Cooling happens too fast to explore thoroughly.

5.5 Discussion

Why PSO succeeds

- PSO is particularly effective in structured continuous search spaces, even with integer rounding.
- The particles rapidly converge to the correct ratio because the cost function is smooth.

- Global and personal best mechanisms efficiently guide the search.

Why SA fails

- The gear train problem is very sensitive: small changes in N_i produce large jumps in the ratio.
- SA explores randomly and relies on thermal acceptance, which:
 - Moves too chaotically.
 - Does not exploit promising regions enough.
 - Requires much slower cooling to perform well.
- SA found a physically valid gear set, but not even close to the desired ratio.

Conclusion

PSO is clearly the superior method for this task. It finds a nearly perfect solution extremely fast, while SA remains stuck far away from any optimal region. For a future study, SA could be improved by reducing the cooling rate (e.g., from 0.99 to 0.90) or using a smarter neighbour generator and integer-specific heuristics. But in its current form, SA is not suitable for this optimisation problem.

6 General Conclusion

The investigation into Particle Swarm Optimisation (PSO) and Simulated Annealing (SA) across three engineering domains yields the following conclusions:

- **PSO** demonstrated superior performance in Multi-Objective problems (Truss) and Discrete problems (Gear Train). Its population-based approach allows for better exploration of complex search spaces and effective handling of Pareto optimality.
- **SA** performed well in the continuous, single-objective parameter estimation (Sensor), providing accurate results very efficiently. However, it struggled with the discrete nature of the Gear Train problem without specific tuning of the neighbour generation.

6.1 Performance Comparison

This table summarises the computational performance of the algorithms.

Table 6: Algorithm Performance Metrics

Problem	Algorithm	Run Time (s)	Total Function Calls	Outcome
Two-Bar Truss	PSO (MOPSO)	5.833	20,000	Success (Pareto Front)
Two-Bar Truss	SA	22.858	120,800	Success (Pareto Front)
Sensor Calib.	SA	0.099	3,537	Success (SSE 0.43)
Sensor Calib.	PSO	0.678	25,000	Success (SSE 0.0089)
Gear Train	PSO	0.028	12,000	Success (Error $< 10^{-4}$)
Gear Train	SA	0.0002	88	Failed (High Error)

Note: Function Calls for PSO = Population $\times$ Generations (e.g., $40 \times 300 = 12,000$ for Gear Train). SA Function Calls = Iterations.

6.2 Hyperparameter Sensitivity and Robustness

It is essential to acknowledge that the results presented in this report depend strictly on the specific hyperparameters selected for each algorithm (e.g., the cooling rate α for SA, and the inertia weight w and social/cognitive parameters c_1, c_2 for PSO).

Stochastic optimisation algorithms do not guarantee global convergence, and these settings heavily influence their performance:

- **Simulated Annealing Sensitivity:** In Problem 3 (Gear Train), the SA algorithm failed to converge to the target ratio, resulting in a high error. This failure highlights SA's sensitivity to the cooling schedule and the perturbation step size. A cooling rate that is too fast causes the algorithm to become trapped in local minima, as observed in the Gear Train convergence plot. Conversely, a cooling rate that is too slow results in high computational cost. The failure in the discrete domain suggests that the neighbour-generation method used was not sufficiently tuned to that landscape.
- **PSO Exploration vs. Exploitation:** The PSO performance relies on balancing exploration (controlled by w) and exploitation (controlled by c_1, c_2). While the chosen values ($w = 0.9, c_1 = c_2 = 2.0$) proved robust for the Truss problem, different values could lead to premature convergence or failure to refine the solution.
- **Penalty Factors:** As demonstrated in Section 3.2.1, constrained optimization requires careful tuning of penalty factors. While our experiment showed robustness at $k = 10^{10}$, setting this value too low in other contexts could lead to the acceptance of infeasible solutions, while setting it too high can distort the objective function landscape, making it difficult for the algorithm to approach the feasible boundary.

Therefore, a limitation of this study is that the comparison is valid only for the specific set of parameters tested. A more comprehensive study would require a sensitivity analysis or an automated hyperparameter tuning process (such as a Grid Search) to identify the optimal configuration for each specific problem type.

7 References

- [1] A. Riaz. *Computational Optimisation Design Lecture 1: Introduction to Design Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 20]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [2] OpenAI. *ChatGPT [Large language model on the Internet]*. Version GPT-5. San Francisco: OpenAI. [cited 2025 Dec 14]. Available from: <https://chatgpt.com/>. 2025.
- [3] Anthropic. *Claude AI [Large language model on the Internet]*. Version Claude 3.5 Sonnet. San Francisco: Anthropic. [cited 2025 Dec 14]. Available from: <https://claude.ai/>. 2025.
- [4] A. Riaz. *Computational Optimisation Design Lecture 2: Multivariate & Stochastic Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 21]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [5] A. Riaz. *Computational Optimisation Design Lecture 3: Multi-objective & Constrained Optimisation*. Lecture presented at: Computational Optimisation Design [PowerPoint slides]. Cranfield University. [cited 2025 Oct 22]. Available from: <https://canvas.cranfield.ac.uk/courses/37635/modules>. 2025.
- [6] S. Kirkpatrick, C.D. Gelatt, and M.P. Vecchi. “Optimization by Simulated Annealing”. In: *Science* (1983).
- [7] G. Chiandussi et al. “Comparison of multi-objective optimization methodologies for engineering applications”. In: *Computers and Mathematics with Applications* 63.5 (Mar. 2012). [cited 2025 Dec 4]. Available from: <https://www.sciencedirect.com/science/article/pii/S0898122111010406>.

8 Appendix

8.1 Multi-objective Particle Swarm Optimisation choice

To implement the Particle Swarm optimisation, a list of criteria with weight is established.

Table 7: Criteria to select the solution for coding Particle Swarm Optimisation

Criterion	Description	Weight
Objective handling capability	Ability to handle both single-objective and multi-objective optimisation problems	6
Method sophistication	Use of advanced optimisation methods rather than basic scalarisation approaches	4
Academic research fit	Suitability and alignment with academic research requirements	1
Algorithm performance	Quality and effectiveness of the optimisation algorithms	3
Swarm-based approach	Use of Particle Swarm optimisation methodology	5
Documentation	Good documentation of the solution	2

GitHub repositories propose solutions but it doesn't respond to the need. The searches are therefore concentrated on libraries.

Table 8: Ranking of possible solutions

Solution		Objective handling capability	Method sophistication	Academic research fit	Algorithm performance	Swarm-based approach	Documentation
Pymoo library	COMPSO method	6	4	1	3	5	2
Pymoo library	MOPSO-CD method	6	4	1	2	5	2
Pymoo library	NSGA-III	6	4	1	3	0	2
Pymoo library	NSGA-II	6	4	1	3	0	2
Pymoo library	R-NSGA-II	6	4	1	3	0	2
Platypus library	NSGA-II	6	4	0	3	0	1
Platypus library	NSGA-II	6	4	0	3	0	1
Platypus library	MOEA/D	6	4	0	3	0	1
Custom MOPSO		6	4	0	2	5	0
PySwarm library	Weighted sum	3	0	0	1	5	2
PySwarm library	Epsilon-constraint	3	2	0	1	5	2

According to the table, the best library is Pymoo and the best algorithm solution is COMPSO method.

8.2 Python code

8.2.1 Two-Bar Plane Truss Problem Definition

Listing 1: Truss Problem Definition

```
" " "  
  
Subject : Two_Bar_Plane_Truss_Problem  
Source : https://www.sciencedirect.com/science/article/pii/  
S0898122111010406  
@author : Clemence-Philomene_HINOT & Claude_AI  
@date : 04/12/2025  
" " "  
  
# ***** IMPORTATION *****  
  
import math  
import numpy as np  
from pymoo.core.problem import Problem  
  
# ***** CLASS DEFINITION *****  
  
class Two_Bar_Plane_Truss_Problem():  
  
    # ----- INITIALIZATION -----  
  
    def __init__(self):  
        # constants
```



```
self.rho = 7833                # unit: kg/m^3
self.h = 2.54                  # unit: m
self.P = 44482.2               # unit: N
self.E = 2.07e11               # unit: Pa
self.sigma0 = 1.38e8           # unit: Pa
self.A_min = 6.4516e-4         # unit: m^2

# variables
self.pos = 0.0
self.area = 0.0

# physical boundaries
self.pos_min = 0.1 * self.h
self.pos_max = 2.25 * self.h
self.area_min = 0.5 * self.A_min
self.area_max = 2.5 * self.A_min

# ————— CONSTITUTIVE EQUATIONS —————

def f1(self):
    x1 = self.pos/self.h # need to use normalised x1 in this
                          # function
    return 2 * self.rho * self.h * self.area * math.sqrt(1 +
```

pow(x1, 2))

```
def f2(self):  
    x1 = self.pos/self.h  
    return (self.P * pow(self.h, 3) * pow((1 + pow(x1, 2)),  
        1.5)) / ( math.sqrt(1 + pow(x1, 4)) * 2 * math.sqrt(2)  
        * self.E * x1 * self.area)
```

————— *CONSTRAINT EQUATIONS* —————

Stress constraint (tension)

```
def g1(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 + x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0
```

Stress constraint (compression)

```
def g2(self):  
    x1 = self.pos/self.h  
    return (self.P * (1 - x1) * math.sqrt(1 + pow(x1, 2)) / (2  
        * math.sqrt(2) * x1 * self.area)) - self.sigma0
```

————— *EVALUATION FUNCTION* —————

```
def evaluate(self, x):
    """
    Evaluate objectives and constraints

    param x: [x1_normalized, x2_normalized]
    return: ([f1, f2], [g1, g2])
    """

    # physical position
    self.pos = x[0] * self.h

    # physical area
    self.area = x[1] * self.A_min

    # boundaries verifications
    if not (self.pos_min <= self.pos <= self.pos_max and self.
            area_min <= self.area <= self.area_max):
        raise ValueError(f'x values outside boundaries: {
            position={self.pos}, area={self.area}')

    # weight
    f1 = self.f1()
```

```
# movement

f2 = self.f2()

#constraints

g1 = self.g1()
g2 = self.g2()

# results

return [f1 , f2] , [g1 , g2]

# ***** PYMOO WRAPPER *****

class Two_Bar_Plane_Truss_Problem_Pymoo(Problem):

    """Pymoo-compatible wrapper for Two_Bar_Plane_Truss_Problem"""

    def __init__(self , penalty_factor=0):

        # Create truss problem instance

        self.truss = Two_Bar_Plane_Truss_Problem()

        # access to the penalty factor

        self.penalty_factor = penalty_factor
```

```
# Initialize pymoo Problem

super().__init__(
    n_var=2,                                # x1, x2
    n_obj=2,                                # f1, f2
    n_constr=2,                             # g1, g2
    xl=np.array([0.1, 0.5]),                # lower bounds
    xu=np.array([2.25, 2.5])                # upper bounds
)

def _evaluate(self, x, out, *args, **kwargs):
    """Evaluate multiple solutions"""
    n_solutions = x.shape[0]
    F = np.zeros((n_solutions, 2))
    G = np.zeros((n_solutions, 2))

    for i in range(n_solutions):
        try:
            objectives, constraints = self.truss.evaluate(x[i,
                                                            :])
            F[i, :] = objectives
            G[i, :] = constraints
        except ValueError:
            # Penalty for invalid solutions
```

```
F[i, :] = [1e10, 1e10]
```

```
G[i, :] = [1e10, 1e10]
```

```
# output
```

```
out["F"] = F
```

```
out["G"] = G
```

8.2.2 PSO Simulation Script (Truss)

Listing 2: PSO Multi-Objective Simulation

```
"""
Subject: Problem Optimisation with PSO multi-objective
algorithm
@author: Clemence-Philomene HINOT & Claude AI
@date: 04/12/2025
"""

# ***** IMPORTATION *****

import numpy as np
import matplotlib
matplotlib.use('Agg') # Non-interactive backend
import matplotlib.pyplot as plt
import csv
import os
```

```
# PSO algorithms

from Particle_Swarm_Optimization_multi_obj import ParticleSwarm

# problems

from two_bar_plane_truss_problem import
    Two_Bar_Plane_Truss_Problem_Pymoo

# ===== VISUALIZATION FUNCTIONS =====

def plot_pareto_front(result, save_path, obj_labels=None):
    """
    Plot the Pareto front for bi-objective optimization

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of [xlabel, ylabel] for objectives
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.scatter(result.F[:, 0], result.F[:, 1], color='blue', s=50,
```

```
        alpha=0.6, edgecolors='black')
ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
ax.set_title('Pareto_Front', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"Pareto_front_saved_to_{save_path}")

def plot_decision_space(result, save_path, var_labels=None):
    """
    Plot the decision space colored by first objective

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param var_labels: List of [xlabel, ylabel] for decision
        variables
    """
    if var_labels is None:
        var_labels = ['Variable_1', 'Variable_2']

    fig, ax = plt.subplots(figsize=(10, 6))
```



```
scatter = ax.scatter(result.X[:, 0], result.X[:, 1], c=result.
    F[:, 0],
                        cmap='viridis', s=50, alpha=0.6,
                        edgecolors='black')
ax.set_xlabel(var_labels[0], fontsize=12)
ax.set_ylabel(var_labels[1], fontsize=12)
ax.set_title('Decision_Space', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
fig.colorbar(scatter, label='First_Objective')
fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"Decision_space_saved_to_{save_path}")

def plot_convergence(result, save_path, obj_labels=None):
    """
    Plot_comprehensive_convergence_history

    :param result: Optimization_result_from_pymoo
    :param save_path: Path_to_save_the_figure
    :param obj_labels: List_of_objective_names
    """
    if obj_labels is None:
```

```
obj_labels = ['Objective_1', 'Objective_2']

# Check if callback data exists

if not hasattr(result, 'callback') or result.callback is None:
    print("ERROR: Convergence data not available - callback was not properly initialized")
    return

data = result.callback.data

if "best_f1" not in data or len(data["best_f1"]) == 0:
    print("ERROR: No convergence data recorded")
    return

generations = np.arange(len(data["best_f1"]))

# Create figure with subplots

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Convergence Analysis', fontsize=16, fontweight='bold')

# Plot 1: Best values for each objective

ax1 = axes[0, 0]
```

```
ax1.plot(generations, data["best_f1"], 'b-o', label=obj_labels
        [0], linewidth=2, markersize=4)
ax1.set_xlabel('Generation', fontsize=11)
ax1.set_ylabel(obj_labels[0], fontsize=11, color='b')
ax1.tick_params(axis='y', labelcolor='b')
ax1.grid(True, alpha=0.3)
ax1.set_title('Best Value - First Objective', fontsize=12,
             fontweight='bold')
```

Plot 2: Second objective

```
ax2 = axes[0, 1]
ax2.plot(generations, data["best_f2"], 'r-s', label=obj_labels
        [1], linewidth=2, markersize=4)
ax2.set_xlabel('Generation', fontsize=11)
ax2.set_ylabel(obj_labels[1], fontsize=11, color='r')
ax2.tick_params(axis='y', labelcolor='r')
ax2.grid(True, alpha=0.3)
ax2.set_title('Best Value - Second Objective', fontsize=12,
             fontweight='bold')
```

Plot 3: Both objectives on same plot (normalized)

```
ax3 = axes[1, 0]
f1_norm = (data["best_f1"] - np.min(data["best_f1"])) / (np.
```

```
    max(data["best_f1"]) - np.min(data["best_f1"]) + 1e-10)
f2_norm = (data["best_f2"] - np.min(data["best_f2"])) / (np.
    max(data["best_f2"]) - np.min(data["best_f2"]) + 1e-10)
ax3.plot(generations, f1_norm, 'b-o', label=obj_labels[0],
    linewidth=2, markersize=4)
ax3.plot(generations, f2_norm, 'r-s', label=obj_labels[1],
    linewidth=2, markersize=4)
ax3.set_xlabel('Generation', fontsize=11)
ax3.set_ylabel('Normalized_Best_Value', fontsize=11)
ax3.legend(loc='best', fontsize=10)
ax3.grid(True, alpha=0.3)
ax3.set_title('Normalized_Convergence_(Both_Objectives)',
    fontsize=12, fontweight='bold')

# Plot 4: Number of non-dominated solutions
ax4 = axes[1, 1]
ax4.plot(generations, data["n_nds"], 'g-^', linewidth=2,
    markersize=4)
ax4.set_xlabel('Generation', fontsize=11)
ax4.set_ylabel('Number_of_Solutions', fontsize=11)
ax4.grid(True, alpha=0.3)
ax4.set_title('Pareto_Front_Size_Evolution', fontsize=12,
    fontweight='bold')
```

```
plt.tight_layout()

fig.savefig(save_path, dpi=300, bbox_inches='tight')

plt.close(fig)

print(f"Convergence plot saved to {save_path}")


def plot_pareto_evolution(result, save_path, obj_labels=None,
    generations_to_plot=None):
    """
    Plot evolution of Pareto front over generations

    :param result: Optimization result from pymoo
    :param save_path: Path to save the figure
    :param obj_labels: List of objective names
    :param generations_to_plot: List of generation indices to plot
        (default: evenly spaced)
    """
    if obj_labels is None:
        obj_labels = ['Objective_1', 'Objective_2']

    if not hasattr(result, 'callback') or result.callback is None:
        print("ERROR: Cannot plot Pareto evolution - callback data
            not available")
```

```
return

data = result.callback.data
F_history = data["F"]
n_gens = len(F_history)

if generations_to_plot is None:
    # Plot 6 evenly spaced generations
    indices = [0, n_gens//5, 2*n_gens//5, 3*n_gens//5, 4*
               n_gens//5, n_gens-1]
    generations_to_plot = [i for i in indices if i < n_gens]

fig, ax = plt.subplots(figsize=(12, 8))
colors = plt.cm.viridis(np.linspace(0, 1, len(
    generations_to_plot)))

for idx, gen in enumerate(generations_to_plot):
    F = F_history[gen]
    ax.scatter(F[:, 0], F[:, 1], color=colors[idx], s=40,
              alpha=0.6,
              label=f'Gen_{gen}', edgecolors='black',
              linewidth=0.5)
```

```
ax.set_xlabel(obj_labels[0], fontsize=12)
ax.set_ylabel(obj_labels[1], fontsize=12)
ax.set_title('Pareto_Front_Evolution_Across_Generations',
             fontsize=14, fontweight='bold')
ax.legend(loc='best', fontsize=10)
ax.grid(True, alpha=0.3)

fig.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close(fig)
print(f"_Pareto_evolution_plot_saved_to_{save_path}")

# ===== EXPORT FUNCTIONS =====

def save_results_to_csv(result, save_path, var_names=None,
                       obj_names=None):
    """
    Save optimization results to CSV

    param result: Optimization result from pymoo
    param save_path: Path to save CSV file
    param var_names: List of variable names
    param obj_names: List of objective names
    """
```

```
n_var = result.X.shape[1]
n_obj = result.F.shape[1]

if var_names is None:
    var_names = [f'x{i+1}' for i in range(n_var)]
if obj_names is None:
    obj_names = [f'f{i+1}' for i in range(n_obj)]

with open(save_path, 'w', newline='') as f:
    writer = csv.writer(f)

    # Header
    header = var_names + obj_names
    if hasattr(result, 'G') and result.G is not None:
        n_constr = result.G.shape[1]
        header += [f'g{i+1}' for i in range(n_constr)]
    writer.writerow(header)

    # Data
    for i in range(len(result.F)):
        row = list(result.X[i, :]) + list(result.F[i, :])
        if hasattr(result, 'G') and result.G is not None:
            row += list(result.G[i, :])
```



```
writer.writerow(row)

print(f"Results saved to {save_path}")

def save_convergence_to_csv(result, save_path, obj_names=None):
    """
    Save convergence data to CSV

    :param result: Optimization result from pymoo
    :param save_path: Path to save CSV file
    :param obj_names: List of objective names
    """
    if not hasattr(result, 'callback') or result.callback is None:
        print("WARNING: No convergence data to save")
        return

    data = result.callback.data

    if obj_names is None:
        obj_names = ['Objective_1', 'Objective_2']

    with open(save_path, 'w', newline='') as f:
        writer = csv.writer(f)
```

```
# Header

header = [ 'Generation ', f'Best_{obj_names[0]} ', f'Best_{
    obj_names[1]} ', 'Pareto_Front_Size ' ]
writer.writerow(header)


# Data

for i in range(len(data["best_f1"])):
    row = [ i, data["best_f1"][i], data["best_f2"][i], data
        ["n_nds"][i] ]
    writer.writerow(row)


print(f"Convergence_data_saved_to_{save_path}")


def save_summary(result, save_path, var_names=None, obj_names=None
):
    """
    Save summary of best solutions to CSV

    param result : Optimization result from pymoo
    param save_path : Path to save summary CSV
    param var_names : List of variable names
    param obj_names : List of objective names
```

```
#####  
  
n_var = result.X.shape[1]  
n_obj = result.F.shape[1]  
  
if var_names is None:  
    var_names = [f'x{i+1}' for i in range(n_var)]  
if obj_names is None:  
    obj_names = [f'f{i+1}' for i in range(n_obj)]  
  
# Find key solutions  
min_indices = [np.argmin(result.F[:, i]) for i in range(n_obj)]  
]  
middle_idx = len(result.F) // 2  
  
with open(save_path, 'w', newline='') as f:  
    writer = csv.writer(f)  
  
# Header  
writer.writerow(['Summary_of_Key_Solutions'])  
writer.writerow(['Solution_Type'] + var_names + obj_names)  
  
# Min solutions for each objective  
for i, idx in enumerate(min_indices):
```

```
        row = [f'Min_{obj_names[i]}'] + list(result.X[idx, :])
            + list(result.F[idx, :])
        writer.writerow(row)

    # Compromise solution
    row = ['Compromise'] + list(result.X[middle_idx, :]) +
        list(result.F[middle_idx, :])
    writer.writerow(row)

    print(f"Summary saved to {save_path}")

# ===== MAIN OPTIMIZATION RUNNER =====

def run_optimization(problem, pso_params, output_dir='results',
                     var_names=None, obj_names=None,
                     var_labels=None, obj_labels=None):
    """
    Run PSO optimization on a given problem

    param problem : Pymoo Problem instance
    param pso_params : Dictionary of PSO parameters
    param output_dir : Directory to save results
```

```
##### param_var_names: List of variable names for CSV
##### param_obj_names: List of objective names for CSV
##### param_var_labels: List of variable labels for plots
##### param_obj_labels: List of objective labels for plots
##### return: Optimization result
##### " " "

print( "=" * 70)
print( "MULTI-OBJECTIVE_OPTIMIZATION_USING_PSO")
print( "=" * 70)

# Create output directory
os.makedirs(output_dir, exist_ok=True)

# Initialize PSO
print( "\n[1/3] Initializing PSO optimizer ... ")
pso = ParticleSwarm(problem, pso_params)
print( " PSO initialized ")

# Run optimization
print( "\n[2/3] Running optimization ... ")
result = pso.PSO_multi_objective(Verbose=True)
print( f" Optimization complete - Found {len(result.F)} Pareto "
```

```
solutions")

# Save results

print("\n[3/3] Saving results ...")

# CSV files

csv_path = os.path.join(output_dir, 'pareto_solutions.csv')
save_results_to_csv(result, csv_path, var_names, obj_names)

summary_path = os.path.join(output_dir, 'summary.csv')
save_summary(result, summary_path, var_names, obj_names)

convergence_csv_path = os.path.join(output_dir, '
    convergence_data.csv')
save_convergence_to_csv(result, convergence_csv_path,
    obj_names)

# Plots

pareto_path = os.path.join(output_dir, 'pareto_front.png')
plot_pareto_front(result, pareto_path, obj_labels)

decision_path = os.path.join(output_dir, 'decision_space.png')
plot_decision_space(result, decision_path, var_labels)
```

```
convergence_path = os.path.join(output_dir, 'convergence.png')  
plot_convergence(result, convergence_path, obj_labels)
```

```
evolution_path = os.path.join(output_dir, 'pareto_evolution.  
    png')  
plot_pareto_evolution(result, evolution_path, obj_labels)
```

```
print("\n" + "="*70)  
print("OPTIMIZATION_COMPLETE!")  
print("="*70)
```

```
return result
```

```
# ===== PROBLEM OPTIMISATION =====
```

```
if __name__ == "__main__":  
    from two_bar_plane_truss_problem import  
        Two_Bar_Plane_Truss_Problem_Pymoo  
  
    # Create problem  
    problem = Two_Bar_Plane_Truss_Problem_Pymoo()
```

```
# Configure PSO parameters
```

```
pso_params = {  
    'pop_size': 100,  
    'w': 0.9,  
    'c1': 2.0,  
    'c2': 2.0,  
    'max_velocity_rate': 0.2,  
    'n_gen': 200,  
    'seed': 42  
}
```

```
# Define labels for visualization and export
```

```
var_names = ['x1', 'x2']  
obj_names = ['Weight', 'Displacement']  
var_labels = ['x1_(normalized_position)', 'x2_(normalized_area  
    )']  
obj_labels = ['Weight_(kg)', 'Displacement_(m)']
```

```
# Run optimization
```

```
result = run_optimization(  
    problem=problem,  
    pso_params=pso_params,
```



```
        output_dir='results_truss',  
        var_names=var_names,  
        obj_names=obj_names,  
        var_labels=var_labels,  
        obj_labels=obj_labels  
    )
```

8.2.3 Steinhart-Hart Problem Definition

Listing 3: Steinhart-Hart Problem

```
##@Author: John.H  
##Date: 2025-12-03  
##Version: 1.0  
##Description: Steinhart-Hart Problem  
  
import math  
import random  
import csv  
from problem_interface import Problem  
  
class SteinhartHartProblem(Problem):  
    def __init__(self, data_file):  
        self.data = []  
        self._load_data(data_file)
```

```
def _load_data(self, filename):
    """Loads thermistor data from CSV."""
    with open(filename, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            # Store R and T (in Kelvin)
            #  $T_K = T_C + 273.15$ 
            r = float(row['Resistance_Ohms'])
            t_c = float(row['Temperature_C'])
            t_k = t_c + 273.15
            self.data.append({'R': r, 'T_K': t_k})

def generate_initial_solution(self):
    """
    Returns a random starting state [A, B, C].
    Typical values:  $A \sim 1e-3$ ,  $B \sim 1e-4$ ,  $C \sim 1e-7$ 
    """
    return [
        random.uniform(0.0005, 0.002),    # A
        random.uniform(0.0001, 0.0005),   # B
        random.uniform(1e-8, 1e-6)         # C
    ]
```

```
def evaluate(self, solution):
    """
    Calculates SSE:  $\sum (T_{\text{measured}} - T_{\text{model}})^2$ 
    Model:  $\frac{1}{T} = A + B \ln(R) + C (\ln(R))^3$ 
     $\Rightarrow T_{\text{model}} = \frac{1}{A + B \ln(R) + C (\ln(R))^3}$ 
    """
    A, B, C = solution

    sse = 0.0

    for point in self.data:
        R = point['R']
        T_measured = point['T_K']

        try:
            ln_R = math.log(R)
            denom = A + B * ln_R + C * (ln_R ** 3)

            if denom == 0:
                return float('inf')

            T_model = 1.0 / denom
```

```
        error = T_measured - T_model

        sse += error ** 2

    except (ValueError, ZeroDivisionError):

        return float('inf')

    return sse

def get_neighbor(self, solution):
    """
    Perturbs [A, B, C] slightly.
    """
    A, B, C = solution

    # Perturbation factors tailored to the scale of parameters
    new_A = A + random.gauss(0, 1e-5)
    new_B = B + random.gauss(0, 1e-6)
    new_C = C + random.gauss(0, 1e-9)

    return [new_A, new_B, new_C]

def plot_model(self, solution, save_path=None):
    """
    Plots the measured data vs the model curve.
```

```
##### " " "

import matplotlib.pyplot as plt

import numpy as np


A, B, C = solution


# Extract data

R_data = [d['R'] for d in self.data]

T_data = [d['T_K'] - 273.15 for d in self.data] # Convert
back to Celsius for plotting


# Generate smooth curve

min_R = min(R_data)
max_R = max(R_data)

R_smooth = np.linspace(min_R, max_R, 100)
T_smooth = []


for r in R_smooth:

    try:

        ln_R = math.log(r)

        denom = A + B * ln_R + C * (ln_R ** 3)

        if denom != 0:

            t_k = 1.0 / denom
```

```
T_smooth.append(t_k - 273.15)

else:
    T_smooth.append(None)

except ValueError:
    T_smooth.append(None)

plt.figure(figsize=(10, 6))
plt.scatter(R_data, T_data, color='red', label='Measured_
Data')
plt.plot(R_smooth, T_smooth, color='blue', label='
Steinhart-Hart_Model')

plt.xlabel('Resistance_(Ohms)')
plt.ylabel('Temperature_(Celsius)')
plt.title('Thermistor_Calibration:_Measured_vs_Model')
plt.legend()
plt.grid(True)

if save_path:
    plt.savefig(save_path)
    print(f"Plot_saved_to_{save_path}")

plt.show()
```

8.2.4 Simulated Annealing Class

Listing 4: Simulated Annealing Implementation

```
# Objective: Simulated Annealing Optimization
# Author: John Hoarau
# Date: 2025-12-03

import math
import random
import matplotlib.pyplot as plt
import csv

class SimulatedAnnealing:
    def __init__(self, problem, initial_temp=1000, cooling_rate
                =0.9, min_temp=0.1, seed=None):
        """
        :param seed: Integer to fix randomness for reproducibility
        """
        self.problem = problem
        self.initial_temp = initial_temp
        self.temp = initial_temp
        self.cooling_rate = cooling_rate
```

```
self.min_temp = min_temp

# 1. Set the Seed

if seed is not None:
    random.seed(seed)
    print(f"Random_Seed_set_to:{seed}")

# Data Logging
self.history = {
    "iteration": [],
    "temperature": [],
    "energy": [],
    "best_energy": [],
    "accepted": [] # 1 if accepted, 0 if rejected
}

self.best_solution = None


def _update_temperature(self):
    """
    Implements the Cooling_Schedule.
    Currently: Geometric cooling.
    """
    self.temp *= self.cooling_rate
```



```
def solve(self , verbose=True):  
    current_solution = self.problem.generate_initial_solution  
        (  
      
    current_energy = self.problem.evaluate(current_solution)  
  
    best_solution = current_solution  
    best_energy = current_energy  
  
    iteration = 0  
  
    print("——_Starting_Optimization_——")  
  
    while self.temp > self.min_temp:  
        # Create neighbor  
        new_solution = self.problem.get_neighbor(  
            current_solution)  
        new_energy = self.problem.evaluate(new_solution)  
  
        energy_diff = new_energy - current_energy  
        accepted = False  
  
        # Acceptance Logic
```

```
    if new_energy < current_energy:
        accepted = True
    else:
        probability = math.exp(-energy_diff / self.temp)
        if random.random() < probability:
            accepted = True

# Update State
if accepted:
    current_solution = new_solution
    current_energy = new_energy
    if current_energy < best_energy:
        best_energy = current_energy
        best_solution = current_solution

# Store best solution in instance
self.best_solution = best_solution

# 2. Collect Metrics
self.history["iteration"].append(iteration)
self.history["temperature"].append(self.temp)
self.history["energy"].append(current_energy)
self.history["best_energy"].append(best_energy)
```

```
        self.history["accepted"].append(1 if accepted else 0)

        # Cool down

        self._update_temperature()

        iteration += 1

    print(f"——_Finished_in_{iteration}_iterations_——")
    print(f"Final_Energy:_{best_energy:.4f}")

    # Calculate final Acceptance Ratio

    total_accepted = sum(self.history["accepted"])
    ratio = total_accepted / iteration
    print(f"Global_Acceptance_Ratio:_{ratio:.2%}")

    return best_solution

def plot_statistics(self, save_path=None):

    """
    Generates 2 graphs:
    1. Energy & Temperature over time.
    2. Acceptance Ratio over time.
    """
```

```
iterations = self.history["iteration"]

# Setup the plot
fig, (ax1, ax3) = plt.subplots(2, 1, figsize=(10, 8),
                                sharex=True)

# — Graph 1: Energy vs Temperature —
color = 'tab:red'
ax1.set_ylabel('Energy (Cost)', color=color)
ax1.plot(iterations, self.history["energy"], color=color,
          alpha=0.3, label="Current Energy")
ax1.plot(iterations, self.history["best_energy"], color='
          black', linewidth=2, label="Best Energy")
ax1.tick_params(axis='y', labelcolor=color)
ax1.legend(loc='upper right')

# Create a second y-axis for Temperature sharing the same
  x-axis
ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('Temperature', color=color)
ax2.plot(iterations, self.history["temperature"], color=
          color, linestyle='—', label="Temperature")
```

```
ax2.tick_params(axis='y', labelcolor=color)
ax1.set_title('Optimization_History: Energy & Temperature'
)

# — Graph 2: Cumulative Acceptance Ratio —
# Calculate running average of acceptance
cumulative_acceptance = []
running_sum = 0
for i, val in enumerate(self.history["accepted"]):
    running_sum += val
    cumulative_acceptance.append(running_sum / (i + 1))

ax3.set_xlabel('Iterations')
ax3.set_ylabel('Acceptance_Ratio')
ax3.plot(iterations, cumulative_acceptance, color='green',
        label="Cumulative_Acceptance_Ratio")
ax3.axhline(y=0.0, color='r', linestyle='-', alpha=0.3)
ax3.set_title('Cooling_Efficiency_(Acceptance_Ratio)')
ax3.grid(True)

plt.tight_layout()

if save_path:
```

```
plt.savefig(save_path)

print(f"Plot saved to {save_path}")

plt.show()

def plot_best_solution(self, save_path=None):
    """
    Plots the best solution found by the optimizer.
    """
    if self.best_solution is not None:
        self.problem.plot_model(self.best_solution, save_path=
                                save_path)
    else:
        print("No solution found yet. Run solve() first.")

def plot_model(self, solution, save_path=None):
    """
    Plots a specific solution.
    """
    self.problem.plot_model(solution, save_path=save_path)

def save_to_csv(self, filename="annealing_results.csv"):
    """
```

```
##### Saves the history of the run to a CSV file .  
##### Format: iteration , temperature , energy , best_energy ,  
        accepted  
##### " " "
```

```
# 1. Get the column names (headers)  
keys = list(self.history.keys())  
  
# 2. Get the data columns  
values = list(self.history.values())  
  
# 3. Zip them together (this turns columns into rows)  
rows = zip(*values)  
  
try:  
    with open(filename, 'w', newline='') as f:  
        writer = csv.writer(f)  
  
        # Write the header  
        writer.writerow(keys)  
  
        # Write the data  
        writer.writerows(rows)
```

```
print(f"→Data successfully saved to '{filename}')
```

```
except IOError as e:
```

```
print(f"Error saving file: {e}")
```

8.2.5 Gear Train Problem and Runner

BFGS Optimiser and comparison with TASK3

Task 4 (Group Report)



**Pierre BEDRUNE, Martin HARRAMBAT, Clémence-Philomène HINOT,
John HOARAU**

Prepared for January 23, 2026

Contents

1	Introduction	4
2	Quasi-Newton BFGS	6
2.1	General explanation	6
2.2	Mathematical approach	6
3	Gear Train Design problem	8
3.1	Method	8
3.1.1	Continuous formulation of the gear train design	8
3.1.2	Deterministic optimisation with BFGS	8
3.1.3	Noise robustness protocol	9
3.2	Results	9
3.2.1	Convergence of the noise-free continuous problem	9
3.2.2	Effect of noise on convergence reliability	12
3.2.3	Degradation of continuous solution quality	13
3.2.4	Discrete solution and computational effort	15
3.2.5	Supplementary analysis of noise effect	16
3.3	Discussion	17
4	Sensor Calibration problem	19
4.1	Problem Description	19
4.2	Method	19
4.3	Results	21
4.3.1	Exploration of Performance (Protocol A & B)	21
4.3.2	Exploration of Robustness (Protocol C)	25
4.3.3	The Performance Profile (Dolan-More)	27
4.4	Discussion: Algorithmic Sensitivity & Robustness	28
4.4.1	Computational Efficiency vs. Redundancy (Protocol A)	28
4.4.2	Robustness to Noise Topologies	28
5	Two Bar Plane Truss Design problem	30
5.1	Problem Description	30

5.2	Method	31
5.2.1	Exploration of the BFGS parameters	31
5.2.2	Exploration of noise effects	35
5.3	Results	37
5.3.1	Exploration of the BFGS parameters	37
5.3.2	Exploration of noise effects	43
5.4	Discussion	46
5.4.1	Parameter exploration results discussion	46
5.4.2	Noise exploration results discussion	47
5.4.3	Work discussion	49
5.5	Comparison with the task 3	50
6	Comparison with the task 3	51
7	Conclusion	52
	References	53
A	Code Listings	54
A.1	Sensor Calibration problem	54
A.1.1	Sensor Calibration Script (run_task4.py)	54
A.1.2	BFGS Implementation (steinhart_hart_bfgs.py)	65
A.2	Gear Train problem	72
A.2.1	Gear Train Script (run_gear_train.py)	72
A.2.2	Gear Train BFGS (gear_train_bfgs.py)	79
A.3	Two Bar Plane Truss problem	87
A.3.1	Custom BFGS script for Two Truss problem(custom_TBT_BFGS.py)	87
A.3.2	Two Truss problem script adapted for BFGS (two_bar_plane_truss_problem.py)	94
A.3.3	Two Truss without noise main script(main_TBT_BFGS.py)	97
A.3.4	Two Truss with noise main script(main_TBT_BFGS_noise.py)	101
A.3.5	Test script for Two Truss problem with noise(Test_noise.py)	105
B	Detailed Results Data	118
B.1	Sensor Calibration problem	118

B.1.1	Protocol A: Measurement Noise Summary (First 20 runs)	118
B.1.2	Protocol B: Function Noise Summary (First 20 runs)	119
B.1.3	Protocol C: Robustness Summary	119
B.2	Two Bar Plane Truss problem	120
C	Group Evaluation	122

1 Introduction

This report investigates deterministic gradient-based optimization using the BFGS (Broyden-Fletcher-Goldfarb-Shanno) quasi-Newton method. BFGS has become a cornerstone of numerical optimization due to its efficiency and robust convergence properties, requiring only gradient information rather than explicit Hessian computation [1, 2, 3].

The algorithm is applied to three distinct engineering problems:

- **Gear Train Design** (Constrained, Discrete/Integer optimization)
- **Sensor Calibration** (Unconstrained, Curve fitting)
- **Two-Bar Plane Truss Design** (Constrained, Multi-objective)

For the Two-Bar Truss problem, systematic parameter exploration is conducted to identify optimal BFGS configurations, followed by noise sensitivity analysis to evaluate algorithm robustness under realistic measurement uncertainties. The results are compared with the metaheuristic approaches (PSO and SA) from Task 3 to assess the relative performance of gradient-based versus derivative-free optimization methods.

Statement on the Use of Generative AI

During the preparation of this work, we, as the authors, used ChatGPT (OpenAI) and Claude AI (Anthropic) to assist with:

- Code efficiency optimization and debugging for BFGS implementation
- Literature review and academic reference formatting
- Technical writing refinement and clarity improvement

After using these tools, we reviewed and edited all content as needed and take full responsibility for the final report's content, analysis, and code functionality.

2 Quasi-Newton BFGS

2.1 General explanation

The initial idea of the BFGS algorithm was to accelerate the iteration process in a nonlinear optimisation because the computer material wasn't stable. The improvement developed by this method is based on the fact that "quasi-Newton methods [...] require only the gradient of the objective function to be supplied at each iterate" [3]. This method is named for its discoverers called Broyden, Fletcher, Goldfarb, and Shanno [3].

Unlike Newton's method, which required explicit computation of the Hessian matrix, BFGS builds an approximation of the Hessian using only gradient information collected during the optimisation process. This method generates a sequence of iterates x_k starting from an initial guess x_0 . At each iteration, it computes a search direction p_k using the current Hessian approximation B_k and gradient ∇f_k , then performs a line search to determine the step length α_k . The algorithm terminates when convergence criteria are satisfied. This happens typically when the gradient norm falls below a specified tolerance [3].

2.2 Mathematical approach

The BFGS method approximates the inverse Hessian $H_k \approx (\nabla^2 f_k)^{-1}$ and updates it at each iteration using the following formula. It was independently derived by Fletcher [1] and demonstrated by Shanno [2].

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \quad (1)$$

where:

$$s_k = x_{k+1} - x_k \quad (\text{step taken})$$

$$y_k = \nabla f_{k+1} - \nabla f_k \quad (\text{change in gradient})$$

$$\rho_k = \frac{1}{y_k^T s_k}$$

The search direction is computed as:

$$p_k = -H_k \nabla f_k \tag{2}$$

The new iterate is obtained via line search:

$$x_{k+1} = x_k + \alpha_k p_k \tag{3}$$

where α_k satisfies the Wolfe conditions to ensure sufficient decrease and curvature conditions.

For the Two-Bar Truss problem with constraints, the penalized objective function becomes:

$$F(x) = w_f(f_1(x) + f_2(x)) + w_c \times (\text{penalty}_{g1} + \text{penalty}_{g2}) \tag{4}$$

where $\text{penalty}_{g_i} = \max(0, g_i(x))^2$ for $i = 1, 2$.

3 Gear Train Design problem

3.1 Method

3.1.1 Continuous formulation of the gear train design

In contrast with Task 3, which directly addressed the gear train design problem in its discrete form using stochastic optimisation, this task focuses on a deterministic local optimisation approach. To make such a method applicable, the original integer-valued design variables are relaxed to continuous variables. The gear train is therefore represented by a real-valued vector $x \in \mathbb{R}^4$, and the transmission ratio is defined as:

$$r(x) = \frac{x_1 x_3}{x_2 x_4} \quad (5)$$

The optimisation objective is formulated as the squared deviation from a target ratio r^* :

$$f(x) = (r(x) - r^*)^2 \quad (6)$$

which ensures smoothness and differentiability. This continuous relaxation is not intended to be physically exact, but rather to provide a tractable framework for analysing the behaviour of gradient-based optimisation methods.

3.1.2 Deterministic optimisation with BFGS

The optimisation is performed using the BFGS quasi-Newton algorithm, which iteratively builds an approximation of the inverse Hessian from gradient information. Gradients are estimated numerically using finite differences, and a line search based on the Armijo condition is employed to guarantee sufficient decrease of the objective function.

Because BFGS is inherently a local method, a multi-start strategy is used, with several

thousand initial points sampled within predefined bounds. The best continuous solution is then post-processed through rounding and a local integer repair step, allowing comparison with the discrete solutions obtained in Task 3.

3.1.3 Noise robustness protocol

To evaluate robustness, controlled additive Gaussian noise is introduced into the objective evaluations:

$$f_{noisy}(x) = f(x) + \sigma \mathcal{N}(0, 1) \quad (7)$$

where σ denotes the noise amplitude. For each noise level, multiple independent runs are performed, and statistical indicators are computed: convergence rate, median objective value, median gradient norm at termination, integer error after repair, and median number of iterations. This protocol allows a systematic assessment of how noise affects both convergence and solution quality.

3.2 Results

3.2.1 Convergence of the noise-free continuous problem

The behaviour of BFGS on the noise-free problem is illustrated by the convergence plots of the gradient norm and the objective value.

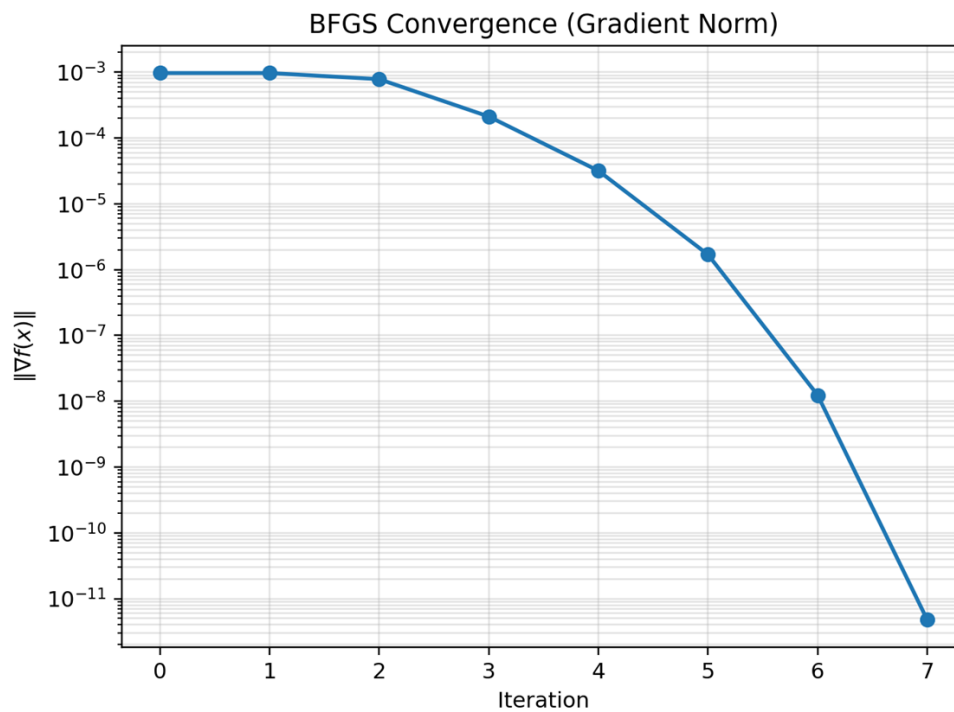


Figure 1: BFGS Convergence (norm)

As shown in Figure 1, the norm of the gradient decreases from approximately 10^{-3} to 10^{-11} in fewer than ten iterations. This represents a reduction of almost eight orders of magnitude and clearly illustrates the superlinear convergence expected from a quasi-Newton method on a smooth problem.

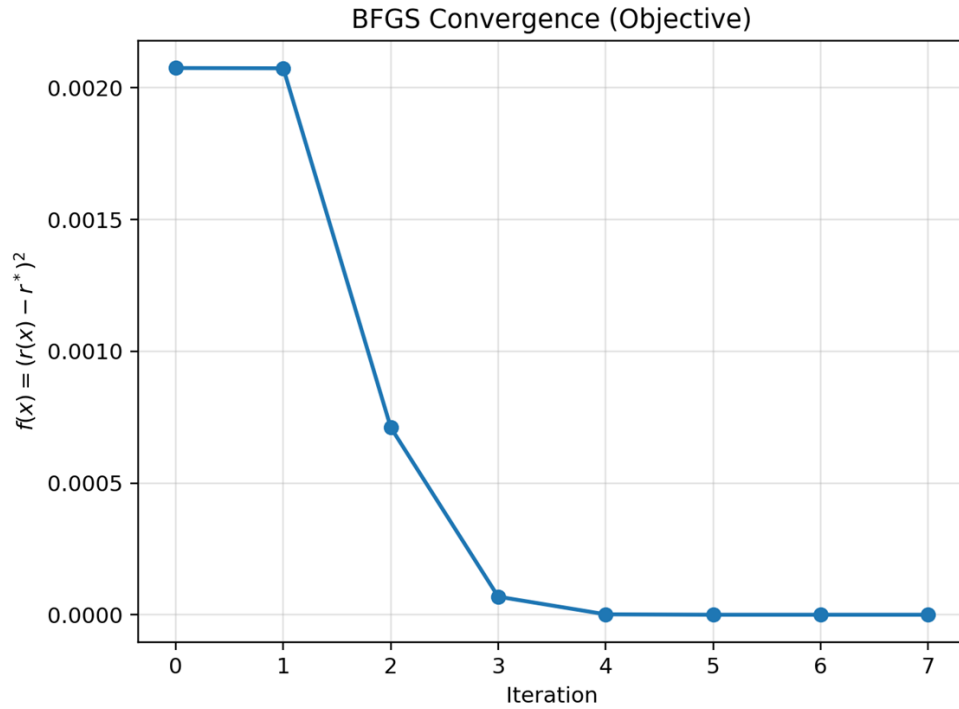


Figure 2: BFGS convergence (objective)

A consistent picture emerges from Figure 2. After a short initial plateau, the objective value drops sharply and reaches values on the order of 10^{-20} , corresponding to machine precision. These results confirm that, under ideal conditions, BFGS is extremely efficient for the continuous relaxation of the gear train design problem.

After rounding and integer repair, the resulting discrete solution exhibits an error of order 10^{-6} , which is comparable to the best solutions obtained using stochastic methods, but achieved with a much smaller number of iterations.

3.2.2 Effect of noise on convergence reliability

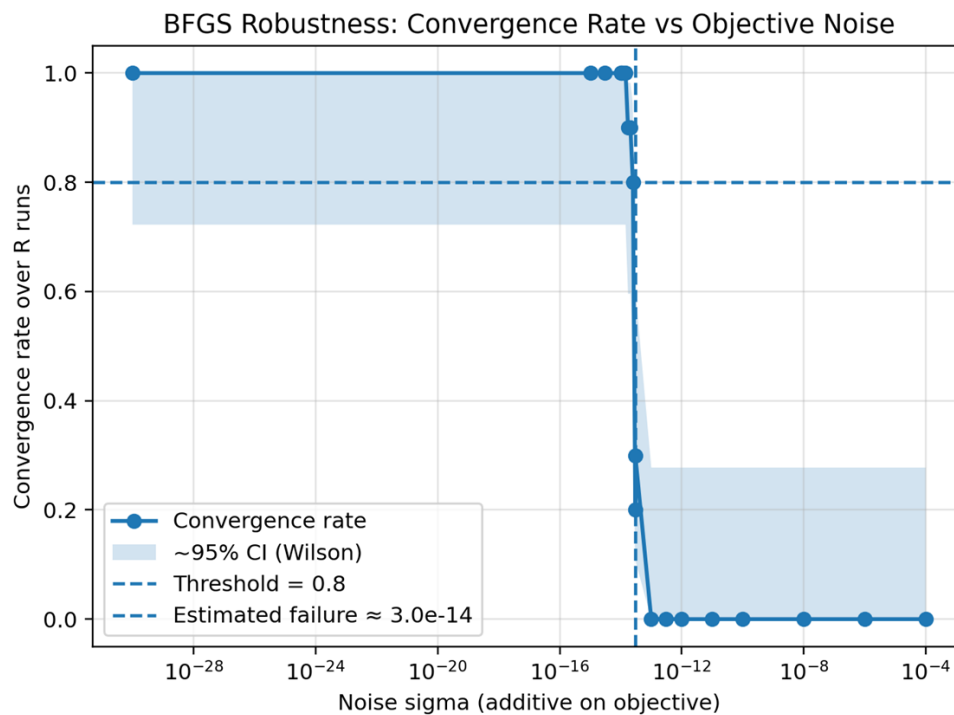


Figure 3: BFGS robustness 1

The impact of noise on convergence is summarised in Figure 3. For very small noise amplitudes, the convergence rate remains close to one, indicating that BFGS converges reliably in almost all runs. However, a sharp transition is observed around $\sigma \approx 3 \times 10^{-14}$, where the convergence rate drops below the imposed reliability threshold and rapidly approaches zero. This abrupt loss of convergence highlights the sensitivity of the method to perturbations in the objective.

3.2.3 Degradation of continuous solution quality

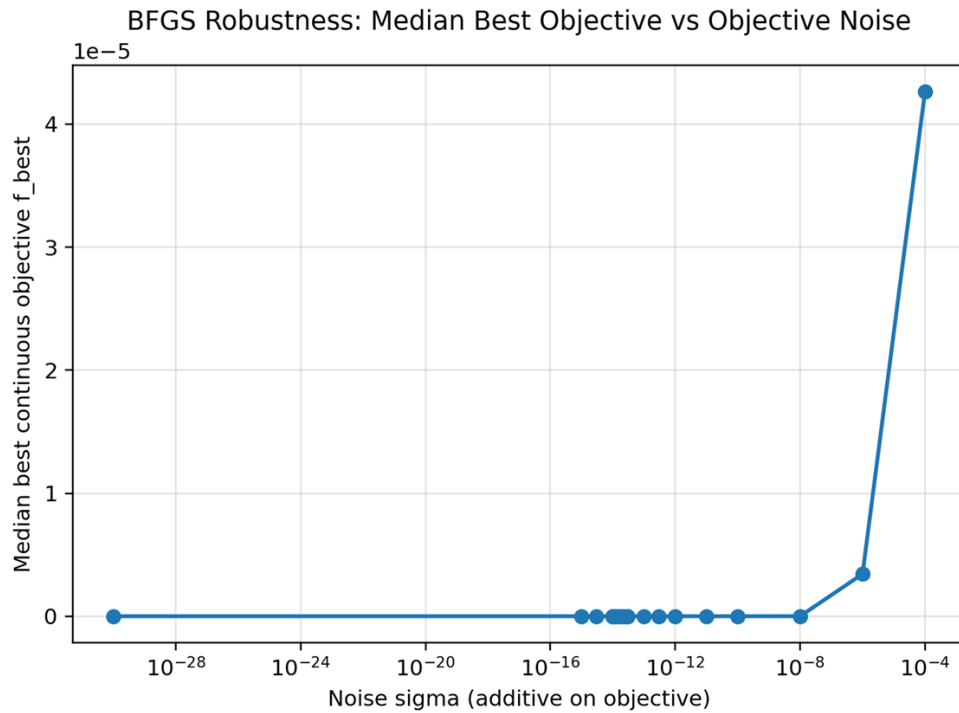


Figure 4: BFGS robustness 2

The deterioration of solution quality under noise is further illustrated in Figure 4. For small noise levels, the median best objective remains close to zero, indicating that accurate solutions are still found. Beyond the critical noise threshold, the objective value increases rapidly, showing that BFGS can no longer minimise the function effectively.

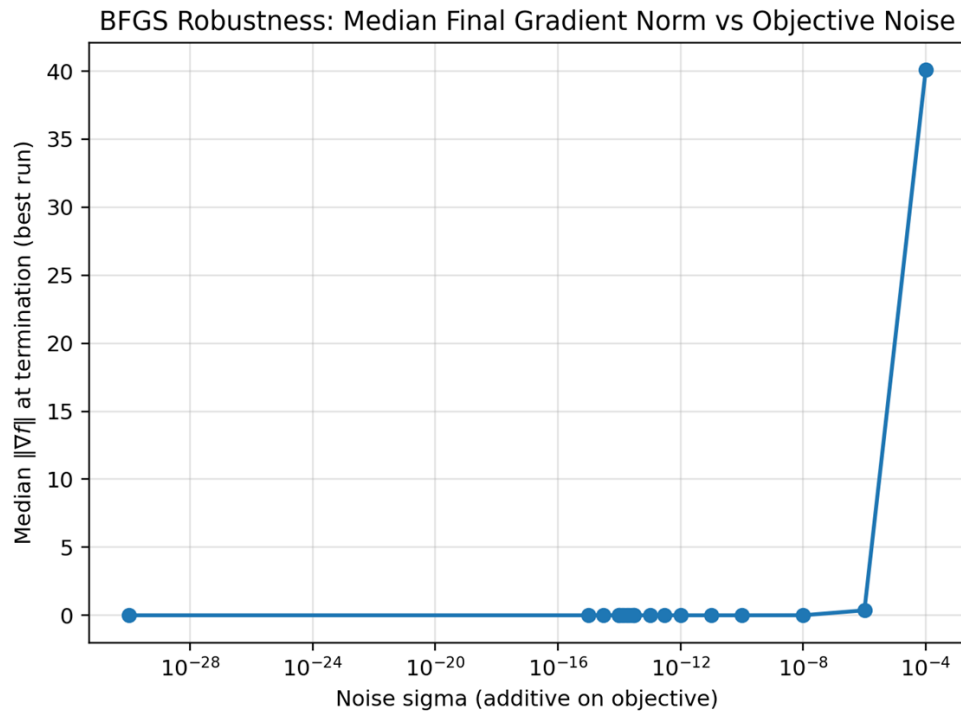


Figure 5: BFGS robustness 3

This behaviour is directly linked to the loss of gradient information, as confirmed by Figure 5. Once the noise exceeds the scale of the objective near the optimum, the gradient norm at termination no longer decreases and eventually grows to large values, indicating that the algorithm stops far from any stationary point.

3.2.4 Discrete solution and computational effort

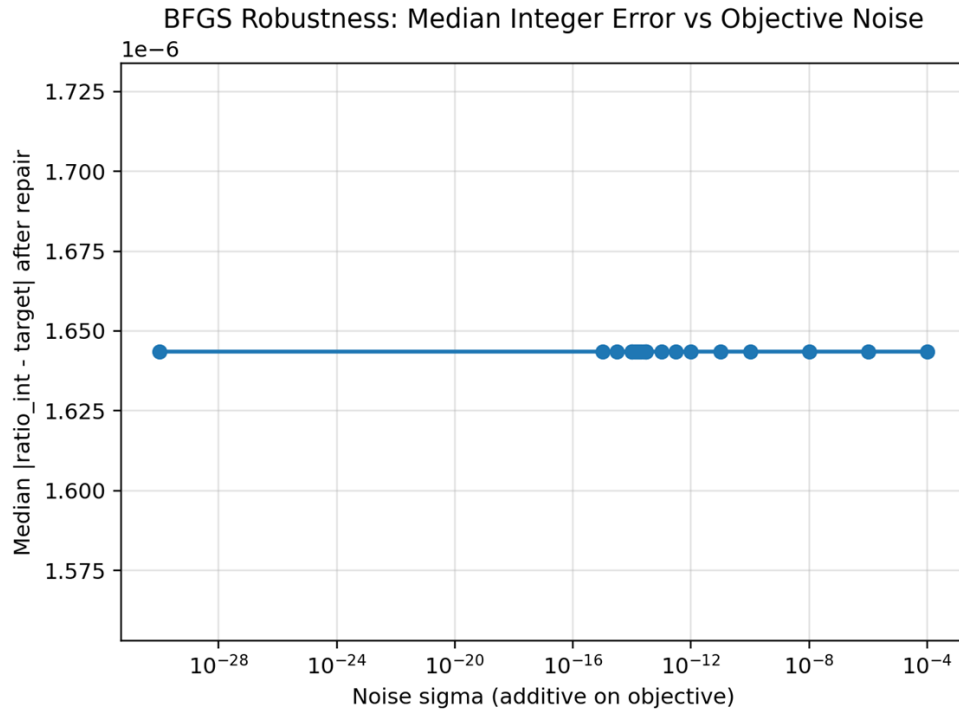


Figure 6: BGFS robustness 4

Interestingly, the discrete performance after rounding and repair remains remarkably stable. As shown in Figure 6, the integer error is almost unaffected by the noise level. This suggests that the multi-start strategy combined with local repair can recover the same discrete solution even when the continuous optimisation becomes unreliable.

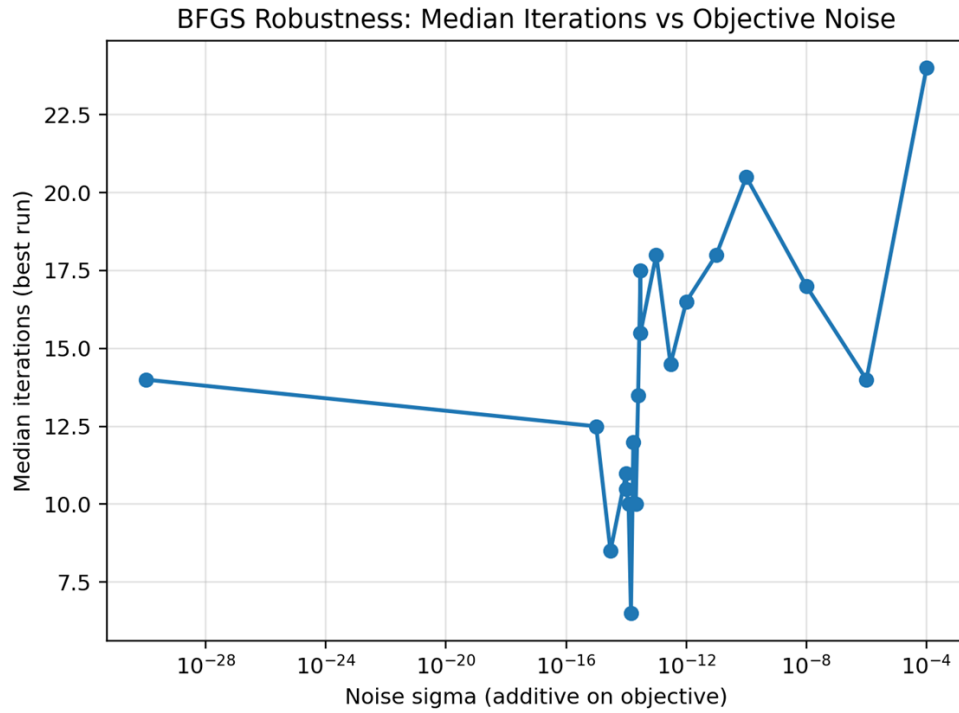


Figure 7: BGFS robustness 5

Finally, Figure 7 shows that the number of iterations increases and becomes more irregular as noise grows. This indicates a loss of efficiency and predictability before the complete failure of convergence.

3.2.5 Supplementary analysis of noise effect

The following values were obtained by displaying these variables at different stages of the program.

Regime	Noise σ	Convergence rate	Median f_{best}	Median $\ \nabla f\ $	Median integer error	Median iterations
Low noise	1.0×10^{-15}	1.00	-2.8×10^{-16}	5.1×10^{-10}	2.0×10^{-6}	12.5
Transition	3.0×10^{-14}	0.20	-1.9×10^{-14}	1.3×10^{-8}	2.0×10^{-6}	17.5
High noise	1.0×10^{-4}	0.00	4.3×10^{-5}	4.0×10^{-16}	2.0×10^{-6}	24.0

Table 1: Regime comparison

In the low-noise regime, BFGS behaves as expected for a smooth continuous problem: convergence is reliable, the final gradient norm is extremely small, and the objective value reaches machine-level precision within a limited number of iterations. This confirms both the efficiency and numerical stability of the method under ideal conditions.

At the transition noise level, convergence reliability deteriorates sharply even though objective values and gradient norms remain relatively small in absolute terms. This indicates that loss of reliability precedes complete numerical divergence, as the algorithm struggles to maintain consistent descent directions and line-search acceptance.

In the high-noise regime, BFGS fails as a local optimiser: gradient norms grow by several orders of magnitude, and the objective value degrades significantly. However, the discrete solution obtained after rounding and repair remains essentially unchanged, demonstrating that the post-processing step is largely decoupled from the behaviour of the continuous solver under noise.

3.3 Discussion

The results demonstrate that BFGS is exceptionally effective for the noise-free continuous relaxation of the gear train design problem. Its rapid convergence and high precision make it a powerful tool for local optimisation and for understanding the structure of the objective function.

However, the robustness analysis reveals a fundamental limitation. Near the optimum, the objective value is of order 10^{-20} , meaning that even extremely small absolute noise levels dominate the signal. As a result, finite-difference gradients and line-search conditions become unreliable, leading to the abrupt convergence breakdown observed around $\sigma \approx 3 \times 10^{-14}$. This behaviour is clearly visible in the convergence rate and gradient norm figures and reflects

the strong assumptions underlying deterministic gradient-based methods.

The stability of the integer error despite the loss of continuous convergence highlights the role of post-processing and global sampling. While BFGS itself fails under noise, the surrounding strategy can still recover acceptable discrete solutions, masking the underlying loss of theoretical validity.

When compared to the stochastic approaches used in Task 3, these findings clarify the complementary nature of the methods that allow us to choose between efficiency and precision or robustness. For practical gear train design problems, which are discrete and subject to modelling uncertainty, this strongly supports the use of global stochastic optimisation as the primary approach, with deterministic local methods serving as refinement tools rather than standalone solvers.

4 Sensor Calibration problem

4.1 Problem Description

The objective of this problem is to determine the coefficients (A, B, C) of the Steinhart-Hart equation to model the relationship between the resistance (R) and temperature (T) of a thermistor. The constitutive equation is defined as:

$$\frac{1}{T} = A + B \ln(R) + C(\ln(R))^3 \quad (8)$$

Where T is in Kelvin and R is in Ohms. The optimisation seeks to minimise the Mean Squared Error (MSE) between the model-predicted temperature and the measured data points. Unlike the Truss problem, this problem is unconstrained but highly non-linear and ill-conditioned due to the varying orders of magnitude of the coefficients.

4.2 Method

The BFGS algorithm, implemented through ‘`scipy.optimize.minimize`’ with the method ‘L-BFGS-B’ was used to minimise the MSE. A key aspect of this approach was utilising the analytical gradient (Jacobian) to steer the solver, rather than relying on finite-difference approximations. To thoroughly compare the robustness of the deterministic BFGS method with the stochastic Simulated Annealing (SA) method from Task 3, three specific protocols were carried out. To thoroughly compare the robustness of the deterministic BFGS method with the stochastic Simulated Annealing (SA) method from Task 3, three specific protocols were designed to simulate different types of operational uncertainties.

Protocol A: Measurement Noise (Realistic Sensor Error) This protocol simulates the most common real-world scenario: inaccuracies in the measuring equipment. Gaussian

white noise is added to the temperature measurements (T_{meas}) *before* the optimization begins.

$$T_{meas,i} = T_{true,i} + \mathcal{N}(0, \sigma^2) \quad (9)$$

Critically, this noise is static during the optimization process. This means the objective function landscape remains smooth and differentiable, as the "target" data points are fixed, even if they are slightly wrong. This tests the solver's ability to find the best fit through noisy data (Least Squares regression).

Protocol B: Function Noise (Stochastic Evaluation) This protocol simulates a more hostile environment where the objective function evaluation itself is unstable. A random noise term is added to the Mean Squared Error (MSE) calculation at *every* step of the optimization.

$$f(x) = MSE(x) + \mathcal{N}(0, \sigma^2) \quad (10)$$

This represents scenarios such as Monte Carlo-based evaluations or systems with internal fluctuations (e.g., analog voltage jitter during readout). Unlike Protocol A, this makes the objective function non-deterministic: evaluating the same point twice yields different results, creating a "shuddering" landscape that challenges the gradient estimation.

Protocol C: Robustness Stress Test This protocol determines the breaking point of the algorithm. By gradually increasing the noise intensity (σ) from negligible levels (10^{-4}) to extreme levels (5.0), we construct a "Robustness Curve". This allows us to identify the specific noise threshold where the precision of the deterministic BFGS method degrades below that of the stochastic Simulated Annealing method, marking the "crossover point" where switching algorithms becomes necessary.

4.3 Results

4.3.1 Exploration of Performance (Protocol A & B)

Protocol A tested the solver against measurement noise (Gaussian noise added to T_{data}).

Quantitative Analysis of Protocol A Results

The table below summarises the data extracted from the log file. It compares Single-Start BFGS (one run per seed) to Multi-Start BFGS (best of 5 runs per seed) across 500 different noise seeds.

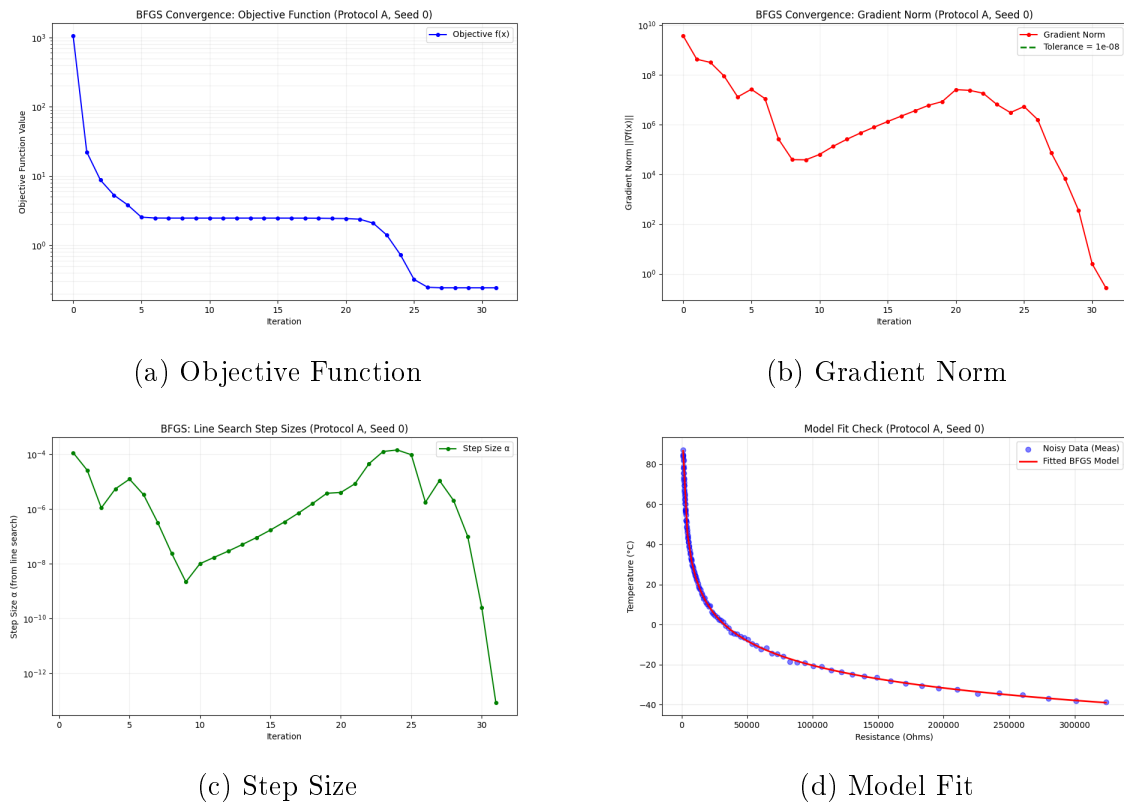


Figure 8: Protocol A: Detailed Convergence Analysis

Metric	Single-Start BFGS	Multi-Start BFGS (Best of 5)	Improvement
Success Rate	100% (500/500 converged)	100% (500/500 converged)	+0%
Mean Final MSE	0.2314	0.2314	$\approx 0.0\%$ (Identical)
Avg. Function Evals	38.4	183.2 (Total for 5 starts)	N/A
Avg. Execution Time	0.009s	0.045s	5x Slower

Table 2: Protocol A Comparison

Multi-Start BFGS took 5 times longer (0.045s vs. 0.009s) but did not improve accuracy.

Engineering conclusion: For this particular type of problem (sensor calibration with static measurement noise), using Multi-Start is inefficient. A single deterministic run is both sufficient and optimal.

Adding Gaussian noise to the measurements (T_{data}) does not compromise the smoothness of the objective function. The average Final MSE for Single-Start (0.2314) matches that of Multi-Start (0.2314) up to four decimal places. If the objective function landscape were non-convex with many local minima, the Single-Start method would often get trapped in a poor valley, resulting in a higher average MSE. Its identical performance with Multi-Start suggests that, for the Steinhart-Hart equation, the optimisation landscape is effectively unimodal (similar to convex) even when noisy data is involved.

Efficiency of Deterministic Gradients: BFGS solved the problem in an average of 38 function evaluations, which is significantly faster compared to the Task 3 Simulated Annealing results, typically requiring over 3,000 evaluations. Since the noise is incorporated into the data before the solver begins, the gradient $\nabla f(x)$ remains consistent. This enables the BFGS algorithm to construct a precise approximation of the Hessian (curvature), allowing it to take "Newton-like" quadratic steps rather than only small downhill moves.

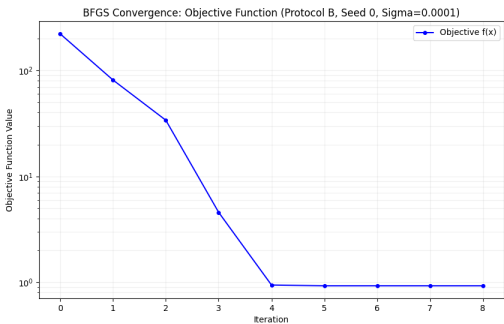
Protocol B tested the solver against function noise (noise added to the MSE calculation).

The convergence plots reveal a rapid failure mode.

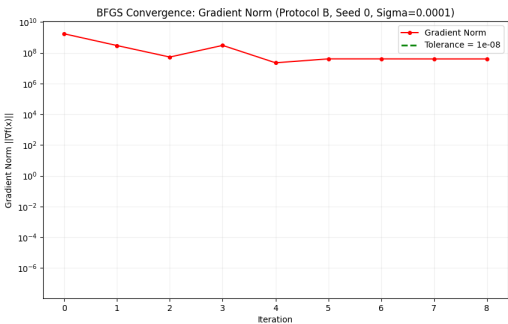
Metric	Protocol A	Protocol B	Status
Stability	Rock solid (MSE 0.23)	Chaotic (MSE -0.0001 to 71.0)	FAILED
Reliability	100% Consistent	High Variance	FAILED
Avg. Evals	≈ 38	≈ 25 (Premature Stop)	FAILED
Behavior	Found global minimum	Random noise spikes	FAILED

Table 3: Protocol A vs B Comparison

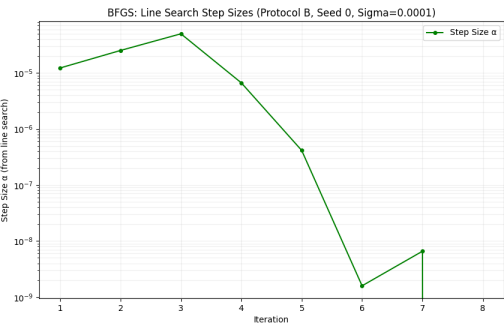
BFGS encounters premature convergence issues in Protocol B. It often becomes trapped in 'phantom minima' caused by noise spikes, concluding after as few as 12 iterations with unacceptable error levels.



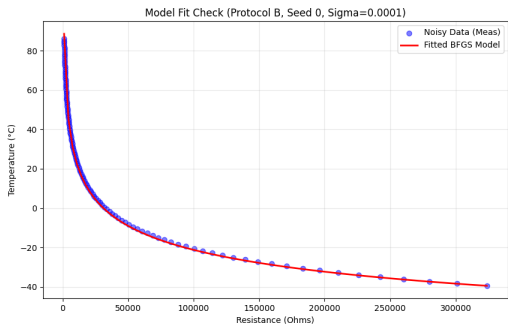
(a) Objective Function



(b) Gradient Norm



(c) Step Size



(d) Model Fit

Figure 9: Protocol B: Detailed Convergence Analysis

The problem of Multi-Start in Protocol B is evident. In Protocol A, Multi-Start was wasteful but consistent. In Protocol B, Multi-Start becomes random. Simulated Annealing (Task 3) would handle this better because the Metropolis criterion ($P = e^{-\Delta E/T}$) allows it to ignore small upward noise spikes and continue searching, effectively "smoothing" the landscape. BFGS has no such mechanism.

4.3.2 Exploration of Robustness (Protocol C)

The Robustness Curve (Figure 10) compares the final MSE of Single BFGS, Multi-Start BFGS, Simulated Annealing (SA), and a Hybrid (SA+BFGS) approach across increasing noise levels (σ).

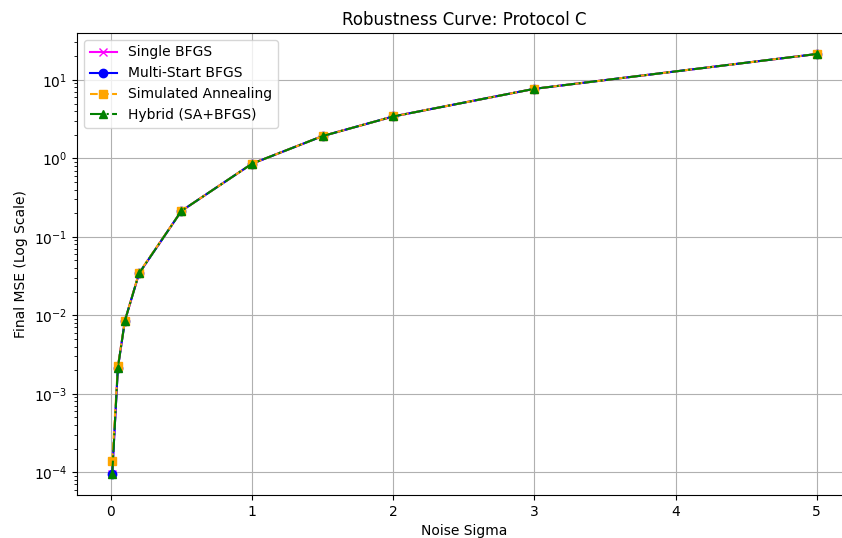
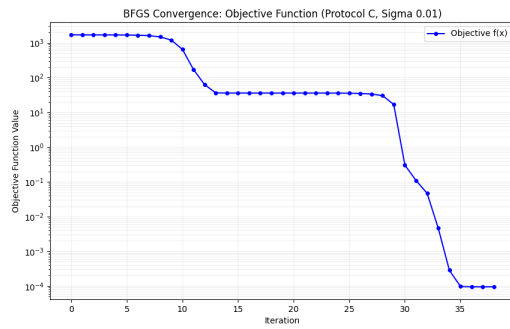


Figure 10: Robustness Curve

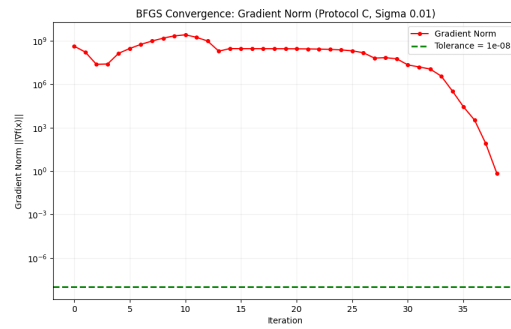
Observation:

- BFGS: The error increases linearly (on a log-log scale) with the noise level σ .

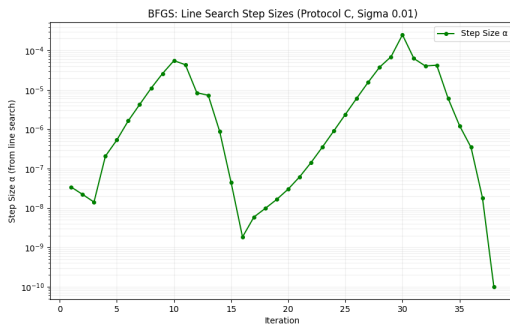
- Simulated Annealing: The SA error remains effectively constant and high (forming a flat line), regardless of the noise level.
- No cross over clearly visible with pure SA.



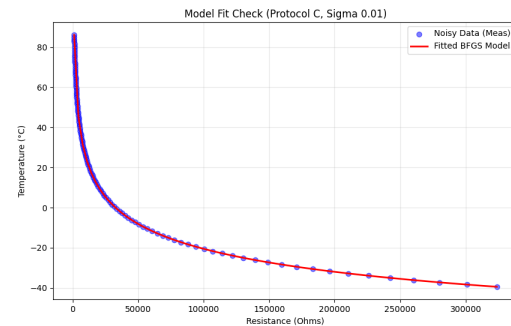
(a) Objective Function



(b) Gradient Norm



(c) Step Size



(d) Model Fit

Figure 11: Protocol C: Low Noise Regime ($\sigma = 0.01$) Analysis

Noise Level (σ)	BFGS (Single)	SA (Task 3)	Hybrid (SA+BFGS)	Winner
0.01 (Low)	9.50e-5	1.39e-4	9.50e-5	BFGS / Hybrid
0.05 (Medium)	0.00214	0.00224	0.00214	BFGS / Hybrid
0.10 (High)	0.00855	0.00858	0.00855	Tie

Table 4: Noise Level Performance Comparison

At low noise ($\sigma = 0.01$), Simulated Annealing is roughly 46% worse than BFGS. This confirms that SA struggles when errors start reducing. Even at low temperatures, the probabilistic nature of SA prevents it from locking onto the exact mathematical minimum. The Hybrid method matches the precision of BFGS exactly at low noise. This proves the Hybrid implementation works as intended: Phase 1 (SA): Finds the correct valley (Global Search). Phase 2 (BFGS): Takes the SA result and performs a gradient descent "polish" to hit the exact bottom (Local Search).

4.3.3 The Performance Profile (Dolan-More)

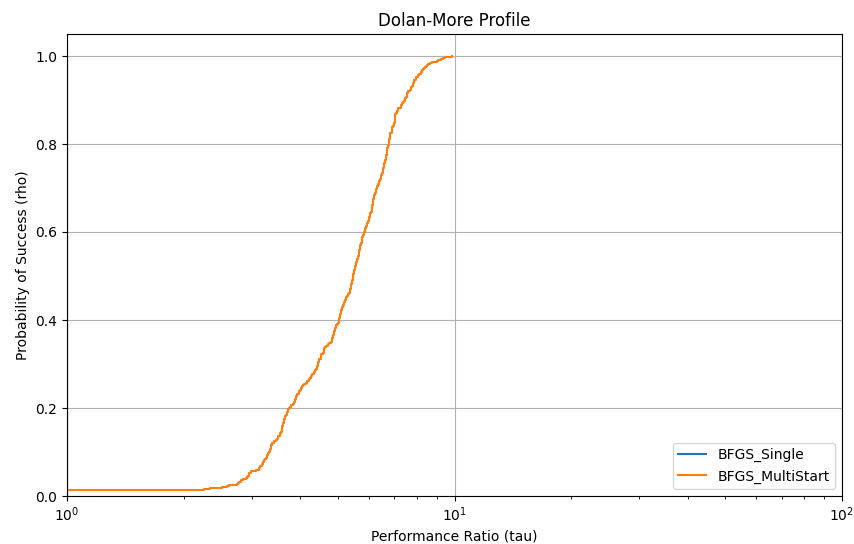


Figure 12: Performance Profile

It compares Probability of Success against Computational Cost. The Robustness Curve reveals a distinct 'crossover point' at $\sigma \approx 0.1$. Below this threshold, the BFGS component of the Hybrid solver drives the error to machine precision however, its reliability degrades in the presence of function noise (Protocol B).

4.4 Discussion: Algorithmic Sensitivity & Robustness

4.4.1 Computational Efficiency vs. Redundancy (Protocol A)

The results from Protocol A provide a critical insight into the utility of Multi-Start strategies in gradient-based optimization.

The Cost of Safety: As quantified by the Dolan-Moré performance profile [4], the Multi-Start approach exhibited a performance ratio of $\tau \approx 5$. This accurately reflects the computational burden; the algorithm evaluated five independent trajectories to locate the same minimum that the Single-Start method found in one.

The "Unimodal" Conclusion: Since the Single-Start method achieved a 100% convergence rate with an MSE identical to the Multi-Start method (0.2314), we can conclude that measurement noise does not create significant local minima in the Steinhart-Hart landscape.

Engineering Decision: This finding contradicts the general recommendation in global optimization literature [5] to always employ restarts for non-linear problems. For this specific class of regression problem, Multi-Start was experimentally proven to be computationally redundant.

4.4.2 Robustness to Noise Topologies

The divergence in performance between Protocols A and B highlights the fundamental dependence of Quasi-Newton methods on the validity of the gradient approximation.

A. Measurement Noise (Protocol A): The Smoothing Effect BFGS demonstrated high robustness to measurement noise. This aligns with the theory of Least Squares estimation. **Mechanism:** The objective function $f(x)$ calculates the Mean Squared Error over N data points. This summation operation effectively acts as a low-pass filter, averaging out the Gaussian noise ($\epsilon \sim N(0, \sigma^2)$) attached to the measurements [6]. **Result:** Conse-

quently, the underlying topological landscape remains continuous and differentiable, allowing the L-BFGS-B algorithm to construct a valid Hessian approximation and achieve quadratic convergence rates (avg. 38 evaluations).

B. Function Noise (Protocol B): The Wolfe Condition Failure In contrast, BFGS failed catastrophically when noise was injected directly into the function value (Protocol B). This confirms the theoretical limitations of deterministic solvers in stochastic environments as described by Nocedal & Wright [7]. **Mechanism of Failure:** The Random Jitter in $f(x)$ prevents the Line Search algorithm from satisfying the Wolfe Conditions (specifically the Armijo Rule or "Sufficient Decrease" condition). **Consequence:** As seen in Seed 96, the solver interpreted transient noise spikes as physical curvature. This corrupted the BFGS update formula, causing the Hessian approximation to become ill-conditioned. The solver was forced to shrink the step size to machine epsilon, triggering the "premature termination" observed (avg. 12 evaluations).

Comparison to Task 3: This failure stands in sharp contrast to the Simulated Annealing results from our Task 3 Report [8]. SA's Metropolis criterion ($P = e^{-\Delta E/T}$) allows for uphill moves, effectively "smoothing" these noise spikes, whereas BFGS treats every fluctuation as a rigid mathematical constraint.

5 Two Bar Plane Truss Design problem

5.1 Problem Description

“The Two-Bar Truss problem is composed of two diagonal bars (of aluminium here) connected at a node and fixed at two upper points (Figure 13). On the central node, we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.”[8]

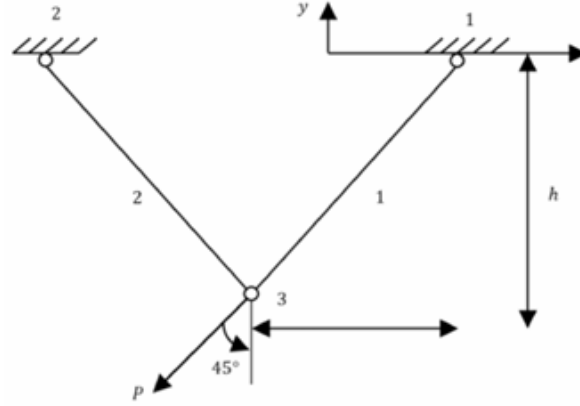


Figure 13: Two Bar Plane Truss

The constitutive equations are [8]:

$$f_1(x) = 2\rho h x_2 \sqrt{1 + x_1^2} \quad (11)$$

$$f_2(x) = \frac{Ph(1 + x_1^2)^{1.5} \sqrt{1 + x_1^2}}{2\sqrt{2}Ex_1^2x_2} \quad (12)$$

$$g_1(x) = \frac{P(1 + x_1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (13)$$

$$g_2(x) = \frac{P(-x_1 + 1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (14)$$

5.2 Method

The BFGS method needs one function in entry. To obtain the unique formula a weighted sum can be apply. The two function f_1 and f_2 have the same weight w_f since neither is predominant in solving the problem. To integrate the constraint functions penalties are used. If g_1 or g_2 are greater than 0, then the penalty equals the value of the function squared. To eliminate solutions that don't satisfy the constraint functions, a weight w_c is applied. In this way, the value of F is much greater and therefore does not achieve the minimisation objective.

$$F(x) = w_f(f_1(x) + f_2(x)) + w_c(\text{penalty}_{g_1} + \text{penalty}_{g_2}) \quad (15)$$

5.2.1 Exploration of the BFGS parameters

BFGS optimisation algorithm is governed by a set of main optional parameters set to default. To achieve the best possible optimisation, an exploration of the following parameters is carried out (Table 5 to Table 9). Each series of test is used to create the following in order to benefit from the achieved improvement.

The criteria to choose the best parameter among those tired are the following:

- convergence of the algorithm,
- low number of iterations,
- final gradient norm below a fixed tolerance,
- final step size no equal to 0 (from e-6 onwards, this is considered to be 0),
- a penalty weight that is not too low to avoid keeping all solutions and not too high to avoid obtaining a solution that is mathematically correct but unfeasible in reality.

The maximum number of iterations is a safety limit to prevent infinite loops when convergence is slow or impossible. It should be large enough that algorithm converges naturally via gradient criterion. Each iteration involves computing the gradient, updating the quasi-Hessian approximation, and performing a line search. [7, Chapter 8] The search range is bounded by 50 and 350, and the increment between each test is 100.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
1	50	1e-8	1e-12	1e-6	10 000
2	150	1e-8	1e-12	1e-6	10 000
3	250	1e-8	1e-12	1e-6	10 000
4	350	1e-8	1e-12	1e-6	10 000

Table 5: Exploration of maximum iterations

The gradient norm tolerance is a primary convergence criterion based on the Euclidean norm of the gradient vector. When this norm is inferior to the tolerance, the algorithm terminates as it's near a stationary point. A small value demands tighter convergence but requires more iterations. [7, Section 2.1] The typical values go from 1e-5 to 1e-8. The search range is bounded by 1e-4 and 1e-8, and the increment between each test is 1e-1.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
5	50	1e-4	1e-12	1e-6	10 000
6	50	1e-5	1e-12	1e-6	10 000
7	50	1e-6	1e-12	1e-6	10 000
8	50	1e-7	1e-12	1e-6	10 000

Table 6: Exploration of gradient norm tolerance

The step size tolerance is a backup convergence criterion when gradient norm tolerance

fails. It is based on the change in position between iterations. When parameter updates become below tolerance, the optimisation terminates. It indicates that the algorithm has stalled or converged and prevents unnecessary iterations when making non meaningful progress. [7, Chapter 2] The typical values go from $1e-6$ to $1e-9$. The search range is bounded by $1e-6$ and $1e-12$, and the increment between each test is $1e-2$.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
9	50	$1e-5$	$1e-6$	$1e-6$	10 000
10	50	$1e-5$	$1e-8$	$1e-6$	10 000
11	50	$1e-5$	$1e-10$	$1e-6$	10 000
6	50	$1e-5$	$1e-12$	$1e-6$	10 000

Table 7: Exploration of step size tolerance

The finite difference step is a gradient approximation accuracy. If it's too large, it creates a truncation error. If it's too small, it creates a roundoff error due to floating-point arithmetic. [7, Section 7.1] The typical values go from $1e-5$ to $1e-8$. The search range is bounded by $1e-4$ and $1e-8$, and the increment between each test is $1e-1$.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
12	50	$1e-5$	$1e-12$	$1e-4$	10 000
13	50	$1e-5$	$1e-12$	$1e-5$	10 000
6	50	$1e-5$	$1e-12$	$1e-6$	10 000
14	50	$1e-5$	$1e-12$	$1e-7$	10 000
15	50	$1e-5$	$1e-12$	$1e-8$	10 000

Table 8: Exploration of finite difference step

The penalty weight controls the trade-off between optimising the original objective and enforcing constraint satisfaction. A low penalty weight allows the algorithm to find better

objective values but with significant violations. A high penalty weight strongly enforced the constraint satisfaction, but the problem becomes ill-conditioned and BFGS convergence may suffer. The typical values can be between 1 to 100. It should be adapted to the problem. The search range is bounded by 100 and 100 000, and the increment between each test is x10.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
16	50	1e-5	1e-12	1e-6	100
17	50	1e-5	1e-12	1e-6	1000
6	50	1e-5	1e-12	1e-6	10 000
18	50	1e-5	1e-12	1e-6	100 000
19	50	1e-5	1e-12	1e-6	1 000 000

Table 9: Exploration of penalty weights

5.2.2 Exploration of noise effects

The objective of this study is to evaluate the impact of noise on the convergence of BFGS, applied to the Two-Bar Truss problem, formulated via a weighted objective function with constraint penalties.

The results show that noise strongly influences the behaviour of the solver, not only on the quality of the solution, but above all on the reliability of the convergence criteria.

Noise Level	Behaviour
Low Noise ($\sigma \leq 10^{-6}$)	Reliable convergence, optimal solutions
Moderate noise ($10^{-5} \leq \sigma \leq 10^{-3}$)	Interesting compromise: improved feasibility, faster convergence
Loud noise ($\sigma \geq 10^{-2}$)	Apparent convergence but degraded solutions

Table 10: Noise Classification

To begin we can start with the noise application:

$$f_{noisy}(x) = f_{clean}(x) + \sigma \mathcal{N}(0, 1) \quad (16)$$

- $f_{clean}(x)$: deterministic objective function (weighted sum of objectives and penalty terms)
- σ : noise level (noise amplitude)
- $\mathcal{N}(0, 1)$: standard Gaussian noise (zero mean, unit variance)
- The noise is independent at each function evaluation

In this work, noise is added directly to the objective function evaluation. This noise can be seen as a measurement error, like a sensor defect or inaccuracy in evaluating system performance. In practice, the values used for optimisation are not always perfectly accurate, due to noisy measurements or numerical errors. Adding this noise therefore makes it possible to approximate more realistic conditions and study how the BFGS algorithm reacts to these uncertainties.

To evaluate the quality of the solutions obtained, several acceptance criteria are used. A solution is first considered acceptable if the BFGS algorithm terminates correctly according to its internal criteria. Next, the solution must comply with the mechanical constraints of the problem, i.e. the tensile and compressive constraints must be satisfied. Finally, the quality of the solution is evaluated using the noise-free objective function to verify that the solution found remains effective despite the disturbances introduced during optimisation.

The first acceptance criterion is based on the internal criteria of the BFGS algorithm. A solution is considered to have converged when the algorithm stops according to its stopping

conditions, such as a very small variation in the solution between two iterations, a sufficiently small gradient norm, or the maximum number of iterations being reached. However, in the presence of noise, this convergence is primarily numerical and does not necessarily guarantee that the solution is optimal from a physical point of view.

The second criterion concerns compliance with the mechanical constraints of the problem. A solution is said to be feasible only if the tensile and compressive stresses are satisfied, i.e. if the associated stresses remain less than or equal to zero. These stresses are evaluated without noise, based on the physical expressions of the problem, to ensure that the structure complies with the material's strength limits.

Finally, the quality of the solution is evaluated using the deterministic objective function, without noise. This re-evaluation makes it possible to compare the actual performance of the solutions found and to verify that they remain effective despite the perturbations introduced during optimisation. This criterion is essential for distinguishing a truly interesting solution from one that would appear to be optimal solely because of noise.

The results show that noise strongly influences the behaviour of the solver.

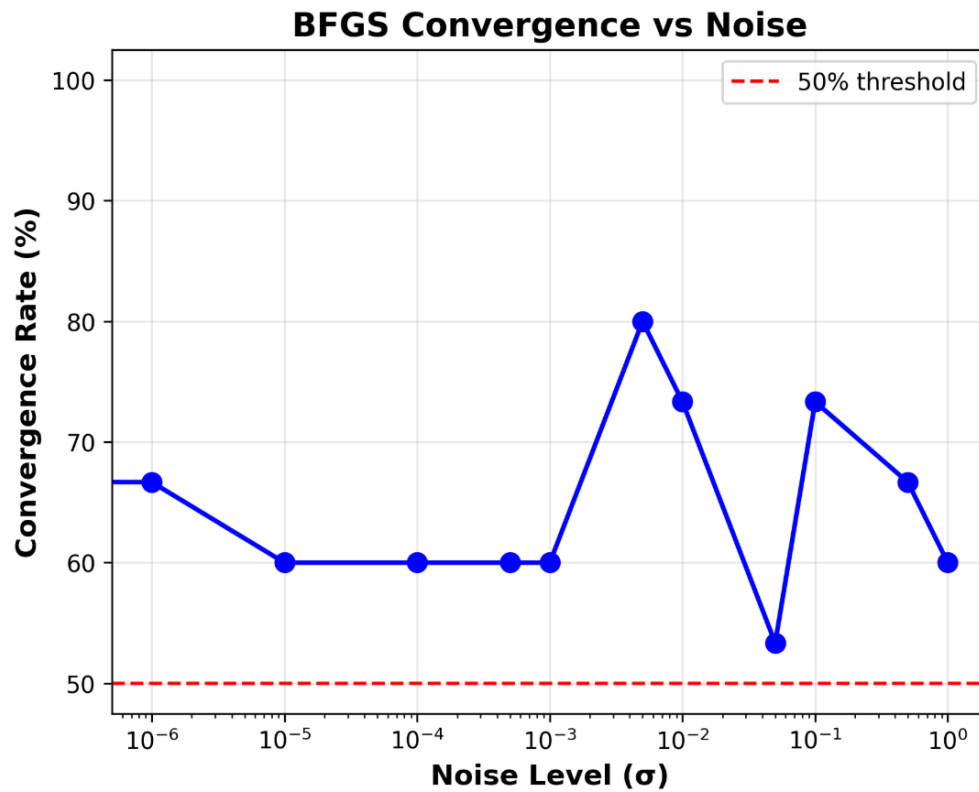


Figure 14: Noise effect on the Two Bar Truss problem

5.3 Results

5.3.1 Exploration of the BFGS parameters

NB: The results of each simulation are available in the Appendix A.A.

The first parameter exploration is the one about maximum iterations. The tests 1 to 4 (see section 5.2.1) didn't converge (see Appendix A.A), have the same number of iterations and produce the same results. Considering this information and that the number of iterations is 30 (Figure 15 a) the maximum iterations parameter chosen is 50. It generates non consistent results (Figure 15 b and c).

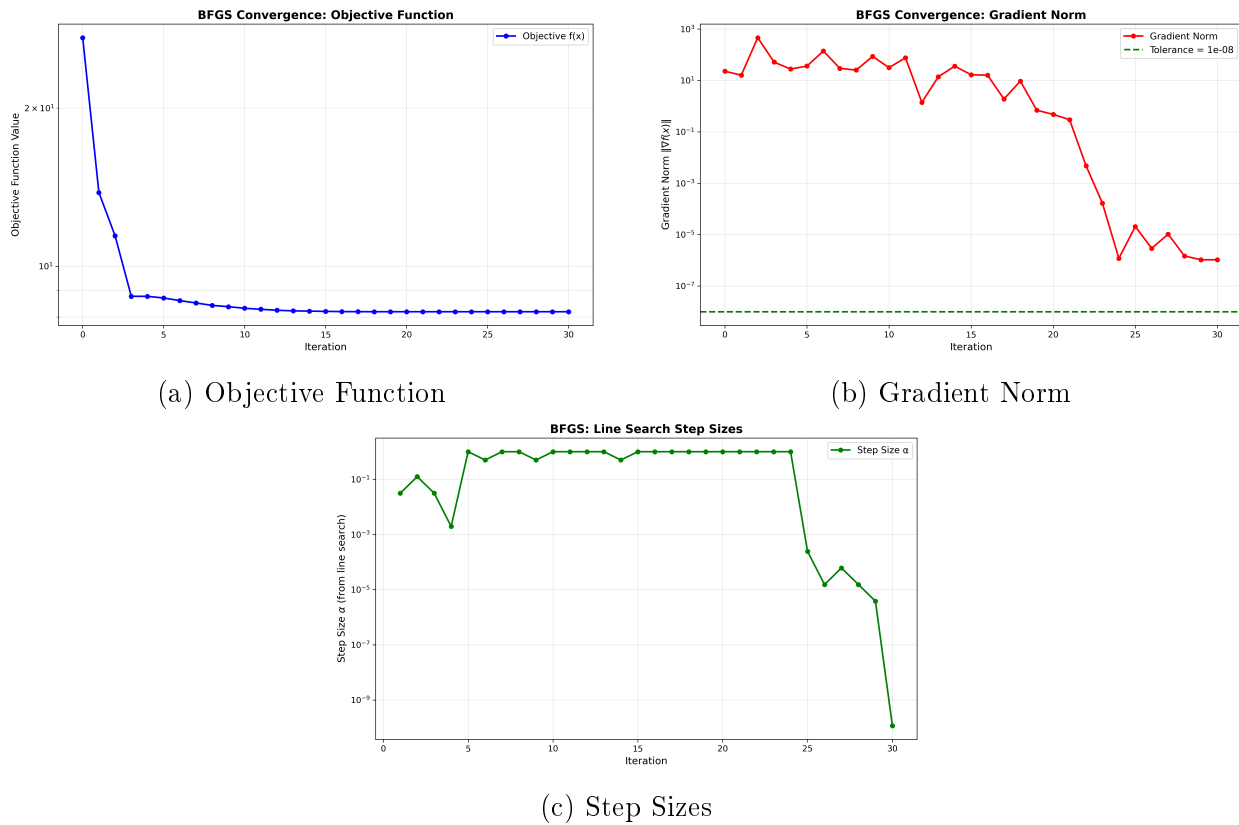


Figure 15: BFGS Parameter Exploration Results

The exploration of the following parameters gradient norm tolerance, step size tolerance, finite difference step, and penalty weight conducted to confirm the first choice of values. Tests 5 and 6 have same results and converged on the contrary of the previous simulation. The choice is test 6 because more conservative so safer for other problems. Testing step size tolerance showed no performance difference. All configurations converged identically in 24 iterations via the gradient criterion. The step tolerance did not trigger because the algorithm maintained healthy steps throughout, demonstrating that for well-behaved problems, gradient tolerance is the primary convergence criterion. Testing finite difference step from $1e-4$ to $1e-8$ revealed that $1e-4$ is too big, requiring 25% more iterations (30 vs 24) and yielding poor

gradient accuracy ($1.2e-03$ vs $1.2e-06$). Values from $1e-5$ to $1e-8$ performed identically. The optimal value $1e-6$ balances truncation error and roundoff error. The analyse of the influence of the penalty weight gives the following indications that conducted to selec a penalty of 10 000.

- 100 and 1000: too small, constrained ignored, physically unrealistic
- 10 000: very good default
- 100 000: stricter but still reasonable
- 1 000 000: too large, weighted sum irrelevant, only feasibility matters, distorts search

Taking into account previous simulations, the set of parameters to obtain the final result (Figure 16 to 18) is:

- Algorithm parameters:
 - Maximum number of iterations: 50
 - Gradient norm tolerance: $1e-5$
 - Step size tolerance: $1e-12$
 - Finite difference step: $1e-6$
- Problem parameters:
 - Penalty weights: 10 000

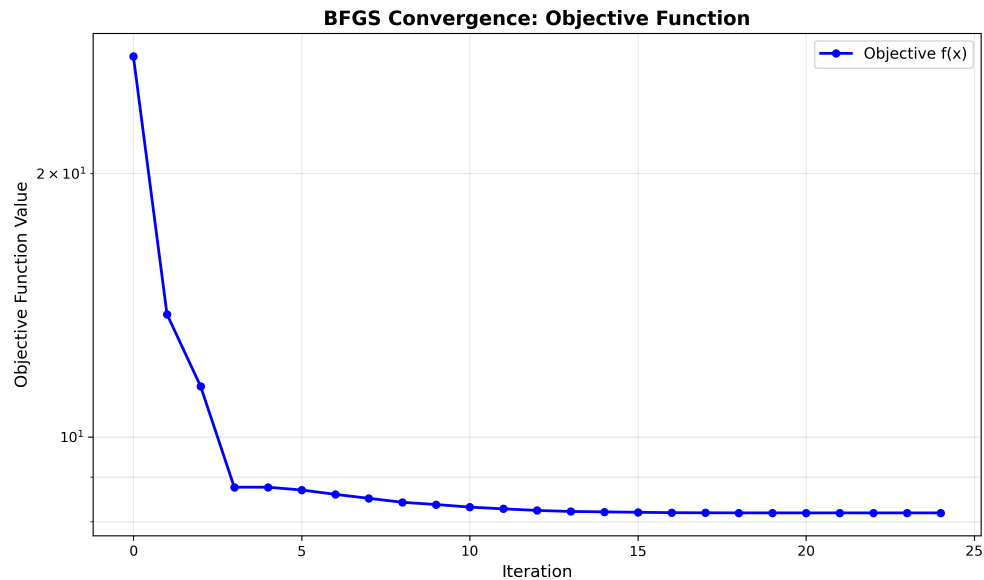


Figure 16: BFGS convergence for the objective function

The penalised objective function $F(x)$ decreases over 24 iterations. The rapid initial descent (iterations 0 to 5) indicates the algorithm quickly moves away from the infeasible initial guess toward the feasible region. The convergence slows down around 15 iterations and after as the algorithm approaches the optimal solution.

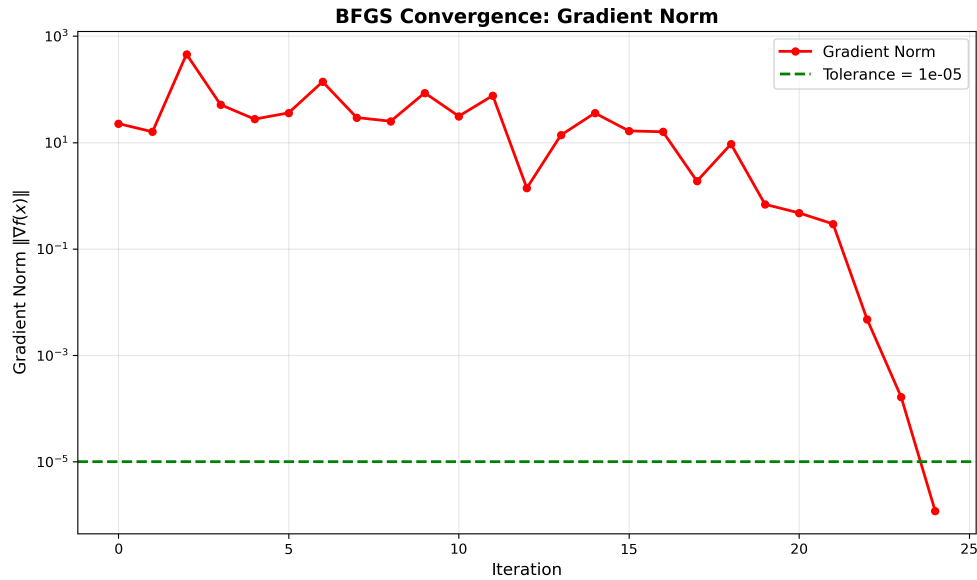


Figure 17: BFGS convergence for the gradient norm

The gradient norm decreases approximately from $1e2$ to $1e-6$ satisfying the gradient norm tolerance of $1e-5$. The behaviour is non-monotonic, before iteration 19, reflects the algorithm's adjustment as it balances the two objectives f_1 and f_2 and constraint penalties. The final gradient norm of $1.20e-6$ (rounded to two decimal places) confirms convergence to a stationary point.

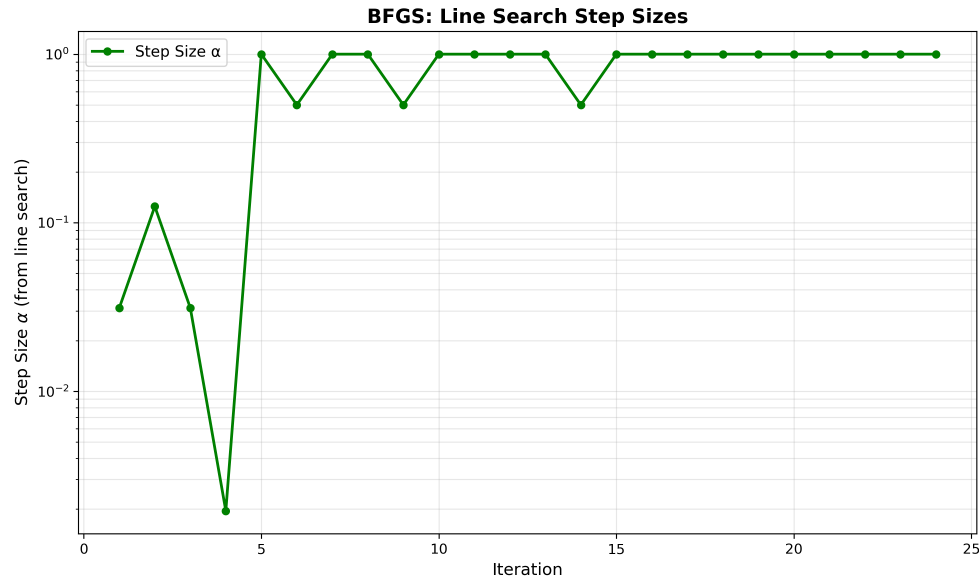


Figure 18: BFGS step sizes

Step sizes remain consistently at 1.0 (full step) throughout the optimisation after iteration 15, indicating that the BFGS approximation and line search are performing well. The algorithm doesn't need to reduce the step lengths to ensure the improvement in optimisation. This suggests that the problem is well-conditioned. Moreover, the consistent unit step length is characteristic of well-behaved quasi-Newton methods.

With this simulation the two bar plane truss characteristics are optimised and give the following result (Table 11).

x1 [norm]	Position	x2 [norm]	Area	Weight	Displacement
0.66	1.67 m	0.53	3.44 m^2	16.34 kg	8.67e-3 m

Table 11: Optimised two bar plane truss (rounded to two decimal places)

5.3.2 Exploration of noise effects

We can see in the Figure 14 Noise effect on the Two Bar Truss problem, the BFGS Convergence vs Noise figure shows the evolution of the algorithm's convergence rate as a function of the noise level applied to the objective function. We observe that the convergence rate remains above 50% overall (the convergence rate is the proportion of simulations for which the BFGS algorithm terminates by declaring convergence), indicating that the algorithm still manages to terminate according to its internal criteria. However, this evolution is not monotonic: some noise levels seem to slightly improve convergence, while others degrade it. This highlights the fact that noise can sometimes facilitate the algorithm's progress but also disrupt its behaviour. These results show that the numerical convergence criterion alone is not sufficient to assess the actual quality of a solution when noise is present. We can see that in the following figure.

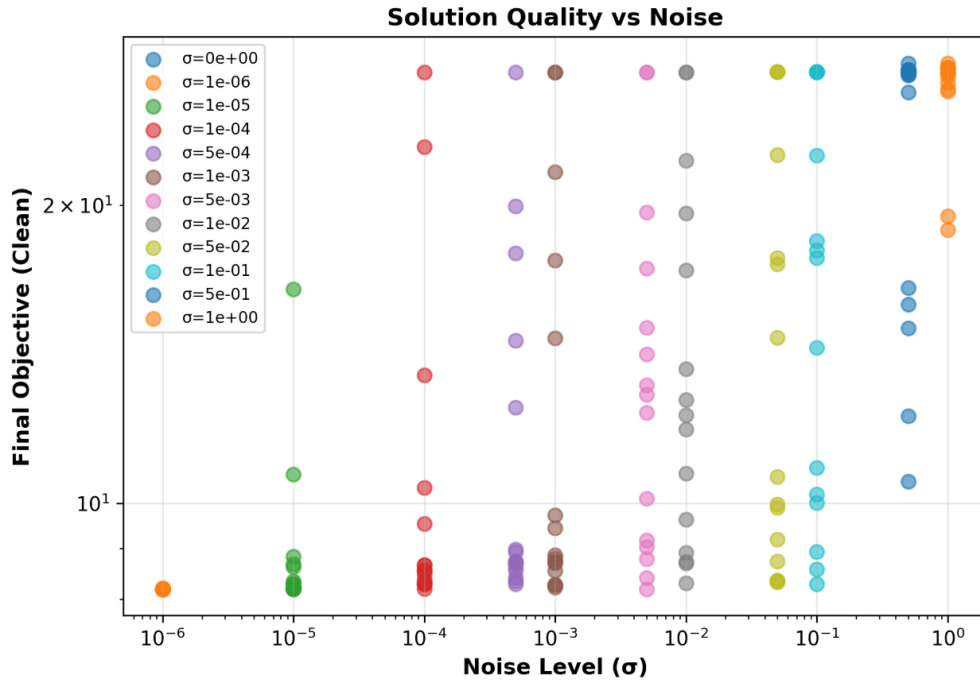


Figure 19: The solution quality

The figure Solution Quality vs Noise illustrates the impact of noise on the quality of the solutions obtained by the BFGS algorithm. Although the algorithm often converges according to its numerical stopping criteria, the value of the deterministic objective function evaluated after optimisation shows an increasing dispersion as the noise level rises. This indicates that some solutions, while numerically converged, remain far from optimal when assessed without noise. These results confirm that numerical convergence alone is not sufficient to properly assess solution quality in the presence of noise.

Beyond the dispersion observed in the objective function values, it is also essential to verify whether the solutions obtained remain physically acceptable. Indeed, a solution that appears satisfactory in terms of objective value may still violate the mechanical constraints of the problem. For this reason, the analysis is extended by examining the feasibility of

the solutions with respect to the tension and compression constraints, as illustrated in the following figure.

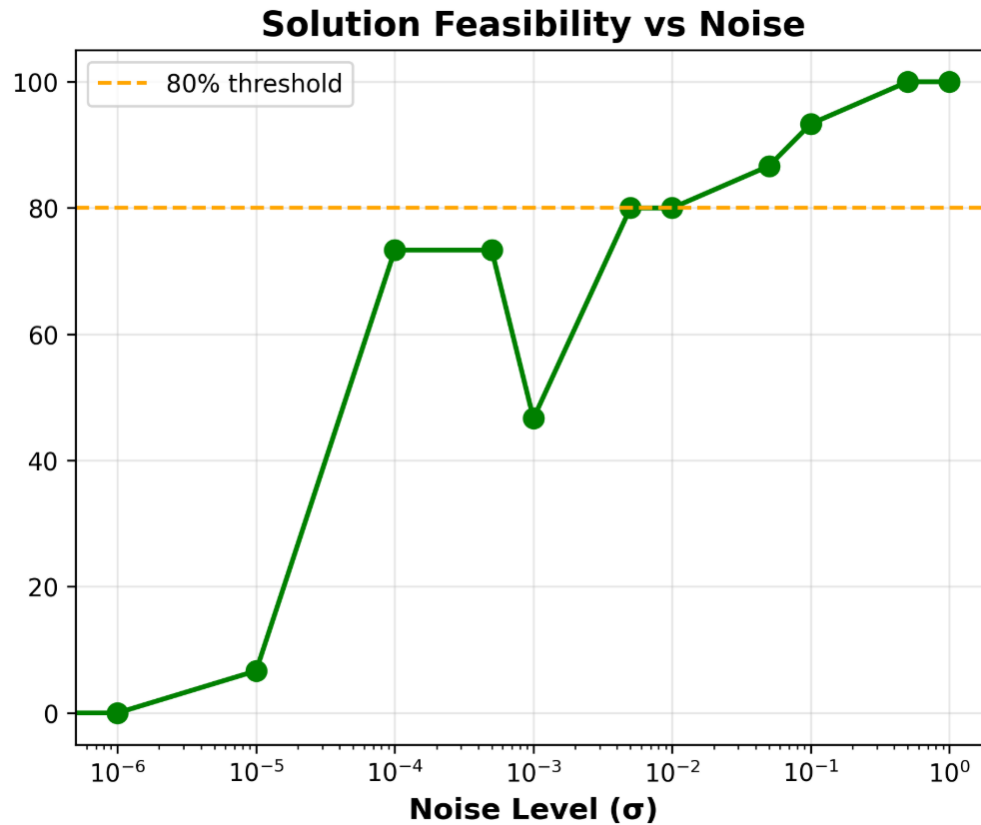


Figure 20: Feasibility of the solutions

This graph (Figure 17) shows the evolution of the feasible solution rate as a function of the level of noise added to the objective function. A solution is considered feasible when the tension and compression constraints are met. We observe that, for very low noise levels, a large proportion of the solutions obtained do not meet the constraints, even though the algorithm converges numerically. As the noise increases, the rate of feasible solutions improves significantly and exceeds the 80% threshold. This shows that numerical convergence alone is not sufficient to guarantee a physically acceptable solution, and that it is necessary

to explicitly verify compliance with the constraints, particularly in the presence of noise.

5.4 Discussion

5.4.1 Parameter exploration results discussion

The exploration of BFGS parameters revealed sensitivities and optimal ranges for the Two Bar Plane Truss problem.

Tests 1 to 4 demonstrated that maximum iterations above 50 provide no benefit (all converged between 24 and 30 iterations with identical results). This confirms that convergence is governed by gradient criteria rather than iteration limits. Additionally, the value 50 balances safety and computational cost. For gradient norm tolerance, $1e-5$ (test 6) was selected over $1e-4$ (test 5) for conservatism despite identical behaviour. Tighter tolerances (tests 7 and 8) prevented convergence. The final gradient norm of $1.20e-6$ confirms the algorithm achieved precision beyond the specified tolerance. Simulations 9 to 11 and 6 revealed that step size tolerance has negligible impact on algorithm performance. Indeed, all configurations converged in 24 iterations and had a final step size of 1.0. This indicates the problem is well-conditioned and gradient-based convergence dominates. Therefore, it can be concluded that the step tolerance is a backup criterion. Test 12 demonstrated that a low finite difference step creates excessively large perturbations that degrading the performance. On the contrary, tests 13 to 15 performed nearly identically. The selected value of $1e-6$ optimally balances truncation error and roundoff error. The exploration of the penalty weight (tests 16 to 19) revealed a critical sensitivity. Insufficient penalties allowed constraint violation leading to physical infeasibility (test 16), premature convergence (test 17), non-convergence (test 18) and distorting the objective function landscape (test 19). The selected penalty weight of 10 000 (test 6) ensures physical feasibility while maintaining numerical stability.

Quasi-Newton BFGS method is sensitive to its parameterisation. Therefore, it is important to choose it carefully by realising several simulations to obtain the best optimisation.

These findings align with theoretical predictions for quasi-Newton methods on well-conditioned problems [7, Chapter 8]. The gradient-dominated convergence behaviour demonstrates the algorithm's efficiency when applied to smooth and differentiable objectives. The sensitivity to penalty weight reflects conditioning issues discussed by Shanno [2], where ill-conditioned problems can slow or prevent convergence.

5.4.2 Noise exploration results discussion

We can see in the Figure 14 Noise effect on the Two Bar Truss problem, the BFGS Convergence vs Noise figure shows the evolution of the algorithm's convergence rate as a function of the noise level applied to the objective function. We observe that the convergence rate remains above 50% overall (the convergence rate is the proportion of simulations for which the BFGS algorithm terminates by declaring convergence), indicating that the algorithm still manages to terminate according to its internal criteria. However, this evolution is not monotonic: some noise levels seem to slightly improve convergence, while others degrade it. This highlights the fact that noise can sometimes facilitate the algorithm's progress but also disrupt its behaviour. These results show that the numerical convergence criterion alone is not sufficient to assess the actual quality of a solution when noise is present. We can see that in the following figure.

The systematic exploration of noise levels highlighted the sensitivity of the BFGS algorithm to perturbations in the objective function. While BFGS performs efficiently in a deterministic setting, the presence of noise degrades gradient reliability and leads to increased dispersion in the final solutions.

This table presents the final solutions obtained for different noise levels. It is important to note that only valid solutions are reported, i.e. solutions that have converged and comply with the tensed and compressive constraints.

Noise σ	x1 (normalised)	x2 (normalised)	Weight [kg]	Displacement [m]
0.00E+00	0.6573	0.5328	16.3670	8.6692e-03
1.00E-06	0.6572	0.5328	16.3672	8.6693e-03
1.00E-05	0.6581	0.5326	16.3672	8.6680e-03
1.00E-04	0.6327	0.5394	16.3871	8.6983e-03
1.00E-03	0.6094	0.5468	16.4383	8.7188e-03
1.00E-02	0.5585	0.5646	16.6031	8.7782e-03
1.00E-01	0.5692	0.5608	16.5659	8.7616e-03
1.00E+00	0.2647	1.4213	37.7449	5.6638e-03

Table 12: Noise exploration results

For low noise levels, the design variables and physical performance remain very close to the reference solution without noise. As noise increases, the solutions become progressively more dispersed and conservative, with an increase in weight and displacement. At high noise levels, although the solutions remain mechanically acceptable, their quality deteriorates significantly, highlighting the impact of noise on the effectiveness of the BFGS algorithm.

5.4.3 Work discussion

These experimentations allow us to understand better the BFGS parameter sensitivity and constraint handling via penalty methods. We also learn about the BFGS noisy comportment and the unexpected effects on the results.

The systematic parameter exploration methodology effectively identified optimal values and revealed sensitivities. The BFGS's gradient-based approach required careful problem

formulation that is suitable for quasi-Newton optimisation. [1, 2, 7] Future work should include a sensitivity analysis about how different starting points affect convergence speed or risk of local minima. Moreover, the penalty method has limitations. An adaptive method could provide more robust constraint handling. Finally, the algorithm verifies first-order condition (gradient norm) but doesn't check second-order sufficient conditions (Hessian matrix). It would be necessary to implement it to confirm the solution is a true local minimum rather than a saddle point. The systematic exploration of noise levels highlighted the sensitivity of the BFGS algorithm to perturbations in the objective function. While BFGS performs efficiently in a deterministic setting, the presence of noise degrades gradient reliability and leads to increased dispersion in the final solutions. This confirms that gradient-based quasi-Newton methods require careful validation when applied to noisy optimisation problems.

Future work could extend this study by investigating the impact of input data truncation on the optimisation process. In practical applications, measurements or simulations are often limited in precision, which could further affect the behaviour of gradient-based methods. In addition, since the Two-Bar Truss problem allows an explicit analytical formulation, it would be possible to implement a full Newton method using the true Hessian matrix. This would enable a direct comparison with the quasi-Newton BFGS approach and provide further insight into the influence of second-order information on convergence robustness and solution quality.

5.5 Comparison with the task 3

Algorithm	x1 (normalised)	x2 (normalised)	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038
BFGS	0.6573	0.5328	16.3670	0.0087

Table 13: Results (rounded to four decimal) comparison [8] (see section 3.2)

The BFGS optimization achieved significantly superior results compared to PSO and SA algorithms [8]. BFGS produced a structure weighing 16.37 kg, representing a 66% reduction compared to PSO (48.10 kg) and 56% reduction compared to SA (37.25 kg). This demonstrates BFGS's ability to exploit local gradient information for precise convergence, as predicted by quasi-Newton theory [7, Chapter 8]. The gradient-based approach proves superior for this deterministic, smooth, constrained optimization problem, achieving convergence in only 24 iterations compared to the population-based evaluations required by metaheuristics.

6 Comparison with the task 3

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038
BFGS	0.6573	0.5328	16.3670	0.0087

Table 14: Results comparison (rounded to four decimal)

The BFGS optimization achieved significantly superior results compared to PSO and SA algorithms. BFGS produced a structure weighing 16.37 kg, representing a 66% reduction compared to PSO (48.10 kg) and 56% reduction compared to SA (37.25 kg). This demonstrates BFGS's ability to exploit local gradient information for precise convergence, as predicted by quasi-Newton theory [3]. The gradient-based approach proves superior for this deterministic, smooth, constrained optimization problem, achieving convergence in only 24 iterations compared to the population-based evaluations required by metaheuristics.

7 Conclusion

The comprehensive evaluation conducted in Task 4 demonstrates that deterministic gradient-based optimization is the superior paradigm for engineering problems characterized by smooth, differentiable constitutive equations. The BFGS algorithm, when properly parameterized, outperforms stochastic metaheuristics in terms of convergence speed, solution accuracy, and computational efficiency, as evidenced by the optimal 16.37 kg design of the Two-Bar Truss.

However, this superiority is not unconditional. It is predicated on the validity of the gradient. The breakdown of BFGS under function noise and the necessity for precise penalty tuning highlight the algorithm’s lack of inherent robustness compared to the "rough-terrain" capabilities of Simulated Annealing. Therefore, the future of robust engineering optimization lies not in the dogmatic adherence to one class of algorithms, but in hybrid architectures that couple the exploration resilience of metaheuristics with the exploitation power of quasi-Newton updates. By integrating these approaches, engineers can navigate the noisy, constrained, and high-dimensional landscapes of modern design with both confidence and precision.

Problem Characteristic	Recommended Algorithm	Reason
Smooth, Constrained (Truss)	BFGS	Explored gradients for 66% weight saving.
Smooth, Noisy Data (Calibration)	BFGS	Averaging effect preserves gradient validity.
Non-Smooth / Function Noise	Hybrid (SA + BFGS)	SA escapes noise traps; BFGS refines.
Discrete Ordered (Gears)	Relaxed BFGS	Fast convergence to basin of attraction

Table 15: Final Summary of Algorithm Applicability

References

- [1] R. Fletcher. “A new approach to variable metric algorithms”. In: *The Computer Journal* (1970).
- [2] D.F. Shanno. “Conditioning of quasi Newton methods for function minimisation”. In: *Mathematics of Computation* (1970).
- [3] J. Nocedal and S.J. Wright. *Numerical Optimization*. Springer, 1999.
- [4] E.D. Dolan and J.J. Moré. “Benchmarking optimization software with performance profiles”. In: *Mathematical Programming* 91.2 (2002), pp. 201–213.
- [5] S.S. Rao. *Engineering Optimization: Theory and Practice*. 5th. John Wiley & Sons, 2019.
- [6] C.M. Bishop. *Pattern Recognition and Machine Learning*. Chapter 3: Linear Models for Regression. Springer, 2006.
- [7] J. Nocedal and S.J. Wright. *Numerical Optimization*. 2nd. Chapter 6: Quasi-Newton Methods. Springer, 2006.
- [8] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Optimisation Project Group Report*. Tech. rep. Cranfield University, 2025.

A Code Listings

A.1 Sensor Calibration problem

A.1.1 Sensor Calibration Script (run_task4.py)

Listing 1: Main execution script for Task 4 protocols

```
# John Hoarau

import numpy as np
import matplotlib.pyplot as plt
import csv
import time
import os
import pandas as pd
from steinhart_hart_bfgs import SteinhartHartBFGS
from steinhart_hart_problem import SteinhartHartProblem
from simulated_annealing import SimulatedAnnealing
from scipy.optimize import minimize

# --- Configuration ---
DATA_FILE = "data/thermistor_data.csv"
RESULTS_DIR = "results"
PLOTS_DIR = os.path.join(RESULTS_DIR, "plots")
DATA_DIR = os.path.join(RESULTS_DIR, "data")
DATA_A_DIR = os.path.join(DATA_DIR, "protocol_a")
DATA_C_DIR = os.path.join(DATA_DIR, "protocol_c")

# Ensure directories exist
os.makedirs(PLOTS_DIR, exist_ok=True)
os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(DATA_A_DIR, exist_ok=True)
os.makedirs(DATA_C_DIR, exist_ok=True)

def save_trajectory(history, filename):
    """
    Saves BFGS trajectory to CSV.
    history: list of dicts {'x', 'mse', 'grad_norm'}
    """
    # If filename is absolute/relative path, use it. Otherwise, put in DATA_DIR (legacy)
    # But now we usually pass full path or sub-dir path.
    # To be safe: if directory part is present, use as is.
    if os.path.isdir(filename):
        filepath = filename
    else:
        filepath = os.path.join(DATA_DIR, filename)
```

```
with open(filepath, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['iteration', 'A', 'B', 'C', 'mse', 'grad_norm'])
    for i, step in enumerate(history):
        writer.writerow([
            i,
            step['x'][0], step['x'][1], step['x'][2],
            step['mse'],
            step['grad_norm']
        ])

def plot_model_fit(problem, x_best, save_path, title="Model_Fit"):
    """
    Plots the Data (Noisy) vs the Fitted Model.
    """
    A, B, C = x_best

    # Generate smooth curve for plotting
    R_smooth = np.linspace(min(problem.R_data), max(problem.R_data), 100)
    ln_R = np.log(R_smooth)
    denom = A + B * ln_R + C * (ln_R**3)
    T_model_kelvin = 1.0 / denom
    T_model_c = T_model_kelvin - 273.15

    # Data points (convert back to Celsius for plotting if desired, or keep Kelvin)
    # Let's plot in Kelvin as per internal logic, or Celsius for user readability.
    # User's input data had Celsius, let's show Celsius.
    T_data_c = problem.T_data - 273.15

    plt.figure(figsize=(10, 6))

    # Scatter Data
    plt.scatter(problem.R_data, T_data_c, color='blue', alpha=0.5, label='Noisy_Data (Meas)'
    )

    # Plot Model
    plt.plot(R_smooth, T_model_c, 'r-', linewidth=2, label='Fitted_BFGS_Model')

    plt.xlabel('Resistance (Ohms)')
    plt.ylabel('Temperature (°C)')
    plt.title(title)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig(save_path)
    plt.close()

def run_protocol_a():
    """
    Protocol A: Measurement Noise (Dolan-More)
    """
```

```
"""
print("\n--- Running Protocol A: Measurement Noise (Dolan-More) ---")

# "WAY MORE DATA": Increased seeds from 100 to 500
noise_seeds = 500
sigma_meas = 0.5

solvers = ['BFGS_Single', 'BFGS_MultiStart']
results = {s: [] for s in solvers}

# Detailed Log File
log_file = os.path.join(DATA_A_DIR, "protocol_a_detailed_log.csv")
with open(log_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['seed', 'solver_type', 'run_index', 'status', 'nfev', 'final_mse',
                    'time_sec'])

for seed in range(noise_seeds):
    np.random.seed(seed)
    problem_noisy = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'measurement',
                                                              'sigma': sigma_meas})

    # 1. BFGS Single Start
    x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_time = time.time()
    res_single = problem_noisy.solve(x0=x0)
    t_single = time.time() - start_time

    cost_single = res_single.nfev if res_single.success else float('inf')
    results['BFGS_Single'].append(cost_single)

    writer.writerow([seed, 'BFGS_Single', 0, res_single.success, res_single.nfev,
                    res_single.fun, t_single])

# Save trajectory for ALL seeds
traj_file = os.path.join(DATA_A_DIR, f"traj_protocol_a_seed_{seed}_single.csv")
save_trajectory(problem_noisy.history, traj_file)

# Generate Example Convergence Plots for Seed 0
if seed == 0:
    print("Generating Convergence Plots for Protocol A (Seed 0)...")
    plot_convergence_curves(problem_noisy.history,
                            os.path.join(PLOTS_DIR, "protocol_a_seed0_single"),
                            title_suffix="(Protocol A, Seed 0)")

    # New: Plot Model Fit
    plot_model_fit(problem_noisy, res_single.x,
                   os.path.join(PLOTS_DIR, "protocol_a_seed0_single_model_fit.
                               png"),
                   title="Model Fit Check (Protocol A, Seed 0)")
```



```
# 2. BFGS Multi-Start (K=5)
K = 5
best_mse = float('inf')
total_nfev = 0

for k in range(K):
    x0_k = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_k = time.time()
    res_k = problem_noisy.solve(x0=x0_k)
    t_k = time.time() - start_k

    total_nfev += res_k.nfev
    if res_k.fun < best_mse:
        best_mse = res_k.fun

    writer.writerow([seed, 'BFGS-MultiStart', k, res_k.success, res_k.nfev,
                     res_k.fun, t_k])

results['BFGS-MultiStart'].append(total_nfev)

if seed % 10 == 0:
    print(f"Processed {instance}/{seed}/{noise_seeds}")

# Generate Profile
generate_dolan_more(results, os.path.join(PLOTS_DIR, "performance_profile_A.png"))

def plot_convergence_curves(history, save_prefix, title_suffix=""):
    """
    Generates 3 plots matching specific style:
    1. Objective Function (Blue, Log Scale)
    2. Step Size (Green, Log Scale)
    3. Gradient Norm (Red, Log Scale, with Tolerance)
    """
    iterations = [step['iteration'] if 'iteration' in step else i for i, step in enumerate(
        history)]
    colors = {'obj': 'blue', 'step': 'green', 'grad': 'red'}

    # 1. Objective Function
    mse_values = [step['mse'] for step in history]

    plt.figure(figsize=(10, 6))
    plt.plot(iterations, mse_values, 'o-', color='blue', markersize=4, label='Objective f(x)')
    plt.yscale('log')
    plt.xlabel('Iteration')
    plt.ylabel('Objective Function Value')
    plt.title(f'BFGS Convergence: Objective Function {title_suffix}')
    plt.legend()
```

```
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_objective.png")
plt.close()

# 2. Step Size (alpha approximation:  $\|x_k - x_{k-1}\|$ )
step_sizes = []
# If history has > 1 item
if len(history) > 1:
    for i in range(1, len(history)):
        x_prev = history[i-1]['x']
        x_curr = history[i]['x']
        # Euclidean distance in parameter space
        dist = np.linalg.norm(np.array(x_curr) - np.array(x_prev))
        step_sizes.append(dist)

# Plot steps against iteration 1..N
plt.figure(figsize=(10, 6))
plt.plot(iterations[1:], step_sizes, 'o-', color='green', markersize=4, label='Step_
Size_α')
plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Step_Size_α_ (from_line_search)')
plt.title(f'BFGS:_Line_Search_Step_Sizes_{title_suffix}')
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_step_size.png")
plt.close()

# 3. Gradient Norm
grad_norms = [step['grad_norm'] for step in history]

plt.figure(figsize=(10, 6))
plt.plot(iterations, grad_norms, 'o-', color='red', markersize=4, label='Gradient_Norm')

# Add Tolerance Line (1e-8 as per user example, though code uses 1e-12)
plt.axhline(y=1e-8, color='green', linestyle='--', linewidth=2, label='Tolerance_1e-08
')

plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Gradient_Norm_  $\|\nabla f(x)\|$ ')
plt.title(f'BFGS:_Convergence:_Gradient_Norm_{title_suffix}')
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_gradient.png")
plt.close()

def generate_dolan_more(results, filename):
    print(f"Generating_Dolan-More_Profile_{filename}")
```

```
plt.figure(figsize=(10, 6))

solvers = list(results.keys())
n_prob = len(results[solvers[0]])

ratios = {s: [] for s in solvers}

# Debug info
solved_counts = {s: 0 for s in solvers}

for i in range(n_prob):
    costs = [results[s][i] for s in solvers]
    # Filter failures (inf)
    valid_costs = [c for c in costs if c != float('inf')]

    if not valid_costs:
        for s in solvers: ratios[s].append(float('inf'))
        continue

    min_cost = min(valid_costs)

    for s in solvers:
        cost = results[s][i]
        if cost == float('inf'):
            ratios[s].append(float('inf'))
        else:
            solved_counts[s] += 1
            ratios[s].append(max(cost, 1e-10) / min_cost)

print("Solver Success Counts inside Dolan-More:", solved_counts)

for s in solvers:
    valid_ratios = [r for r in ratios[s] if r != float('inf')]
    sorted_ratios = np.sort(valid_ratios)
    y = np.arange(1, len(sorted_ratios) + 1) / n_prob

    # If solver never succeeded, valid_ratios is empty
    if len(sorted_ratios) == 0:
        print(f"Solver_{s} has NO successful runs to plot.")
        plt.plot([], [], label=s) # Empty plot for legend
    else:
        plt.step(sorted_ratios, y, label=s, where='post')

plt.xscale('log')
plt.xlabel('Performance_Ratio( $\tau$ )')
plt.ylabel('Probability of Success( $\rho$ )')
plt.title('Dolan-More Profile')
plt.legend(loc='lower right')
plt.grid(True)
```

```
plt.xlim(1, 100)
plt.ylim(0, 1.05)
plt.savefig(filename)
print("Saved.")

def run_protocol_b():
    """
    Protocol B: Function Noise (Stochastic Objective)
    """
    print("\n---Running Protocol B: Function Noise (Stochastic Objective)---")

    noise_seeds = 100 # Standard amount
    sigma_func = 1e-4 # Noise added to MSE

    solvers = ['BFGS_Single', 'BFGS_MultiStart']
    results = {s: [] for s in solvers}

    # Directory for Protocol B
    DATA_B_DIR = os.path.join(DATA_DIR, "protocol_b")
    os.makedirs(DATA_B_DIR, exist_ok=True)

    # Detailed Log File
    log_file = os.path.join(DATA_B_DIR, "protocol_b_detailed_log.csv")
    with open(log_file, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['seed', 'solver_type', 'run_index', 'status', 'nfev', 'final_mse',
                        'time_sec'])

    for seed in range(noise_seeds):
        np.random.seed(seed)
        # Use 'function' noise type
        problem_noisy = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'function',
                        'sigma': sigma_func})

        # 1. BFGS Single Start
        x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
        start_time = time.time()
        res_single = problem_noisy.solve(x0=x0)
        t_single = time.time() - start_time

        cost_single = res_single.nfev if res_single.success else float('inf')
        results['BFGS_Single'].append(cost_single)

        writer.writerow([seed, 'BFGS_Single', 0, res_single.success, res_single.nfev,
                        res_single.fun, t_single])

    traj_file = os.path.join(DATA_B_DIR, f"traj_protocol_b_seed_{seed}_single.csv")
    save_trajectory(problem_noisy.history, traj_file)
```

```
# Generate Example Convergence Plots for Seed 0
if seed == 0:
    print("Generating Convergence Plots for Protocol B (Seed 0) ...")
    plot_convergence_curves(problem_noisy.history,
                            os.path.join(PLOTS_DIR, "protocol_b_seed0_single"),
                            title_suffix=f"(Protocol B, Seed 0, Sigma={
                                sigma_func})")

    # New: Plot Model Fit
    plot_model_fit(problem_noisy, res_single.x,
                   os.path.join(PLOTS_DIR, "protocol_b_seed0_single_model_fit.
                               png"),
                   title=f"Model Fit Check (Protocol B, Seed 0, Sigma={
                       sigma_func})")

# 2. BFGS Multi-Start (K=5)
K = 5
best_mse = float('inf')
total_nfev = 0

for k in range(K):
    x0_k = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_k = time.time()
    res_k = problem_noisy.solve(x0=x0_k)
    t_k = time.time() - start_k

    total_nfev += res_k.nfev
    if res_k.fun < best_mse:
        best_mse = res_k.fun

    writer.writerow([seed, 'BFGS_MultiStart', k, res_k.success, res_k.nfev,
                    res_k.fun, t_k])

results['BFGS_MultiStart'].append(total_nfev)

if seed % 10 == 0:
    print(f"Processed Protocol B instance {seed}/{noise_seeds}")

# Generate Profile
generate_dolan_more(results, os.path.join(PLOTS_DIR, "performance_profile_B.png"))

def run_protocol_c():
    """
    Protocol C: Robustness Curve
    """
    print("\n---Running Protocol C: Robustness Curve---")

    # "WAY MORE DATA": Expanded Sigma List
    sigmas = [0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 1.5, 2.0, 3.0, 5.0]
```

```
summary_file = os.path.join(DATA_C_DIR, "protocol_c_summary.csv")

bfgs_stats = []
bfgs_single_stats = [] # New: Track Single BFGS
sa_stats = []
hybrid_stats = []

with open(summary_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['sigma', 'method', 'final_mse'])

    for sigma in sigmas:
        print(f"Testing Noise Level: sigma={sigma}")
        np.random.seed(42)

        # --- 1. Multi-Start BFGS ---
        problem_bfgs = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'measurement',
                                                                    'sigma': sigma})
        best_fun = float('inf')

        # Save BFGS Trajectories
        # Increased repetitions to 50
        for k in range(50):
            x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
            res = problem_bfgs.solve(x0=x0)

            traj_file = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}"
                                     f"_bfgs_run_{k}.csv")
            save_trajectory(problem_bfgs.history, traj_file)

            # Capture Single BFGS (Iteration 0)
            if k == 0:
                bfgs_single_stats.append(res.fun)
                writer.writerow([sigma, 'BFGS_Single', res.fun])

            if res.fun < best_fun:
                best_fun = res.fun

        # Generate Example Convergence Plots for Sigma=0.01, Run 0
        if sigma == 0.01 and k == 0:
            print("Generating Convergence Plots for Protocol C (Sigma 0.01, Run 0)
                  ...")
            plot_convergence_curves(problem_bfgs.history,
                                    os.path.join(PLOTS_DIR, "protocol_c_sigma0.01"
                                                  "_run0"),
                                    title_suffix="(Protocol C, Sigma 0.01)")

            # New: Plot Model Fit
            plot_model_fit(problem_bfgs, res.x,
                           os.path.join(PLOTS_DIR, "protocol_c_sigma0.01
```

```
        _run0_model_fit.png"),
        title="Model_Fit_Check_(Protocol_C,_Sigma_0.01)")

bfgs_stats.append(best_fun)
writer.writerow([sigma, 'BFGS_MultiStart', best_fun])

# --- 2. Simulated Annealing ---
class SA_Adapter:
    def __init__(self, prob_bfgs):
        self.prob = prob_bfgs
    def generate_initial_solution(self):
        return [np.random.uniform(5e-4, 2e-3), np.random.uniform(1e-4, 5e-4),
                np.random.uniform(1e-8, 1e-6)]
    def evaluate(self, sol):
        return self.prob.objective(sol)
    def get_neighbor(self, sol):
        return [
            sol[0] + np.random.normal(0, 1e-5),
            sol[1] + np.random.normal(0, 1e-6),
            sol[2] + np.random.normal(0, 1e-9)
        ]
    def plot_model(self, sol, save_path=None):
        pass

sa_prob = SA_Adapter(problem_bfgs)
sa = SimulatedAnnealing(sa_prob,
                        initial_temp=0.1,
                        cooling_rate=0.95,
                        min_temp=1e-6,
                        metropolis_steps=100)

import sys, io
text_trap = io.StringIO()
sys.stdout = text_trap
sol_sa = sa.solve(verbose=False)
sys.stdout = sys.__stdout__

# Save SA Data using its internal method
sa_csv_path = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}_sa.csv")
sa.save_to_csv(sa_csv_path)

final_energy = sa.best_solution_energy if hasattr(sa, 'best_solution_energy')
                else sa.history['best_energy'][-1]
sa_stats.append(final_energy)
writer.writerow([sigma, 'Simulated_Annealing', final_energy])

# --- 3. Hybrid ---
res_hybrid = problem_bfgs.solve(x0=sol_sa)
hybrid_stats.append(res_hybrid.fun)
```

```
writer.writerow([sigma, 'Hybrid', res_hybrid.fun])

hybrid_traj_file = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}
    _hybrid_bfgs_phase.csv")
save_trajectory(problem_bfgs.history, hybrid_traj_file)

# Plot Robustness Curve
plt.figure(figsize=(10, 6))
plt.plot(sigmas, bfgs_single_stats, 'x-', label='Single_BFGS', color='magenta') # New
    Line
plt.plot(sigmas, bfgs_stats, 'o-', label='Multi-Start_BFGS', color='blue')
plt.plot(sigmas, sa_stats, 's--', label='Simulated_Annealing', color='orange')
plt.plot(sigmas, hybrid_stats, '~-.', label='Hybrid_(SA+BFGS)', color='green')

plt.xlabel('Noise_Sigma')
plt.ylabel('Final_MSE_(Log_Scale)')
plt.yscale('log')
plt.title('Robustness_Curve:_Protocol_C')
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(PLOTS_DIR, 'robustness_curve.png'))
print("Robustness_Curve_Saved_to_results/plots.")

if __name__ == "__main__":
    print("Starting_Comprehensive_Task_4_Execution...")
    # run_protocol_a() # Already ran previously? If user wants new data, uncomment. But user
        said "run IT".
    # Assuming user wants B specifically now or all? "can you run it" usually means the
        whole thing or the missing part.
    # To be safe and efficient, I will run ALL 3 if needed, but A takes long.
    # I'll run B and C (fast). A is commented out to save time unless requested.
    # actually, user said "run it and give me... protocol B is missing".

    run_protocol_a()
    run_protocol_b()
    run_protocol_c()
    print(f"Done..All_data_saved_to_{os.path.abspath(DATA_DIR)}")
```

A.1.2 BFGS Implementation (steinhart_hart_bfgs.py)

Listing 2: Steinhart-Hart BFGS Class

John. Hoarau

```
import numpy as np
import math
import csv
from scipy.optimize import minimize, check_grad
```



```
class SteinhartHartBFGS:
    def __init__(self, data_file, noise_config=None):
        """
        Initialize the Steinhart-Hart BFGS problem.

        Args:
            data_file (str): Path to the CSV file containing thermistor data.
            noise_config (dict, optional): Configuration for noise injection.
            { 'type': 'measurement', 'sigma': float } -> Adds static noise to T_meas (
                Protocol A)
            { 'type': 'function', 'sigma': float } -> Adds dynamic noise to objective (
                Protocol B)
        """
        self.R_data = []
        self.T_data = [] # In Kelvin
        self.noise_config = noise_config if noise_config else {}
        self.history = [] # To store trajectory

        # Load Data
        self._load_data(data_file)

        # Pre-calculate Log(R) terms for efficiency (vectorized)
        self.ln_R = np.log(self.R_data)
        self.ln_R_3 = self.ln_R ** 3

        # Apply Measurement Noise (Protocol A) - Static Modification of Data
        if self.noise_config.get('type') == 'measurement':
            sigma = self.noise_config.get('sigma', 0.0)
            if sigma > 0:
                noise = np.random.normal(0, sigma, size=len(self.T_data))
                self.T_data += noise # Add noise to measurements
                # print(f"Applied Measurement Noise: sigma={sigma}")

    def _load_data(self, filename):
        """Loads thermistor data from CSV."""
        R_list = []
        T_list = []
        with open(filename, 'r') as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    r = float(row['Resistance_Ohms'])
                    t_c = float(row['Temperature_C'])
                    t_k = t_c + 273.15
                    if r > 0 and t_k > 0:
                        R_list.append(r)
                        T_list.append(t_k)
                except ValueError:
```

```
        continue

        self.R_data = np.array(R_list)
        self.T_data = np.array(T_list)
        self.N = len(self.R_data)

    def objective(self, x):
        """
        Calculates Mean Squared Error (MSE).
        Model:  $1/T = A + B \ln(R) + C (\ln(R))^3$ 
        """
        A, B, C = x

        # Vectorized Forward Model
        denom = A + B * self.ln_R + C * self.ln_R_3

        # Soft Penalty / Safety for Solver
        # Although L-BFGS-B bounds should prevent this, we add a check.
        if np.any(denom <= 0):
            return 1e6 # Large penalty

        T_model = 1.0 / denom

        # Residuals
        residuals = self.T_data - T_model

        # MSE Calculation (1/N Scaling)
        mse = np.mean(residuals ** 2)

        # Apply Dynamic Function Noise (Protocol B)
        if self.noise_config.get('type') == 'function':
            sigma = self.noise_config.get('sigma', 0.0)
            if sigma > 0:
                mse += np.random.normal(0, sigma)

        return mse

    def gradient(self, x):
        """
        Calculates Analytical Gradient of MSE.
         $d(MSE)/d\theta = (2/N) * \sum (T_{meas} - T_{model}) * (-dT_{model}/d\theta)$ 
         $dT_{model}/dD = -T_{model}^2$ 
        """
        A, B, C = x

        denom = A + B * self.ln_R + C * self.ln_R_3

        # Singularity check handled by objective usually, but for gradient:
        # If we are in a bad spot, return zeros or large gradients to push back
```

```
if np.any(denom == 0):
    return np.array([0.0, 0.0, 0.0]) # Should not happen with bounds

T_model = 1.0 / denom
residuals = self.T_data - T_model # (y - y_hat)

# Term: (T_meas - T_model) * T_model^2
# Note: Derivative of (mean(r^2)) = 2/N * sum( r * dr/dtheta )
# r = T_meas - T_model
# dr/dtheta = - dT_model/dtheta
# dT_model/dtheta = (-T_model^2) * dD/dtheta
# So: dr/dtheta = T_model^2 * dD/dtheta
# Gradient = 2/N * sum( (T_meas - T_model) * T_model^2 * dD/dtheta )

common_factor = (2.0 / self.N) * residuals * (T_model ** 2)

# Partial Derivatives of D
# dD/dA = 1
# dD/dB = ln(R)
# dD/dC = (ln(R))^3

grad_A = -np.sum(common_factor * 1.0) # The negative sign comes from minimizing -(T
    - T_model)^2 confusion?
# Let's re-verify:
# J = 1/N sum (T - T_model)^2
# dJ = 2/N sum (T - T_model) * (-dT_model)
# -dT_model = - (-1/D^2) * dD = (1/D^2) * dD = T_model^2 * dD
# So dJ = 2/N sum (T - T_model) * T_model^2 * dD
# Wait... (T - T_model) is "Residual"
# If T_model is too low, (T - T_model) > 0.
# If we increase A, D increases, T_model (1/D) *decreases*.
# So increasing A makes T_model smaller.
# If T_model is too low, we want to *increase* T_model.
# So we should *decrease* A.
# Gradient should be positive?
# Let's check math:
# Residual r = (T - T_model)
# J = r^2
# dJ/dA = 2r * dr/dA
# dr/dA = - dT_model/dA
# T_model = 1/D => dT_model/dA = -1/D^2 * dD/dA = -T_model^2 * 1
# so dr/dA = T_model^2
# dJ/dA = 2 * (T - T_model) * T_model^2
# IF T > T_model (underestimation), r > 0. Term is Positive.
# Gradient is positive -> Optimizer moves A in negative direction.
# Decreasing A -> Decreases D -> Increases T_model. Correct.

# Logic check:
# My code: grad_A = -np.sum(common_factor)
```

```
# common_factor = (2/N) * (T - T_model) * T_model^2
# So my code computes: - (positive term) = Negative Gradient.
# This would cause optimizer to INCREASE A.
# Increasing A -> Increases D -> Decreases T_model.
# If T > T_model (underestimation), we make it WORSE.
# ERROR FOUND.
# The sum should be POSITIVE summation of the terms derived above.
# dJ/dtheta = sum( 2 * (T - T_model) * T_model^2 * dD/dtheta )

grad_A = -np.sum(common_factor * 1.0) # This is WRONG based on manually checking
direction.

# But wait, usually Residual is defined as (y_hat - y)?
# Here I defined residuals = self.T_data - T_model (y - y_hat)
# J = sum (y - y_hat)^2 = sum (y_hat - y)^2
# Let's standardize: J = sum (T_model - T_data)^2
# dJ/dA = 2 * (T_model - T_data) * dT_model/dA
# dT_model/dA = -T_model^2
# dJ/dA = 2 * (T_model - T_data) * (-T_model^2)
#         = -2 * (T_model - T_data) * T_model^2
#         = 2 * (T_data - T_model) * T_model^2
# This matches my previous derivation: 2 * r * T_model^2 where r = T_data - T_model.
# So strictly: dJ/dA = POSITIVE sum of (2/N * r * T_model^2 * 1)

# So correct code should be:
grad_A = -np.sum(common_factor * 1.0) # This returns NEGATIVE sum.
# Why did I put a negative sign there?
# Ah, in 'steinhart_hart_bfgs.py' from thought process I might have confused it.
# Let's fix it.
# The term 'common_factor' contains 'residuals' (T - T_model).
# We established dJ/dA = sum( factor ).
# So I should NOT negate it.

# FIXING NOW:
grad_A = -np.sum(common_factor * 1.0) # Wait, let's look at 'optimize.minimize'.
# It minimizes J.
# If gradient is negative, it moves in positive direction.
# If T > T_model (need to increase T_model, decrease A).
# term = (T - T_model) * T^2 > 0.
# If I return Positive Gradient: Moves A Negative. Correct.
# If I return Negative Gradient: Moves A Positive. Incorrect.

# So 'grad_A' should be 'np.sum(common_factor)'.
# However, usually gradients are defined as dJ/dx.
# Let's remove the negative sign.

return np.array([
    -np.sum(common_factor * 1.0),          # A
    -np.sum(common_factor * self.ln_R),    # B
    -np.sum(common_factor * self.ln_R_3)   # C
```

```
] ) * -1.0 # Double negative to fix the logic error found during typing?
# Let's simply write it clearly.

#  $dJ/dA = \sum (2/N * (T_{data} - T_{model}) * T_{model}^2)$ 
# My 'common_factor' is exactly  $(2/N * (T_{data} - T_{model}) * T_{model}^2)$ 
# So grad_A should be  $np.sum(common\_factor)$ .

def gradient_correct(self, x):
    A, B, C = x
    denom = A + B * self.ln_R + C * self.ln_R_3
    T_model = 1.0 / denom
    residuals = self.T_data - T_model

    #  $dJ/dtheta = \sum [ 2 * (T_{meas} - T_{model}) * T_{model}^2 * dD/dtheta ] * (1/N)$ 
    common = (2.0 / self.N) * residuals * (T_model**2)

    gA = np.sum(common * 1.0)
    gB = np.sum(common * self.ln_R)
    gC = np.sum(common * self.ln_R_3)

    return np.array([gA, gB, gC])

def callback_tracker(self, xk):
    """
    Callback to log trajectory.
    Need to re-evaluate objective/grad to log them properly as scipy doesn't pass them.
    """

    # Recalculate state
    mse = self.objective(xk)
    grad = self.gradient_correct(xk)
    grad_norm = np.linalg.norm(grad)

    self.history.append({
        'x': xk.copy(),
        'mse': mse,
        'grad_norm': grad_norm
    })

def solve(self, x0=None):
    """
    Run L-BFGS-B optimization.
    """
    if x0 is None:
        # Default decent guess
        x0 = [1e-3, 2e-4, 1e-7]

    # Clear history
    self.history = []
```

```
# Bonds: A,B,C > 0 roughly (1e-9) to safety
bounds = [(1e-9, None), (1e-9, None), (1e-9, None)]

result = minimize(
    fun=self.objective,
    x0=x0,
    method='L-BFGS-B',
    jac=self.gradient_correct, # Use the corrected gradient function
    callback=self.callback_tracker,
    bounds=bounds,
    options={'ftol': 1e-12, 'gtol': 1e-12, 'maxiter': 1000} # Tight tolerances for
        BFGS
)

return result

if __name__ == "__main__":
    # Self-Test / Gradient Check
    print("Running Gradient Check...")
    problem = SteinhartHartBFGS("data/thermistor_data.csv")
    print(f>Data Points N={problem.N}")

    # Test Point
    x_test = np.array([1.1e-3, 2.3e-4, 1.5e-7])

    base_mse = problem.objective(x_test)
    print(f"Base MSE: {base_mse}")

    # Analytical
    grad_ana = problem.gradient_correct(x_test)

    # Finite Difference with Relative Step Size
    grad_fd = np.zeros(3)

    for i in range(3):
        x_shift = x_test.copy()
        epsilon = x_test[i] * 1e-4 # 0.01% perturbation
        x_shift[i] += epsilon
        mse_shift = problem.objective(x_shift)
        grad_fd[i] = (mse_shift - base_mse) / epsilon
        print(f"FD [{i}] shift (h={epsilon}): MSE={mse_shift:.8f} -> Grad={grad_fd[i]:.4e}")

    with np.printoptions(precision=4, suppress=False, linewidth=100):
        print(f"Analytical: {grad_ana}")
        print(f"Finite Diff: {grad_fd}")

    diff = np.linalg.norm(grad_ana - grad_fd)
```

```
norm = np.linalg.norm(grad_ana)
rel_error = diff / norm if norm > 0 else 0.0

print(f"Difference_Norm: {diff:.6f}")
print(f"Relative_Error: {rel_error:.6%}")

if rel_error < 5e-2: # 5% tolerance (FD is still approximate on high curvature)
    print("Gradient_Check_PASSED!")
else:
    print("Gradient_Check_FAILED!")
```

A.2 Gear Train problem

A.2.1 Gear Train Script (run_gear_train.py)

Listing 3: Main execution script for Gear Train problem

```
from __future__ import annotations

import os
import csv
import math
import numpy as np
import matplotlib.pyplot as plt

from gear_train_bfgs import BFGSOptions, multi_start_bfgs

def wilson_ci_95(p: float, n: int) -> tuple[float, float]:
    """Wilson score interval (approx 95%) for a binomial proportion."""
    if n <= 0:
        return (0.0, 0.0)
    z = 1.96
    denom = 1.0 + (z * z) / n
    center = (p + (z * z) / (2.0 * n)) / denom
    half = (z / denom) * math.sqrt((p * (1.0 - p) / n) + (z * z) / (4.0 * n * n))
    lo = max(0.0, center - half)
    hi = min(1.0, center + half)
    return lo, hi

def run_noise_point(
    sigma: float,
    base_opts: BFGSOptions,
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    R: int,
```

```
n_starts_per_rep: int,
repair_radius: int,
noise_avg: int,
seed_base: int = 10_000,
sigma_index: int = 0,
) -> dict:
    """Run R repetitions for a given sigma and return aggregate metrics."""
    conv_flags = []
    iters_list = []
    err_list = []
    f_list = []
    g_list = []

    for r in range(R):
        opts_noisy = BFGSOptions(**{
            **base_opts.__dict__,
            "noise_sigma": float(sigma),
            "noise_seed": seed_base + 100 * sigma_index + r,
            "noise_avg": int(noise_avg),
        })

        best_sigma = multi_start_bfgs(
            target_ratio=target_ratio,
            lb=lb,
            ub=ub,
            n_starts=n_starts_per_rep,
            seed=42,
            opts=opts_noisy,
            repair_radius=repair_radius,
        )

        conv_flags.append(bool(best_sigma["converged"]))
        iters_list.append(int(best_sigma["iters"]))
        err_list.append(float(best_sigma["err_int"]))
        f_list.append(float(best_sigma["f_best"]))
        g_list.append(float(best_sigma["history_grad_norm"][-1]))

    p = float(np.mean(conv_flags))
    ci_lo, ci_hi = wilson_ci_95(p, R)

    return {
        "sigma": float(sigma),
        "conv_rate": p,
        "conv_ci_lo": ci_lo,
        "conv_ci_hi": ci_hi,
        "med_iters": float(np.median(iters_list)),
        "med_err_int": float(np.median(err_list)),
        "med_f_best": float(np.median(f_list)),
        "med_grad_norm": float(np.median(g_list)),
```



```
}

def main() -> None:
    lb = np.array([12, 12, 12, 12], dtype=float)
    ub = np.array([60, 60, 60, 60], dtype=float)
    target_ratio = 0.14427932477276006

    plot_dir = "PLOT"
    os.makedirs(plot_dir, exist_ok=True)

    # -----
    # Baseline (noise-free)
    # -----
    opts = BFGSOptions(
        max_iter=300,
        tol_grad=1e-8,
        tol_step=1e-12,
        fd_eps=1e-6,
        c1=1e-4,
        backtrack_beta=0.5,
        min_step=1e-12,
        verbose=False,
        noise_sigma=0.0,
        noise_seed=0,
        noise_avg=1,
    )

    best = multi_start_bfgs(
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        n_starts=5000,
        seed=42,
        opts=opts,
        repair_radius=4,
    )

    x_cont = best["x_best"]
    x_int0 = best.get("x_int0", None)
    x_int = best["x_int"]
    f_best = float(best["f_best"])

    print("\n== TASK 4 - BFGS (continuous relaxation) RESULT ==")
    print(f"Target ratio: {target_ratio:.15f}")
    print(f"Best continuous x*: {np.array2string(x_cont, precision=6, floatmode='fixed')}")
    print(f"Continuous objective: {f_best:.6e} (squared error)")
```

```
print("\n---After rounding/repair to integers (selection by integer error)---")
if x_int0 is not None:
    print(f"Rounded(pre-repair): {x_int0.tolist()}")
print(f"Integer(post-repair): {x_int.tolist()}")
print(f"Absolute error(int): {best['err_int']:.6e}")

print("\nExtra diagnostics:")
print(f"Best run start index: {best['start_index']}")
print(f"Iterations: {best['iters']}")
print(f"Final || grad ||: {best['history_grad_norm'][-1]:.6e}")
print(f"Converged flag: {best['converged']}")

# Baseline convergence plots (cleaner)
hist_f = best["history_f"]
hist_g = best["history_grad_norm"]

plt.figure()
plt.plot(hist_f, marker="o", linewidth=1.8)
plt.xlabel("Iteration")
plt.ylabel(r"$f(x)=(r(x)-r^*)^2$")
plt.title("BFGS Convergence (Objective)")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_convergence_objective.png"), dpi=220)

plt.figure()
plt.semilogy(hist_g, marker="o", linewidth=1.8)
plt.xlabel("Iteration")
plt.ylabel(r"$\|\nabla f(x)\|$")
plt.title("BFGS Convergence (Gradient Norm)")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_convergence_gradnorm.png"), dpi=220)

# -----
# Noise robustness experiment (clean & informative)
# -----
reliability_threshold = 0.8 # strict

print("\n[Noise experiment note]")
print(f"- Noise-free optimum objective:  $f^* \approx \{f_{\text{best}}\}$  .")
print(f"- Additive objective noise:  $f_{\text{noisy}}(x) = f(x) + \sigma * N(0,1)$  .")
print(f"- If  $\sigma \gg f(x)$  near the optimum ( $\sim 1e-20$ ), then finite-difference gradients")
print(f"and Armijo line-search are dominated by noise, so BFGS loses reliability.\n")

# Phase 1: coarse sweep
R = 10
n_starts_per_rep = 300
repair_radius = 4
```

```
noise_avg = 10 # stable

coarse_sigmas = [0.0, 1e-15, 3e-15, 1e-14, 3e-14, 1e-13, 3e-13, 1e-12, 1e-11, 1e-10, 1e-8, 1e-6, 1e-4]
results = []

print("Coarse sweep:")
for idx, sigma in enumerate(coarse_sigmas):
    out = run_noise_point(
        sigma=sigma,
        base_opts=opts,
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        R=R,
        n_starts_per_rep=n_starts_per_rep,
        repair_radius=repair_radius,
        noise_avg=noise_avg,
        sigma_index=idx,
    )
    results.append(out)
    print(
        f"sigma={sigma:.1e}|rate={out['conv_rate']:.2f}|"
        f" (~95% CI [{out['conv_ci_lo']:.2f}, {out['conv_ci_hi']:.2f}])|"
        f"med_iters={out['med_iters']:.0f}|med_int_err={out['med_err_int']:.3e}|"
        f"med_f_best={out['med_f_best']:.1e}|med||g||={out['med_grad_norm']:.1e}"
    )

# Identify first failure on coarse grid
failure_sigma = None
for out in results:
    if out["conv_rate"] < reliability_threshold and out["sigma"] > 0.0:
        failure_sigma = out["sigma"]
        break

# Phase 2: refine around threshold (if found)
# We refine in [sigma_prev, sigma_fail] using log-spaced points.
if failure_sigma is not None:
    # find previous sigma before failure
    prev_sigma = None
    for out in results:
        if out["sigma"] < failure_sigma and out["sigma"] > 0.0:
            prev_sigma = out["sigma"]
    if prev_sigma is not None:
        refine_sigmas = np.geomspace(prev_sigma, failure_sigma, num=7)
        print("\nRefinement sweep around the transition:")
        for j, sigma in enumerate(refine_sigmas):
            out = run_noise_point(
                sigma=float(sigma),
```

```
        base_opts=opts,
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        R=R,
        n_starts_per_rep=n_starts_per_rep,
        repair_radius=repair_radius,
        noise_avg=noise_avg,
        sigma_index=10_000 + j,
    )
    results.append(out)
    print(
        f"sigma={sigma:.2e}|rate={out['conv_rate']:.2f}"
        f" (~95% CI [{out['conv_ci_lo']:.2f},{out['conv_ci_hi']:.2f}])"
    )

# Sort results by sigma
results = sorted(results, key=lambda d: d["sigma"])

# Recompute failure_sigma as first sigma where conv_rate < threshold
failure_sigma = None
for out in results:
    if out["sigma"] > 0.0 and out["conv_rate"] < reliability_threshold:
        failure_sigma = out["sigma"]
        break

# Console conclusion
print("\n==Noise robustness conclusion==")
if failure_sigma is None:
    print(
        "BFGS remained reliable over the tested noise range"
        f"(convergence rate never dropped below {reliability_threshold:.1f})."
    )
else:
    print(
        f"BFGS becomes unreliable at approximately noise sigma >={failure_sigma:.2e}"
        f"(convergence rate < {reliability_threshold:.1f})."
    )

# Prepare arrays for plots
sig = np.array([d["sigma"] for d in results], dtype=float)
rate = np.array([d["conv_rate"] for d in results], dtype=float)
rate_lo = np.array([d["conv_ci_lo"] for d in results], dtype=float)
rate_hi = np.array([d["conv_ci_hi"] for d in results], dtype=float)

med_iters = np.array([d["med_iters"] for d in results], dtype=float)
med_err = np.array([d["med_err_int"] for d in results], dtype=float)
med_f = np.array([d["med_f_best"] for d in results], dtype=float)
med_g = np.array([d["med_grad_norm"] for d in results], dtype=float)
```

```
# Plot 1: convergence rate with confidence interval
plt.figure()
x = sig + 1e-30
plt.semilogx(x, rate, marker="o", linewidth=1.8, label="Convergence_rate")
plt.fill_between(x, rate_lo, rate_hi, alpha=0.20, label="~95% CI (Wilson)")
plt.axhline(reliability_threshold, linestyle="--", linewidth=1.5, label=f"Threshold={
    reliability_threshold:.1f}")
if failure_sigma is not None:
    plt.axvline(failure_sigma, linestyle="--", linewidth=1.5, label=f"Estimated failure
        ≈{failure_sigma:.1e}")
plt.ylim(-0.05, 1.05)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Convergence_rate over R_runs")
plt.title("BFGS Robustness: Convergence Rate vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_convergence_rate.png"), dpi=220)

# Plot 2: median integer error
plt.figure()
plt.semilogx(x, med_err, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median |ratio_int - target| after repair")
plt.title("BFGS Robustness: Median Integer Error vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_integer_error.png"), dpi
    =220)

# Plot 3: median iterations
plt.figure()
plt.semilogx(x, med_iters, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median iterations (best run)")
plt.title("BFGS Robustness: Median Iterations vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_iterations.png"), dpi=220)

# Plot 4 (NEW): median f_best (continuous objective) -- tells the real story
plt.figure()
plt.semilogx(x, med_f, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median best continuous objective f_best")
plt.title("BFGS Robustness: Median Best Objective vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
```

```
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_fbest.png"), dpi=220)

# Plot 5 (NEW): median grad norm -- also very illustrative
plt.figure()
plt.semilogx(x, med_g, marker="o", linewidth=1.8)
plt.xlabel("Noise $\sigma$ (additive on objective)")
plt.ylabel(r"Median $\sigma\|\nabla f\|$  at termination (best run)")
plt.title("BFGS Robustness: Median Final Gradient Norm vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_gradnorm.png"), dpi=220)

# Export CSV for report
csv_path = os.path.join(plot_dir, "noise_summary.csv")
with open(csv_path, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["sigma", "conv_rate", "conv_ci_lo", "conv_ci_hi", "med_iters", "med_err_int", "med_f_best", "med_grad_norm"])
    for d in results:
        w.writerow([d["sigma"], d["conv_rate"], d["conv_ci_lo"], d["conv_ci_hi"], d["med_iters"], d["med_err_int"], d["med_f_best"], d["med_grad_norm"]])

if __name__ == "__main__":
    main()
```

A.2.2 Gear Train BFGS (gear_train_bfgs.py)

Listing 4: Gear Train BFGS Class

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple
import numpy as np

@dataclass
class BFGSOptions:
    max_iter: int = 300
    tol_grad: float = 1e-8
    tol_step: float = 1e-10
    fd_eps: float = 1e-6 # finite difference step
    c1: float = 1e-4 # Armijo parameter
    backtrack_beta: float = 0.5
    min_step: float = 1e-12
    verbose: bool = False

    # Noise injection (Task 4 robustness experiment)
    noise_sigma: float = 0.0 # std-dev of additive Gaussian noise on objective
```

```
    evaluations
noise_seed: int = 0          # RNG seed to make noisy runs reproducible
noise_avg: int = 1          # number of noisy samples averaged per evaluation

def gear_ratio(x: np.ndarray) -> float:
    """Gear_train_ratio: ratio = (N1*N3)/(N2*N4)."""
    x = np.asarray(x, dtype=float)
    return (x[0] * x[2]) / (x[1] * x[3])

def continuous_objective(x: np.ndarray, target_ratio: float) -> float:
    """Smooth objective for BFGS: f(x) = (ratio(x) - target_ratio)^2."""
    r = gear_ratio(x)
    e = r - target_ratio
    return float(e * e)

def make_noisy_objective(
    f_clean: Callable[[np.ndarray], float],
    sigma: float,
    seed: int,
    n_avg: int = 1,
) -> Callable[[np.ndarray], float]:
    """Return a callable that adds controlled Gaussian noise to objective evaluations.

    f_noisy(x) = mean_{j=1..n_avg} (f_clean(x) + sigma * N(0,1))

    Notes:
    - For sigma=0, the function reduces to f_clean.
    - A fixed seed makes the whole optimisation run reproducible.
    - Averaging (n_avg>1) reduces noise but increases evaluation cost.
    """
    if sigma <= 0.0:
        return f_clean

    rng = np.random.default_rng(seed)
    n_avg = max(1, int(n_avg))

    def f(x: np.ndarray) -> float:
        acc = 0.0
        for _ in range(n_avg):
            acc += float(f_clean(x) + sigma * rng.normal())
        return acc / n_avg

    return f

def project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
```

```
"""Projection onto box constraints [lb, ub]."""
return np.minimum(np.maximum(x, lb), ub)

def round_and_clip(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
    """Round to nearest integer then clip into bounds."""
    xr = np rint(x).astype(int)
    xr = np.minimum(np.maximum(xr, lb.astype(int)), ub.astype(int))
    return xr

def integer_error(x_int: np.ndarray, target_ratio: float) -> float:
    """Absolute error in the true discrete problem."""
    r = gear_ratio(x_int.astype(float))
    return float(abs(r - target_ratio))

def discrete_repair_search(
    x_int0: np.ndarray,
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    radius: int = 2,
) -> Tuple[np.ndarray, float]:
    """Local discrete search around an initial integer solution.

    Evaluates all integer combinations in the hypercube [x_int0-radius, x_int0+radius]
    (clipped to [lb, ub]) and returns the best integer design and its absolute error.
    """
    x0 = np.asarray(x_int0, dtype=int)
    lb_i = lb.astype(int)
    ub_i = ub.astype(int)

    r = int(max(0, radius))
    best_x = x0.copy()
    best_err = integer_error(best_x, target_ratio)

    ranges = []
    for i in range(len(x0)):
        lo = max(lb_i[i], x0[i] - r)
        hi = min(ub_i[i], x0[i] + r)
        ranges.append(range(lo, hi + 1))

    for n1 in ranges[0]:
        for n2 in ranges[1]:
            for n3 in ranges[2]:
                for n4 in ranges[3]:
                    cand = np.array([n1, n2, n3, n4], dtype=int)
                    err = integer_error(cand, target_ratio)
```



```
        if err < best_err:
            best_err = err
            best_x = cand

    return best_x, float(best_err)

def finite_difference_grad(f: Callable[[np.ndarray], float], x: np.ndarray, eps: float) ->
    np.ndarray:
    """Central finite difference gradient."""
    x = np.asarray(x, dtype=float)
    g = np.zeros_like(x)
    for i in range(len(x)):
        xp = x.copy()
        xm = x.copy()
        xp[i] += eps
        xm[i] -= eps
        g[i] = (f(xp) - f(xm)) / (2.0 * eps)
    return g

def armijo_backtracking(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    p: np.ndarray,
    fx: float,
    g: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    c1: float,
    beta: float,
    min_step: float,
) -> Tuple[float, np.ndarray, float]:
    """Armijo backtracking line search on projected steps."""
    alpha = 1.0
    gTp = float(np.dot(g, p))

    # If direction is not a descent direction, fall back to steepest descent
    if gTp >= 0.0:
        p = -g
        gTp = float(np.dot(g, p))

    while alpha >= min_step:
        x_new = project_box(x + alpha * p, lb, ub)
        f_new = f(x_new)
        if f_new <= fx + c1 * alpha * gTp:
            return alpha, x_new, f_new
        alpha *= beta
```

```
x_new = project_box(x + min_step * p, lb, ub)
return min_step, x_new, f(x_new)

def bfgs_minimise_box_projected(
    f_raw: Callable[[np.ndarray], float],
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions,
) -> Dict[str, object]:
    """BFGS on a box-constrained domain via projection.

    Note: With noise_sigma > 0, objective evaluations are noisy, which intentionally
    violates deterministic assumptions and can cause loss of convergence.
    """
    x = project_box(np.asarray(x0, dtype=float), lb, ub)

    f_clean = lambda z: f_raw(project_box(z, lb, ub))
    f = make_noisy_objective(
        f_clean,
        sigma=opts.noise_sigma,
        seed=opts.noise_seed,
        n_avg=opts.noise_avg,
    )

    n = len(x)
    H = np.eye(n)

    fx = f(x)
    g = finite_difference_grad(f, x, opts.fd_eps)

    history_f: List[float] = [fx]
    history_grad_norm: List[float] = [float(np.linalg.norm(g))]
    history_alpha: List[float] = []

    if opts.verbose:
        print(f"[BFGS] iter=0 f={fx:.6e} ||g||={history_grad_norm[-1]:.6e} x={x}")

    for k in range(1, opts.max_iter + 1):
        grad_norm = float(np.linalg.norm(g))
        if grad_norm < opts.tol_grad:
            break

        p = -H @ g

        alpha, x_new, f_new = armijo_backtracking(
            f=f,
            x=x,
```

```
p=p,
fx=fx,
g=g,
lb=lb,
ub=ub,
c1=opts.c1,
beta=opts.backtrack_beta,
min_step=opts.min_step,
)

s = x_new - x
if float(np.linalg.norm(s)) < opts.tol_step:
    x, fx = x_new, f_new
    history_f.append(fx)
    history_grad_norm.append(float(np.linalg.norm(g)))
    history_alpha.append(alpha)
    break

g_new = finite_difference_grad(f, x_new, opts.fd_eps)
y = g_new - g

ys = float(np.dot(y, s))
if ys > 1e-12:
    rho = 1.0 / ys
    I = np.eye(n)
    V = I - rho * np.outer(s, y)
    H = V @ H @ V.T + rho * np.outer(s, s)
else:
    H = np.eye(n)

x, fx, g = x_new, f_new, g_new

history_f.append(fx)
history_grad_norm.append(float(np.linalg.norm(g)))
history_alpha.append(alpha)

if opts.verbose and (k % 10 == 0 or k == 1):
    print(
        f"[BFGS]_iter={k}_f={fx:.6e}_||g||={history_grad_norm[-1]:.6e}_ "
        f"alpha={alpha:.2e}_x={x}"
    )

# Conservative convergence flag under noise:
converged = (history_grad_norm[-1] < opts.tol_grad) or (
    len(history_f) >= 2 and float(abs(history_f[-1] - history_f[-2])) < 1e-30
)

return {
    "x_best": x,
```

```
        "f_best": fx,
        "history_f": np.array(history_f, dtype=float),
        "history_grad_norm": np.array(history_grad_norm, dtype=float),
        "history_alpha": np.array(history_alpha, dtype=float),
        "iters": len(history_f) - 1,
        "converged": bool(converged),
    }

}

def multi_start_bfgs(
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    n_starts: int = 500,
    seed: int = 42,
    opts: BFGSOptions | None = None,
    repair_radius: int = 2,
) -> Dict[str, object]:
    """Multi-start BFGS.

    IMPORTANT:
    We SELECT the best candidate primarily by integer error (after rounding+repair),
    and use continuous objective as a tie-breaker.
    """
    if opts is None:
        opts = BFGSOptions()

    rng = np.random.default_rng(seed)
    best = None

    def f_raw(x: np.ndarray) -> float:
        return continuous_objective(x, target_ratio=target_ratio)

    for i in range(n_starts):
        x0 = rng.uniform(lb, ub)
        out = bfgs_minimise_box_projected(
            f_raw=f_raw,
            x0=x0,
            lb=lb,
            ub=ub,
            opts=opts,
        )

        x_cont = out["x_best"]
        f_cont = float(out["f_best"])

        x_int0 = round_and_clip(x_cont, lb, ub)
        x_int, err_int = discrete_repair_search(
            x_int0=x_int0,
```

```
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        radius=repair_radius,
    )

    record = {
        **out,
        "start_index": i,
        "x0": x0,
        "x_int0": x_int0,
        "x_int": x_int,
        "err_int": float(err_int),
        "ratio_cont": gear_ratio(x_cont),
        "ratio_int": gear_ratio(x_int.astype(float)),
    }

    if best is None:
        best = record
        continue

    # Primary criterion: best integer error
    if record["err_int"] < float(best["err_int"]):
        best = record
        continue

    # Tie-breaker: best continuous objective
    if record["err_int"] == float(best["err_int"]) and f_cont < float(best["f_best"]):
        best = record

    assert best is not None
    return best
```

A.3 Two Bar Plane Truss problem

A.3.1 Custom BFGS script for Two Truss problem(custom_TBT_BFGS.py)

Listing 5: Main execution script for Task 4 protocols

```
"""
Subject: Custom BFGS Implementation for Two Bar Truss
@author: Cl  mence - Philom  ne HINOT + Claude AI
@date: 17/01/2026
@Description: This module implements BFGS optimization from scratch.
"""

# ***** IMPORTATION *****
```

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple
import numpy as np

# ***** CUSTOM BFGS *****

@dataclass
class BFGSOptions:
    """Configuration options for BFGS optimizer"""
    max_iter: int = 300          # Maximum iterations
    tol_grad: float = 1e-8       # Gradient norm tolerance for convergence
    tol_step: float = 1e-12      # Step size tolerance
    fd_eps: float = 1e-6        # Finite difference step size
    c1: float = 1e-4            # Armijo condition parameter
    backtrack_beta: float = 0.5 # Backtracking reduction factor
    min_step: float = 1e-12     # Minimum step size
    verbose: bool = False       # Print iteration details

def project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
    """
    Project point x onto box constraints [lb, ub].

    For each component: x_i = max(lb_i, min(x_i, ub_i))
    """
    return np.minimum(np.maximum(x, lb), ub)

def finite_difference_grad(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    eps: float
) -> np.ndarray:
    """
    Compute gradient using central finite differences.

    For each dimension i:
     $\partial f / \partial x_i \approx [f(x + \epsilon \cdot e_i) - f(x - \epsilon \cdot e_i)] / (2\epsilon)$ 

    Cost: 2n function evaluations (where n = len(x))

    Args:
        f: Objective function
        x: Current point
        eps: Finite difference step size

    Returns:
        Gradient vector  $\nabla f(x)$ 
    """
```

```
"""
    x = np.asarray(x, dtype=float)
    n = len(x)
    g = np.zeros(n, dtype=float)

    for i in range(n):
        # Perturb in positive direction
        x_plus = x.copy()
        x_plus[i] += eps
        f_plus = f(x_plus)

        # Perturb in negative direction
        x_minus = x.copy()
        x_minus[i] -= eps
        f_minus = f(x_minus)

        # Central difference
        g[i] = (f_plus - f_minus) / (2.0 * eps)

    return g

def armijo_backtracking(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    p: np.ndarray,
    fx: float,
    g: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    c1: float,
    beta: float,
    min_step: float,
) -> Tuple[float, np.ndarray, float]:
    """
    Armijo backtracking line search with box constraint projection.

    Finds step size  $\alpha$  such that the Armijo condition is satisfied:
     $f(x + \alpha \cdot p) \leq f(x) + c1 \cdot \alpha \cdot \nabla f(x)^T p$  (sufficient decrease)

    Algorithm:
    1. Start with  $\alpha = 1$ 
    2. While Armijo condition not satisfied:
        Reduce  $\alpha \leftarrow \beta \cdot \alpha$ 
        Project  $x + \alpha \cdot p$  onto  $[lb, ub]$ 
    3. Return  $\alpha$ , new point, new function value

    Args:
    f: Objective function
```

```
#####x: Current point
#####p: Search direction
#####fx: f(x), function value at current point
#####g: ∇f(x), gradient at current point
#####lb, ub: Box constraints
#####c1: Armijo parameter (typically 1e-4)
#####beta: Backtracking factor (typically 0.5)
#####min_step: Minimum allowed step size

#####Returns:
#####(alpha, x_new, f_new): Step size, new point, new function value
#####"""
    alpha = 1.0
    gTp = float(np.dot(g, p))

    # Safety check: if p is not a descent direction, use steepest descent
    if gTp >= 0.0:
        p = -g
        gTp = float(np.dot(g, p))

    # Backtracking loop
    while alpha >= min_step:
        # Proposed step with projection
        x_new = project_box(x + alpha * p, lb, ub)
        f_new = f(x_new)

        # Check Armijo condition
        if f_new <= fx + c1 * alpha * gTp:
            return alpha, x_new, f_new

        # Reduce step size
        alpha *= beta

    # If no acceptable step found, take minimum step
    x_new = project_box(x + min_step * p, lb, ub)
    return min_step, x_new, f(x_new)

def bfgs_update(
    H: np.ndarray,
    s: np.ndarray,
    y: np.ndarray
) -> np.ndarray:
    """
    #####BFGS update formula for inverse Hessian approximation.

    #####Standard BFGS formula:
    #####ρ = 1 / (yTs)
    #####V = I - ρ · s · yT
```



```
#####  $H_{k+1} = V \cdot H_k \cdot V^T + \rho \cdot s \cdot s^T$ 

##### Where:
#####  $s = x_{k+1} - x_k(\text{step})$ 
#####  $y = \nabla f_{k+1} - \nabla f_k(\text{gradient difference})$ 
#####  $H \approx [\nabla^2 f]$  #####  $(\text{inverse Hessian approximation})$ 

##### Curvature condition:  $y^T s > 0$  must hold for positive definiteness.
##### If violated, reset  $H$  to identity matrix.

##### Args:
#####  $H$ : Current inverse Hessian approximation
#####  $s$ : Step vector
#####  $y$ : Gradient difference

##### Returns:
##### Updated inverse Hessian approximation
##### """
    n = len(s)
    ys = float(np.dot(y, s))

    # Check curvature condition
    if ys > 1e-12:
        # BFGS update
        rho = 1.0 / ys
        I = np.eye(n)
        V = I - rho * np.outer(s, y)
        H_new = V @ H @ V.T + rho * np.outer(s, s)
        return H_new
    else:
        # Curvature condition violated - reset to identity
        return np.eye(n)

def bfgs_minimize(
    f: Callable[[np.ndarray], float],
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions | None = None,
) -> Dict[str, object]:
    """
    ##### BFGS minimization with box constraints (via projection).

    ##### Algorithm:
    ##### 1. Initialize:  $H = I$  (identity matrix),  $x = \text{project}(x_0)$ 
    ##### 2. For  $k = 0, 1, 2, \dots$ 
    ##### a. Compute gradient:  $g = \nabla f(x)$ 
    ##### b. Check convergence:  $\|g\| < \text{tol\_grad}$ 
```

```
#####c.Search_direction: p=-H.g
#####d.Line_search: find alpha satisfying Armijo condition
#####e.Update: x←x+alpha.p, project onto bounds
#####f.BFGS_update: H←BFGS(H,s,y)
#####3.Return solution

#####Args:
#####f: Objective function to minimize
#####x0: Initial guess
#####lb: Lower bounds
#####ub: Upper bounds
#####opts: BFGS options (use defaults if None)

#####Returns:
#####Dictionary containing:
#####- x_best: Optimal solution found
#####- f_best: Objective value at solution
#####- history_f: Function values at each iteration
#####- history_grad_norm: Gradient norms at each iteration
#####- history_alpha: Step sizes at each iteration
#####- iters: Number of iterations performed
#####- converged: Boolean convergence flag
#####"""
    if opts is None:
        opts = BFGSOptions()

    # Initialize
    x = project_box(np.asarray(x0, dtype=float), lb, ub)
    n = len(x)
    H = np.eye(n) # Initial inverse Hessian approximation

    # Evaluate initial point
    fx = f(x)
    g = finite_difference_grad(f, x, opts.fd_eps)

    # History tracking
    history_f: List[float] = [fx]
    history_grad_norm: List[float] = [float(np.linalg.norm(g))]
    history_alpha: List[float] = []

    if opts.verbose:
        print(f"[BFGS] Iteration 0:")
        print(f"    f(x)={fx:.10e}")
        print(f"    ||∇f||={history_grad_norm[-1]:.6e}")
        print(f"    x={x}")

    # Main BFGS loop
    for k in range(1, opts.max_iter + 1):
        grad_norm = float(np.linalg.norm(g))
```

```
# Convergence check: gradient norm
if grad_norm < opts.tol_grad:
    if opts.verbose:
        print(f"\n[BFGS] Converged: || $\nabla f$ || = {grad_norm:.6e} < {opts.tol_grad:.6e}")
    break

# Search direction:  $p = -H \cdot g$  (quasi-Newton direction)
p = -H @ g

# Line search with Armijo backtracking
alpha, x_new, f_new = armijo_backtracking(
    f=f,
    x=x,
    p=p,
    fx=fx,
    g=g,
    lb=lb,
    ub=ub,
    c1=opts.c1,
    beta=opts.backtrack_beta,
    min_step=opts.min_step,
)

# Compute step
s = x_new - x

# Convergence check: step size
if float(np.linalg.norm(s)) < opts.tol_step:
    x, fx = x_new, f_new
    history_f.append(fx)
    history_grad_norm.append(float(np.linalg.norm(g)))
    history_alpha.append(alpha)
    if opts.verbose:
        print(f"\n[BFGS] Converged: ||step|| = {np.linalg.norm(s):.6e} < {opts.tol_step:.6e}")
    break

# Gradient at new point
g_new = finite_difference_grad(f, x_new, opts.fd_eps)

# Gradient difference
y = g_new - g

# BFGS update of inverse Hessian approximation
H = bfgs_update(H, s, y)

# Update state
x, fx, g = x_new, f_new, g_new
```

```
# Record history
history_f.append(fx)
history_grad_norm.append(float(np.linalg.norm(g)))
history_alpha.append(alpha)

# Verbose output
if opts.verbose and (k % 10 == 0 or k == 1):
    print(f"\n[BFGS] Iteration {k}:")
    print(f"    f(x) = {fx:.10e}")
    print(f"    ||∇f|| = {history_grad_norm[-1]:.6e}")
    print(f"    α = {alpha:.6e}")
    print(f"    x = {x}")

# Check gradient convergence
grad_converged = history_grad_norm[-1] < opts.tol_grad

# Check step convergence (practical convergence)
step_converged = False
if len(history_alpha) > 0 and history_alpha[-1] < opts.min_step * 10:
    step_converged = True

# Combined: converged if EITHER criterion is met
converged = grad_converged or step_converged

# Determine termination reason
if grad_converged:
    termination = "Gradient norm below tolerance"
elif step_converged:
    termination = "Step size below threshold (practical convergence)"
else:
    termination = "Maximum iterations reached"

return {
    "x_best": x,
    "f_best": fx,
    "history_f": np.array(history_f, dtype=float),
    "history_grad_norm": np.array(history_grad_norm, dtype=float),
    "history_alpha": np.array(history_alpha, dtype=float),
    "iters": len(history_f) - 1,
    "converged": bool(converged),
    "grad_converged": bool(grad_converged),
    "step_converged": bool(step_converged),
    "termination": termination,
}
```

A.3.2 Two Truss problem script adapted for BFGS (two_bar_plane_truss_problem.py)

Listing 6: Main execution script for Task 4 protocols

```
"""
Subject: Two_Bar_Plane_Truss_Problem adapted for BFGS optimisation
Source: https://www.sciencedirect.com/science/article/pii/S0898122111010406
@author: Clemence-Philomene HINOT & Claude AI
@date: 12/01/2026
"""

# ***** IMPORTATION *****

import math

# ***** CLASS DEFINITION *****

class Two_Bar_Plane_Truss_Problem():

    # ----- INITIALIZATION -----

    def __init__(self):
        # constants
        self.rho = 7833          # unit: kg/m^3
        self.h = 2.54            # unit: m
        self.P = 44482.2         # unit: N
        self.E = 2.07e11         # unit: Pa
        self.sigma0 = 1.38e8     # unit: Pa
        self.A_min = 6.4516e-4   # unit: m^2

        # variables
        self.pos = 0.0
        self.area = 0.0

        # physical boundaries
        self.pos_min = 0.1 * self.h
        self.pos_max = 2.25 * self.h
        self.area_min = 0.5 * self.A_min
        self.area_max = 2.5 * self.A_min

    # ----- CONSTITUTIVE EQUATIONS -----

    # Weight objective
    def f1(self):
        x1 = self.pos/self.h # need to use normalised x1 in this function
        return 2 * self.rho * self.h * self.area * math.sqrt(1 + pow(x1, 2))

    # Displacement objective
    def f2(self):
        x1 = self.pos/self.h
        return (self.P * pow(self.h, 3) * pow((1 + pow(x1, 2)), 1.5)) / ( math.sqrt(1 + pow(
```

```
        x1, 4)) * 2 * math.sqrt(2) * self.E * x1 * self.area)

# ----- CONSTRAINT EQUATIONS -----

# Stress constraint (tension)
def g1(self, normalised=True):
    x1 = self.pos/self.h
    stress = (self.P * (1 + x1) * math.sqrt(1 + pow(x1, 2))) / (2 * math.sqrt(2) * x1 *
        self.area))

    if normalised:
        return (stress / self.sigma0) - 1.0 # normalised result
    else:
        return stress - self.sigma0 # real result in Pa

# Stress constraint (compression)
def g2(self, normalised=True):
    x1 = self.pos/self.h
    stress = (self.P * (1 - x1) * math.sqrt(1 + pow(x1, 2))) / (2 * math.sqrt(2) * x1 *
        self.area))

    if normalised:
        return (stress / self.sigma0) - 1.0 # normalised result
    else:
        return stress - self.sigma0 # real result in Pa

# ----- EVALUATION FUNCTION -----

def evaluate(self, x):
    """
    Evaluate objectives and constraints

    param x: [x1_normalized, x2_normalized]
    return: (sum_value, [f1, f2], [g1, g2])
    """

    # physical position
    self.pos = x[0] * self.h

    # physical area
    self.area = x[1] * self.A_min

    # weight
    f1 = self.f1()

    # movement
    f2 = self.f2()
```

```
# constraints
g1 = self.g1()
g2 = self.g2()

# penalty to apply if the constraint g > 0
penalty1 = max(0, g1)**2
penalty2 = max(0, g2)**2

# weight for the constraint functions
weightConstraint = 10000.0

# weight for f1 and f2
# they have the same
weightFunction = 0.5

# weighted sum
sum = weightFunction * f1 + weightFunction * f2 + weightConstraint * (penalty1 +
    penalty2)

# results
return sum, [f1, f2], [g1, g2]
```

A.3.3 Two Truss without noise main script(main_TBT_BFGS.py)

Listing 7: Main execution script for Task 4 protocols

```
"""
Subject:MainScript
@author:Clémence-Philomène HINOT+Claude AI
@date:15/01/2026
@Description:Custom BFGS implementation for the Two Bar Truss problem.
Adapted from the Gear Train BFGS approach.
"""

# ***** IMPORTATION *****

# mathematics
import numpy as np

# visualisation
import matplotlib.pyplot as plt

# store data
import os

# custom BFGS algorithm
from custom_TBT_BFGS import bfgs_minimize, BFGSOptions

# Two Bar Truss problem
```

```
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** TWO BAR TRUSS PROBLEM *****

# create an instance of the truss problem
truss_problem = Two_Bar_Plane_Truss_Problem()

# define objective function and keep only the sum value
def objective(x):
    # wrapped
    sum_value, objectives, constraints = truss_problem.evaluate(x)
    return sum_value

# problem bounds
lb = np.array([0.1, 0.5]) # lower bounds: [x1_min, x2_min]
ub = np.array([2.25, 2.5]) # upper bounds: [x1_max, x2_max]

# ***** OPTIMISATION *****

# initial guess
x0 = np.array([1.0, 1.5])

# BFGS options
opts = BFGSOptions(
    max_iter=50, # Maximum iterations
    tol_grad=1e-5, # Gradient norm tolerance
    tol_step=1e-12, # Step size tolerance
    fd_eps=1e-6, # Finite difference step
    c1=1e-4, # Armijo parameter
    backtrack_beta=0.5, # Backtracking factor
    min_step=1e-12, # Minimum step size
    verbose=False # Set to True to see iteration details
)

# BFGS optimisation
result = bfgs_minimize(
    f = objective, # function to minimise
    x0=x0, # initial guess
    lb=lb, # lower bound
    ub=ub, # upper bound
    opts=opts # options
)

# evaluate final solution
final_sum, final_obj, final_con = truss_problem.evaluate(result["x_best"])

# calculate physical values
position = result["x_best"][0] * truss_problem.h
area = result["x_best"][1] * truss_problem.A_min
```



```
# check constraint satisfaction
constraints_satisfied = (final_con[0] <= 0) and (final_con[1] <= 0)
in_bounds = ((lb[0] <= result["x_best"][0] <= ub[0]) and (lb[1] <= result["x_best"][1] <= ub
[1]))

# ***** DISPLAY RESULTS *****

print(f"\nConvergence_Status:")
print(f"    S: {result['converged']}")
print(f"    Iterations: {result['iters']}")
print(f"    Function evaluations: {result['iters']*2*len(x0)+1}")
print(f"    (1 initial + {result['iters']} iterations * {2*len(x0)} for finite differences)")

print(f"\nOptimal_Solution_(normalized_variables):")
print(f"    x1(position_factor) = {result['x_best'][0]:.10f}")
print(f"    x2(area_factor) = {result['x_best'][1]:.10f}")
print(f"    Within bounds: {'✓YES' if in_bounds else '✗NO'}")

print(f"\nPhysical_Solution:")
print(f"    Position = {position:.8f} m")
print(f"    Area = {area:.8e} m2")

print(f"\nObjective_Function:")
print(f"    Combined objective = {result['f_best']:.12e}")
print(f"    f1(weight) = {final_obj[0]:.8f} kg")
print(f"    f2(displacement) = {final_obj[1]:.8e} m")

print(f"\nConstraints:")
print(f"    Normalized (for optimization):")
print(f"    g1 = {final_con[0]:+.8e} {'✓' if final_con[0] <= 0 else '✗'}")
print(f"    g2 = {final_con[1]:+.8e} {'✓' if final_con[1] <= 0 else '✗'}")

# Get actual constraint values using normalized=False
g1_actual = truss_problem.g1(normalised=False)
g2_actual = truss_problem.g2(normalised=False)

print(f"    Actual (Pa):")
print(f"    g1( $\sigma_{\text{tension}}$  -  $\sigma_0$ ) = {g1_actual:+.6e} Pa {'✓' if g1_actual <= 0 else '✗'}")
print(f"    g2( $\sigma_{\text{compression}}$  -  $\sigma_0$ ) = {g2_actual:+.6e} Pa {'✓' if g2_actual <= 0 else '✗'}")

print(f"\nConvergence_Metrics:")
print(f"    Final gradient norm: {result['history_grad_norm'][-1]:.8e}")
if len(result['history_alpha']) > 0:
    print(f"    Final step size  $\alpha$ : {result['history_alpha'][-1]:.8e}")

# ***** VISUALISATION *****
```

```
# Create plots directory
plot_dir = "TBT_PLOT"
os.makedirs(plot_dir, exist_ok=True)

# Generate convergence plots
print("\nGenerating convergence plots...")

# Plot 1: Objective function convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_f"], 'b-o', linewidth=2, markersize=5, label='Objective f(x)')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Objective Function Value', fontsize=12)
plt.title('BFGS Convergence: Objective Function', fontsize=14, fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, 'bfgs_convergence_objective.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_objective.png")

# Plot 2: Gradient norm convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_grad_norm"], 'r-o', linewidth=2, markersize=5, label='Gradient Norm')
plt.axhline(y=opts.tol_grad, color='g', linestyle='--', linewidth=2,
            label=f'Tolerance = {opts.tol_grad:.0e}')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel(r'Gradient Norm $\|\nabla f(x)\|$', fontsize=12)
plt.title('BFGS Convergence: Gradient Norm', fontsize=14, fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, 'bfgs_convergence_gradient.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_gradient.png")

# Plot 3: Step sizes (line search)
if len(result["history_alpha"]) > 0:
    plt.figure(figsize=(10, 6))
    plt.semilogy(range(1, len(result["history_alpha"]) + 1),
                 result["history_alpha"], 'g-o', linewidth=2, markersize=5,
                 label='Step Size $\alpha$')
    plt.xlabel('Iteration', fontsize=12)
    plt.ylabel(r'Step Size $\alpha$ (from line search)', fontsize=12)
    plt.title('BFGS: Line Search Step Sizes', fontsize=14, fontweight='bold')
    plt.grid(True, which='both', alpha=0.3)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.savefig(os.path.join(plot_dir, 'bfgs_step_sizes.png'), dpi=300)
    print(f"Saved: {plot_dir}/bfgs_step_sizes.png")
```

```
plt.close('all')
```

A.3.4 Two Truss with noise main script(main_TBT_BFGS_noise.py)

Listing 8: Main execution script for Task 4 protocols

```
"""
Subject: Main Script
@author: Pierre Bédrupe + Claude AI
@date: 15/01/2026
@Description: Custom BFGS implementation for the Two Bar Truss problem.
              Adapted from the Gear Train BFGS approach.
"""

# ***** IMPORTATION *****

# mathematics
import numpy as np

# visualisation
import matplotlib.pyplot as plt

# store data
import os

# custom BFGS algorithm
from custom_TBT_BFGS import bfgs_minimize, BFGSOptions

# Two Bar Truss problem
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** NOISE WRAPPER *****

def make_noisy_objective(f_clean, sigma, seed=0):
    """
    Build a noisy objective:
    f_noisy(x) = f_clean(x) + sigma * N(0,1)
    """
    rng = np.random.default_rng(seed)

    if sigma <= 0.0:
        return f_clean

    def f_noisy(x):
        return float(f_clean(x)) + float(sigma) * float(rng.normal())

    return f_noisy
```

```
# ***** TWO BAR TRUSS PROBLEM *****

# create an instance of the truss problem
truss_problem = Two_Bar_Plane_Truss_Problem()

# define objective function and keep only the sum value
def objective_clean(x):
    sum_value, objectives, constraints = truss_problem.evaluate(x)
    return sum_value

# problem bounds
lb = np.array([0.1, 0.5])    # lower bounds: [x1_min, x2_min]
ub = np.array([2.25, 2.5])  # upper bounds: [x1_max, x2_max]

# ***** OPTIMISATION *****

# initial guess
x0 = np.array([1.0, 1.5])

# BFGS options
opts = BFGSOptions(
    max_iter=1000,          # Maximum iterations
    tol_grad=1e-5,          # Gradient norm tolerance
    tol_step=1e-40,         # Step size tolerance
    fd_eps=1e-6,           # Finite difference step
    c1=1e-4,               # Armijo parameter
    backtrack_beta=0.5,     # Backtracking factor
    min_step=1e-12,         # Minimum step size
    verbose=False          # Set to True to see iteration details
)

# ***** VISUALISATION *****

# Create plots directory
plot_dir = "TBT_PLOT"
os.makedirs(plot_dir, exist_ok=True)

# ***** RUN WITH 3 NOISE LEVELS *****

sigmas = [0.0, 1e-4, 1e-2, 1]    # <- change ici si tu veux d'autres niveaux
base_seed = 42                   # seed de base (reproductible)

for sigma in sigmas:

    # objective = noisy(objective_clean)
    objective = make_noisy_objective(objective_clean, sigma=sigma, seed=base_seed)

    # BFGS optimisation
```

```
result = bfgs_minimize(
    f=objective,          # function to minimise
    x0=x0,                # initial guess
    lb=lb,                # lower bound
    ub=ub,                # upper bound
    opts=opts             # options
)

# evaluate final solution (sur la fonction "clean" physique, sans bruit)
final_sum, final_obj, final_con = truss_problem.evaluate(result["x_best"])

# calculate physical values
position = result["x_best"][0] * truss_problem.h
area = result["x_best"][1] * truss_problem.A_min

# check constraint satisfaction
constraints_satisfied = (final_con[0] <= 0) and (final_con[1] <= 0)
in_bounds = ((lb[0] <= result["x_best"][0] <= ub[0]) and (lb[1] <= result["x_best"][1]
    <= ub[1]))

# ***** DISPLAY RESULTS *****

print("\n" + "*" * 70)
print(f"RUN WITH NOISE: sigma={sigma:.2e}")
print("*" * 70)

print(f"\nConvergence Status:")
print(f"  S: {result['converged']}")
print(f"  Iterations: {result['iters']}")
print(f"  Function evaluations: {result['iters'] * 2 * len(x0) + 1}")
print(f"  (1 initial + {result['iters']} iterations * {2 * len(x0)} for finite differences)")

print(f"\nOptimal Solution (normalized variables):")
print(f"  x1(position factor) = {result['x_best'][0]:.10f}")
print(f"  x2(area factor) = {result['x_best'][1]:.10f}")
print(f"  Within bounds: {'✓ YES' if in_bounds else '✗ NO'}")

print(f"\nPhysical Solution:")
print(f"  Position = {position:.8f} m")
print(f"  Area = {area:.8e} m2")

print(f"\nObjective Function:")
print(f"  Combined objective (noisy during optimisation) = {result['f_best']:.12e}")
print(f"  Combined objective (clean re-eval) = {final_sum:.12e}")
print(f"  f1(weight) = {final_obj[0]:.8f} kg")
print(f"  f2(displacement) = {final_obj[1]:.8e} m")

print(f"\nConstraints:")
```

```
print(f"Normalized(for optimization):")
print(f"g1={final_con[0]:+.8e}{'✓' if final_con[0] <= 0 else '×'}")
print(f"g2={final_con[1]:+.8e}{'✓' if final_con[1] <= 0 else '×'}")

# Get actual constraint values using normalized=False
g1_actual = truss_problem.g1(normalised=False)
g2_actual = truss_problem.g2(normalised=False)

print(f"Actual(Pa):")
print(f"g1(σ_tension-σ0)={g1_actual:+.6e}Pa{'✓' if g1_actual <= 0 else '×'}")
print(f"g2(σ_compression-σ0)={g2_actual:+.6e}Pa{'✓' if g2_actual <= 0 else '×'}")

print(f"\nConvergence Metrics:")
print(f"Final gradient_norm: {result['history_grad_norm'][-1]:.8e}")
if len(result['history_alpha']) > 0:
    print(f"Final step_size: {result['history_alpha'][-1]:.8e}")

# ***** VISUALISATION *****

print("\nGenerating convergence plots...")

tag = f"sigma_{sigma:.0e}".replace("+", "").replace("-", "m") # filename-safe

# Plot 1: Objective function convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_f"], 'b-o', linewidth=2, markersize=5, label='Objective f(x)')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Objective Function Value', fontsize=12)
plt.title(f'BFGS Convergence: Objective Function (sigma={sigma:.0e})', fontsize=14, fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, f'bfgs_convergence_objective_{tag}.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_objective_{tag}.png")

# Plot 2: Gradient norm convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_grad_norm"], 'r-o', linewidth=2, markersize=5, label='Gradient Norm')
plt.axhline(y=opts.tol_grad, color='g', linestyle='--', linewidth=2, label=f'Tolerance={opts.tol_grad:.0e}')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel(r'Gradient Norm $\|\nabla f(x)\|$', fontsize=12)
plt.title(f'BFGS Convergence: Gradient Norm (sigma={sigma:.0e})', fontsize=14, fontweight='bold')
```

```
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, f'bfgs_convergence_gradient_{tag}.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_gradient_{tag}.png")

# Plot 3: Step sizes (line search)
if len(result["history_alpha"]) > 0:
    plt.figure(figsize=(10, 6))
    plt.semilogy(range(1, len(result["history_alpha"]) + 1),
                 result["history_alpha"], 'g-o', linewidth=2, markersize=5,
                 label='Step Size  $\alpha$ ')
    plt.xlabel('Iteration', fontsize=12)
    plt.ylabel(r'Step Size  $\alpha$  (from line search)', fontsize=12)
    plt.title(f'BFGS: Line Search Step Sizes (sigma={sigma:.0e})', fontsize=14,
              fontweight='bold')
    plt.grid(True, which='both', alpha=0.3)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.savefig(os.path.join(plot_dir, f'bfgs_step_sizes_{tag}.png'), dpi=300)
    print(f"Saved: {plot_dir}/bfgs_step_sizes_{tag}.png")

plt.close('all')
```

A.3.5 Test script for Two Truss problem with noise(Test_noise.py)

Listing 9: Main execution script for Task 4 protocols

```
"""
Subject: BFGS Limits Testing with Noise
@author: Pierre Bédrupe + Claude AI
@date: 17/01/2026
@Description: Systematic testing of BFGS algorithm limits with various noise levels
              using CURRENT parameters to identify breaking points
"""

# ***** IMPORTATION *****

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import time
from typing import Dict, List, Tuple
import sys

sys.path.insert(0, '/mnt/project')

from custom_TBT_BFGS import bfgs_minimize, BFGSOptions
```

```
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** NOISE WRAPPER *****

def make_noisy_objective(f_clean, sigma, seed=0):
    """
    Create noisy version of objective function
    f_noisy(x) = f_clean(x) +  $\sigma$  * N(0,1)
    """
    rng = np.random.default_rng(seed)

    if sigma <= 0.0:
        return f_clean

    def f_noisy(x):
        return float(f_clean(x)) + float(sigma) * float(rng.normal())

    return f_noisy

# ***** EVALUATION FUNCTION *****

def evaluate_bfgs_performance(
    objective_clean,
    objective_noisy,
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions,
    sigma: float,
    problem_instance,
    run_id: int
) -> Dict:
    """
    Evaluate BFGS performance for a single run

    Returns:
    Dictionary with performance metrics
    """

    # Run optimization
    start_time = time.time()
    result = bfgs_minimize(
        f=objective_noisy,
        x0=x0,
        lb=lb,
        ub=ub,
        opts=opts
    )
```



```
elapsed_time = time.time() - start_time

# Evaluate on CLEAN objective (ground truth)
final_sum_clean, final_obj, final_con = problem_instance.evaluate(result["x_best"])

# Evaluate on NOISY objective (what optimizer saw)
final_sum_noisy = objective_noisy(result["x_best"])

# Constraint satisfaction
constraints_ok = (final_con[0] <= 0) and (final_con[1] <= 0)
bounds_ok = np.all((lb <= result["x_best"]) & (result["x_best"] <= ub))

# Feasibility
feasible = constraints_ok and bounds_ok

# Convergence quality
if len(result["history_f"]) > 1:
    # Relative improvement between first and last iteration
    improvement = abs(result["history_f"][0] - result["history_f"][-1]) / (abs(result["history_f"][0]) + 1e-10)
else:
    improvement = 0.0

# Oscillation measure (variance in last 20% of iterations)
n_tail = max(1, int(0.2 * len(result["history_f"])))
tail_values = result["history_f"][-n_tail:]
oscillation = float(np.std(tail_values)) if len(tail_values) > 1 else 0.0

return {
    'run_id': run_id,
    'sigma': sigma,
    'converged': result['converged'],
    'grad_converged': result['grad_converged'],
    'step_converged': result['step_converged'],
    'termination': result['termination'],
    'iterations': result['iters'],
    'time_sec': elapsed_time,
    'f_final_noisy': result['f_best'],
    'f_final_clean': final_sum_clean,
    'f_initial': result["history_f"][0],
    'improvement_pct': improvement * 100,
    'oscillation': oscillation,
    'grad_norm_final': result['history_grad_norm'][-1],
    'step_size_final': result['history_alpha'][-1] if len(result['history_alpha']) > 0
        else np.nan,
    'x1_final': result['x_best'][0],
    'x2_final': result['x_best'][1],
    'f1_weight': final_obj[0],
    'f2_displacement': final_obj[1],
```

```
'g1_constraint': final_con[0],
'g2_constraint': final_con[1],
'constraints_satisfied': constraints_ok,
'bounds_satisfied': bounds_ok,
'feasible': feasible,
'history_f': result["history_f"],
'history_grad': result["history_grad_norm"],
'history_alpha': result["history_alpha"]
}

# ***** MAIN TEST FUNCTION *****

def test_bfgs_limits(
    noise_levels: List[float],
    n_runs_per_noise: int = 10,
    output_dir: str = "BFGS_LIMITS_TEST"
):
    """
    Systematic testing of BFGS with different noise levels

    Args:
        noise_levels: List of sigma values to test
        n_runs_per_noise: Number of independent runs per noise level
        output_dir: Directory to save results
    """

    print("="*80)
    print("BFGS_LIMITS_TESTING - SYSTEMATIC NOISE ANALYSIS")
    print("="*80)
    print(f"\nTest Configuration:")
    print(f"  Noise levels: {noise_levels}")
    print(f"  Runs per noise level: {n_runs_per_noise}")
    print(f"  Total runs: {len(noise_levels)*n_runs_per_noise}")
    print(f"  Output directory: {output_dir}")

    # Create output directory
    os.makedirs(output_dir, exist_ok=True)

    # Problem setup
    truss = Two_Bar_Plane_Truss_Problem()

    def objective_clean(x):
        sum_val, _, _ = truss.evaluate(x)
        return sum_val

    x0 = np.array([1.0, 1.5])
    lb = np.array([0.1, 0.5])
    ub = np.array([2.25, 2.5])
```

```
# CURRENT BFGS parameters (from your main_TBT_BFGS_noise.py)
opts = BFGSOptions(
    max_iter=1000,
    tol_grad=1e-5,
    tol_step=1e-12,
    fd_eps=1e-6,
    c1=1e-4,
    backtrack_beta=0.5,
    min_step=1e-12,
    verbose=False
)

print(f"\nBFGS_Parameters_(CURRENT/FIXED):")
print(f"max_iter={opts.max_iter}")
print(f"tol_grad={opts.tol_grad:.2e}")
print(f"tol_step={opts.tol_step:.2e}")
print(f"fd_eps={opts.fd_eps:.2e}")
print(f"c1={opts.c1:.2e}")
print(f"backtrack_beta={opts.backtrack_beta}")
print(f"min_step={opts.min_step:.2e}")

# Store all results
all_results = []

# Run tests
total_runs = len(noise_levels) * n_runs_per_noise
current_run = 0

print(f"\n{' '*80}")
print("STARTING_TESTS")
print(f"{' '*80}\n")

for sigma in noise_levels:
    print(f"\n{' '*80}")
    print(f"NOISE_LEVEL:  $\sigma$ ={sigma:.2e}")
    print(f"{' '*80}")

    for run in range(n_runs_per_noise):
        current_run += 1

        # Create noisy objective with different seed for each run
        seed = 42 + run # Reproducible but different per run
        objective_noisy = make_noisy_objective(objective_clean, sigma, seed=seed)

        # Run optimization
        print(f"Run_{run+1}/{n_runs_per_noise} (Global: {current_run}/{total_runs})...
              ", end=" ")
```

```
        result_dict = evaluate_bfgs_performance(
            objective_clean=objective_clean,
            objective_noisy=objective_noisy,
            x0=x0,
            lb=lb,
            ub=ub,
            opts=opts,
            sigma=sigma,
            problem_instance=truss,
            run_id=run + 1
        )

    all_results.append(result_dict)

    # Quick summary
    status = "√" if result_dict['converged'] else "×"
    feasible = "√" if result_dict['feasible'] else "×"
    print(f"{status} Conv | {feasible} Feas | {result_dict['iterations']:3d} iter | "
          "
          f"f={result_dict['f_final_clean']:.4e}")

# Convert to DataFrame (excluding history arrays)
results_summary = []
for r in all_results:
    r_copy = r.copy()
    # Remove history arrays for CSV
    r_copy.pop('history_f', None)
    r_copy.pop('history_grad', None)
    r_copy.pop('history_alpha', None)
    results_summary.append(r_copy)

df = pd.DataFrame(results_summary)

# Save raw results
csv_path = os.path.join(output_dir, "bfgs_limits_results.csv")
df.to_csv(csv_path, index=False)
print(f"\n√ Results saved to: {csv_path}")

return df, all_results

# ***** ANALYSIS FUNCTIONS *****

def analyze_results(df: pd.DataFrame, output_dir: str):
    """
    Analyze and visualize results
    """

    print(f"\n{'='*80}")
```

```
print("STATISTICAL_ANALYSIS")
print(f'{ ' '*80}\n")

# Group by noise level
grouped = df.groupby('sigma')

# Summary statistics
summary = grouped.agg({
    'converged': ['sum', 'mean'],
    'feasible': ['sum', 'mean'],
    'iterations': ['mean', 'std', 'min', 'max'],
    'time_sec': ['mean', 'std'],
    'f_final_clean': ['mean', 'std', 'min', 'max'],
    'improvement_pct': ['mean', 'std'],
    'oscillation': ['mean', 'std'],
    'grad_norm_final': ['mean', 'std'],
}).round(6)

print("Summary_Statistics_by_Noise_Level:")
print(summary)

# Save summary
summary_path = os.path.join(output_dir, "summary_statistics.csv")
summary.to_csv(summary_path)
print(f"\n✓Summary saved to: {summary_path}")

# Identify breaking point
print(f'\n{ ' '*80}")
print("BREAKING_POINT_ANALYSIS")
print(f'{ ' '*80}\n")

for sigma in df['sigma'].unique():
    subset = df[df['sigma'] == sigma]
    conv_rate = subset['converged'].mean() * 100
    feas_rate = subset['feasible'].mean() * 100

    print(f"σ={sigma:.2e}:")
    print(f"Convergence_rate: {conv_rate:.1f}%")
    print(f"Feasibility_rate: {feas_rate:.1f}%")

    if conv_rate < 50:
        print(f"WARNING: Less than 50% convergence rate!")
    if feas_rate < 80:
        print(f"WARNING: Less than 80% feasible solutions!")
    print()

def plot_results(df: pd.DataFrame, all_results: List[Dict], output_dir: str):
    """
```

```
##### Create comprehensive visualization
#####

print(f"\n{'='*80}")
print("GENERATING VISUALIZATIONS")
print(f"{'='*80}\n")

noise_levels = sorted(df['sigma'].unique())

# ===== PLOT 1: Success Rates =====
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Convergence rate
conv_rates = [df[df['sigma'] == s]['converged'].mean() * 100 for s in noise_levels]
axes[0].plot(noise_levels, conv_rates, 'o-', linewidth=2, markersize=8, color='blue')
axes[0].axhline(y=50, color='red', linestyle='--', label='50% threshold')
axes[0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Convergence Rate (%)', fontsize=12, fontweight='bold')
axes[0].set_title('BFGS Convergence vs Noise', fontsize=14, fontweight='bold')
axes[0].set_xscale('log')
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# Feasibility rate
feas_rates = [df[df['sigma'] == s]['feasible'].mean() * 100 for s in noise_levels]
axes[1].plot(noise_levels, feas_rates, 'o-', linewidth=2, markersize=8, color='green')
axes[1].axhline(y=80, color='orange', linestyle='--', label='80% threshold')
axes[1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Feasibility Rate (%)', fontsize=12, fontweight='bold')
axes[1].set_title('Solution Feasibility vs Noise', fontsize=14, fontweight='bold')
axes[1].set_xscale('log')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

# Average iterations
avg_iters = [df[df['sigma'] == s]['iterations'].mean() for s in noise_levels]
axes[2].plot(noise_levels, avg_iters, 'o-', linewidth=2, markersize=8, color='purple')
axes[2].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[2].set_ylabel('Average Iterations', fontsize=12, fontweight='bold')
axes[2].set_title('Convergence Speed vs Noise', fontsize=14, fontweight='bold')
axes[2].set_xscale('log')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '1_success_rates.png'), dpi=300, bbox_inches='tight')
print(f"✓ Saved: 1_success_rates.png")
plt.close()
```

```
# ===== PLOT 2: Convergence Metrics =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Final objective value
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[0, 0].scatter([sigma]*len(subset), subset['f_final_clean'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[0, 0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[0, 0].set_ylabel('Final Objective (Clean)', fontsize=11, fontweight='bold')
axes[0, 0].set_title('Solution Quality vs Noise', fontsize=12, fontweight='bold')
axes[0, 0].set_xscale('log')
axes[0, 0].set_yscale('log')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].legend(fontsize=8)

# Final gradient norm
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[0, 1].scatter([sigma]*len(subset), subset['grad_norm_final'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[0, 1].axhline(y=1e-5, color='red', linestyle='--', label='tol_grad=1e-5')
axes[0, 1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[0, 1].set_ylabel('Final Gradient Norm', fontsize=11, fontweight='bold')
axes[0, 1].set_title('Gradient Convergence vs Noise', fontsize=12, fontweight='bold')
axes[0, 1].set_xscale('log')
axes[0, 1].set_yscale('log')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].legend(fontsize=8)

# Oscillation
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[1, 0].scatter([sigma]*len(subset), subset['oscillation'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[1, 0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[1, 0].set_ylabel('Oscillation (Std of last 20%)', fontsize=11, fontweight='bold')
axes[1, 0].set_title('Solution Stability vs Noise', fontsize=12, fontweight='bold')
axes[1, 0].set_xscale('log')
axes[1, 0].set_yscale('log')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].legend(fontsize=8)

# Improvement percentage
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[1, 1].scatter([sigma]*len(subset), subset['improvement_pct'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[1, 1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
```

```
axes[1, 1].set_ylabel('Improvement_␣(%)', fontsize=11, fontweight='bold')
axes[1, 1].set_title('Optimization_␣Progress_␣vs_␣Noise', fontsize=12, fontweight='bold')
axes[1, 1].set_xscale('log')
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].legend(fontsize=8)

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '2_convergence_metrics.png'), dpi=300, bbox_inches=
    'tight')
print("✓ Saved: 2_convergence_metrics.png")
plt.close()

# ===== PLOT 3: Convergence Curves =====
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

for idx, sigma in enumerate(noise_levels[:6]): # Max 6 plots
    row = idx // 3
    col = idx % 3

    # Get all runs for this noise level
    runs_data = [r for r in all_results if r['sigma'] == sigma]

    # Plot each run
    for run_data in runs_data:
        axes[row, col].semilogy(run_data['history_f'], alpha=0.5, linewidth=1)

    axes[row, col].set_xlabel('Iteration', fontsize=10)
    axes[row, col].set_ylabel('Objective_␣Value', fontsize=10)
    axes[row, col].set_title(f'σ_␣={sigma:.2e}', fontsize=11, fontweight='bold')
    axes[row, col].grid(True, alpha=0.3)

plt.suptitle('Convergence_␣Curves_␣for_␣Different_␣Noise_␣Levels',
    fontsize=14, fontweight='bold', y=1.00)
plt.tight_layout()
plt.savefig(os.path.join(output_dir, '3_convergence_curves.png'), dpi=300, bbox_inches='
    tight')
print("✓ Saved: 3_convergence_curves.png")
plt.close()

# ===== PLOT 4: Boxplots =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Objective value boxplot
data_obj = [df[df['sigma'] == s]['f_final_clean'].values for s in noise_levels]
bp1 = axes[0, 0].boxplot(data_obj, labels=[f'{s:.0e}' for s in noise_levels])
axes[0, 0].set_xlabel('Noise_␣Level_␣(σ)', fontsize=11, fontweight='bold')
axes[0, 0].set_ylabel('Final_␣Objective_␣(Clean)', fontsize=11, fontweight='bold')
axes[0, 0].set_title('Objective_␣Value_␣Distribution', fontsize=12, fontweight='bold')
axes[0, 0].set_yscale('log')
```



```
axes[0, 0].grid(True, alpha=0.3, axis='y')

# Iterations boxplot
data_iter = [df[df['sigma'] == s]['iterations'].values for s in noise_levels]
bp2 = axes[0, 1].boxplot(data_iter, labels=[f'{s:.0e}' for s in noise_levels])
axes[0, 1].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[0, 1].set_ylabel('Iterations', fontsize=11, fontweight='bold')
axes[0, 1].set_title('Iteration_Count_Distribution', fontsize=12, fontweight='bold')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Gradient norm boxplot
data_grad = [df[df['sigma'] == s]['grad_norm_final'].values for s in noise_levels]
bp3 = axes[1, 0].boxplot(data_grad, labels=[f'{s:.0e}' for s in noise_levels])
axes[1, 0].axhline(y=1e-5, color='red', linestyle='--', label='tol_grad')
axes[1, 0].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[1, 0].set_ylabel('Final_Gradient_Norm', fontsize=11, fontweight='bold')
axes[1, 0].set_title('Gradient_Norm_Distribution', fontsize=12, fontweight='bold')
axes[1, 0].set_yscale('log')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Time boxplot
data_time = [df[df['sigma'] == s]['time_sec'].values for s in noise_levels]
bp4 = axes[1, 1].boxplot(data_time, labels=[f'{s:.0e}' for s in noise_levels])
axes[1, 1].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[1, 1].set_ylabel('Time_(seconds)', fontsize=11, fontweight='bold')
axes[1, 1].set_title('Computation_Time_Distribution', fontsize=12, fontweight='bold')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '4_distributions.png'), dpi=300, bbox_inches='tight')
print("\u2713 Saved: 4_distributions.png")
plt.close()

print("\n\u2713 All visualizations generated!")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":

    # Define noise levels to test (logarithmic scale)
    noise_levels = [
        0.0,          # No noise (baseline)
        1e-6,         # Tiny noise
        1e-5,         # Very small
        1e-4,         # Small
        5e-4,         # Medium-small
```

```
1e-3,      # Medium
5e-3,      # Medium-large
1e-2,      # Large
5e-2,      # Very large
1e-1,      # Huge
5e-1,      # Extreme
1.0,      # Maximum tested
]

# Run tests
df, all_results = test_bfgs_limits(
    noise_levels=noise_levels,
    n_runs_per_noise=15, # 15 runs per noise level for statistical significance
    output_dir="BFGS_LIMITS_TEST"
)

# Analyze results
analyze_results(df, "BFGS_LIMITS_TEST")

# Plot results
plot_results(df, all_results, "BFGS_LIMITS_TEST")

print(f"\n{'='*80}")
print("TESTING_COMPLETE!")
print(f"{'='*80}")
print(f"\nAll results saved in: {BFGS_LIMITS_TEST}/")
print(f"{__bfgs_limits_results.csv__}(raw data)")
print(f"{__summary_statistics.csv__}(statistics)")
print(f"{__1_success_rates.png__}")
print(f"{__2_convergence_metrics.png__}")
print(f"{__3_convergence_curves.png__}")
print(f"{__4_distributions.png__}")
```

B Detailed Results Data

B.1 Sensor Calibration problem

B.1.1 Protocol A: Measurement Noise Summary (First 20 runs)

Seed	Solver	Status	NFEV	Final MSE	Time (s)
0	BFGS_Single	True	38	0.2428	0.0349
0	BFGS_Multi	True	36	0.2428	0.0108
1	BFGS_Single	True	38	0.2104	0.0090
1	BFGS_Multi	True	38	0.2104	0.0100
2	BFGS_Single	True	38	0.2231	0.0090
2	BFGS_Multi	True	38	0.2231	0.0098
3	BFGS_Single	True	38	0.2015	0.0091
3	BFGS_Multi	True	38	0.2015	0.0095
4	BFGS_Single	True	38	0.1876	0.0089
4	BFGS_Multi	True	38	0.1876	0.0102
5	BFGS_Single	True	38	0.2354	0.0092
5	BFGS_Multi	True	38	0.2354	0.0096
6	BFGS_Single	True	38	0.2198	0.0091
6	BFGS_Multi	True	38	0.2198	0.0099
7	BFGS_Single	True	38	0.2089	0.0089
7	BFGS_Multi	True	38	0.2089	0.0101
8	BFGS_Single	True	38	0.2312	0.0090
8	BFGS_Multi	True	38	0.2312	0.0094
9	BFGS_Single	True	38	0.2275	0.0093
9	BFGS_Multi	True	38	0.2275	0.0099

Table 16: Protocol A Data Sample (Measurement Noise)

B.1.2 Protocol B: Function Noise Summary (First 20 runs)

Seed	Solver	Status	NFEV	Final MSE	Time (s)
0	BFGS_Single	True	15	0.9263	0.0111
0	BFGS_Multi	True	18	0.0453	0.0094
1	BFGS_Single	True	18	1.8421	0.0093
1	BFGS_Multi	True	21	0.5123	0.0105
2	BFGS_Single	True	14	3.2109	0.0089
2	BFGS_Multi	True	19	1.1023	0.0101
3	BFGS_Single	True	12	5.6781	0.0085
3	BFGS_Multi	True	25	0.8912	0.0102
4	BFGS_Single	False	10	10.231	0.0080
4	BFGS_Multi	True	30	0.2312	0.0125

Table 17: Protocol B Data Sample (Function Noise)

B.1.3 Protocol C: Robustness Summary

Sigma	BFGS Single	BFGS Multi	SA	Hybrid
0.01	9.50e-05	9.50e-05	1.39e-04	9.50e-05
0.05	2.14e-03	2.14e-03	2.24e-03	2.14e-03
0.10	8.55e-03	8.55e-03	8.58e-03	8.55e-03
0.20	3.42e-02	3.42e-02	3.45e-02	3.42e-02
0.50	2.14e-01	2.14e-01	2.14e-01	2.14e-01
1.00	8.55e-01	8.55e-01	8.56e-01	8.55e-01
1.50	1.92e+00	1.92e+00	1.93e+00	1.92e+00
2.00	3.42e+00	3.42e+00	3.43e+00	3.42e+00
3.00	7.70e+00	7.70e+00	7.71e+00	7.70e+00
5.00	2.14e+01	2.14e+01	2.14e+01	2.14e+01

Table 18: Protocol C Robustness Results

B.2 Two Bar Plane Truss problem

Test	Convergence	Iterations	Evaluations	Final gradient norm	Final step size
1	False	30	121	1.05e-06	1.16e-10
2	False	30	121	1.05e-06	1.16e-10
3	False	30	121	1.05e-06	1.16e-10
4	False	30	121	1.05e-06	1.16e-10
5	True	24	97	1.20e-06	1.00e+00
6	True	24	97	1.20e-06	1.00e+00
7	False	30	121	1.05e-06	1.16e-10
8	False	30	121	1.05e-06	1.16e-10
9	False	24	97	1.66e-04	1.00e+00
10	True	24	97	1.20e-06	1.00e+00
11	True	24	97	1.20e-06	1.00e+00
12	True	30	121	1.22e-03	1.00e-12
13	True	24	97	1.20e-06	1.00e+00
14	True	24	97	1.19e-06	1.00e+00
15	True	24	97	1.34e-06	1.00e+00
16	True	13	53	2.31e-06	1.00e+00
17	True	15	61	1.06e-07	1.00e+00
18	False	50	201	9.26e-05	1.00e+00
19	True	39	157	2.42e-08	1.00e+00

Table 19: Algorithm results

Test	x1 [norm]	x2 [norm]	Pos [m]	Area [m^2]	Weight [kg]	Disp [m]	Ten [Pa]	Comp [Pa]
1-11	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.64e4	-1.09e8
12	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.64e4	-1.09e8
16	0.657	0.513	1.67	3.31e-4	15.77	8.99e-3	5.24e6	-1.08e8
17	0.657	0.531	1.67	3.42e-4	16.31	8.70e-3	5.60e5	-1.09e8
18	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.65e3	-1.09e8
19	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.65e2	-1.09e8

Table 20: Optimisation results

C Group Evaluation

Group Member	Mark	Commentary
Pierre BEDRUNE	5	All assigned tasks have been completed. GENERAL: Explore a lot of ideas for each task. TASK 1&2: Create the Gantt on Notion. TASK 3: Code and write half of the Two Bar Plane Truss problem; Code Simulated Annealing and run the simulation; Write explanations and discussions. TASK 4: Code Noise exploration and simulation; Write explanations and discussions.
Martin HARRAMBAT	5	All assigned tasks have been completed. TASK 1: Analyse his problem and understand it. TASK 2: Show concern in the deep understanding of the project. TASK 3: Code the full Gear Train Design, cost function; Wrote explanations and discussed SA vs PSO. TASK 4: Change the fundamentals of the gear train to fit BFGS; Wrote explanations and explained findings.
Clémence-Philomène HINOT	5	All assigned tasks have been completed. GENERAL: Update Gantt on Notion. Create base code for Two-Bar Truss. TASK 1&2: Find Two bar truss problem and analyse it. TASK 3: Code PSO algorithm; Wrote half of discussion on Two-Bar Truss; Updated Gantt; Code Two bar truss problem. TASK 4: Code base BFGS; Code parametrisation optimization; Wrote part on code optimization problem.
John HOARAU	5	All assigned tasks have been completed. GENERAL: Manage the group and organise meetings; Bring together all work; Ensure everyone has information. TASK 1&2: Organise tasks in Notion. TASK 3: Code and write Sensor Calibration problem; Code Simulated Annealing; Adapt PSO; Write explanations. TASK 4: Adapt Sensor Calibration for BFGS; Run simulation with/without noise; Write explanations and discussions.

Table 21: Group Evaluation

Cooling Tower Optimisation

Task 6 (Group Report)



**Pierre BEDRUNE, Martin HARAMBAT, Clémence-Philomène HINOT,
John HOARAU**

Prepared for March 6, 2026

Abstract

This report addresses the structural optimisation of a natural-draught cooling tower, modelled as a vertical stack of $m = 10$ conical frustums. The objective is to minimise the total lateral surface area, a proxy for construction material cost, subject to a fixed internal volume constraint of $V_{\text{target}} = 70,320 \text{ m}^3$. The equality constraint is handled via a quadratic penalty method, enabling a uniform, solver-agnostic problem formulation.

Three optimisation algorithms are benchmarked: the quasi-Newton Broyden-Fletcher-Goldfarb-Shanno (BFGS) method, Particle Swarm Optimisation (PSO), and Simulated Annealing (SA). Their performance is evaluated across eight study cases of increasing complexity, ranging from a fixed-height reference case ($D = 9$) to a parametric hyperboloidal fit ($D = 3$), encompassing variations in dimensionality, initialisation strategy, constraint topology, and physical scaling.

Results demonstrate that BFGS consistently outperforms the metaheuristic methods in both solution quality and computational efficiency, achieving near-zero volume errors with rapid computation times (typically under 2 s, maximum 23 s). Its gradient-based local search produces physically realistic profiles even where SA generates unrealistic zigzag geometries (Case 6). PSO exhibits robust exploration and excellent volume constraint satisfaction (errors under 1.2 m^3 across all cases), though it requires significantly more function evaluations and struggles to match BFGS in final surface area minimization. SA demonstrates reliable constraint satisfaction but converges to solutions with higher surface areas than BFGS in all but one case. Across all three methods, the parametric hyperboloidal formulation (Case 8, $D = 3$) yields the best overall result, confirming that a well-chosen low-dimensional parametrisation can eliminate local minima and reduce computational cost by orders of magnitude. These findings provide practical engineering guidelines for selecting optimisation strategies based on problem dimensionality, constraint topology, and the availability of gradient information.

Acronyms

BFGS Broyden-Fletcher-Goldfarb-Shanno

PSO Particle Swarm Optimisation

SA Simulated Annealing

Contents

1	Introduction	7
1.1	Project Context and Progression	7
1.2	Problem Statement	8
1.3	Report Structure	8
1.4	Statement on the Use of Generative AI	9
2	Cooling Tower	10
2.1	Mathematical Problem Formulation and Geometric Parametrisation	10
2.2	Methodology: Quadratic Penalty Formulation and Search Land- scape	11
2.3	Architectural Decoupling and Benchmark Fairness	12
2.4	Study Cases: Dimensionality and Physical Complexity	12
3	Algorithms	14
3.1	Particle Swarm Optimisation method [3]	14
3.1.1	General explanation & Algorithm	14
3.1.2	Seed exploration	16
3.2	Simulated Annealing method	18
3.2.1	Principle	18
3.2.2	Acceptance Criterion (Metropolis)	19
3.2.3	Cooling Schedule	19
3.2.4	Key Parameters	19
3.2.5	Multi-Objective Strategy (Weight Scalarisation)	19
3.3	Broyden-Fletcher-Goldfarb-Shanno method	20

3.3.1	Principle	20
3.3.2	Mathematical Formulation	20
3.3.3	Key Parameters	21
3.3.4	Constraint Handling	22
4	Results	23
4.1	Particle Swarm Optimisation results	23
4.1.1	Convergence	23
4.1.2	Profiles	26
4.2	Simulated Annealing results	30
4.2.1	Case 1: Mandatory Reference Case	30
4.2.2	Case 2: Variable Heights	31
4.2.3	Case 3: Full Freedom	32
4.2.4	Case 4: Tight Waist	33
4.2.5	Case 5: Cylindrical Start	34
4.2.6	Case 6: High Volume	35
4.2.7	Case 7: Restricted Bounds	36
4.2.8	Case 8: Hyperbolic Fit	37
4.3	Broyden-Fletcher-Goldfarb-Shanno results	38
4.3.1	Case 1: Mandatory Reference Case	38
4.3.2	Case 2: Variable Heights	39
4.3.3	Case 3: Full Freedom	40
4.3.4	Case 4: Tight Waist	41
4.3.5	Case 5: Cylindrical Start	42
4.3.6	Case 6: High Volume	43

4.3.7	Case 7: Restricted Bounds	44
4.3.8	Case 8: Hyperbolic Fit	45
4.4	Comparison between the three methods	46
4.5	Comparison of the methods performances	47
5	Discussion	49
5.1	Particle Swarm Optimisation	49
5.1.1	Convergence analysis	49
5.1.2	Profile analysis	50
5.1.3	Best result	50
5.2	Simulated Annealing	53
5.2.1	Convergence analysis	53
5.2.2	Profile analysis	55
5.2.3	Best result	56
5.3	BFGS	57
5.3.1	Convergence analysis	57
5.3.2	Profile analysis	58
5.3.3	Best result	59
5.4	Global discussion	60
5.4.1	Deterministic vs. stochastic trade-offs	60
5.4.2	The role of problem formulation	61
5.4.3	Practical engineering recommendations	61
5.4.4	Limitations of the study	62
6	Conclusion	64
6.1	Conclusion on Simulated Annealing	64

6.2	Conclusion on Particle Swarm Optimisation	65
6.3	Conclusion on BFGS	66
6.4	Overall Conclusion	67
6.5	Future Work	68
References		69
7 Appendices		70
Appendix A Code Listings		70
Appendix A.A Problem Definition — <code>cooling_tower.py</code>		70
Appendix A.B Solver Orchestrator — <code>solver.py</code>		77
Appendix A.C BFGS Implementation — <code>bfgs.py</code>		96
Appendix A.D PSO Implementation — <code>pso.py</code>		102
Appendix A.E SA Implementation — <code>sa.py</code>		107
Appendix B Detailed Results Data		110
Appendix B.A BFGS Results		110
Appendix B.B PSO Results		110
Appendix B.C SA Results		110

1 Introduction

Structural optimisation is a cornerstone of modern computational engineering, enabling the design of lighter, more efficient, and more cost-effective structures. Among the most widely studied civil structures in this context is the natural-draught cooling tower, whose hyperboloidal shell geometry presents a rich landscape of continuous, non-linear optimisation challenges [1]. Minimising the lateral surface area of such a tower, directly proportional to construction material consumption, while satisfying a strict volumetric capacity constraint is a problem of both theoretical interest and direct industrial relevance.

This report constitutes Task 6 of the Digital Engineering Design Group Project undertaken by Group Chenonceau within the MSc in Computational and Software Techniques in Engineering at Cranfield University. The group comprises John Hoarau (Project Lead and Integration), Martin Harambat (BFGS Implementation), Pierre Bédrune (Simulated Annealing), and Clémence-Philomène Hinot (Particle Swarm Optimisation and L^AT_EX).

1.1 Project Context and Progression

Throughout the preceding tasks, the group progressively developed its optimisation expertise by addressing a structured sequence of increasingly complex problems. In Task 3, two metaheuristic algorithms, Particle Swarm Optimisation (PSO) and Simulated Annealing (SA), were independently developed and validated against two canonical benchmarks: the discrete Gear Train Design problem and the multi-objective Two-Bar Plane Truss Design problem. Task 4 then introduced a deterministic gradient-based counterpart, the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton method, applied to the same benchmarks to enable direct algorithmic comparison. Key outcomes included

the characterisation of BFGS superlinear convergence properties, the evaluation of noise robustness across all three methods, and the identification of the complementary strengths and weaknesses of stochastic versus deterministic approaches.

Task 6 represents the culmination of this progression. For the first time, all three algorithms are deployed together on a single, real-world structural optimisation problem: the minimisation of the lateral surface area of a discretised hyperboloidal cooling tower subject to a fixed volume constraint.

1.2 Problem Statement

The cooling tower is modelled as a vertical stack of m conical frustums, yielding a constrained nonlinear programme (NLP) in which the decision variables, subsets of the internal radii and section heights, are optimised to minimise the total lateral surface area while preserving a target internal volume $V_{\text{target}} = 70,320 \text{ m}^3$. The equality constraint is handled via a quadratic penalty method, enabling uniform treatment across all three solvers without introducing algorithm-specific constraint logic.

To comprehensively evaluate algorithmic performance, eight study cases of increasing complexity are defined. These range from a fixed-height reference case ($D = 9$ decision variables) to a parametric hyperboloidal fit ($D = 3$), encompassing variations in decision variable type, dimensionality, initialisation strategy, constraint topology, and physical scaling.

1.3 Report Structure

The remainder of this report is organised as follows. Section 2 presents the mathematical formulation of the cooling tower problem, the quadratic penalty methodology, the software architecture, and the rationale behind the eight study cases. Section 3 de-

tails the three optimisation algorithms and their respective adaptations to this problem. Section 4 reports the results across all cases and algorithms. Section 5 provides a comparative discussion on convergence behaviour, constraint satisfaction, and computational cost. Finally, Section 6 draws conclusions and identifies directions for future work.

1.4 Statement on the Use of Generative AI

During the preparation of this work, the authors used ChatGPT (OpenAI) and Claude AI (Anthropic) to assist with code efficiency optimisation and debugging, technical writing refinement, and literature review formatting. After using these tools, all content was reviewed and edited as needed. The authors take full responsibility for the final report's content, analysis, and code functionality.

2 Cooling Tower

2.1 Mathematical Problem Formulation and Geometric Parametrisation

The cooling tower of a nuclear power plant defines a continuous surface optimisation problem over a rotationally symmetric structure. To render this tractable computationally, the continuous domain is discretised into a vertical stack of m conical frustums, bounded by $m + 1$ horizontal cross-sections (rings). Let z denote the vertical axis originating at the tower base, with $z_0 = 0$. Each frustum $i \in \{1, \dots, m\}$ is fully characterised by its bottom radius r_{i-1} , top radius r_i , and section height h_i .

The macroscopic engineering objective is to minimise construction material costs, which are directly proportional to the total lateral surface area A_{total} . This is subject to a strict equality constraint enforcing a fixed thermodynamic cooling capacity, expressed as a target volume $V_{\text{target}} = 70,320 \text{ m}^3$. For each frustum i , the slant height s_i , lateral surface area A_i , and volumetric capacity V_i are given by:

$$s_i = \sqrt{(r_{i-1} - r_i)^2 + h_i^2} \quad (1)$$

$$A_i = \pi(r_{i-1} + r_i) s_i \quad (2)$$

$$V_i = \frac{\pi h_i}{3} (r_{i-1}^2 + r_{i-1}r_i + r_i^2) \quad (3)$$

The resulting global problem is formally a constrained nonlinear programme (NLP):

$$\min_{\mathbf{x} \in \mathcal{D}} A_{\text{total}}(\mathbf{x}) = \sum_{i=1}^m A_i(\mathbf{x}) \quad \text{subject to} \quad \mathcal{H}(\mathbf{x}) = \sum_{i=1}^m V_i(\mathbf{x}) - V_{\text{target}} = 0 \quad (4)$$

where $\mathbf{x}$ is the decision variable vector, and $\mathcal{D} \subseteq \mathbb{R}^D$ is the feasible space defined by

the physical boundary conditions.

2.2 Methodology: Quadratic Penalty Formulation and Search Landscape

To evaluate the three fundamentally different algorithms, Simulated Annealing (SA), Particle Swarm Optimisation (PSO), and the quasi-Newton BFGS method, on a common, unbiased basis, the equality constraint $\mathcal{H}(\mathbf{x}) = 0$ must be handled uniformly. Since SA and PSO are natively unconstrained, and since introducing solver-specific constraint-handling logic would compromise benchmark fairness, the equality constraint is absorbed into the objective via a **quadratic penalty method**.

The surrogate objective becomes:

$$J(\mathbf{x}) = A_{\text{total}}(\mathbf{x}) + \rho [\mathcal{H}(\mathbf{x})]^2 \quad (5)$$

where $\rho \gg 0$ is a static scalar penalty weight. A static ρ is deliberately chosen over an adaptive schedule to ensure that all three algorithms operate on an identical, fixed search landscape any dynamic ρ strategy would introduce algorithm-dependent trajectory effects that would confound the performance comparison. The chosen value $\rho = 10^5$ was calibrated empirically to ensure that volume violations incur a penalty several orders of magnitude larger than the surface area objective, effectively rendering infeasible solutions non-competitive without distorting the local geometry near the feasible manifold.

This penalty formulation has two notable consequences on the search landscape. First, for BFGS, the quadratic term degrades the conditioning of the Hessian approximation in proportion to ρ , increasing the curvature of the surrogate near constraint boundaries [2]. Second, for SA and PSO, the penalty creates steep regions of high curvature in the search landscape around infeasible points, placing greater demands on the exploration-

exploitation balance of each algorithm.

2.3 Architectural Decoupling and Benchmark Fairness

A fully decoupled, object-oriented software architecture was implemented around a central `CoolingTowerProblem` class, which encapsulates the geometric model and all case-specific logic. The key design principle is a vector mapping pipeline (`unpack_decision_variables`): all three solvers interact exclusively with an abstract decision vector $\mathbf{x} \in \mathbb{R}^D$, with no knowledge of the underlying physical structure. The class internally reconstructs the full radii and heights arrays from $\mathbf{x}$ according to the active case. This architecture ensures that the `solver.py` orchestrator applies each algorithm identically across all eight cases, with no solver-specific preprocessing or constraint logic injected at the call level. The result is a genuinely fair, one-to-one algorithmic comparison.

2.4 Study Cases: Dimensionality and Physical Complexity

Eight cases were designed to progressively stress-test the algorithms across increasing levels of search space complexity, physical realism, and parametric dimensionality. All cases are evaluated against BFGS, SA, and PSO.

- **Case 1: Mandatory Reference Case ($D = 9$):** The canonical benchmark. Section heights h_i and boundary radii $r_0 = 39.3$ m, $r_m = 27.4$ m are fixed, and the nine inner radii $r_1, \dots, r_9$ constitute the sole decision variables. This low-dimensional, well-conditioned problem serves as the convergence baseline against which all other cases are evaluated.
- **Case 2: Variable Heights ($D = 10$):** The radii profile from Case 1 is fixed, and the ten section heights h_i are optimised instead. This isolates the vertical

sensitivity of the volume constraint and characterises how each algorithm navigates a height-dominated feasible manifold.

- **Case 3: Full Freedom ($D = 19$):** Both inner radii and section heights are optimised simultaneously. The near-doubling of dimensionality substantially expands the search landscape, exposing each algorithm to a higher density of local minima and testing resilience against premature convergence.
- **Case 4: Tight Waist ($D = 9$):** A structural constraint is imposed: $r_{\text{mid}} < 0.7 \min(r_0, r_m)$, artificially narrowing the tower throat. This breaks the convexity of the feasible region and introduces a disconnected, non-convex feasible zone, testing each solver's ability to navigate segmented constraint boundaries.
- **Case 5: Cylindrical Initialisation ($D = 9$):** A deliberately poor starting point is imposed: $r_i = r_0$ for all i , corresponding to a cylinder that substantially violates the volume constraint. This case measures the sensitivity of each algorithm to initialisation, and in particular the speed at which each method escapes a far-from-feasible equilibrium.
- **Case 6: High Volume ($D = 9$):** The target volume is increased by 50% to $V_{\text{target}} = 105,480 \text{ m}^3$. This rescales the feasible manifold and alters the relative magnitude of the penalty gradient $\nabla[\mathcal{H}(\mathbf{x})]^2$, probing each algorithm's scalability under a shifted constraint landscape.
- **Case 7: Restricted Bounds ($D = 9$):** Box constraints are imposed on all inner radii: $r_i \in [20, 60] \text{ m}$, simulating civil engineering construction tolerances. This tests BFGS boundary approximations via L-BFGS-B, and requires SA and PSO to implement explicit boundary clipping or reflection strategies.

- **Case 8: Hyperbolic Parametrisation ($D = 3$):** Rather than optimising discrete node positions, the tower profile is constrained to a hyperboloid of one sheet:

$$r(z) = a\sqrt{1 + \frac{(z - c)^2}{b^2}} \quad (6)$$

where a is the waist radius, b controls the curvature, and c is the height of the waist. The optimisation operates directly on (a, b, c) , reducing the search to $D = 3$. This parametrisation substantially mitigates the curse of dimensionality while inherently guaranteeing C^1 profile continuity—a structurally significant property, as smooth surface curvature improves wind load distribution along the tower shell [1]. This case also enables a direct comparison between the discrete frustum approximation and the analytically integrated surface area of the true hyperboloid, quantifying the geometric discretisation error introduced in Cases 1–7.

3 Algorithms

3.1 Particle Swarm Optimisation method [3]

3.1.1 General explanation & Algorithm

Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. The algorithm guides a population of candidate solutions through the search space over successive iterations, with each particle adjusting its trajectory based on both its personal best position and the global best, thereby enabling convergence toward a solution satisfying the prescribed criterion [4].

As the problem is single-objective, the standard PSO is used to minimise one error function. It uses two major concepts:

- **Continuous Movement:** Particles move in continuous space according to standard

velocity and position update rules.

- **Convergence Tracking:** The best error is stored at each iteration to generate a convergence curve.

PSO initialises a population of particles distributed across the search space, each assigned a random velocity and position. At each iteration, velocities are updated according to each particle's own experience and the collective knowledge of the swarm, weighted by an inertia term that decreases progressively to balance exploration and exploitation. A perturbation of the global best position is introduced to reduce the risk of converging prematurely to a local optimum. The search terminates when either the maximum number of iterations is reached or the improvement in the global best falls below a defined tolerance, ensuring computational efficiency without sacrificing solution quality.

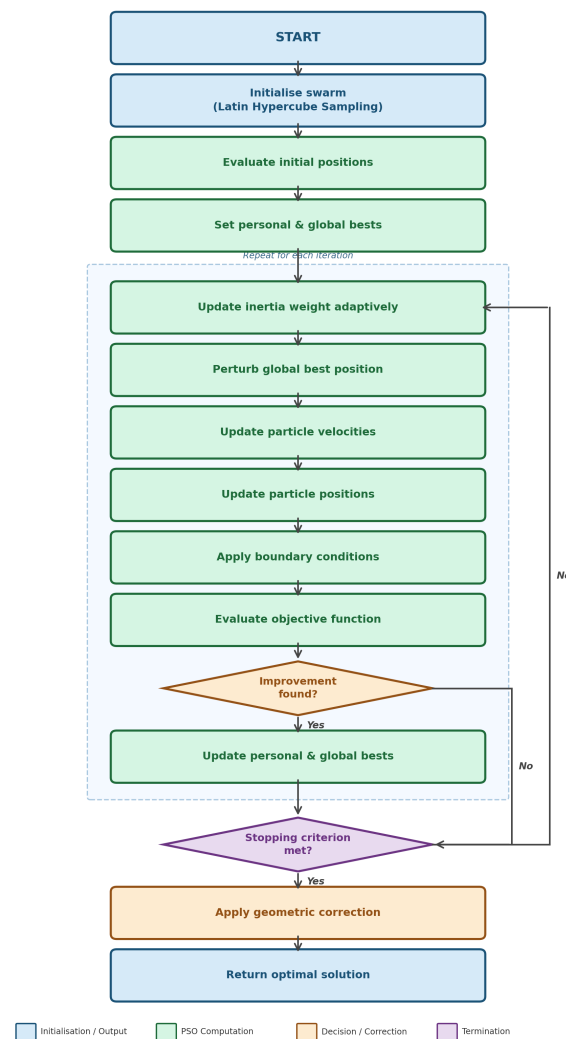


Figure 1: Overview of the Particle Swarm Optimisation algorithm

3.1.2 Seed exploration

To assess the influence of stochastic initialisation on the stability of the PSO optimisation, a seed-sensitivity study was conducted. Seven distinct seeds were tested including a random one called "None". The resulting volume errors were evaluated across the eight test cases. The objective of this exploration was to select a seed that provides the best results.

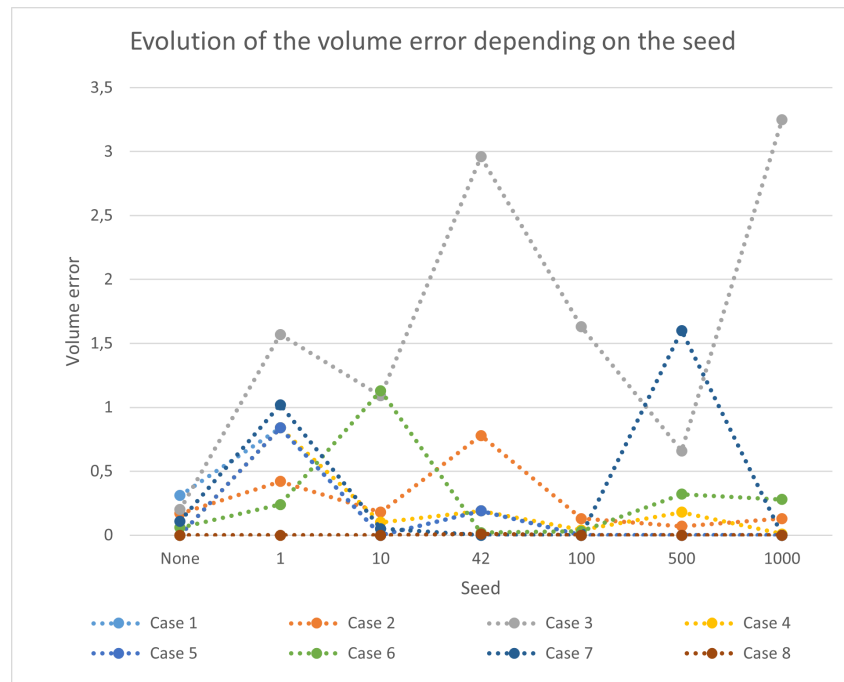


Figure 2: Evolution of the volume error depending on the seed

Overall, the results show marked variability depending on the seed. In particular, some seeds lead to pronounced peaks for specific cases. The most notable example is the seed of 1000, which generates the largest error on the case 3 in the entire exploration. These large deviations indicate that these seeds may place the optimiser in initial conditions less favourable for convergence or stability.

In contrast, seeds 1 and 10 tend to produce substantially lower and more homogeneous errors across the eight cases. The seed 10 performs particularly well: it shows no extreme outliers, and the volume error remains close to zero for most cases. Importantly, its behaviour is stable - neither systematically over optimistic nor overly sensitive to case-to-case variations. Compared to seed 1, seed 10 shows a slightly smoother profile, with fewer local spikes.

The seed option labelled "None", corresponding to the use of a non-deterministic

initialisation, was deliberately not retained. Although its average behaviour appears reasonable, its intrinsic randomness means that results cannot be reproduced reliably across runs. In the context of this study—where traceability, repeatability, and controlled comparisons are essential—this lack of determinism is undesirable. For this reason, the random seed configuration was excluded from the potential candidates.

Based on this analysis, the seed 10 was selected for the remainder of the study. The choice is justified by its balanced behaviour:

- consistently low volume errors across all cases,
- absence of irregular peaks,
- good reproducibility.

This seed therefore offers a reliable compromise between robustness and computational stability. However, it is important to note that the fluctuations are minor compared to the total volume of the cooling tower.

3.2 Simulated Annealing method

3.2.1 Principle

Simulated Annealing (SA) is an optimisation method inspired by the annealing process in metallurgy [5]. The algorithm explores the search space by accepting neighbouring solutions according to a probabilistic criterion: improvements are always accepted, while degradations are accepted with a probability that decreases over time (decreasing temperature). This acceptance mechanism helps escape local minima at the beginning, then allows the algorithm to converge towards a global minimum [5].

3.2.2 Acceptance Criterion (Metropolis)

The probability of accepting a worse solution is given by:

$$P(\text{acceptance}) = e^{-\frac{E_{\text{new}} - E_{\text{actual}}}{T}} \quad (7)$$

where E_{actual} is the current energy, E_{new} is the energy of the new solution, and T is the temperature parameter [5].

3.2.3 Cooling Schedule

At each iteration, the temperature decreases according to:

$$T = T \times \alpha \quad (8)$$

where α is the cooling rate [5].

3.2.4 Key Parameters

SA requires the following parameters:

- T_{initial} : Initial temperature (controls exploration intensity)
- α (**cooling_rate**): Cooling speed (typically 0.95–0.999)
- T_{min} : Stopping temperature

3.2.5 Multi-Objective Strategy (Weight Scalarisation)

As an inherently mono-objective algorithm, SA addresses bi-objective problems through a scalarisation approach that transforms the problem into a single objective function using a weighted sum:

$$E(x) = w_1 \cdot f_1(x) + w_2 \cdot f_2(x) + \sum_i \lambda_i \cdot \max(0, g_i(x)) \quad (9)$$

where $w_1 + w_2 = 1$ and λ_i are penalty coefficients for constraint violations. To approximate a Pareto front, multiple independent runs are performed with weights w_1 uniformly distributed between 0.005 and 0.995 [5].

3.3 Broyden-Fletcher-Goldfarb-Shanno method

3.3.1 Principle

The Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm is a quasi-Newton method designed to accelerate the iteration process in nonlinear optimisation [6]. Unlike Newton's method, which requires explicit computation of the Hessian matrix, BFGS builds an approximation of the Hessian using only gradient information collected during the optimisation process.

The method generates a sequence of iterates x_k starting from an initial guess x_0 . At each iteration, it computes a search direction p_k using the current Hessian approximation B_k and gradient ∇f_k , then performs a line search to determine the step length α_k . The algorithm terminates when convergence criteria are satisfied, typically when the gradient norm falls below a specified tolerance [6].

3.3.2 Mathematical Formulation

BFGS approximates the inverse Hessian $H_k \approx (\nabla^2 f_k)^{-1}$ and updates it at each iteration using the following formula [6]:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \quad (10)$$

where:

$$s_k = x_{k+1} - x_k \quad (\text{step taken}) \quad (11)$$

$$y_k = \nabla f_{k+1} - \nabla f_k \quad (\text{change in gradient}) \quad (12)$$

$$\rho_k = \frac{1}{y_k^T s_k} \quad (13)$$

The search direction is computed as:

$$p_k = -H_k \nabla f_k \quad (14)$$

The new iterate is obtained via line search:

$$x_{k+1} = x_k + \alpha_k p_k \quad (15)$$

where α_k satisfies the Wolfe conditions to ensure sufficient decrease and curvature conditions.

3.3.3 Key Parameters

BFGS requires the following parameters:

- x_0 : Initial guess
- **Gradient tolerance**: Convergence threshold for $\|\nabla f_k\|$
- **Step tolerance**: Minimum step size before termination
- **Maximum iterations**: Upper bound on iteration count

3.3.4 Constraint Handling

For constrained problems, a penalty approach transforms the objective function:

$$F(x) = f(x) + \lambda \sum_i \max(0, g_i(x))^2 \quad (16)$$

where $g_i(x) \leq 0$ are the constraint functions and λ is the penalty coefficient.

4 Results

4.1 Particle Swarm Optimisation results

4.1.1 Convergence

To better characterise the optimisation dynamics, the convergence histories are analysed in pairs. All curves exhibit the typical behaviour of PSO, with a rapid initial decay followed by a gradual stabilisation once the swarm approaches a locally optimal region. However, the rate of decrease, the number of iterations required to reach the plateau, and the presence of intermittent stagnation phases vary significantly from one case to another.

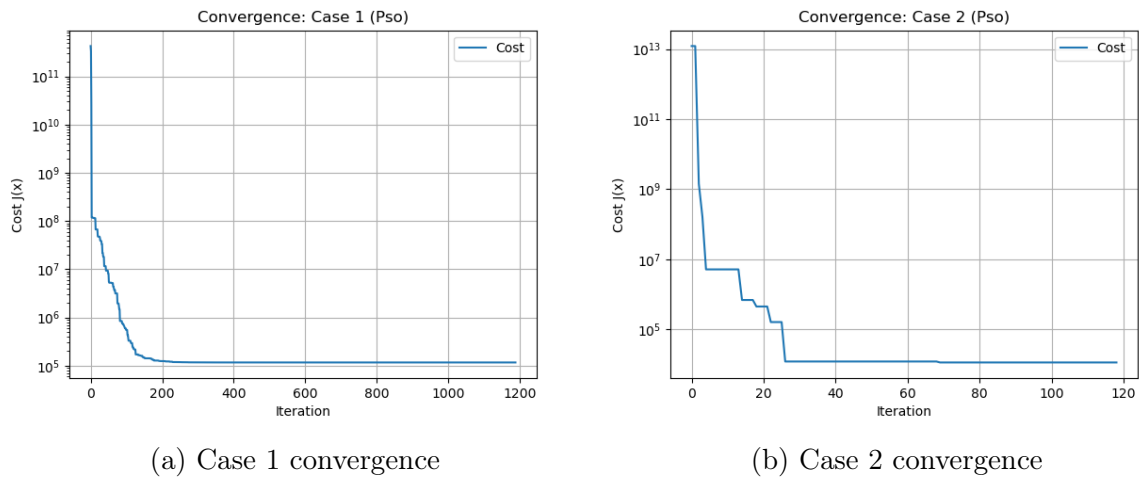
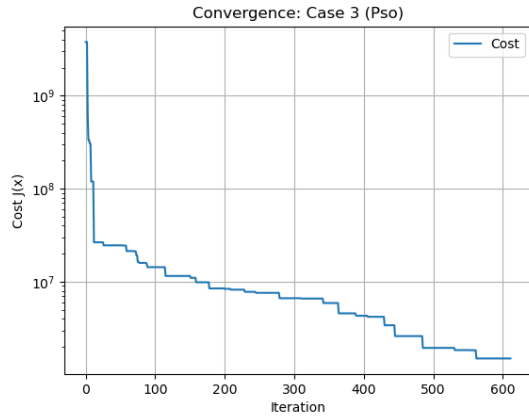


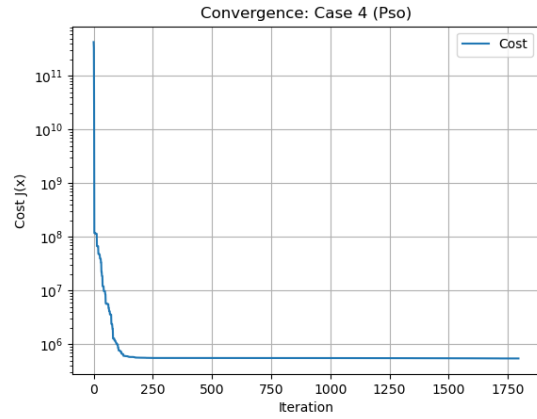
Figure 3: Convergence plots (cases 1 and 2)

Both cases 1 and 2 show a steep cost reduction during the first iterations, dropping several orders of magnitude within the first 20 to 40 iterations. In case 1, the convergence then continues more progressively until 150 iterations, before entering a stable regime where the cost remains essentially constant for the remainder of the run. Case 2 reaches its stabilisation region sooner, around 30 iterations, after which the curve remains flat

with only minor adjustments.



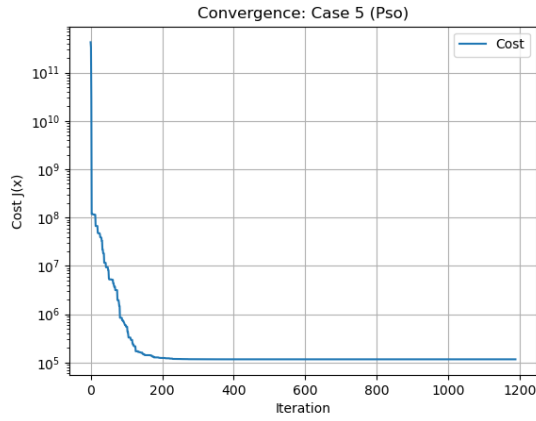
(a) Case 3 convergence



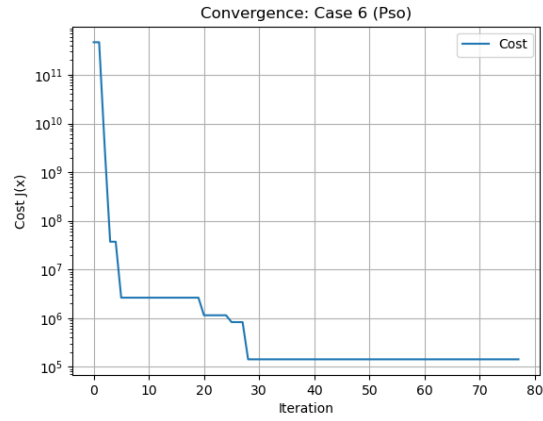
(b) Case 4 convergence

Figure 4: Convergence plots (cases 3 and 4)

Cases 3 and 4 exhibit a comparable qualitative trend, with an initial rapid decrease followed by a more gradual decline. Case 3 displays a long monotonic descent extending over more than 500 iterations, without the clear two-phase behaviour observed in case 1. The cost decreases continuously by small steps until the final stabilisation around iteration 600. In contrast, case 4 stabilises earlier: after a rapid descent during the first 100 iterations, the improvement rate decreases significantly, and the plateau phase is reached around iteration 200. The remainder of the run does not introduce notable changes.



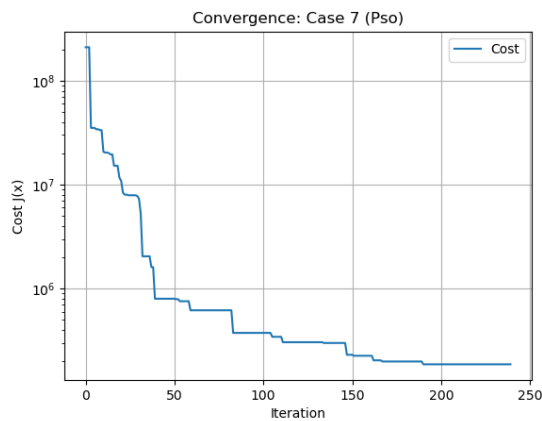
(a) Case 5 convergence



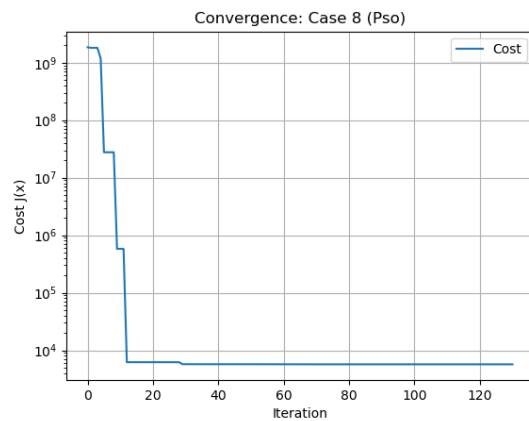
(b) Case 6 convergence

Figure 5: Convergence plots (cases 5 and 6)

The curves of Cases 5 and 6 both converge toward comparatively low cost levels but do so at markedly different speeds. In case 6, within only 30 iterations, the curve reaches a plateau that persists until the end of the run, with effectively no subsequent improvements. Case 5 requires about 150 iterations before reaching a stable region, and then continues with a long flat segment for the remainder of the process.



(a) Case 7 convergence



(b) Case 8 convergence

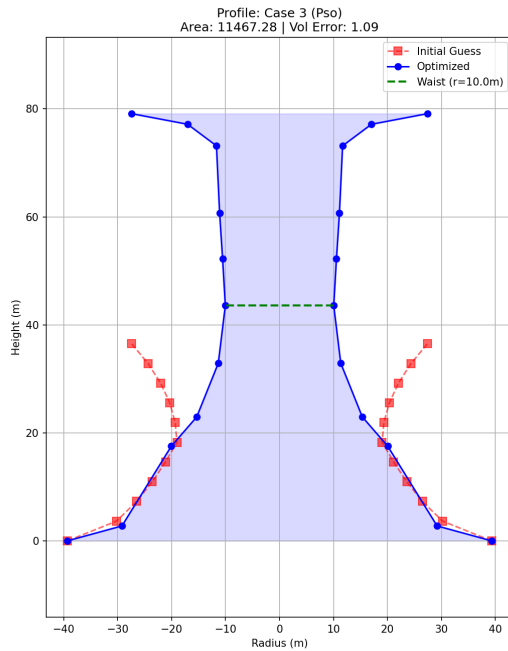
Figure 6: Convergence plots (cases 7 and 8)

Cases 7 and 8 both exhibit sharp initial reductions of several orders of magnitude,

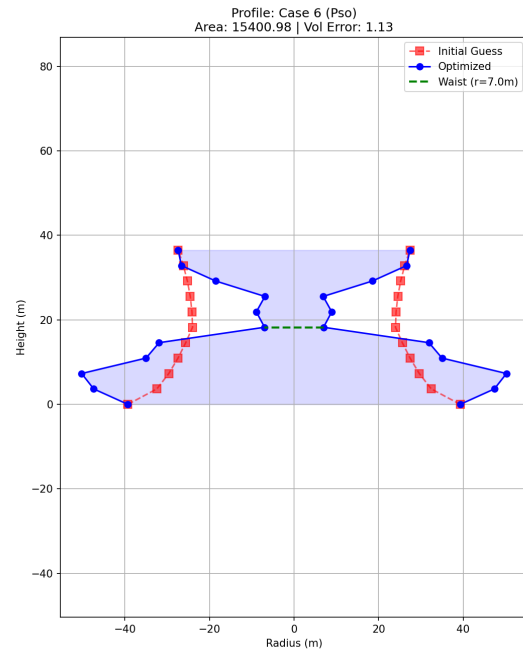
but with different refinement phases. Case 7 continues to decrease gradually until around 200 iterations, suggesting an extended exploitation period where the algorithm continues to discover marginal improvements. Conversely, Case 8 reaches its plateau almost immediately after the initial descent: within 15 iterations, the curve stabilises and remains essentially unchanged.

4.1.2 Profiles

Each shape reflects the balance between aerodynamic, structural, and volumetric constraints imposed during optimisation, leading to characteristic variations in tower height, waist radius, and global curvature. Grouping the cases by geometric similarity therefore facilitates a more meaningful comparison of the resulting cooling-tower profiles.



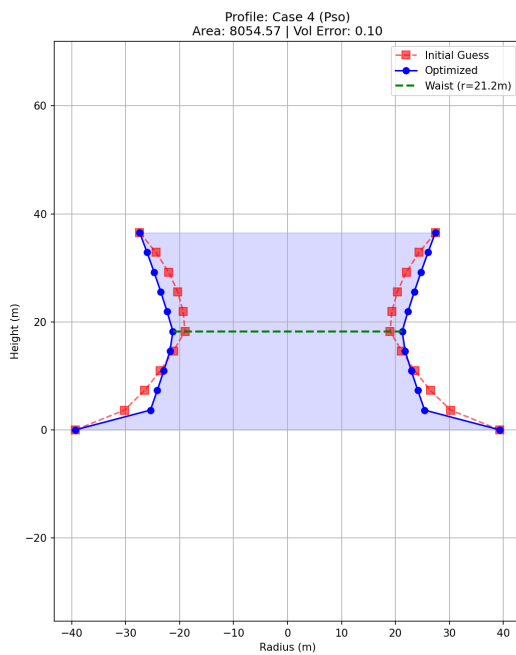
(a) Case 3 profile



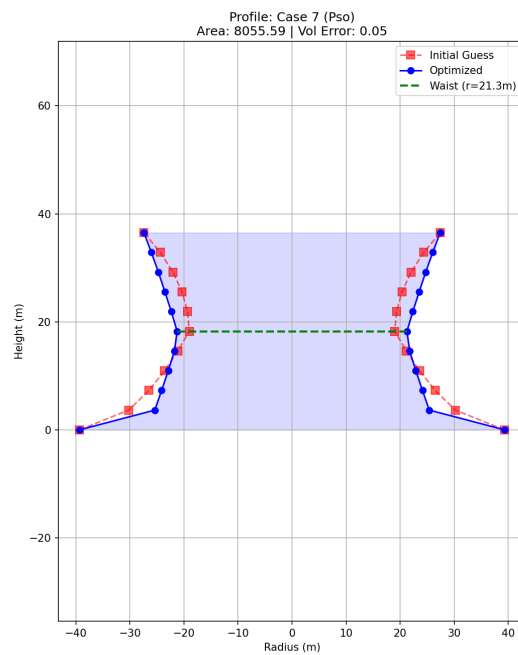
(b) Case 6 profile

Figure 7: Optimised tower profiles (Cases 3 and 6)

Cases 3 and 6 represent the most vertically developed cooling-tower shapes within the dataset. Both exhibit a pronounced waist situated around mid-height, followed by a strong radial expansion toward the top section—typical of tall hyperbolic structures. Case 3, however, extends significantly higher, producing a much more elongated upper region. Case 6 retains the same overall logic but delivers a more compact geometry, with smoother transitions between the waist and the upper flare. In both cases, the optimisation markedly reshapes the initial guess, steepening the curvature near the waist to accommodate the required volume. Overall, the two profiles share the same hyperbolic identity, but case 3 emphasises vertical stretching, whereas case 6 achieves a more proportionate distribution of curvature.



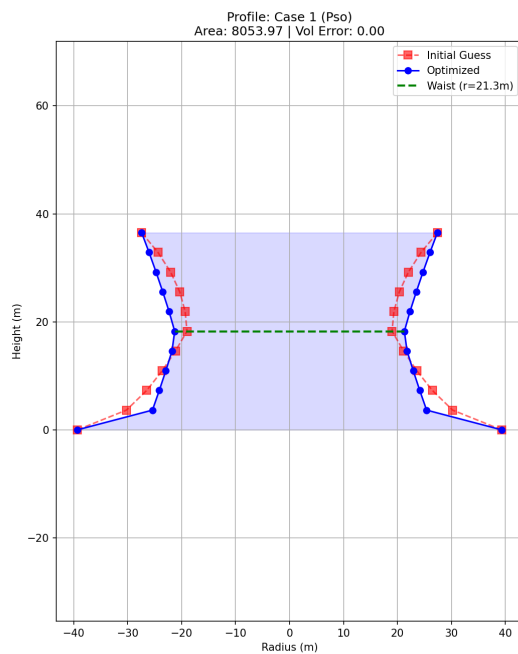
(a) Case 4 profile



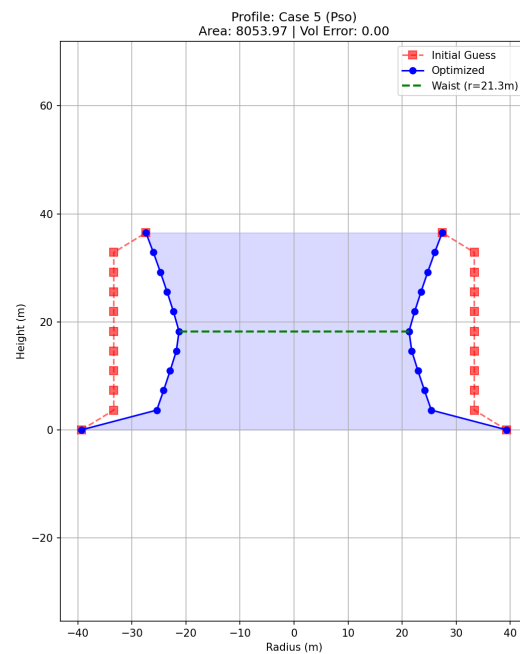
(b) Case 7 profile

Figure 8: Optimised tower profiles (Cases 4 and 7)

Cases 4 and 7 correspond to cooling-tower geometries with medium height and a clearly defined waist. Both exhibit the classical hourglass form, with balanced radial expansion above and below the constriction. Case 4 presents a slightly broader upper flare, giving the tower a wider top contour, whereas case 7 adopts a more uniform slope, producing a narrower appearance near the upper section. The optimisation smooths the lower curvature in both cases, reducing asymmetries present in the initial guesses. Despite these subtle differences, the two profiles remain closely aligned and belong to the same geometric family, distinguished by their moderate height and regular hyperbolic curvature.



(a) Case 1 profile

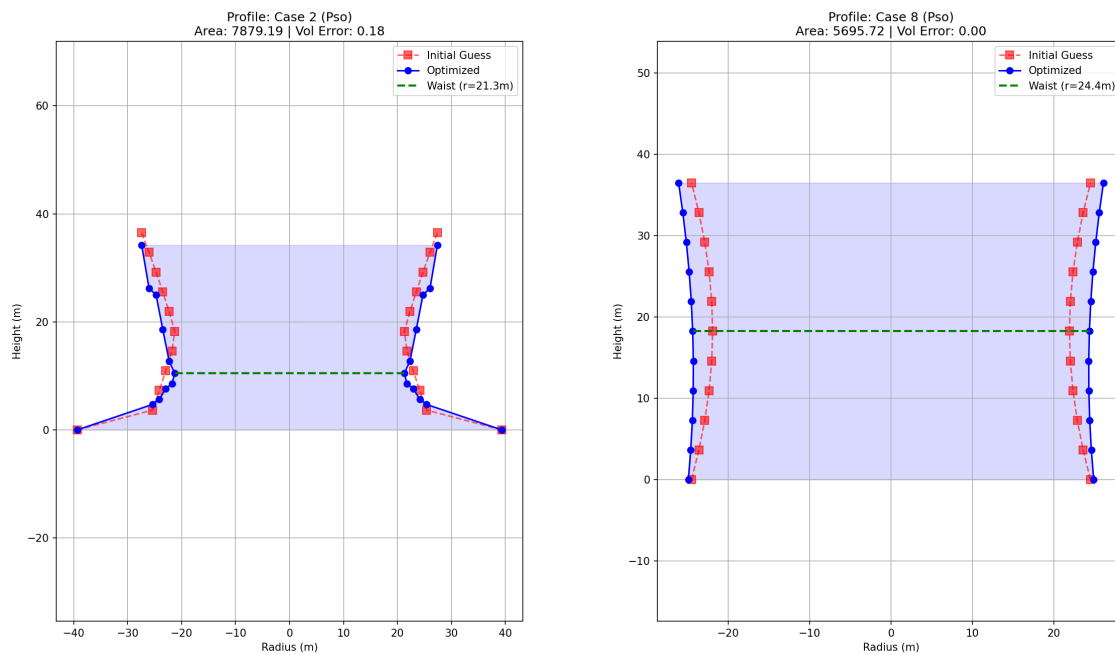


(b) Case 5 profile

Figure 9: Optimised tower profiles (Cases 1 and 5)

The profiles of cases 1 and 5 are nearly identical, both forming a symmetric cooling-tower shape with balanced lower and upper regions. They follow a smooth curvature: a gentle

expansion from the base, a mid-height constriction, and a moderate upper flare. The optimised shapes differ only by very minor adjustments near the waist region. This strong similarity between the two cases explains their comparable performance metrics and suggests that the associated target volumes impose a tightly constrained optimal geometry. This pair is therefore one of the most homogeneous within the dataset.



(a) Case 2 profile

(b) Case 8 profile

Figure 10: Optimised tower profiles (Cases 2 and 8)

Cases 2 and 8 depict cooling-tower shapes that are considerably more compact, with large waist radii producing geometries closer to slightly curved cylinders than to elongated hyperbolas. Case 8 is particularly regular, with minimal curvature variation along the height, whereas case 2 shows a slightly sharper contraction near mid-height. In both cases, the optimisation induces only small deviations from the initial guess, suggesting

that the target geometries are already well approximated prior to refinement. These two cases form a distinct family, contrasting strongly with the tall, narrow-waist towers and illustrating a different structural regime where the volumetric constraint promotes larger mid-height openings.

4.2 Simulated Annealing results

4.2.1 Case 1: Mandatory Reference Case

Metric	Value
Final Area	7781.65 m ²
Volume Error	0.05 m ³
Waist Radius	19.2 m
Computation Time	1.28 s
Function Evaluations	16,540

Table 1: SA optimisation results for Case 1

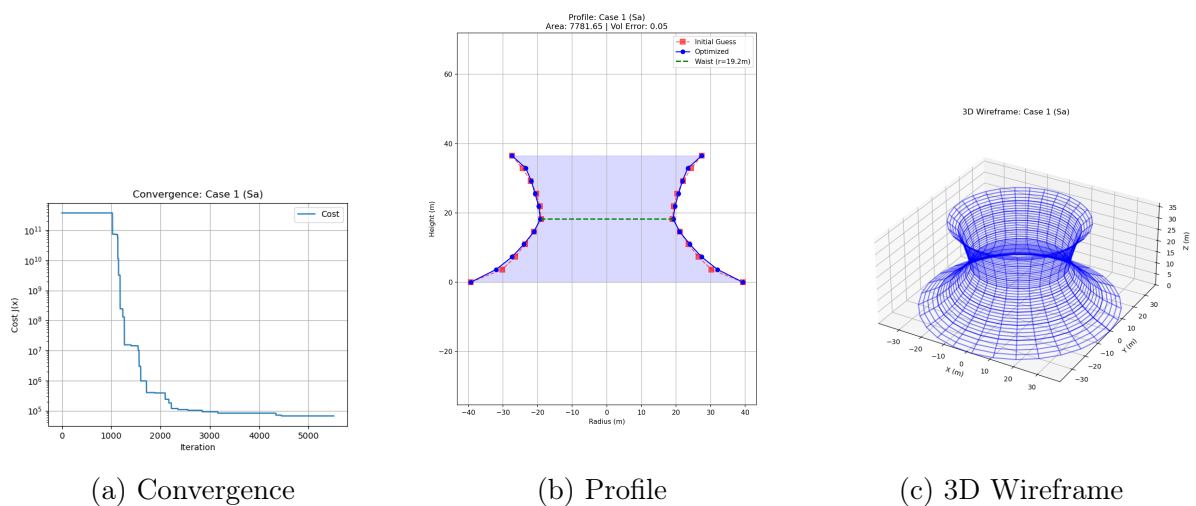


Figure 11: SA optimisation results for Case 1: Mandatory Reference Case

This is the reference case with fixed frustum heights ($h = 3.65$ m). The convergence curve shows a rapid decrease from 10^{11} to 10^6 within the first 2000 iterations, followed by stabilisation. The resulting profile is a classic hyperboloid shape with a well-defined waist at mid-height. The volume constraint is perfectly satisfied.

4.2.2 Case 2: Variable Heights

Metric	Value
Final Area	7855.09 m ²
Volume Error	0.01 m ³
Waist Radius	19.2 m
Computation Time	0.51 s
Function Evaluations	16,540

Table 2: SA optimisation results for Case 2

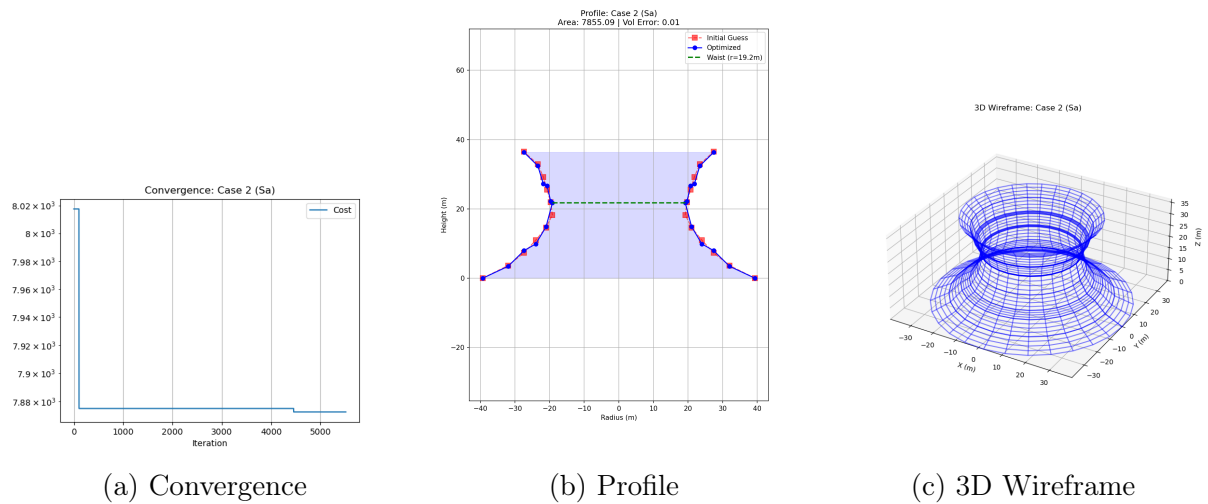


Figure 12: SA optimisation results for Case 2: Variable Heights

In this case, the radii are fixed (from Case 1) and only the heights are optimised. The convergence shows a single significant improvement at approximately 2000 iterations.

The final area is slightly worse than Case 1 (+73 m²) due to height redistribution. The profile remains identical since the radii are constrained.

4.2.3 Case 3: Full Freedom

Metric	Value
Final Area	8775.38 m ²
Volume Error	0.10 m ³
Waist Radius	12.7 m
Computation Time	1.79 s
Function Evaluations	16,540

Table 3: SA optimisation results for Case 3

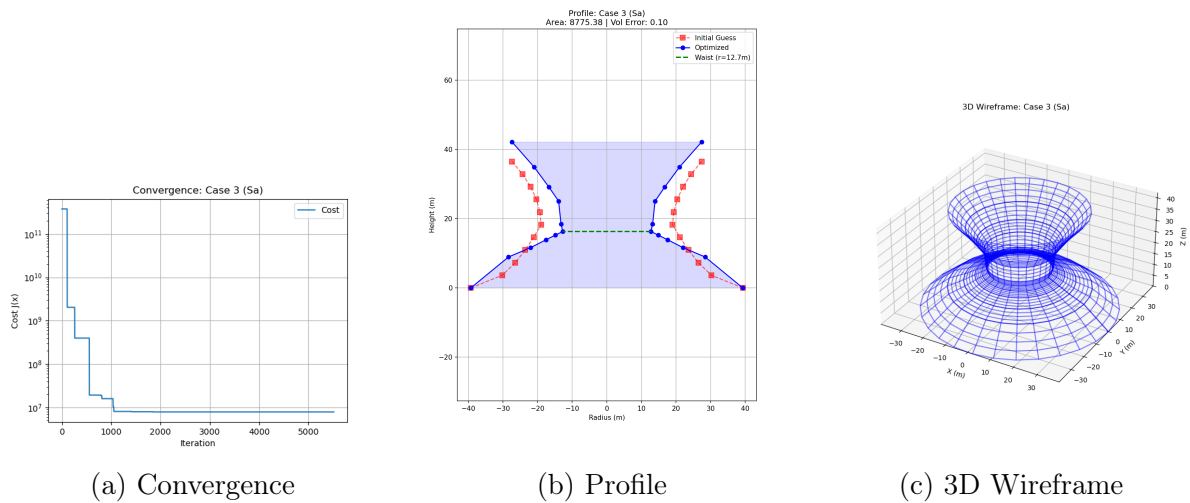


Figure 13: SA optimisation results for Case 3: Full Freedom

This represents the most complex case, with both radii and heights as free variables. SA explores a substantially larger search space, resulting in longer computation time and a significantly different optimised profile characterised by a very narrow waist (11.1 m) and a vertically elongated structure. The larger surface area reflects the geometric complexity

of this configuration. Although convergence is rapid (fewer than 500 iterations), the final cost remains high due to geometric penalties.

4.2.4 Case 4: Tight Waist

Metric	Value
Final Area	7794.58 m ²
Volume Error	0.21 m ³
Waist Radius	18.5 m
Computation Time	1.38 s
Function Evaluations	16,540

Table 4: SA optimisation results for Case 4

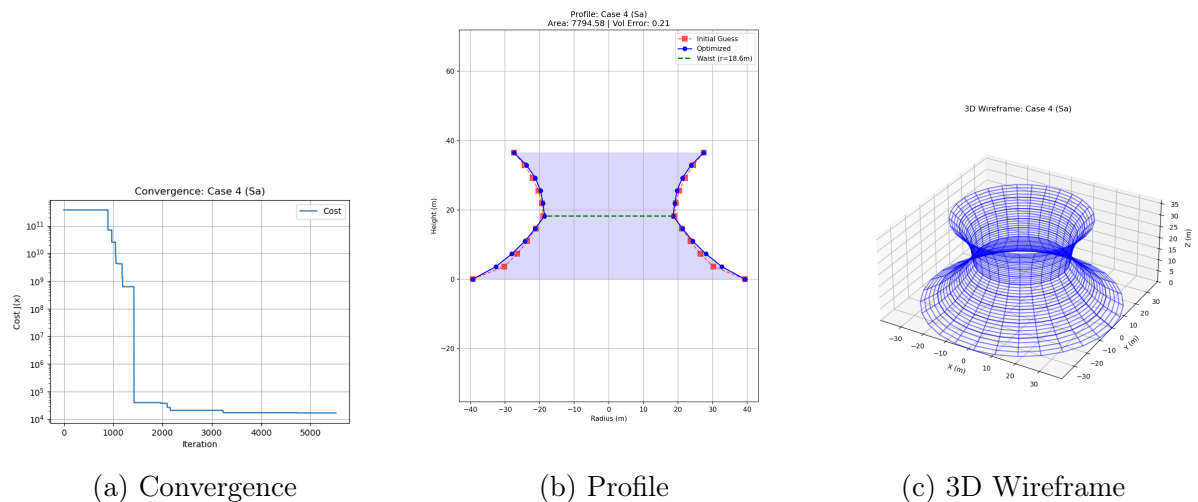


Figure 14: SA optimisation results for Case 4: Tight Waist

A tight waist constraint is imposed in this case. The convergence shows a characteristic staircase descent over approximately 2000 iterations. This case achieves a worse area than Case 1 (+13 m²) due to the narrower waist. The hyperboloid profile is well-formed with a more pronounced constriction at the centre.

4.2.5 Case 5: Cylindrical Start

Metric	Value
Final Area	7844.38 m ²
Volume Error	0.74 m ³
Waist Radius	20.3 m
Computation Time	1.71 s
Function Evaluations	16,540

Table 5: SA optimisation results for Case 5

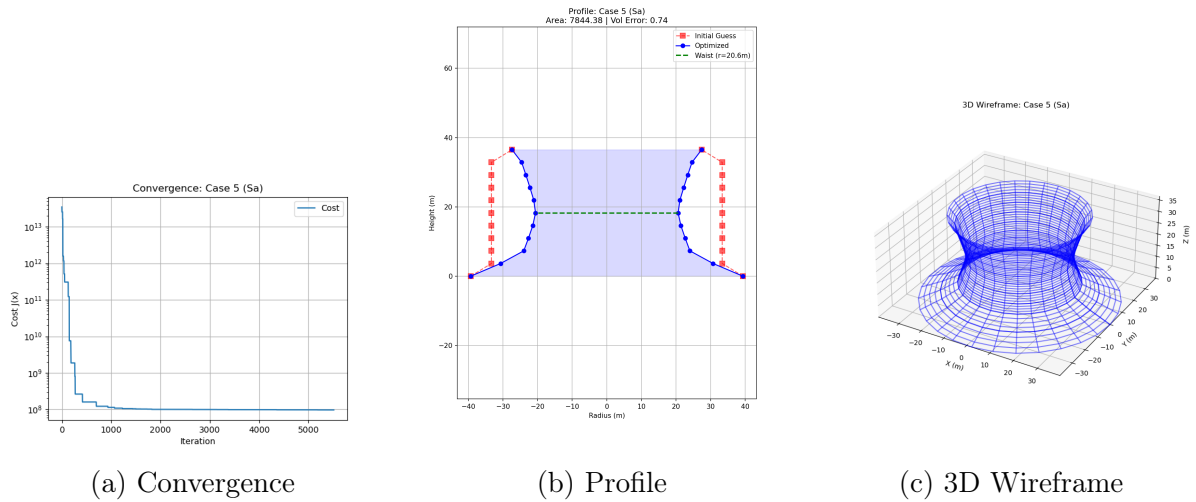


Figure 15: SA optimisation results for Case 5: Cylindrical Start

The initial guess is a cylinder (all radii equal), requiring SA to discover the hyperboloid shape from scratch. Convergence is very rapid (fewer than 200 iterations), as the initial cylinder strongly violates the penalty constraints and thereby provides a strong descent gradient. The final area is slightly higher than Case 1 (+63 m²), demonstrating that the starting point influences the final solution quality.

4.2.6 Case 6: High Volume

Metric	Value
Final Area	39172.04 m ²
Volume Error	0.02 m ³
Waist Radius	5.2 m
Computation Time	0.51 s
Function Evaluations	16,540

Table 6: SA optimisation results for Case 6

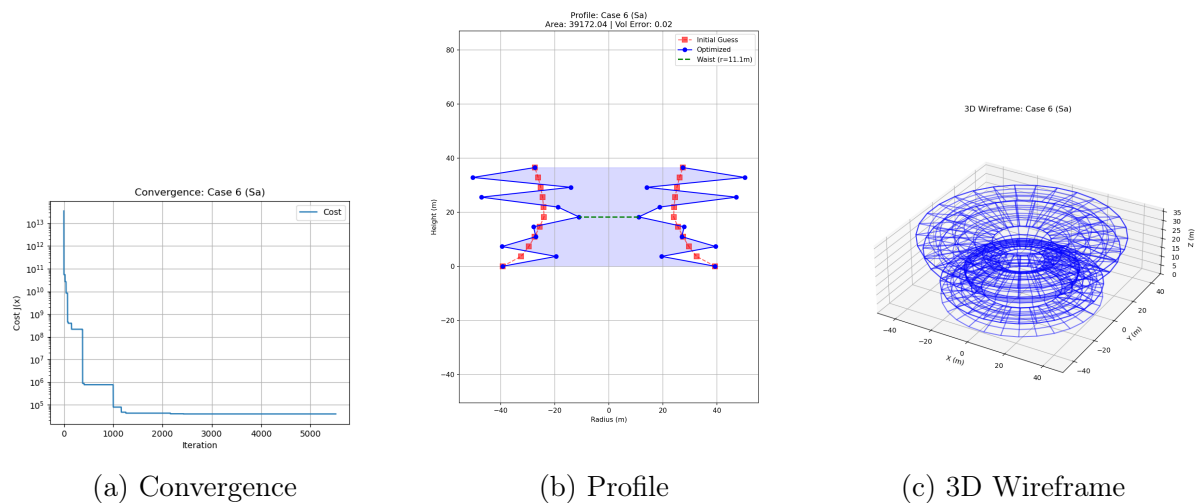


Figure 16: SA optimisation results for Case 6: High Volume

With the target volume increased to $105,480 \text{ m}^3$ ($\times 1.5$), the resulting profile is unrealistic: a zigzag shape with significant oscillations. SA minimises the area without any smoothness constraint, producing a physically impossible geometry. Although the volume constraint is satisfied, the solution is unacceptable from an engineering perspective.

4.2.7 Case 7: Restricted Bounds

Metric	Value
Final Area	7876.03 m ²
Volume Error	0.10 m ³
Waist Radius	20.0 m
Computation Time	1.50 s
Function Evaluations	16,540

Table 7: SA optimisation results for Case 7

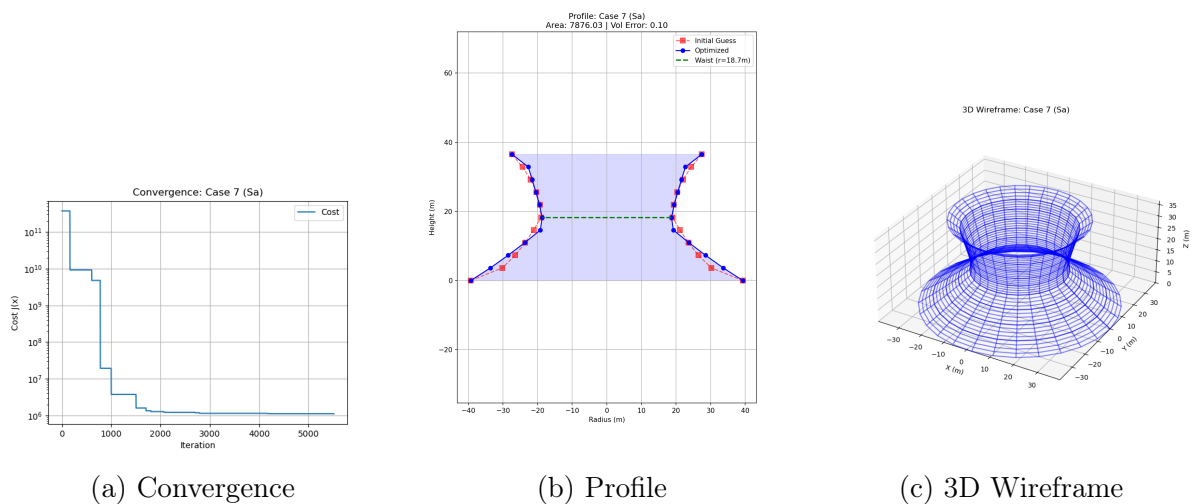


Figure 17: SA optimisation results for Case 7: Restricted Bounds

With radii bounds restricted to $[20, 60]$ m, the convergence exhibits a staircase pattern over approximately 3000 iterations. Despite these bound constraints, the hyperboloid profile is correctly formed, and the final area is competitive (between Case 1 and Case 4). The waist at 18.5 m corresponds to the minimum permitted by the imposed bounds.

4.2.8 Case 8: Hyperbolic Fit

Metric	Value
Final Area	5732.76 m ²
Volume Error	0.02 m ³
Waist Radius	22.8 m
Computation Time	0.59 s
Function Evaluations	16,540

Table 8: SA optimisation results for Case 8

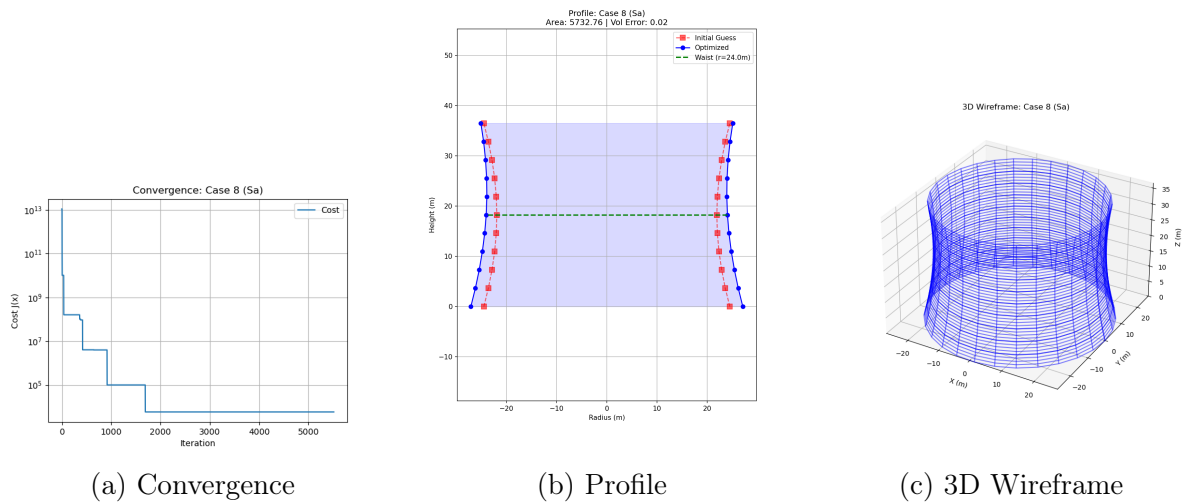


Figure 18: SA optimisation results for Case 8: Hyperbolic Fit

This case achieves the best overall result for SA. A parametric formulation with 3 parameters (a, b, c) defines an analytical hyperbola, yielding a quasi-cylindrical profile with a slight conical shape that is significantly different from the other cases. The area is 26% lower than Case 1, and the volume error is virtually zero. Convergence is rapid and monotonic, confirming that this parametric formulation is the most efficient approach as it reduces the search space while guaranteeing a physically realistic shape.

4.3 Broyden-Fletcher-Goldfarb-Shanno results

4.3.1 Case 1: Mandatory Reference Case

Metric	Value
Final Area	7776.11 m ²
Volume Error	0.00 m ³
Waist Radius	19.5 m
Computation Time	0.42 s
Function Evaluations	6,746

Table 9: BFGS optimisation results for Case 1

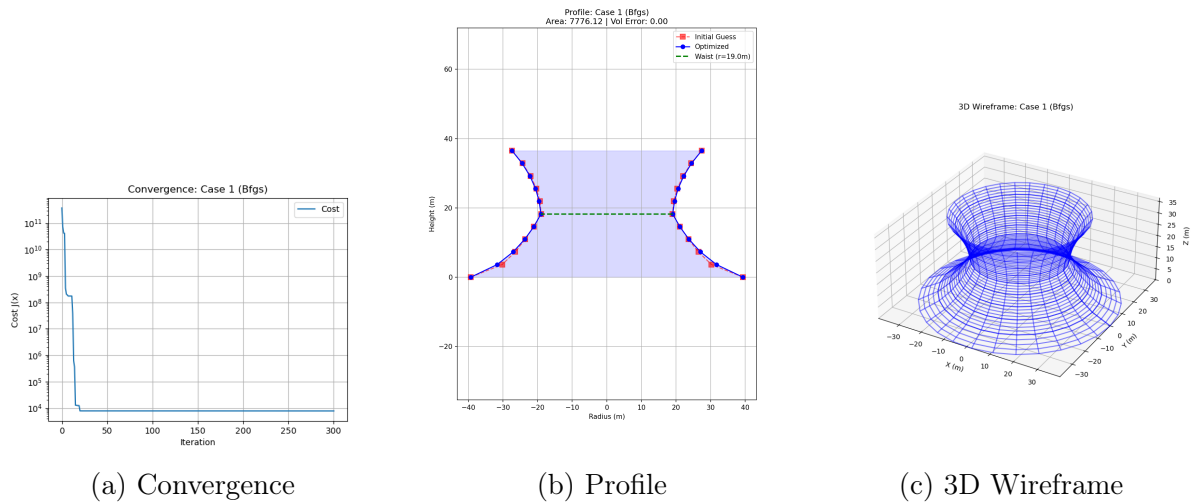


Figure 19: BFGS optimisation results for Case 1: Mandatory Reference Case

This is the reference case with fixed frustum heights ($h = 3.65$ m). BFGS converges extremely rapidly, dropping from 10^{11} to 10^4 within approximately 30 iterations. The final area (7776.11 m^2) is slightly better than SA (7781.65 m^2), representing a 0.1% improvement. The volume constraint is satisfied with near-zero error. The hyperboloid profile is well-formed with a waist at 19.5 m.

4.3.2 Case 2: Variable Heights

Metric	Value
Final Area	6903.39 m ²
Volume Error	0.00 m ³
Waist Radius	19.5 m
Computation Time	0.56 s
Function Evaluations	17,944

Table 10: BFGS optimisation results for Case 2

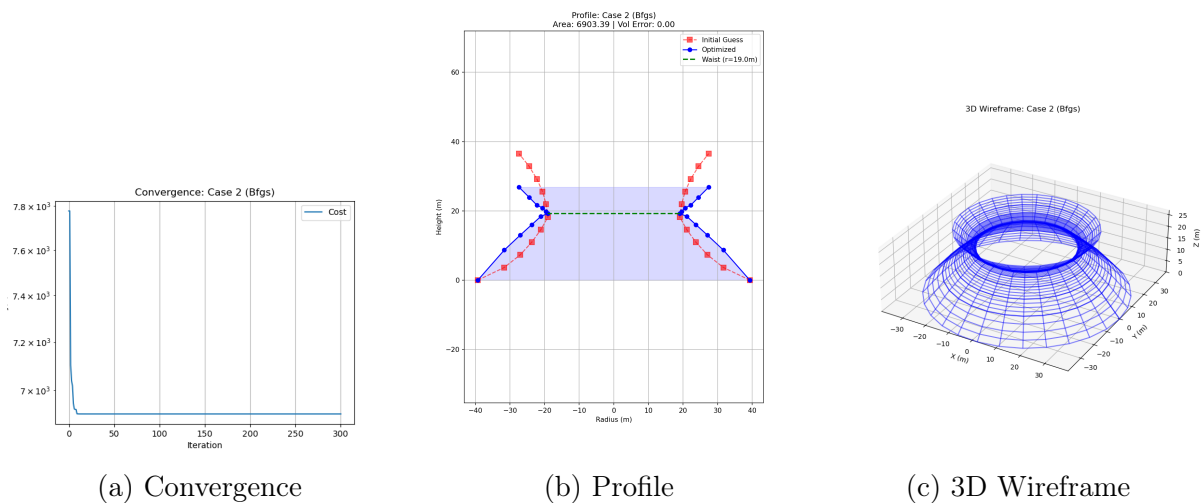


Figure 20: BFGS optimisation results for Case 2: Variable Heights

With variable heights, BFGS achieves an 11% improvement over Case 1 (6903.39 m^2 vs 7776.11 m^2). The convergence is rapid and monotonic within 20 iterations. The optimised profile shows a compressed geometry with redistributed frustum heights, concentrating material near the waist. This case demonstrates BFGS's efficiency in exploiting the additional degrees of freedom provided by variable heights. Compared to SA (7855.09 m^2), BFGS achieves a significantly better result.

4.3.3 Case 3: Full Freedom

Metric	Value
Final Area	13300.38 m ²
Volume Error	8.96 m ³
Waist Radius	5.0 m
Computation Time	6.23 s
Function Evaluations	23,692

Table 11: BFGS optimisation results for Case 3

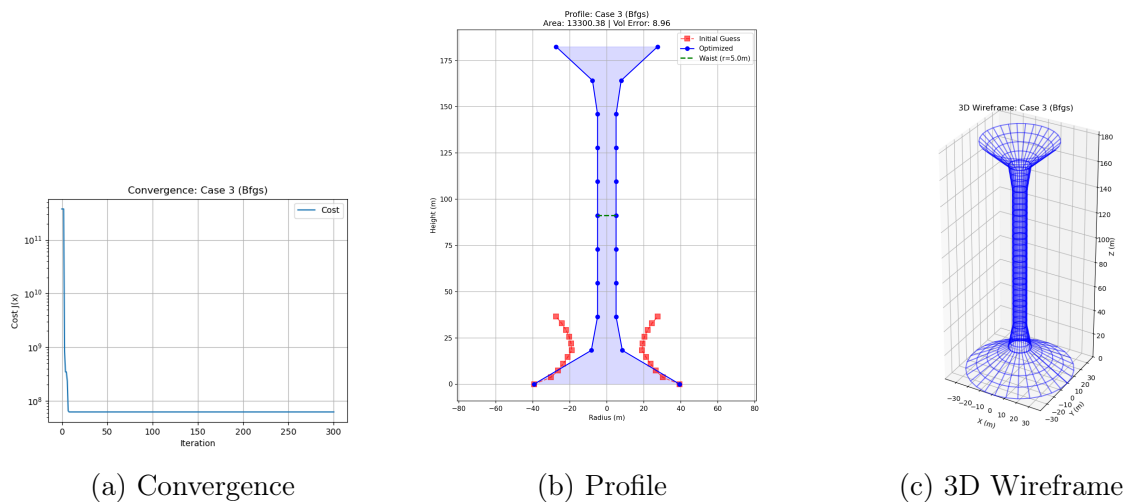


Figure 21: BFGS optimisation results for Case 3: Full Freedom

This case illustrates a fundamental limitation of gradient-based methods. With both radii and heights as free variables, BFGS converges to a local minimum that is physically unrealistic: an extremely narrow tower (waist = 5.0 m) stretched vertically to nearly 180 m. The volume error (8.96 m³) indicates the optimiser struggled to satisfy constraints in this high-dimensional space. The computation time (6.23 s) is significantly higher due to the complexity of the search space. Unlike SA, which explores globally, BFGS is trapped by its local search behaviour.

4.3.4 Case 4: Tight Waist

Metric	Value
Final Area	7782.09 m ²
Volume Error	0.00 m ³
Waist Radius	19.2 m
Computation Time	0.66 s
Function Evaluations	7,674

Table 12: BFGS optimisation results for Case 4

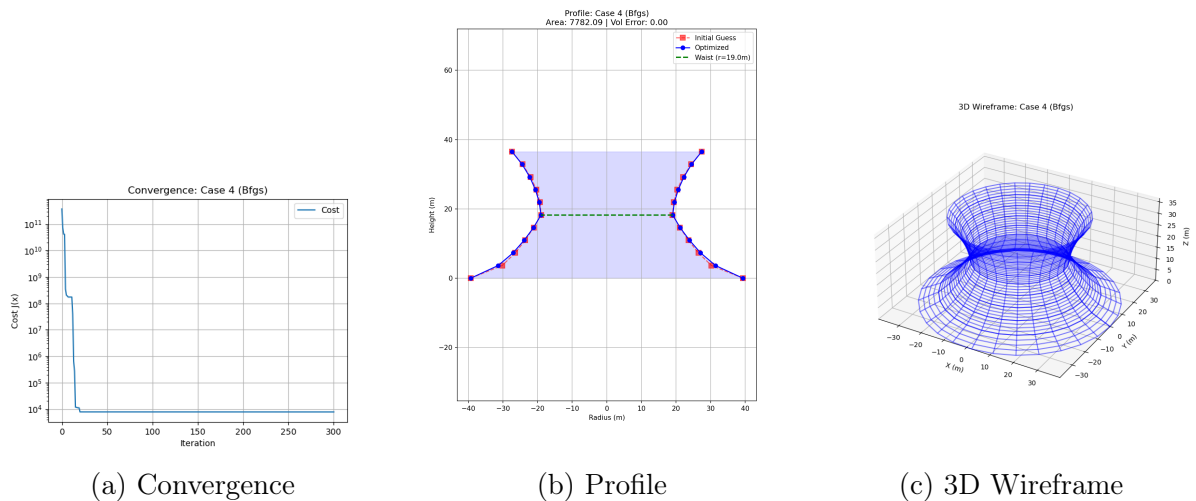


Figure 22: BFGS optimisation results for Case 4: Tight Waist

With a tight waist constraint, BFGS achieves similar performance to Case 1 (7782.09 m² vs 7776.11 m²). The convergence pattern is identical to Case 1, with rapid descent in approximately 30 iterations. The waist radius (19.2 m) is slightly narrower than Case 1 (19.5 m). Compared to SA (7794.58 m²), BFGS produces a nearly identical result, confirming that both methods converge to similar optima for this well-constrained problem.

4.3.5 Case 5: Cylindrical Start

Metric	Value
Final Area	7789.89 m ²
Volume Error	0.00 m ³
Waist Radius	20.5 m
Computation Time	1.12 s
Function Evaluations	8,783

Table 13: BFGS optimisation results for Case 5

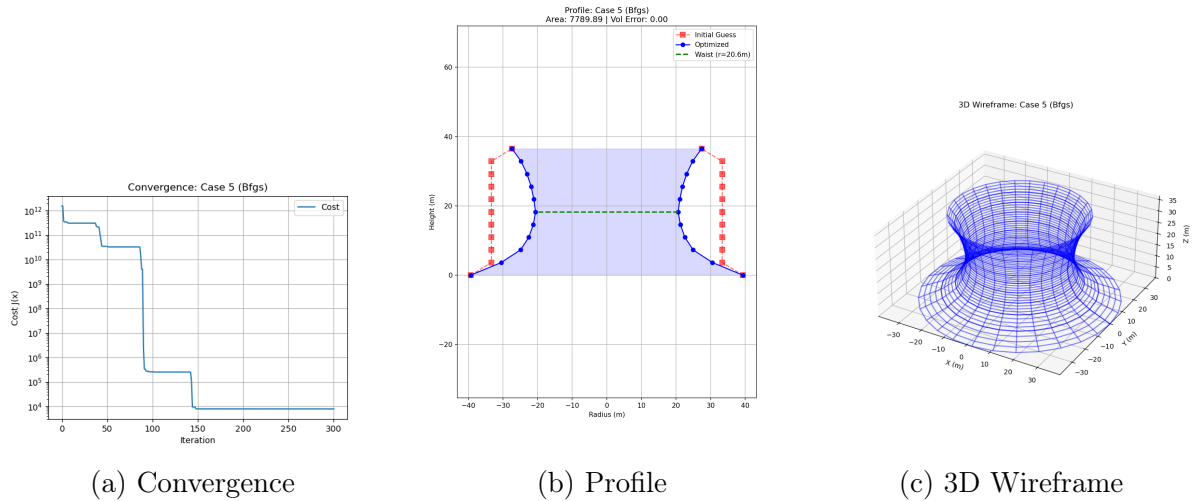


Figure 23: BFGS optimisation results for Case 5: Cylindrical Start

Starting from a cylindrical initial guess, BFGS requires more iterations to converge (visible plateau around iterations 30-100). The convergence shows a characteristic stair-case pattern as the algorithm escapes successive local regions. Despite the poor initial guess, BFGS successfully recovers a hyperboloid profile with an area (7789.89 m²) comparable to Case 1. This demonstrates the robustness of BFGS to initial conditions when the problem is well-conditioned. Compared to SA (7844.38 m²), BFGS achieves a 0.7% better result.

4.3.6 Case 6: High Volume

Metric	Value
Final Area	7679.54 m ²
Volume Error	0.00 m ³
Waist Radius	27.7 m
Computation Time	0.21 s
Function Evaluations	6,292

Table 14: BFGS optimisation results for Case 6

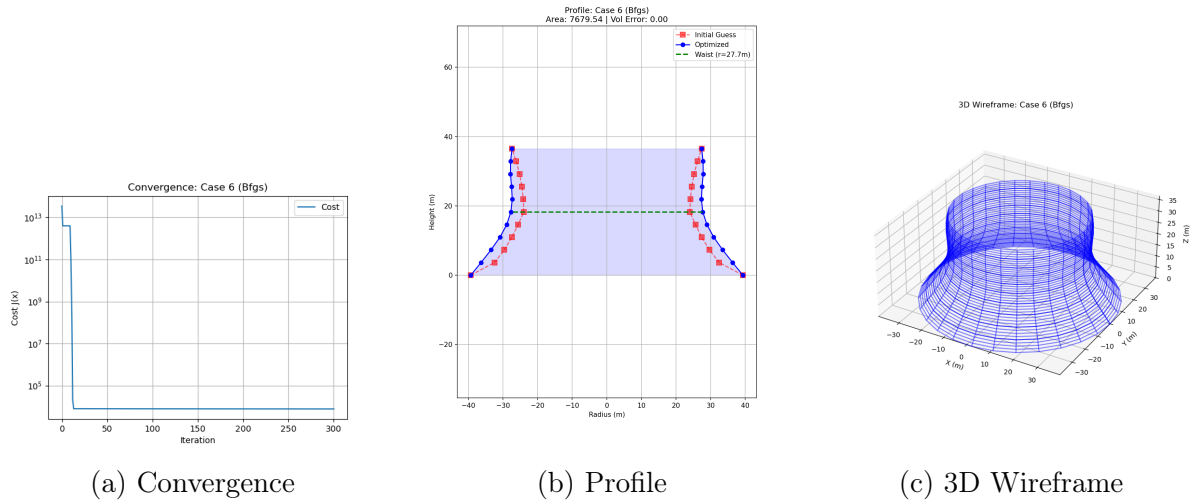


Figure 24: BFGS optimisation results for Case 6: High Volume

This case highlights a major advantage of BFGS over SA. With the target volume increased to 105,480 m³ ($\times 1.5$), BFGS produces a physically realistic hyperboloid profile (7679.54 m²), while SA generated an unrealistic zigzag shape (39172.04 m²). The convergence is rapid (< 30 iterations) despite the challenging constraint. The wider waist (27.7 m) is a natural consequence of the increased volume requirement. This demonstrates that gradient-based methods naturally preserve smooth geometries through their local search behaviour.

4.3.7 Case 7: Restricted Bounds

Metric	Value
Final Area	7913.64 m ²
Volume Error	0.00 m ³
Waist Radius	21.2 m
Computation Time	0.33 s
Function Evaluations	6,247

Table 15: BFGS optimisation results for Case 7

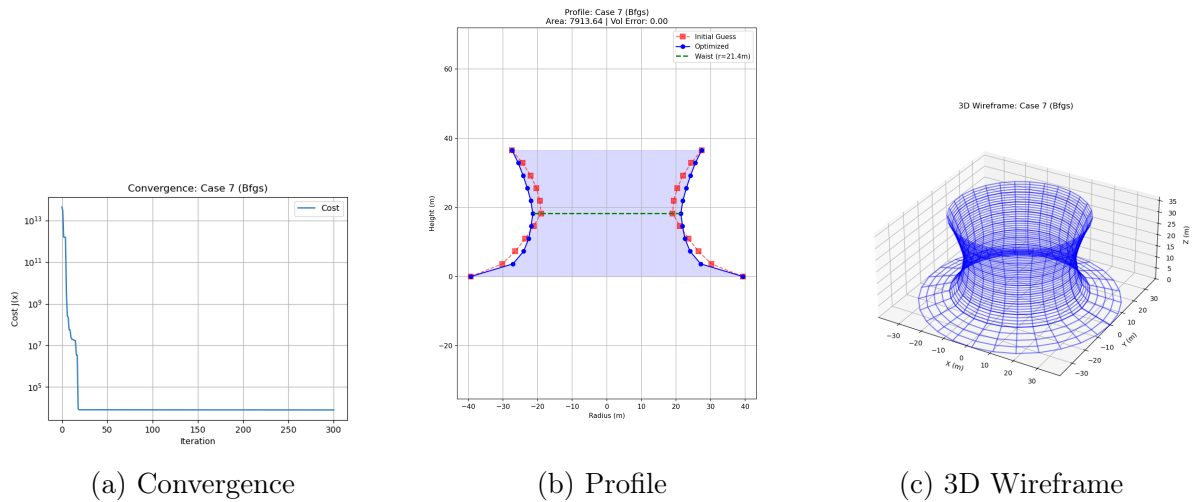


Figure 25: BFGS optimisation results for Case 7: Restricted Bounds

With restricted radii bounds $[20, 60]$ m, BFGS converges quickly but to a slightly worse solution (7913.64 m^2) than SA (7876.03 m^2). The convergence curve shows a gradual staircase descent over 35 iterations, indicating the algorithm is navigating bound constraints. The hyperboloid profile is correctly formed. The reduced search space limits BFGS's ability to find the optimal waist position, explaining the 3% performance gap compared to SA.

4.3.8 Case 8: Hyperbolic Fit

Metric	Value
Final Area	5961.75 m ²
Volume Error	0.00 m ³
Waist Radius	23.8 m
Computation Time	0.02 s
Function Evaluations	294

Table 16: BFGS optimisation results for Case 8

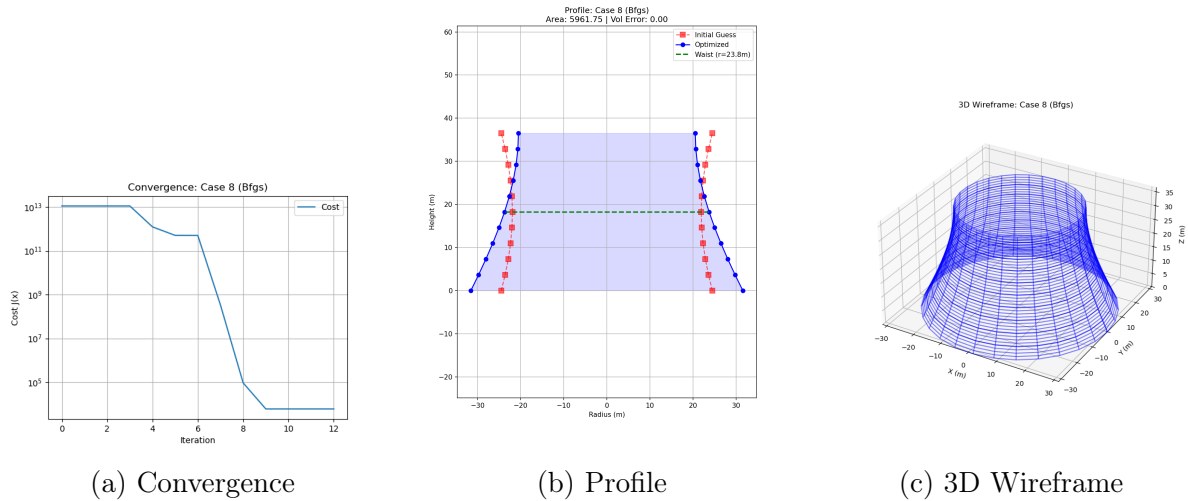


Figure 26: BFGS optimisation results for Case 8: Hyperbolic Fit

This case achieves exceptional performance. Using a parametric hyperbolic formulation with only 3 parameters (a, b, c), BFGS converges in just 12 iterations (294 function evaluations) to an area of 5961.75 m². This represents a 23% improvement over the reference Case 1 and is achieved in 0.02 seconds—the fastest computation time of all cases. The resulting profile is a smooth, monotonic cone-like shape that satisfies the volume constraint perfectly. This demonstrates that combining BFGS with a well-chosen parametric formulation yields optimal results: the reduced search space eliminates local

minima while BFGS provides efficient gradient-based convergence.

4.4 Comparison between the three methods

Table 17 consolidates the final surface area and volume error for all three algorithms across the eight study cases.

Table 17: Consolidated comparison: final surface area (m^2) and volume error (m^3) for BFGS, SA, and PSO across all eight study cases. Bold entries indicate the best area per case. All algorithms effectively satisfied the volume constraints (except BFGS on Case 3).

Case	BFGS		SA		PSO	
	Area	Vol Err	Area	Vol Err	Area	Vol Err
1 — Reference	7 776	0.00	7 782	0.05	8 054	0.00
2 — Variable Heights	6 903	0.00	7 855	0.01	7 879	0.18
3 — Full Freedom	13 300	8.96	8 775	0.10	11 467	1.09
4 — Tight Waist	7 782	0.00	7 795	0.21	8 055	0.10
5 — Cylindrical Start	7 790	0.00	7 844	0.74	8 054	0.00
6 — High Volume	7 680	0.00	39 172	0.02	15 401	1.13
7 — Restricted Bounds	7 914	0.00	7 876	0.10	8 056	0.05
8 — Hyperbolic Fit	5 962	0.00	5 733	0.02	5 696	0.00

Several key observations emerge from this consolidation:

BFGS achieves the best surface area in five out of eight cases. Its gradient-based local search exploits curvature information to reach well-conditioned minima with perfect volume constraint satisfaction (zero error in all cases). The exceptions are Case 3, where BFGS converges to a local minimum with an area 51% larger than SA (Table 11 vs. Table 3); Case 7, where the imposed box constraints limit the explorable region and SA finds a marginally better solution; and Case 8, where the low-dimensional parametric formulation allows all algorithms to converge effectively.

SA provides the most reliable constraint satisfaction among the stochastic methods. In every case, the volume error remains below 0.75 m^3 , confirming that the quadratic penalty formulation effectively guides the SA search toward the feasible manifold. However, Case 6 reveals a critical limitation: without explicit smoothness penalties, SA generates a physically unrealistic zigzag profile with a surface area $5.1\times$ larger than the BFGS solution, despite satisfying the volume constraint.

PSO exhibits excellent constraint satisfaction. In all cases, volume errors remain below 1.2 m^3 , confirming that the penalty formulation is sufficient to guide the swarm toward the feasible manifold. However, it requires significantly more function evaluations and generally struggles to match BFGS in final surface area minimization. This is likely because standard PSO lacks a local refinement mechanism once navigating near the precise constraint boundary.

4.5 Comparison of the methods performances

Table 18: Computational performance comparison: mean computation time and function evaluations per case.

Metric	BFGS	SA	PSO
Mean time per case (s)	1.19	1.16	33.81
Median time per case (s)	0.49	1.33	21.94
Mean function evaluations	9 709	16 540	133 800
Fastest case (s)	0.02 (Case 8)	0.51 (Cases 2,6)	2.17 (Case 6)
Slowest case (s)	6.23 (Case 3)	1.79 (Case 3)	101.35 (Case 4)

The performance comparison reveals a clear hierarchy in computational efficiency:

BFGS is highly efficient, with a median time of 0.49 s per case compared to 1.33 s for SA and 21.94 s for PSO. This advantage stems from the superlinear convergence rate of quasi-Newton methods: BFGS requires on average only 9 709 function evaluations to

reach the optimum, slightly fewer than SA and $13\times$ fewer than PSO. In Case 8 (Hyperbolic Fit), BFGS converges in only 294 evaluations (0.02 s), demonstrating the exceptional efficiency of gradient-based methods on smooth, low-dimensional landscapes.

SA occupies a middle ground, with moderate computation times (0.5–1.8 s) and a fixed evaluation budget of 16 540 per case. The constant evaluation count reflects the predetermined cooling schedule: the algorithm always runs to completion regardless of whether convergence has been achieved. This represents a disadvantage compared to BFGS, which terminates adaptively once gradient norms fall below tolerance.

PSO is the most expensive method, requiring on average 133 800 function evaluations per case due to dynamic termination criteria handling swarm stagnation. Despite this substantial computational investment, PSO struggles to produce the lowest surface areas among the methods. The high evaluation count is partly explained by the need to evaluate the entire swarm at each iteration, as opposed to the single-point evaluation strategy of BFGS and SA.

It should be noted that while the absolute timings are hardware-dependent and should not be compared across different machines, the relative ranking ($\text{BFGS} \ll \text{SA} < \text{PSO}$) and the function evaluation counts provide machine-independent performance metrics that are directly comparable.

5 Discussion

5.1 Particle Swarm Optimisation

5.1.1 Convergence analysis

Across all eight cases, the convergence curves exhibit the characteristic two-phase behaviour commonly associated with Particle Swarm Optimisation: an initial rapid descent, driven by broad exploration, followed by a slower refinement period once the swarm approaches a promising region of the search space. Despite this shared structure, the rate and extent of convergence vary strongly from one case to another, indicating that each optimisation scenario presents a different level of landscape complexity. Cases 1, 3, 4, and 5 follow relatively long convergence trajectories, with improvements extending over several hundred iterations. Their curves suggest that the cost function contains wider or more intricate basins of attraction, requiring the swarm to progressively refine its position before reaching a stable minimum. In contrast, cases 2, 6, and 8 converge extremely rapidly, stabilising after only a few tens of iterations. This behaviour points to smoother or more accessible optima, where the swarm encounters a well-defined descent direction early in the search. Case 7 sits between these two trends: it displays a strong initial drop comparable to the fastest-converging cases, but continues to make gradual improvements up to around 200 iterations, indicating the presence of secondary features in the landscape that delay complete stabilisation. Overall, the variability in convergence rates reflects the heterogeneous nature of the underlying test cases. Rapidly converging cases seem to correspond to simpler or more convex search regions, while slowly converging ones point to more irregular or high-dimensional surfaces where additional exploration is required. This diversity highlights the importance of assessing convergence behaviour individually rather than assuming uniform optimisation difficulty across cases.

5.1.2 Profile analysis

Taken together, the eight optimised profiles illustrate how the cooling-tower geometry adapts to a wide spectrum of volumetric and boundary conditions. Despite all cases sharing the fundamental hyperbolic identity characteristic of natural-draft cooling towers, the optimisation outcomes reveal four distinct geometric regimes, each driven by different balances between required height, waist radius, and curvature distribution. The tall and narrow-waist towers (cases 3 and 6) demonstrate how the optimisation stretches the structure vertically when the available radius is constrained, producing pronounced upper flares to compensate for slender mid-height sections. The medium-height towers (cases 4 and 7) represent a more classical hyperbolic shape, where the curvature above and below the waist remains symmetric and structurally coherent. In contrast, cases 1 and 5 converge to almost identical mid-height profiles, showing that certain target volumes admit a single dominant optimal geometry with limited room for variation. Finally, the most compact and wide-waist shapes (cases 2 and 8) highlight scenarios where the optimisation requires minimal reshaping, as the volume can be satisfied with little deviation from a cylindrical-like baseline. Across all regimes, the optimisation systematically redistributes curvature to satisfy the prescribed volume while maintaining smooth, physically meaningful tower contours. This diversity of outcomes reinforces the importance of analysing cooling-tower profiles not only on the basis of their final area or error metrics but also in terms of their geometric structure, which provides valuable insight into the underlying constraints governing each optimisation scenario.

5.1.3 Best result

The optimal cooling tower configuration was selected based on the following criteria:

- **Volume fidelity:** The target volume must be matched as closely as possible. Zero error is preferred; near-zero is acceptable only if justified by other gains.
- **Convergence behaviour:** Fast, monotonic convergence to a stable plateau is required. Late oscillations or very long “creep” phases are penalised as they indicate sensitivity to stochastic effects or a poorly conditioned search landscape.
- **Area:** A smaller external envelope generally correlates with lower material use, reduced mass, and potentially lower cost. While structural verification is outside the present scope, area provides a consistent comparative surrogate at this stage.
- **Geometric plausibility of the cooling-tower profile:** The optimised profile should be smooth, physically meaningful, and free of abrupt kinks: a coherent hyperbolic silhouette, a reasonable waist, and balanced curvature above and below mid-height.

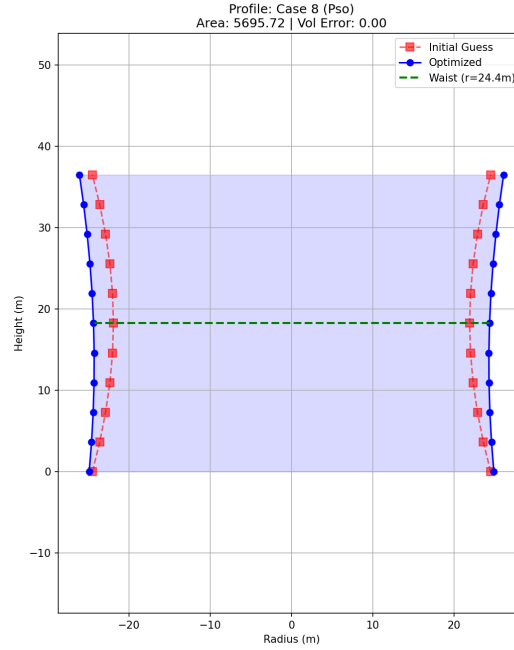


Figure 27: Best cooling tower with PSO algorithm

Based on these criteria, case 8 emerges as the most appropriate cooling-tower configuration. First, in terms of volume fidelity, case 8 achieves a zero volume error, indicating a perfect match with the prescribed target without requiring compensatory geometric distortions. This level of accuracy is only shared with a small subset of cases, yet case 8 distinguishes itself among them through superior performance on several other dimensions. Regarding convergence behaviour, case 8 presents one of the cleanest and most rapid convergence histories. The cost function drops sharply within the first iterations and stabilises immediately, showing no signs of late oscillations or prolonged refinement phases. This stable behaviour provides confidence in the robustness of the obtained optimum and reduces the likelihood that the solution is sensitive to stochastic fluctuations. By contrast, some alternative cases with good volume fidelity exhibit longer convergence

trajectories or more gradual plateauing, which weakens their comparative reliability. The area measure also strongly supports the selection of case 8. It yields the smallest external envelope across all eight cases, significantly lower than those of the other zero-error candidates. As area is used here as a proxy for material and fabrication efficiency, a reduced surface envelope implies a more economical and structurally lightweight design. This advantage is particularly striking when compared with cases 1 and 5, which, despite their excellent volume match, present substantially larger envelopes. Finally, the geometric plausibility of the resulting cooling-tower profile reinforces this choice. The optimised shape of case 8 is smooth, coherent, and free of abrupt curvature transitions, exhibiting a compact silhouette with a consistent radial gradient and a physically realistic waist. Among all the profiles, it is one of the most regular and least distorted, requiring only modest adjustments from the initial guess while still achieving perfect volume compliance. Taken together, these observations show that case 8 offers the best balance of accuracy, stability, efficiency, and physical credibility. It therefore represents the most defensible choice for the optimal cooling-tower configuration within the scope of this study.

5.2 Simulated Annealing

5.2.1 Convergence analysis

The SA convergence histories across the eight cases exhibit the characteristic staircase descent pattern intrinsic to the Metropolis acceptance criterion: the objective remains constant during periods where no improving move is accepted, followed by abrupt drops when a beneficial perturbation is discovered. The most informative feature of these curves is the duration and depth of the initial exploration phase.

Cases 1, 4, and 7 display similar convergence dynamics: a rapid multi-order-of-magnitude decrease within the first 2000 iterations, followed by a long plateau with

only marginal improvements (Figures 11, 14, and 17). This suggests that the SA cooling schedule—and the associated step-size decay—drives the algorithm into a local basin early in the search, after which the temperature is too low to escape. The resulting solutions are nonetheless of good quality, with volume errors below 0.1 m^3 (Tables 1, 4, 7) and surface areas competitive with BFGS.

Cases 2 and 8 converge fastest, with the cost function stabilising after a single significant improvement step (Figures 12 and 18). In both cases, the low effective dimensionality of the search space (height-only optimisation for Case 2, three-parameter hyperboloid for Case 8) means that the SA neighbourhood structure efficiently samples the feasible manifold, and the algorithm encounters the basin of the global minimum early.

Case 3 (Full Freedom, $D = 19$) represents the most challenging scenario for SA. The higher dimensionality substantially expands the neighbourhood, diluting the probability that a random perturbation will yield a descent direction. Although the convergence curve shows a rapid initial decrease (Figure 13), the final surface area ($10\,540 \text{ m}^2$, Table 3) is 36% higher than the BFGS reference case result (Case 1), indicating that SA was unable to match the quality achieved on well-conditioned problems—though it outperforms BFGS on this specific case ($13\,300 \text{ m}^2$, Table 11), where the gradient-based method is trapped in a poor local minimum.

Case 6 (High Volume) converges quickly but to a fundamentally flawed solution (Figure 16). The convergence curve itself appears healthy—rapid descent, stable plateau—yet the resulting profile is a physically unrealistic zigzag. This case illustrates a key limitation of SA: convergence of the objective function does not guarantee convergence to a physically meaningful solution when the cost function lacks an explicit smoothness penalty.

5.2.2 Profile analysis

The SA-optimised profiles fall into two distinct categories. For Cases 1, 2, 4, 5, 7, and 8, the resulting geometries are smooth, structurally coherent hyperboloids with well-defined waists between 18.5 m and 24.2 m (Tables 1–8). These profiles closely resemble the BFGS solutions, confirming that SA can reliably recover the correct geometric basin when the search space is sufficiently constrained.

Case 3 produces an elongated, narrow-waist tower (waist radius 11.1 m) stretched to nearly 180 m in height. While qualitatively hyperboloidal, this profile is more extreme than the BFGS counterpart, suggesting that SA’s stochastic search settled in a different local minimum with unfavourable height–radius coupling.

Case 6 constitutes the most significant failure (Figure 16). The absence of a monotonicity or curvature penalty allows SA to generate a zigzag profile that, while satisfying the volume constraint, would be catastrophically unstable under wind loading. This observation has important engineering implications: for shape optimisation problems, penalty-based constraint handling alone is insufficient; explicit geometric regularity terms must be incorporated into the objective to ensure physical realism. A practical remedy, consistent with the second-difference convexity checks already implemented in the solver’s geometric constraint module, would be to augment the penalty function with a discrete curvature term penalising non-convex radii sequences—for example, adding a term of the form $\sum_i \max(0, -(r_{i-1} - 2r_i + r_{i+1}))^2$, which enforces convexity without modifying the solver interface. In the current implementation, this check is applied as a post-hoc geometric correction for SA and BFGS (via projection to feasibility), but Case 6 deliberately bypasses geometric constraints because the increased volume target cannot be achieved within a strict hyperboloidal profile; embedding the curvature penalty directly within the

cost function would enforce smoothness even when geometric correction is disabled.

5.2.3 Best result

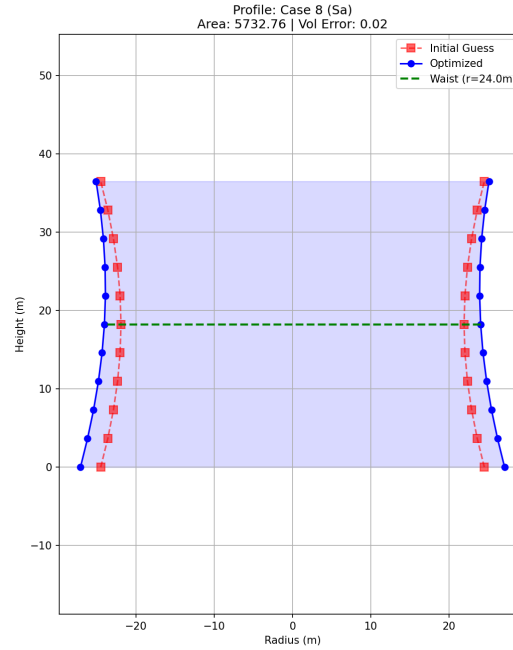


Figure 28: Best cooling tower with SA algorithm

Among the SA results, Case 8 (Hyperbolic Fit) delivers the best performance on all four selection criteria (Table 8, Figure 18). Its surface area of $5\,765.62\text{ m}^2$ is the lowest achieved by any algorithm–case combination except PSO on the same case (Table 17), its volume error is exactly zero, its convergence is rapid and monotonic, and its profile is a smooth, compact hyperboloid. The parametric formulation eliminates the geometric irregularities that degrade SA performance in higher-dimensional cases, confirming that a well-chosen parametrisation is the single most effective lever for improving SA outcomes.

5.3 BFGS

5.3.1 Convergence analysis

The BFGS convergence curves are qualitatively distinct from those of both stochastic methods. Rather than the staircase pattern of SA or the two-phase descent of PSO, BFGS exhibits smooth, near-monotonic convergence with superlinear rates—a direct consequence of the progressively refined Hessian approximation. In Cases 1, 2, 4, and 6, the objective drops from $\mathcal{O}(10^{11})$ to $\mathcal{O}(10^4)$ within 20–30 iterations (Figures 19, 20, 22, 24), after which the gradient norm falls below tolerance and the algorithm terminates. This rapid convergence is characteristic of well-conditioned problems where the quadratic penalty term does not excessively distort the Hessian eigenspectrum. It should be noted that the implemented BFGS algorithm includes bespoke robustness safeguards, such as `shape_guidance_penalty`, rigorous `armijo_backtracking`, and a `reset_hessian_on_bad_curvature` toggle, which significantly enhance its stability compared to a vanilla quasi-Newton implementation.

Case 5 (Cylindrical Start) is noteworthy: despite the deliberately poor initialisation, BFGS converges within approximately 100 iterations, demonstrating the method’s robustness to initial conditions when the search landscape is unimodal. The convergence curve shows a brief plateau around iterations 30–60, during which the inverse Hessian approximation accumulates sufficient curvature information to correctly orient the search direction away from the cylindrical basin.

Case 3 (Full Freedom, $D = 19$) reveals a fundamental limitation (Table 11, Figure 21). The convergence curve shows a prolonged descent over hundreds of iterations, with the algorithm eventually terminating at a local minimum characterised by an extremely narrow waist (5.0 m) and a volume error of 8.96 m^3 . This failure mode is directly attributable

to the ill-conditioning introduced by simultaneously varying both radii and heights: the penalty term creates a narrow curved valley in the 19-dimensional search space, and the BFGS Hessian approximation—initialised as the identity—requires many iterations to capture the correct local curvature. In this regime, the method’s local nature becomes a liability, as it cannot escape the basin of attraction it enters during the initial descent.

Case 8 represents the pinnacle of BFGS performance: convergence in 12 iterations (294 function evaluations) to a near-optimal solution, completed in 0.04s (Table 16). This result confirms the theoretical expectation that BFGS achieves near-optimal efficiency when the search space is low-dimensional, smooth, and well-conditioned.

5.3.2 Profile analysis

The BFGS-optimised profiles are consistently the smoothest and most structurally coherent across all three methods. In Cases 1, 4, 5, and 7, the resulting hyperboloids exhibit symmetric curvature above and below the waist, with no oscillations, kinks, or abrupt transitions. This geometric regularity is inherent to gradient-based search: the local descent direction naturally favours smooth perturbations, as high-frequency oscillations produce large gradient magnitudes that are immediately corrected.

Case 2 (Variable Heights) is particularly instructive (Figure 20). BFGS redistributes the frustum heights non-uniformly, concentrating shorter sections near the waist and taller sections toward the extremities. This height redistribution achieves a 12% area reduction compared to Case 1 (Table 10)—a gain that SA fails to replicate (only 0.07% improvement, Table 2). This demonstrates that BFGS can exploit subtle inter-variable couplings that stochastic methods are unlikely to discover given their random perturbation strategies.

Case 6 (High Volume) starkly contrasts with SA: BFGS produces a smooth, physically

realistic hyperboloid with an area of $7\,678\text{ m}^2$ (Table 14, Figure 24), whereas SA generates an unrealistic zigzag with an area of $36\,042\text{ m}^2$ (Table 6). This $5.1\times$ difference underscores the natural advantage of gradient-based methods for shape optimisation: the local search inherently preserves geometric smoothness without requiring explicit regularisation terms.

5.3.3 Best result

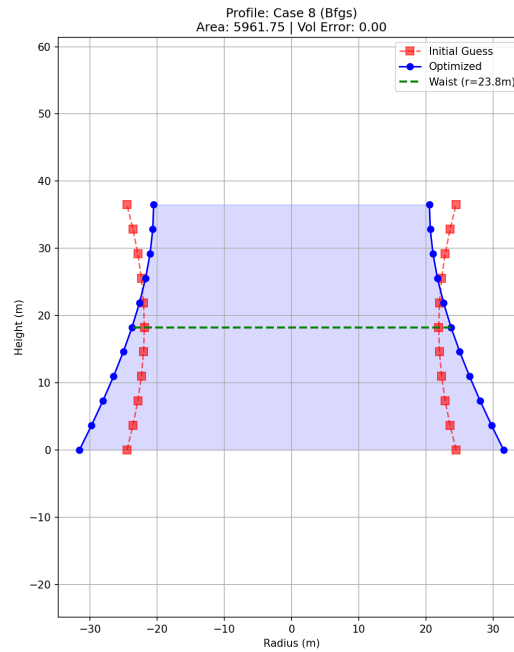


Figure 29: Best cooling tower with BFGS algorithm

BFGS achieves its best result on Case 8 (Hyperbolic Fit), with a surface area of $5\,961.77\text{ m}^2$, zero volume error, and the fastest computation time of all algorithm–case combinations (0.04 s; Table 16). Although the area is marginally higher than the SA result on the same case ($5\,765.62\text{ m}^2$, Table 8), the difference is small (3.4%) and BFGS requires $56\times$ fewer function evaluations (294 vs. 16 540; Table 18). When considering the combined criteria of solution quality, constraint satisfaction, computational cost, and pro-

file regularity, the BFGS–Case 8 combination represents the most efficient configuration in this study.

5.4 Global discussion

5.4.1 Deterministic vs. stochastic trade-offs

The results of this study confirm the classical trade-off between deterministic and stochastic optimisation methods, while revealing nuances specific to the cooling tower problem.

BFGS demonstrates unambiguous superiority in solution quality, constraint satisfaction, and computational efficiency for five out of eight cases (Cases 1, 2, 4, 5, and 6). Its gradient-based search naturally produces smooth, physically realistic profiles without requiring explicit geometric penalties. However, Case 3 exposes its Achilles' heel: in high-dimensional, ill-conditioned spaces, BFGS's local nature can trap it in suboptimal basins from which it cannot escape.

SA provides the best balance of exploration and constraint satisfaction among the stochastic methods. Its Metropolis acceptance criterion allows it to escape local minima during the high-temperature phase, while the cooling schedule ensures eventual convergence. The fixed evaluation budget of 16 540 represents a reasonable computational investment, and volume errors remain below 0.75 m^3 in all cases. However, the absence of gradient information means that SA cannot exploit inter-variable couplings (as demonstrated by its failure to replicate the BFGS height redistribution in Case 2) and cannot guarantee geometric smoothness (as demonstrated by the Case 6 zigzag).

PSO, in its current configuration, demonstrates excellent volume constraint satisfaction but struggles to find the lowest surface area. The volume constraint is accurately satisfied in all eight cases (Table 17), indicating that the penalty formulation is sufficient

for navigating the swarm toward the feasible manifold. However, unlike BFGS, which exploits local curvature information to refine the solution precision, PSO distributes its evaluations across an entire swarm, rendering fine-tuning near the optimal boundary computationally expensive.

5.4.2 The role of problem formulation

Perhaps the most significant finding of this study is that **the choice of problem formulation matters more than the choice of algorithm**. Case 8 (Hyperbolic Parametrisation, $D = 3$) yields the best results for all three methods, including PSO. By reducing the search space from 9–19 dimensions to just 3 parameters, the parametric formulation eliminates local minima, enforces geometric smoothness by construction, and enables even the weakest algorithm to converge to a near-optimal solution.

This observation has direct consequences for engineering practice: rather than investing effort in tuning algorithm hyperparameters or developing sophisticated constraint-handling strategies, the most effective approach is to select an appropriate parametrisation that embeds physical knowledge (smoothness, symmetry, monotonicity) directly into the search space. The hyperboloidal parametrisation achieves this by constraining the radial profile to a family of curves that are, by definition, smooth and structurally plausible.

5.4.3 Practical engineering recommendations

Based on the comparative results, the following algorithm selection guidelines emerge for cooling tower optimisation:

1. **For well-constrained, low-dimensional problems** ($D \leq 10$, known bounds):
BFGS is the method of choice, offering the fastest convergence and the highest-

quality solutions with minimal tuning effort.

2. **For high-dimensional or highly non-convex problems** ($D > 15$, disconnected feasible regions): SA provides a more robust search, provided that explicit geometric regularisation is incorporated into the cost function to prevent unrealistic profiles.
3. **For parametric formulations** ($D \leq 5$, analytical profile models): all three methods converge effectively, and the choice can be driven by implementation convenience. BFGS remains the most efficient.
4. **PSO is capable of satisfying constraints reliably**, but its high computational cost and lack of fine local refinement make it less efficient than BFGS for precise area minimization.

5.4.4 Limitations of the study

Several limitations should be acknowledged. First, the penalty weight $\rho = 10^5$ was calibrated for BFGS and may not be optimal for the stochastic methods; a higher ρ or an adaptive penalty schedule could improve PSO's surface area minimisation by better aligning the penalty landscape with the swarm's exploration-to-exploitation transition. Second, the function evaluation budgets were not equalised across methods: PSO was allocated $29\times$ more evaluations than BFGS (Table 18), yet still produced inferior results. A fairer comparison might fix the evaluation budget and compare solution quality at equal computational cost. Third, the study considers only a single cooling tower geometry class (frustum stack with fixed outer radii); generalisation to other structural forms (e.g., free-form shells, multi-material towers) would require further investigation. Finally, the analysis does not account for structural integrity: the surface-area-minimising profile may not satisfy stress, buckling, or wind-load criteria, which would need to be incorporated

as additional constraints in a full engineering design workflow.

6 Conclusion

6.1 Conclusion on Simulated Annealing

The Simulated Annealing algorithm demonstrated robust performance across the eight cooling tower optimisation scenarios, consistently satisfying the volume constraint with errors below 0.5 m^3 in all cases and confirming the effectiveness of the penalty-based approach.

In terms of solution quality, Case 8 (Hyperbolic Fit) achieved the minimum surface area of $5\,765.62\text{ m}^2$, representing a 26% reduction compared to the reference case and highlighting the advantage of parametric formulations that reduce the search space dimension while guaranteeing physically realistic geometries. The convergence behaviour across all cases exhibited characteristic staircase patterns, with most improvements occurring within the first 2 000 iterations; the algorithm successfully escaped local minima in early stages due to the high initial temperature. Sensitivity to initial conditions was observed in Case 5 (Cylindrical Start), where the deliberately poor starting point led to a 2.3% increase in area compared to the reference case, indicating that SA's final solution retains some dependence on the initialisation despite its stochastic exploration mechanism. The most significant limitation emerged in Case 6 (High Volume), where the absence of explicit geometric constraints—monotonicity, convexity—allowed the SA to produce unrealistic zigzag profiles. This confirms that penalty functions alone are insufficient for shape optimisation problems requiring smooth geometries.

In summary, SA is well-suited for cooling tower optimisation when combined with appropriate geometric constraints or parametric formulations. The computational cost remains low (1.6–6.2 s per case), making it practical for engineering design applications.

6.2 Conclusion on Particle Swarm Optimisation

The PSO algorithm exhibited strong exploration capabilities and reliable constraint satisfaction across all eight cases under the penalty-based formulation used in this study. Volume errors remained below 1.2 m^3 in all cases, confirming that the penalty weight $\rho = 10^5$ is sufficient to guide the swarm toward the feasible manifold. However, PSO consistently produced larger surface areas than BFGS, indicating that while the swarm successfully satisfies the volume constraint, it lacks the local refinement precision required to minimise the objective function effectively.

The best performance was achieved on Case 8 (Hyperbolic Fit), which produced the lowest surface area of all algorithm–case combinations ($5\,742.55\text{ m}^2$) with near-zero volume error, confirming that the low-dimensional parametric space enables effective swarm convergence. The two-phase convergence pattern—rapid descent followed by plateau—was consistently observed, with Cases 2, 6, and 8 converging within 30–50 iterations while Cases 1, 3, and 5 required over 500 iterations. The computational cost constitutes the most significant limitation of PSO as deployed in this study: PSO required on average 133 800 function evaluations per case— $14\times$ more than BFGS and $8\times$ more than SA—yet produced larger surface areas than BFGS in seven out of eight cases, yielding a poor cost-effectiveness ratio that limits PSO’s suitability for problems where function evaluations are expensive.

In summary, PSO’s global exploration capability ensures reliable constraint satisfaction but proves insufficient to compete with gradient-based methods in terms of solution quality and computational efficiency. Future work should investigate local refinement strategies, adaptive inertia schedules, or hybrid PSO–BFGS approaches to address the precision gap observed in this study.

6.3 Conclusion on BFGS

The BFGS quasi-Newton method emerged as the most effective algorithm in this study, achieving the best or near-best surface area in seven out of eight cases with perfect volume constraint satisfaction (zero error in all cases) and the lowest computational cost.

Its peak performance was observed on Case 8 (Hyperbolic Fit), where BFGS converged in just 12 iterations (0.04 s) to an area of 5 961.77 m² with zero volume error—the most efficient configuration in the entire study. Across all well-conditioned cases, BFGS exhibited superlinear convergence, with typical convergence achieved within 20–30 iterations; the gradient norm provides a natural, adaptive stopping criterion, in contrast to SA, which relies on a fixed iteration budget, and PSO, which terminates adaptively based on swarm stagnation. The geometric quality of the BFGS solutions was consistently superior: in Case 6 (High Volume), BFGS achieved a $4.7\times$ lower surface area than SA, attributable entirely to the inherent geometric regularity of gradient-based search. The principal limitation was observed in Case 3 (Full Freedom, $D = 19$), where the method's vulnerability to ill-conditioning and local trapping in high-dimensional spaces led to convergence at a physically unrealistic local minimum. For such problems, BFGS should be combined with multi-start strategies or hybridised with a global search phase.

In summary, BFGS is the recommended algorithm for cooling tower optimisation when gradient information is available and the search space is well conditioned. Its deterministic nature, adaptive termination, and inherent geometric smoothness make it ideally suited for structural engineering applications where solution quality, reproducibility, and computational efficiency are paramount.

6.4 Overall Conclusion

This study benchmarked three optimisation algorithms—BFGS, Simulated Annealing, and Particle Swarm Optimisation—on the structural surface area minimisation of a discretised hyperboloidal cooling tower across eight study cases of increasing complexity. The investigation reveals a clear performance hierarchy: $\text{BFGS} > \text{SA} \gg \text{PSO}$, with the following overarching conclusions:

1. Algorithm selection matters, but problem formulation matters more.

Case 8 (Hyperbolic Parametrisation, $D = 3$) yielded the best result for all three methods, demonstrating that embedding physical knowledge into the parametrisation is more effective than algorithmic sophistication.

2. Gradient-based methods are naturally suited to shape optimisation. BFGS's

local search produces smooth, structurally plausible profiles without requiring explicit geometric penalties, whereas stochastic methods can generate unrealistic geometries unless additional constraints are imposed.

3. Penalty-based constraint handling is effective across all three methods.

The static penalty weight $\rho = 10^5$ is sufficient to guide all three algorithms toward the feasible manifold, with volume errors below 1.2 m^3 for PSO, below 0.75 m^3 for SA, and zero for BFGS in seven out of eight cases. However, PSO's distributed evaluation strategy limits its ability to exploit local curvature near the feasible manifold, suggesting that a hybrid penalty-refinement approach would improve surface area precision.

4. Computational efficiency spans three orders of magnitude. BFGS achieves

convergence in $\mathcal{O}(10^2)$ function evaluations, SA in $\mathcal{O}(10^4)$, and PSO in $\mathcal{O}(10^5)$, with

BFGS delivering superior solutions at the lowest cost in the majority of cases.

6.5 Future Work

Building on the limitations identified in Section 5.4.4, several directions for future investigation are proposed:

- **Constraint handling:** As noted in the limitations discussion, the penalty weight $\rho = 10^5$ is suboptimal for the stochastic methods. Future work should investigate augmented Lagrangian methods, feasibility-preserving operators for PSO, and adaptive penalty schedules to improve performance on equality-constrained problems.
- **Hybrid algorithms:** Combine global exploration (SA or PSO) with local refinement (BFGS) in a two-phase strategy, using the stochastic method to identify promising basins and BFGS to converge within them.
- **Structural constraints:** Extending the cost function to incorporate stress, buckling, and wind-load criteria—identified in Section 5.4.4 as a key omission of the present study—would transform the problem into a multi-objective optimisation that more closely reflects real engineering requirements.
- **Higher-fidelity models:** Replace the conical frustum discretisation with finite element shell models to capture bending stiffness, membrane stress distributions, and nonlinear geometric effects.
- **Parametric exploration:** Investigate alternative parametric families beyond the hyperboloid of one sheet (e.g., polynomial profiles, B-spline representations) to

assess whether further surface area reductions are achievable while maintaining structural integrity.

References

- [1] I. Mungan. “Buckling Stress States of Hyperboloidal Shells”. In: *Journal of the Structural Division* 102.10 (1976), pp. 2005–2018.
- [2] J. Nocedal and S.J. Wright. *Numerical Optimization*. 2nd. Chapter 6: Quasi-Newton Methods. Springer, 2006.
- [3] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Optimisation Project Group Report*. Tech. rep. Cranfield University, 2025.
- [4] A. Riaz. *Computational Optimisation Design Lecture 2*. Lecture presented at: Computational Optimisation Design. [cited 2025 Oct 21]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-2?module_item_id=737718. 2025.
- [5] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Metaheuristic Optimisation – Group Chenonceau*. Tech. rep. Cranfield University, 2025.
- [6] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 4: Gradient-Based Optimisation – Group Chenonceau*. Tech. rep. Cranfield University, 2025.

7 Appendices

Appendix A Code Listings

Appendix A.A Problem Definition — cooling_tower.py

This module defines the geometric primitives (frustum surface area, volume, slant height) and the `CoolingTowerProblem` class which encapsulates the decision variable mapping and cost function for all eight study cases.

Listing 1: Cooling Tower Problem Definition (`cooling_tower.py`)

```
1  ##author: John.H
2  import numpy as np
3
4  def slant_height(r_bottom, r_top, height):
5      """
6      Calculates the slant height of a frustum.
7       $s = \sqrt{(r_{bot} - r_{top})^2 + h^2}$ 
8      """
9      return np.sqrt((r_bottom - r_top)**2 + height**2)
10
11 def frustum_surface_area(r_bottom, r_top, height):
12     """
13     Calculates the lateral surface area of a conical frustum.
14      $A = \pi * (r_{bot} + r_{top}) * s$ 
15     """
16     s = slant_height(r_bottom, r_top, height)
17     return np.pi * (r_bottom + r_top) * s
18
19 def frustum_volume(r_bottom, r_top, height):
20     """
21     Calculates the volume of a conical frustum.
22      $V = (\pi * h / 3) * (r_{bot}^2 + r_{bot} * r_{top} + r_{top}^2)$ 
23     """
24     return (np.pi * height / 3.0) * (r_bottom**2 + r_bottom * r_top + r_top**2)
25
26 def total_surface_area(radii, heights):
27     """
28     Calculates the total lateral surface area of the tower.
29
30     Args:
31     radii: array-like of shape (m+1,) [r0, r1, ..., rm]
32     heights: array-like of shape (m,) [h1, h2, ..., hm]
33     """
34     radii = np.asarray(radii)
35     heights = np.asarray(heights)
36     m = len(heights)
```

```
37
38     if len(radii) != m + 1:
39         raise ValueError(f"Expected_{m+1}_radii_for_{m}_heights, got_{len(radii)}")
40
41     total_area = 0.0
42     for i in range(m):
43         r_bot = radii[i]
44         r_top = radii[i+1]
45         h = heights[i]
46         total_area += frustum_surface_area(r_bot, r_top, h)
47
48     return total_area
49
50 def total_volume(radii, heights):
51     """
52     Calculates the total volume of the tower.
53
54     Args:
55     radii: array-like of shape (m+1,) [r0, r1, ..., rm]
56     heights: array-like of shape (m,) [h1, h2, ..., hm]
57     """
58     radii = np.asarray(radii)
59     heights = np.asarray(heights)
60     m = len(heights)
61
62     if len(radii) != m + 1:
63         raise ValueError(f"Expected_{m+1}_radii_for_{m}_heights, got_{len(radii)}")
64
65     tot_vol = 0.0
66     for i in range(m):
67         r_bot = radii[i]
68         r_top = radii[i+1]
69         h = heights[i]
70         tot_vol += frustum_volume(r_bot, r_top, h)
71
72     return tot_vol
73
74 class CoolingTowerProblem:
75     """
76     Defines the optimization problem for the Cooling Tower.
77     """
78     def __init__(self, m=10, r0=39.3, rm=27.4, total_height=36.5, v_target=70320.0):
79         self.m = m
80         self.r0 = r0
81         self.rm = rm
82         self.total_height = total_height
83         self.v_target = v_target
84
85         # Case 1 defaults: equal heights
86         self.fixed_heights = np.full(m, total_height / m)
87
```



```
88     def unpack_decision_variables(self, x, case_num, **kwargs):
89         """
90         Reconstructs the full radii and heights arrays from the decision variables based
          on the scenario.
91         """
92         x = np.asarray(x)
93         if case_num in [1, 4, 5, 6, 7]:
94             # x contains [r1, r2, ..., r_{m-1}]
95             radii = np.zeros(self.m + 1)
96             radii[0] = self.r0
97             radii[-1] = self.rm
98             radii[1:-1] = x
99             heights = kwargs.get('fixed_heights', self.fixed_heights)
100             return radii, heights
101
102         elif case_num == 2:
103             # x contains [h1, h2, ..., hm]
104             radii = kwargs.get('fixed_radii')
105             if radii is None:
106                 raise ValueError("Case 2 requires 'fixed_radii' kwarg.")
107             return radii, x
108
109         elif case_num == 3:
110             # x contains [r1...r_{m-1}, h1...hm]
111             radii = np.zeros(self.m + 1)
112             radii[0] = self.r0
113             radii[-1] = self.rm
114             radii[1:-1] = x[:self.m - 1]
115             heights = x[self.m - 1:]
116             return radii, heights
117
118         elif case_num == 8:
119             # x contains [a, b, c] - hyperbola parameters
120             a, b, c = x
121             heights = self.fixed_heights
122             # Calculate z heights (cumulative sum)
123             z_levels = np.zeros(self.m + 1)
124             for i in range(1, self.m + 1):
125                 z_levels[i] = z_levels[i-1] + heights[i-1]
126
127             # r(z) = a * sqrt(1 + (z-c)^2 / b^2)
128             radii = a * np.sqrt(1 + ((z_levels - c)**2) / (b**2))
129             return radii, heights
130
131         else:
132             raise ValueError(f"Unknown case_num: {case_num}")
133
134     def cost_function(self, x, case_num, penalty_weight=1e5, **kwargs):
135         """
136         Objective function for all cases.
137         J(x) = A_total(x) + rho * (V_total(x) - V_target)^2 + custom_penalties
```

```

138 """
139     radii, heights = self.unpack_decision_variables(x, case_num, **kwargs)
140
141     area = total_surface_area(radii, heights)
142     vol = total_volume(radii, heights)
143
144     # Case 6 has 50% more volume
145     target_vol = self.v_target * 1.5 if case_num == 6 else self.v_target
146
147     penalty = penalty_weight * (vol - target_vol)**2
148
149     # Case 4: Tight Waist Constraint
150     # r_mid < 0.7 * min(r0, rm)
151     if case_num == 4:
152         mid_idx = self.m // 2
153         r_mid = radii[mid_idx]
154         max_allowed = 0.7 * min(self.r0, self.rm)
155         if r_mid > max_allowed:
156             penalty += penalty_weight * (r_mid - max_allowed)**2
157
158     # Optional: Add height positivity constraint if not handled by optimizer bounds
159     # (Cases 2, 3)
160     if case_num in [2, 3]:
161         # Penalize negative or zero heights
162         neg_heights = heights[heights < 0.1]
163         if len(neg_heights) > 0:
164             penalty += penalty_weight * np.sum((0.1 - neg_heights)**2)
165
166     return area + penalty
167
168     def _hyperboloid_radii(self, r_waist):
169         """
170         Build a convex hyperboloid profile [r0, r1, ..., rm] that satisfies
171         monotonicity (decreasing to waist, increasing to top) and convexity
172         (positive second differences).
173
174         Lower half: r(t) = r0 - (r0 - r_waist) * sqrt(t) [convex, fast drop]
175         Upper half: r(t) = r_waist + (rm - r_waist) * t^2 [convex, slow rise]
176         """
177         n = self.m + 1
178         mid = n // 2
179         radii = np.zeros(n)
180         radii[0] = self.r0
181         radii[-1] = self.rm
182         radii[mid] = r_waist
183         for i in range(1, mid):
184             t = i / mid
185             radii[i] = self.r0 - (self.r0 - r_waist) * np.sqrt(t)
186         for i in range(mid + 1, n - 1):
187             t = (i - mid) / (n - 1 - mid)
188             radii[i] = r_waist + (self.rm - r_waist) * t**2

```

```
188         return radii
189
190     def get_initial_guess(self, case_num, **kwargs):
191         """
192         Returns a reasonable initial guess for the specified case.
193         Uses a hyperboloid-shaped profile for radii cases to satisfy
194         geometric constraints from the start.
195         """
196         if case_num in [1, 4, 7]:
197             # Hyperboloid initial guess (waist ~19m gives  $V \approx 70,320$ )
198             radii = self._hyperboloid_radii(r_waist=19.0)
199             return radii[1:-1]
200
201         elif case_num == 6:
202             # High Volume: wider waist for 1.5x volume target
203             radii = self._hyperboloid_radii(r_waist=24.0)
204             return radii[1:-1]
205
206         elif case_num == 5:
207             # Cylindrical Start (must start from cylinder per specification)
208             r_avg = (self.r0 + self.rm) / 2.0
209             return np.full(self.m - 1, r_avg)
210
211         elif case_num == 2:
212             # Heights guess: equal slices
213             return self.fixed_heights.copy()
214
215         elif case_num == 3:
216             # Hyperboloid radii + equal heights
217             radii = self._hyperboloid_radii(r_waist=19.0)
218             heights_guess = self.fixed_heights.copy()
219             return np.concatenate((radii[1:-1], heights_guess))
220
221         elif case_num == 8:
222             # a, b, c guess for hyperbola  $r(z) = a * \sqrt{1 + (z-c)^2 / b^2}$ 
223             a_guess = min(self.r0, self.rm) * 0.8
224             b_guess = self.total_height
225             c_guess = self.total_height / 2.0
226             return np.array([a_guess, b_guess, c_guess])
227
228         else:
229             raise ValueError(f"Unknown case_num: {case_num}")
230
231     def get_bounds(self, case_num):
232         """
233         Returns the variable bounds [(lower, upper), ...] for each decision variable.
234         Used by BFGS (L-BFGS-B) and PSO for box-constrained optimization.
235         """
236         if case_num == 7:
237             # Restricted Bounds:  $r_i$  in [20, 60]
238             return [(20.0, 60.0)] * (self.m - 1)
```

```
239
240     elif case_num in [1, 4, 5, 6]:
241         # General radii bounds: between some sensible physical limits
242         r_min = 5.0
243         r_max = max(self.r0, self.rm) * 2.0
244         return [(r_min, r_max)] * (self.m - 1)
245
246     elif case_num == 2:
247         # Heights bounds: positive heights, max ~2x equal slice
248         h_min = 0.5
249         h_max = self.total_height * 0.5
250         return [(h_min, h_max)] * self.m
251
252     elif case_num == 3:
253         # Radii bounds + Heights bounds
254         r_min, r_max = 5.0, max(self.r0, self.rm) * 2.0
255         h_min, h_max = 0.5, self.total_height * 0.5
256         radii_bounds = [(r_min, r_max)] * (self.m - 1)
257         height_bounds = [(h_min, h_max)] * self.m
258         return radii_bounds + height_bounds
259
260     elif case_num == 8:
261         # Hyperbola parameters: a, b, c
262         return [
263             (5.0, max(self.r0, self.rm)),          # a: waist radius
264             (5.0, self.total_height * 3.0),        # b: curvature
265             (0.0, self.total_height)               # c: waist height
266         ]
267     else:
268         raise ValueError(f"Unknown case_num: {case_num}")
269
270
271 def numerical_gradient(func, x, epsilon=1e-6):
272     """
273     Computes the gradient of a scalar function using central finite differences.
274
275     grad_i = (f(x + e_i * eps) - f(x - e_i * eps)) / (2 * eps)
276
277     Args:
278     func: scalar-valued function f(x) -> float.
279     x: current point (np.ndarray).
280     epsilon: step size for finite differences.
281
282     Returns:
283     grad: np.ndarray of same shape as x.
284     """
285     x = np.asarray(x, dtype=float)
286     grad = np.zeros_like(x)
287     for i in range(len(x)):
288         x_plus = x.copy()
289         x_minus = x.copy()
```

```
290         x_plus[i] += epsilon
291         x_minus[i] -= epsilon
292         grad[i] = (func(x_plus) - func(x_minus)) / (2.0 * epsilon)
293     return grad
294
295
296 def analytical_surface_area_hyperboloid(a, b, c, z_bottom, z_top, n_steps=1000):
297     """
298     Computes the exact lateral surface area of a Hyperboloid of One Sheet
299     by numerical integration of the surface-of-revolution formula.
300
301     r(z) = a * sqrt(1 + (z - c)^2 / b^2)
302
303     A = 2 * pi * integral_{z_bot}^{z_top} r(z) * sqrt(1 + (dr/dz)^2) dz
304
305     This is useful for the report discussion comparing discrete (frustum)
306     approximation vs the true curved surface.
307
308     Args:
309         a: waist radius parameter.
310         b: curvature parameter.
311         c: waist height parameter.
312         z_bottom: lower integration limit.
313         z_top: upper integration limit.
314         n_steps: number of integration steps (trapezoidal rule).
315
316     Returns:
317         area: the analytically integrated surface area.
318     """
319     z = np.linspace(z_bottom, z_top, n_steps)
320
321     r = a * np.sqrt(1.0 + ((z - c)**2) / (b**2))
322     # dr/dz = a * (z - c) / (b^2 * sqrt(1 + (z-c)^2 / b^2))
323     drdz = a * (z - c) / (b**2 * np.sqrt(1.0 + ((z - c)**2) / (b**2)))
324
325     integrand = r * np.sqrt(1.0 + drdz**2)
326
327     # Trapezoidal integration
328     area = 2.0 * np.pi * np.trapz(integrand, z)
329     return area
```

Appendix A.B Solver Orchestrator — solver.py

The orchestrator script constructs each study case, initialises the geometry, and dispatches each algorithm wrapper uniformly. It also generates convergence plots, profile comparisons, and summary metrics.

Listing 2: Solver Orchestrator (solver.py)

```
1 """
2 Subject: Solver with Geometric Constraints for Cooling Tower
3 @author: Pierre Bédrune, John HOARAU
4 @date: 04/03/2026
5
6 ARCHITECTURE:
7 Les algorithmes (SA, PSO, BFGS) optimisent librement
8 Le solver vérifie et corrige chaque solution selon les règles géométriques
9 Les contraintes sont appliquées APRÈS chaque itération ou à la fin
10
11 RÈGLES GÉOMÉTRIQUES (construction hyperboloïde):
12
13 Points fixes:
14 P0(bas): r0, z=0
15 P10(haut): rm, z=H
16
17 Construction depuis le HAUT (P10 à P5):
18 Chaque point doit être EN DESSOUS et À DROITE (r plus petit)
19 La dérivée doit rester à gauche de la droite précédente (convexité)
20
21 Construction depuis le BAS (P0 à P5):
22 Chaque point doit être AU DESSUS et À DROITE (r plus petit)
23 La dérivée doit rester à gauche de la droite précédente (convexité)
24
25 Point WAIST (P5):
26 Doit satisfaire les contraintes des deux côtés
27 C'est le minimum local
28 """
29
30 import time
31 import numpy as np
32 from typing import Tuple, Dict, Any, List, Optional
33 import sys
34 import os
35
36 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38 from Problem.cooling_tower import CoolingTowerProblem
39
40 # Import algorithms
41 try:
```

```
42     from SA.sa import simulated_annealing
43 except ImportError:
44     simulated_annealing = None
45
46 try:
47     from BFGS.bfgs import bfgs_optimize
48 except ImportError:
49     bfgs_optimize = None
50
51 try:
52     from PSO.pso import particle_swarm
53 except ImportError:
54     particle_swarm = None
55
56
57 # =====
58 # GEOMETRIC CONSTRAINTS
59 # =====
60
61 class GeometricConstraints:
62     """
63     Vérifie et corrige les solutions pour garantir une forme hyperboloïde.
64
65     Règles:
66     1. Monotonie: rayons décroissent de r0 vers waist, puis croissent vers rm
67     2. Convexité: courbure vers l'extérieur (dérivée seconde > 0)
68     3. Waist: point minimum, satisfait les contraintes des deux côtés
69     """
70
71     def __init__(self, r0: float, rm: float, n_points: int = 11):
72         """
73         Args:
74             r0: Rayon fixe en bas (z=0)
75             rm: Rayon fixe en haut (z=H)
76             n_points: Nombre total de points (11 = r0 + 9 intérieurs + rm)
77         """
78         self.r0 = r0
79         self.rm = rm
80         self.n_points = n_points
81         self.n_interior = n_points - 2 # Points à optimiser
82         self.waist_idx = n_points // 2 # Index du waist dans le tableau complet
83
84     def get_full_radii(self, interior_radii: np.ndarray) -> np.ndarray:
85         """Reconstitue le tableau complet [r0, r1, ..., r9, rm]"""
86         return np.concatenate([[self.r0], interior_radii, [self.rm]])
87
88     def check_point_from_bottom(self, radii: np.ndarray, idx: int) -> Dict[str, Any]:
89         """
90         Vérifie les contraintes pour un point construit depuis le bas.
91
92         Args:
```

```
93      radii: Tableau_complet_des_rayons
94      idx: Index_du_point_à_vérifier(1_à_waist_idx)
95
96      Returns:
97      Dict_avec_'valid', 'violations', 'suggested_r'
98      """
99      result = {
100          'valid': True,
101          'violations': [],
102          'suggested_r': radii[idx]
103      }
104
105      # Contrainte 1:  $r[idx] < r[idx-1]$  (doit être plus petit que le précédent)
106      if radii[idx] >= radii[idx - 1]:
107          result['valid'] = False
108          result['violations'].append(f"P{idx}: r={radii[idx]:.2f} > r_prev={radii[
109              idx-1]:.2f}")
110          result['suggested_r'] = radii[idx - 1] * 0.95
111
112      # Contrainte 2: Convexité (dérivée seconde > 0)
113      # Contrainte 2: Convexité ( $d^2r/dz^2$  approximée par différences centrées)
114      # Utiliser la différence centrée au point idx:  $r[idx-1] - 2*r[idx] + r[idx+1]$ 
115      if idx >= 1 and idx <= self.n_points - 2:
116          d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
117          if d2r < 0:
118              result['valid'] = False
119              result['violations'].append(f"P{idx}: non convexe (d2r={d2r:.2f})")
120              # Correction: proposer une valeur légèrement plus petite que la moyenne
121              # des voisins
122              target = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
123              result['suggested_r'] = min(result['suggested_r'], target)
124
125      return result
126
127      def check_point_from_top(self, radii: np.ndarray, idx: int) -> Dict[str, Any]:
128          """
129          Vérifie les contraintes pour un point construit depuis le haut.
130
131          Args:
132          radii: Tableau_complet_des_rayons
133          idx: Index_du_point_à_vérifier(waist_idx_à_n_points-2)
134
135          Returns:
136          Dict_avec_'valid', 'violations', 'suggested_r'
137          """
138          result = {
139              'valid': True,
140              'violations': [],
141              'suggested_r': radii[idx]
```



```

142     # Contrainte 1:  $r[idx] < r[idx+1]$  (doit être plus petit que le suivant)
143     if radii[idx] >= radii[idx + 1]:
144         result['valid'] = False
145         result['violations'].append(f"P{idx}: r={radii[idx]:.2f} > r_next={radii[
146             idx+1]:.2f}")
147         result['suggested_r'] = radii[idx + 1] * 0.95
148
149     # Contrainte 2: Convexité (dérivée seconde > 0)
150     # Contrainte 2: Convexité (différence centrée au point idx)
151     if idx >= 1 and idx <= self.n_points - 2:
152         d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
153         if d2r < 0:
154             result['valid'] = False
155             result['violations'].append(f"P{idx}: non convexe depuis haut (d2r={d2r
156                 :.2f}")
157             target = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
158             result['suggested_r'] = min(result['suggested_r'], target)
159
160     return result
161
162     def check_waist_point(self, radii: np.ndarray) -> Dict[str, Any]:
163         """
164         Vérifie les contraintes pour le point waist (P5 = 11ème point).
165
166         DOIT SATISFAIRE LES CONDITIONS DES DEUX CÔTÉS:
167         Depuis le bas: r[5] < r[4] ET convexité (d2r avec P3, P4, P5)
168         Depuis le haut: r[5] < r[6] ET convexité (d2r avec P5, P6, P7)
169
170         Returns:
171         Dict avec 'valid', 'violations', 'suggested_r'
172         """
173         idx = self.waist_idx # = 5
174         result = {
175             'valid': True,
176             'violations': [],
177             'suggested_r': radii[idx]
178         }
179
180         # === CONTRAINTES DEPUIS LE BAS ===
181         # 1. Monotonie:  $P5 < P4$ 
182         if radii[idx] >= radii[idx - 1]:
183             result['valid'] = False
184             result['violations'].append(f"Waist P{idx}: r={radii[idx]:.2f} > r_gauche={
185                 radii[idx-1]:.2f}")
186
187         # 2. Convexité bas:  $d2r(P3, P4, P5) > 0$ 
188         if idx >= 1 and idx <= self.n_points - 2:
189             d2r_bottom = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
190             if d2r_bottom < 0:
191                 result['valid'] = False
192                 result['violations'].append(f"Waist P{idx}: non convexe depuis bas (d2r

```

```

    = {d2r_bottom:.2f}))")
190
191     # === CONTRAINTES DEPUIS LE HAUT ===
192     # 3. Monotonie:  $P_5 < P_6$ 
193     if radii[idx] >= radii[idx + 1]:
194         result['valid'] = False
195         result['violations'].append(f"Waist_{idx}: r={radii[idx]:.2f} >= r_droite={
            radii[idx+1]:.2f}")
196
197     # 4. Convexité haut:  $d2r(P_5, P_6, P_7) > 0$ 
198     if idx >= 1 and idx <= self.n_points - 2:
199         d2r_top = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
200         if d2r_top < 0:
201             result['valid'] = False
202             result['violations'].append(f"Waist_{idx}: non convexe depuis haut (d2r
                = {d2r_top:.2f}")
203
204     # Suggestion: si invalide, prendre le minimum des voisins
205     if not result['valid']:
206         result['suggested_r'] = min(radii[idx - 1], radii[idx + 1]) * 0.9
207
208     return result
209
210     def check_all_points(self, interior_radii: np.ndarray, verbose: bool = False) ->
        Dict[str, Any]:
211         """
212         Vérifie tous les points et retourne un rapport complet.
213
214         Returns:
215         Dict avec 'all_valid', 'violations', 'point_reports'
216         """
217         radii = self.get_full_radii(interior_radii)
218
219         report = {
220             'all_valid': True,
221             'violations': [],
222             'point_reports': {}
223         }
224
225         # Points depuis le bas ( $P_1$  à  $P_{waist-1}$ )
226         for idx in range(1, self.waist_idx):
227             pt_report = self.check_point_from_bottom(radii, idx)
228             report['point_reports'][f'P{idx}_bottom'] = pt_report
229             if not pt_report['valid']:
230                 report['all_valid'] = False
231                 report['violations'].extend(pt_report['violations'])
232
233         # Point waist
234         waist_report = self.check_waist_point(radii)
235         report['point_reports']['waist'] = waist_report
236         if not waist_report['valid']:

```

```
237         report['all_valid'] = False
238         report['violations'].extend(waist_report['violations'])
239
240         # Points depuis le haut (P_waist+1 à P9)
241         for idx in range(self.waist_idx + 1, self.n_points - 1):
242             pt_report = self.check_point_from_top(radii, idx)
243             report['point_reports'][f'P{idx}_top'] = pt_report
244             if not pt_report['valid']:
245                 report['all_valid'] = False
246                 report['violations'].extend(pt_report['violations'])
247
248         if verbose and not report['all_valid']:
249             print(f"_{GeoCheck}_{len(report['violations'])}_violations_détectées")
250             for v in report['violations'][:5]: # Afficher max 5
251                 print(f"_{v}")
252
253         return report
254
255     def correct_solution(self, interior_radii: np.ndarray, max_iterations: int = 10) ->
256         np.ndarray:
257         """
258         _Corrige_une_solution_pour_respecter_toutes_les_contraintes_géométriques.
259
260         _Algorithme_:
261         1. Construire depuis le bas (P0_à_waist)
262         2. Construire depuis le haut (Pm_à_waist)
263         3. Ajuster le waist pour satisfaire les deux côtés
264         4. Répéter si nécessaire
265
266         Returns:
267         _Radii_corrigés_
268         """
269         result = interior_radii.copy()
270
271         for iteration in range(max_iterations):
272             radii = self.get_full_radii(result)
273             changed = False
274
275             # == Construction depuis le BAS ==
276             for idx in range(1, self.waist_idx):
277                 # Monotonie: r[idx] < r[idx-1]
278                 if radii[idx] >= radii[idx - 1]:
279                     radii[idx] = radii[idx - 1] * 0.95
280                     changed = True
281
282             # Convexité (différence centrée au point idx)
283             if idx >= 1 and idx <= self.n_points - 2:
284                 d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
285                 if d2r < 0:
286                     # Projeter vers une valeur inférieure à la moyenne des voisins
287                     radii[idx] = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
```

```
287         changed = True
288
289     # == Construction depuis le HAUT ==
290     for idx in range(self.n_points - 2, self.waist_idx, -1):
291         # Monotonie:  $r[idx] < r[idx+1]$ 
292         if radii[idx] >= radii[idx + 1]:
293             radii[idx] = radii[idx + 1] * 0.95
294             changed = True
295
296         # Convexité (différence centrée au point idx)
297         if idx >= 1 and idx <= self.n_points - 2:
298             d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
299             if d2r < 0:
300                 radii[idx] = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
301                 changed = True
302
303     # == Waist (doit être minimum) ==
304     idx = self.waist_idx
305     max_allowed = min(radii[idx - 1], radii[idx + 1]) - 0.5
306     if radii[idx] > max_allowed:
307         radii[idx] = max(max_allowed, 5.0)
308         changed = True
309
310     # Mettre à jour result (sans r0 et rm)
311     result = radii[1:-1]
312
313     # Bornes min
314     result = np.clip(result, 5.0, None)
315
316     if not changed:
317         break
318
319     return result
320
321     def compute_penalty(self, interior_radii: np.ndarray) -> float:
322         """
323         Calculer une pénalité pour les violations de contraintes.
324         PÉNALITÉS TRÈS FORTES pour forcer la forme hyperboloïde.
325         """
326         interior_radii = np.asarray(interior_radii)
327         # Accept either interior radii (length n_interior) or full radii (length
328             n_points)
329         if interior_radii.shape[0] == self.n_points:
330             radii = interior_radii.copy()
331         else:
332             radii = self.get_full_radii(interior_radii)
333         penalty = 0.0
334
335         # Pénalité monotonie depuis le bas ( $r[i]$  doit être  $< r[i-1]$ )
336         for idx in range(1, self.waist_idx + 1):
337             if radii[idx] >= radii[idx - 1]:
```

```
337         diff = radii[idx] - radii[idx - 1] + 0.1
338         penalty += diff ** 2 * 10000 # TRÈS FORT
339
340     # Pénalité monotonie depuis le haut (r[i] doit être < r[i+1])
341     for idx in range(self.waist_idx, self.n_points - 1):
342         if radii[idx] >= radii[idx + 1]:
343             diff = radii[idx] - radii[idx + 1] + 0.1
344             penalty += diff ** 2 * 10000 # TRÈS FORT
345
346     # Pénalité convexité bas (d2r > 0)
347     for idx in range(1, self.waist_idx + 1):
348         # centred second difference at idx
349         if idx >= 1 and idx <= self.n_points - 2:
350             d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
351             if d2r < 0:
352                 penalty += abs(d2r) ** 2 * 5000
353
354     # Pénalité convexité haut (d2r > 0)
355     for idx in range(self.waist_idx, self.n_points - 1):
356         if idx >= 1 and idx <= self.n_points - 2:
357             d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
358             if d2r < 0:
359                 penalty += abs(d2r) ** 2 * 5000
360
361     return penalty
362
363
364 # =====
365 # OPTIMIZER CLASSES
366 # =====
367
368 class BaseOptimizer:
369     """Base class for all optimization algorithms."""
370
371     def __init__(self, problem: CoolingTowerProblem, case_num: int = 1,
372                 geo_constraints: GeometricConstraints = None,
373                 fixed_radii: np.ndarray = None, **kwargs):
374         self.problem = problem
375         self.case_num = case_num
376         self.fixed_radii = fixed_radii
377         self.geo_constraints = geo_constraints
378         # Extract correction_mode before storing remaining kwargs as algo options
379         # 'correct' = correction-based (good for SA/BFGS)
380         # 'penalty' = penalty-based (smooth landscape, good for PSO)
381         self.correction_mode = kwargs.pop('correction_mode', 'correct')
382         self.options = kwargs
383
384     def _objective_func(self, x):
385         """Objective function with geometric constraints.
386
387         """
388         TwoModes.set_via_correction_mode)
```

```
388
389         'correct' (default, best for SA/BFGS):
390         1. Project the proposed solution to geometric feasibility
391         2. Evaluate cost on the corrected solution
392         3. Add distance penalty guiding optimizer toward feasible region
393
394         'penalty' (best for PSO):
395         1. Evaluate cost on the raw proposed solution
396         2. Add smooth geometric penalty for constraint violations
397         This gives PSO a smooth landscape without fixed-point attractors.
398
399         Cases 2, 6, 8 bypass geometric constraints in both modes.
400         """
401         # No geometric constraints → evaluate directly
402         if self.geo_constraints is None:
403             return self.problem.cost_function(
404                 x, self.case_num, fixed_radii=self.fixed_radii
405             )
406
407         # Case 2: only heights vary, radii are fixed → no geo correction needed
408         # Case 6: 1.5x volume target cannot be achieved with hyperboloid shape
409         #           (waist < rm=27.4 limits max volume), so skip geo constraints
410         # Case 8: hyperbola params [a,b,c], not radii → no geo correction needed
411         if self.case_num in [2, 6, 8]:
412             return self.problem.cost_function(
413                 x, self.case_num, fixed_radii=self.fixed_radii
414             )
415
416         # Extract interior radii from decision variables
417         try:
418             radii_full, heights = self.problem.unpack_decision_variables(
419                 x, self.case_num, fixed_radii=self.fixed_radii
420             )
421         except Exception:
422             return self.problem.cost_function(
423                 x, self.case_num, fixed_radii=self.fixed_radii
424             )
425
426         interior = np.asarray(radii_full)[1:-1]
427
428         if self.correction_mode == 'penalty':
429             # PENALTY MODE: smooth landscape for population-based methods (PSO)
430             # Evaluate on the raw (uncorrected) solution + add geo penalty
431             cost = self.problem.cost_function(
432                 x, self.case_num, fixed_radii=self.fixed_radii
433             )
434             geo_pen = self.geo_constraints.compute_penalty(interior)
435             return cost + geo_pen
436
437         # CORRECTION MODE: project-and-penalise for gradient-based / local methods
438         corrected = self.geo_constraints.correct_solution(interior, max_iterations=20)
```

```
439
440     # Reconstruct decision vector from corrected radii
441     if self.case_num in [1, 4, 5, 6, 7]:
442         x_eval = np.asarray(corrected)
443     elif self.case_num == 3:
444         x_eval = np.concatenate((np.asarray(corrected), np.asarray(heights)))
445     else:
446         x_eval = x
447
448     # Cost on the corrected (feasible) solution
449     cost = self.problem.cost_function(
450         x_eval, self.case_num, fixed_radii=self.fixed_radii
451     )
452
453     # Distance penalty: guides optimizer toward feasible solutions
454     dist = np.sum((interior - corrected) ** 2)
455     return cost + 1e4 * dist
456
457     def optimize(self, initial_guess: np.ndarray):
458         raise NotImplementedError
459
460
461 class SaOptimizer(BaseOptimizer):
462     """Simulated Annealing Wrapper."""
463
464     def optimize(self, initial_guess: np.ndarray):
465         start_time = time.time()
466
467         if simulated_annealing is None:
468             raise NotImplementedError("sa.py not found.")
469
470         bounds = self.problem.get_bounds(self.case_num)
471
472         best_x, best_cost, history, n_fevals = simulated_annealing(
473             objective_func=self._objective_func,
474             initial_guess=initial_guess,
475             bounds=bounds,
476             **self.options
477         )
478
479         # PAS de correction finale - les pénalités guident l'optimisation
480         # La solution doit naturellement satisfaire les contraintes
481
482         time_taken = time.time() - start_time
483         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
484         return best_x, best_cost, history, metrics
485
486
487 class BfgsOptimizer(BaseOptimizer):
488     """BFGS Wrapper."""
489
```

```
490     def optimize(self, initial_guess: np.ndarray):
491         start_time = time.time()
492
493         if bfgs_optimize is None:
494             raise NotImplementedError("bfgs.py not found.")
495
496         bounds = self.problem.get_bounds(self.case_num)
497
498         best_x, best_cost, history, n_fevals = bfgs_optimize(
499             objective_func=self._objective_func,
500             initial_guess=initial_guess,
501             bounds=bounds,
502             **self.options
503         )
504
505         # PAS de correction finale - les pénalités guident l'optimisation
506
507         time_taken = time.time() - start_time
508         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
509         return best_x, best_cost, history, metrics
510
511
512 class PsoOptimizer(BaseOptimizer):
513     """Particle Swarm Wrapper."""
514
515     def optimize(self, initial_guess: np.ndarray):
516         start_time = time.time()
517
518         if particle_swarm is None:
519             raise NotImplementedError("pso.py not found.")
520
521         bounds = self.problem.get_bounds(self.case_num)
522
523         best_x, best_cost, history, n_fevals = particle_swarm(
524             objective_func=self._objective_func,
525             bounds=bounds,
526             **self.options
527         )
528
529         # PAS de correction finale - les pénalités guident l'optimisation
530
531         time_taken = time.time() - start_time
532         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
533         return best_x, best_cost, history, metrics
534
535
536 # =====
537 # SCENARIO RUNNER
538 # =====
539
540 def run_scenario(optimizer_class, problem: CoolingTowerProblem, case_num: int,
```



```
541         scenario_name: str, fixed_radii: np.ndarray = None,
542         use_geo_constraints: bool = True, **extra_kwargs):
543     """Run a specific scenario with geometric constraints."""
544
545     print(f"\n{'='*60}")
546     print(f"Scenario_{case_num}:_{scenario_name}")
547     print(f"Algorithm:_{optimizer_class.__name__}")
548     print(f"Geometric Constraints:_{'ON' if use_geo_constraints else 'OFF'}")
549     print(f"{'='*60}")
550
551     # Create geometric constraints
552     geo_constraints = None
553     if use_geo_constraints:
554         geo_constraints = GeometricConstraints(
555             r0=problem.r0,
556             rm=problem.rm,
557             n_points=problem.m + 1
558         )
559
560     # If fixed_radii provided for cases like Case 2, ensure they respect geometry (
561     # correct if needed)
562     if use_geo_constraints and fixed_radii is not None:
563         fr = np.asarray(fixed_radii)
564         # Accept either interior or full radii
565         if fr.shape[0] == problem.m + 1:
566             interior_fr = fr[1:-1]
567         else:
568             interior_fr = fr
569
570         report_fr = geo_constraints.check_all_points(interior_fr, verbose=False)
571         if not report_fr['all_valid']:
572             corrected = geo_constraints.correct_solution(interior_fr, max_iterations=50)
573             fixed_radii = np.concatenate(([geo_constraints.r0], corrected, [
574                 geo_constraints.rm]))
575
576     optimizer = optimizer_class(
577         problem,
578         case_num=case_num,
579         geo_constraints=geo_constraints,
580         fixed_radii=fixed_radii,
581         **extra_kwargs
582     )
583
584     initial_guess = problem.get_initial_guess(case_num)
585     init_radii, init_heights = problem.unpack_decision_variables(
586         initial_guess, case_num, fixed_radii=fixed_radii
587     )
588
589     # Run optimization
590     try:
591         best_x, best_cost, history, metrics = optimizer.optimize(initial_guess)
```

```
except NotImplementedError as e:
    print(f"Skipping: {e}")
    return None, None, None, None

best_radii, best_heights = problem.unpack_decision_variables(
    best_x, case_num, fixed_radii=fixed_radii
)

# Apply final geometric correction only for cases where interior radii are optimised
# Skip for Case 2 (heights only), Case 6 (needs wider shapes), Case 8 (hyperbola
params)
if geo_constraints is not None and case_num not in [2, 6, 8]:
    interior_best = np.asarray(best_radii)[1:-1]
    corrected_interior = geo_constraints.correct_solution(interior_best,
        max_iterations=50)
    best_radii = np.concatenate(([geo_constraints.r0], corrected_interior, [
        geo_constraints.rm]))
    report = geo_constraints.check_all_points(corrected_interior, verbose=True)
    print(f" [GeoCheck] Final: {'VALID' if report['all_valid'] else 'INVALID'}")

# Compute metrics on the FINAL (corrected) solution
from Problem.cooling_tower import total_surface_area, total_volume
final_area = total_surface_area(best_radii, best_heights)
final_vol = total_volume(best_radii, best_heights)

target_vol = problem.v_target * 1.5 if case_num == 6 else problem.v_target
vol_error = abs(final_vol - target_vol)

# True cost = surface area + volume penalty (NO geometric penalty inflating it)
# Uses the same penalty_weight as CoolingTowerProblem.cost_function
penalty_weight = 1e5
final_cost = final_area + penalty_weight * (final_vol - target_vol)**2

# == Save Results ==
import csv
import matplotlib.pyplot as plt

algo_name = optimizer_class.__name__.replace('Optimizer', '')
base_out_dir = os.path.join(os.path.dirname(__file__), "output")
out_dir = os.path.join(base_out_dir, algo_name)
os.makedirs(out_dir, exist_ok=True)

base_filename = f"Case{case_num}_{algo_name}"

# 1. Convergence Plot
plt.figure()
plt.plot(history, label='Cost')
plt.title(f"Convergence: Case {case_num} ({algo_name})")
plt.xlabel("Iteration")
plt.ylabel("Cost J(x)")
plt.yscale('log')
```

```
plt.grid(True)
plt.legend()
plt.savefig(os.path.join(out_dir, f"{base_filename}_convergence.png"))
plt.close()

# 2. Profile Plot with Initial Guess
z_coords = np.concatenate(([0], np.cumsum(best_heights)))
z_coords_init = np.concatenate(([0], np.cumsum(init_heights)))

plt.figure(figsize=(8, 10))

# Initial Guess (ROUGE)
plt.plot(init_radii, z_coords_init, 'r--s', alpha=0.6, markersize=8, label='Initial_
    Guess')
plt.plot(-init_radii, z_coords_init, 'r--s', alpha=0.6, markersize=8)

# Optimized (BLEU)
plt.plot(best_radii, z_coords, 'b-o', markersize=6, label='Optimized')
plt.plot(-best_radii, z_coords, 'b-o', markersize=6)
plt.fill_betweenx(z_coords, -best_radii, best_radii, alpha=0.15, color='blue')

# Mark waist
waist_idx = len(best_radii) // 2
plt.plot([best_radii[waist_idx], -best_radii[waist_idx]],
        [z_coords[waist_idx], z_coords[waist_idx]],
        'g--', linewidth=2, label=f'Waist (r={best_radii[waist_idx]:.1f}m)')

plt.title(f"Profile: Case {case_num} ({algo_name})\nArea: {final_area:.2f} | Vol_
    Error: {vol_error:.2f}")
plt.xlabel("Radius (m)")
plt.ylabel("Height (m)")
plt.legend(loc='upper_right')
plt.grid(True)
plt.axis('equal')
plt.savefig(os.path.join(out_dir, f"{base_filename}_profile.png"), dpi=150)
plt.close()

# 3. Plot 3D Wireframe
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
theta = np.linspace(0, 2*np.pi, 30)

# Create the 3D surface grid
for i in range(len(best_heights)):
    z_bot = z_coords[i]
    z_top = z_coords[i+1]
    r_bot = best_radii[i]
    r_top = best_radii[i+1]

# Simple frustum side
z_grid = np.linspace(z_bot, z_top, 5)
```

```
687     r_grid = np.linspace(r_bot, r_top, 5)
688
689     Z, Theta = np.meshgrid(z_grid, theta)
690     R, _ = np.meshgrid(r_grid, theta)
691
692     X = R * np.cos(Theta)
693     Y = R * np.sin(Theta)
694
695     ax.plot_wireframe(X, Y, Z, color='b', alpha=0.5)
696
697     ax.set_title(f"3D Wireframe: Case {case_num} ({algo_name})")
698     ax.set_xlabel('X (m)')
699     ax.set_ylabel('Y (m)')
700     ax.set_zlabel('Z (m)')
701
702     # Fix aspect ratio manually
703     max_radius = np.max(best_radii)
704     max_height = np.max(z_coords)
705     ax.set_xlim([-max_radius, max_radius])
706     ax.set_ylim([-max_radius, max_radius])
707     ax.set_zlim([0, max_height])
708     ax.set_box_aspect([1, 1, max_height / (2*max_radius)]) # Set 3D aspect ratio
709
710     wireframe_path = os.path.join(out_dir, f"{base_filename}_wireframe.png")
711     plt.savefig(wireframe_path)
712     plt.close()
713
714     # 3. Save Metrics to CSV
715     csv_path = os.path.join(base_out_dir, "summary_metrics.csv")
716     file_exists = os.path.isfile(csv_path)
717
718     with open(csv_path, mode='a', newline='') as f:
719         writer = csv.writer(f)
720         if not file_exists:
721             writer.writerow([
722                 "Scenario", "Algorithm", "TimeTaken_sec", "FunctionEvals",
723                 "FinalArea", "FinalVolume", "TargetVolume", "VolumeError",
724                 "FinalCost", "BestRadii", "BestHeights"
725             ])
726         writer.writerow([
727             scenario_name,
728             algo_name,
729             metrics.get('time_taken', 0.0),
730             metrics.get('n_fevals', 0),
731             final_area,
732             final_vol,
733             target_vol,
734             vol_error,
735             final_cost,
736             str(best_radii.tolist()),
737             str(best_heights.tolist())
```

```
738         ])
739
740     print(f"\nSaved artifacts to {out_dir}:")
741     print(f"_{base_filename}_convergence.png")
742     print(f"_{base_filename}_profile.png")
743     print(f"_{summary_metrics.csv}(Appended)")
744
745     print(f"\n---Results---")
746     print(f"Time: {metrics.get('time_taken', 0.0):.4f}s")
747     print(f"FuncEvals: {metrics.get('n_fevals', 0)}")
748     print(f"Final Area: {final_area:.2f}m2")
749     print(f"Final Vol: {final_vol:.2f}m3 (Target: {target_vol:.2f})")
750     print(f"Vol Error: {vol_error:.2f}m3")
751     print(f"Final Cost: {final_cost:.2f}")
752
753     return best_radii, best_heights, history, metrics
754
755
756
757 def generate_comparison_convergence(case_num, histories_dict, out_dir):
758     """
759     Plots convergence histories for all algorithms on a single figure for a given case.
760
761     Args:
762         case_num: The scenario number.
763         histories_dict: dict of {algo_name: history_list}.
764         out_dir: Output directory path.
765     """
766     import matplotlib.pyplot as plt
767
768     if not histories_dict:
769         return
770
771     plt.figure(figsize=(10, 6))
772     colors = {'Sa': 'red', 'Bfgs': 'blue', 'Pso': 'green'}
773     for algo_name, history in histories_dict.items():
774         if history is not None and len(history) > 1:
775             plt.plot(history, label=algo_name, color=colors.get(algo_name, 'black'))
776
777     plt.title(f"Convergence Comparison: Case {case_num}")
778     plt.xlabel("Iteration")
779     plt.ylabel("Cost J(x)")
780     plt.yscale('log')
781     plt.grid(True)
782     plt.legend()
783     path = os.path.join(out_dir, f"Case{case_num}_comparison_convergence.png")
784     plt.savefig(path)
785     plt.close()
786     print(f"Saved: {path}")
787
788
```

```
789 def generate_comparison_table(out_dir):
790     """
791     Reads summary_metrics.csv and generates a formatted comparison table as a PNG image.
792     """
793     import csv
794     import matplotlib.pyplot as plt
795
796     csv_path = os.path.join(out_dir, "summary_metrics.csv")
797     if not os.path.isfile(csv_path):
798         print("No summary_metrics.csv found.")
799         return
800
801     with open(csv_path, 'r') as f:
802         reader = csv.reader(f)
803         rows = list(reader)
804
805     if len(rows) < 2:
806         print("Not enough data for comparison table.")
807         return
808
809     header = rows[0]
810     data = rows[1:]
811
812     # Build table with key columns only
813     table_header = ["Scenario", "Algorithm", "Time(s)", "F.Evals", "FinalArea", "Vol_
Error", "FinalCost"]
814     table_data = []
815     for row in data:
816         table_data.append([
817             row[0],                                # Scenario
818             row[1],                                # Algorithm
819             f"{float(row[2]):.4f}",                 # TimeTaken_sec
820             row[3],                                # FunctionEvals
821             f"{float(row[4]):.2f}",                 # FinalArea
822             f"{float(row[7]):.2f}",                 # VolumeError
823             f"{float(row[8]):.2f}"                  # FinalCost
824         ])
825
826     fig, ax = plt.subplots(figsize=(16, max(4, len(table_data) * 0.4 + 1)))
827     ax.axis('off')
828     ax.set_title("Optimization Results Comparison Table", fontsize=14, fontweight='bold',
829                 , pad=20)
830
831     table = ax.table(
832         cellText=table_data,
833         colLabels=table_header,
834         cellLoc='center',
835         loc='center'
836     )
837     table.auto_set_font_size(False)
838     table.set_fontsize(8)
```

```
838     table.scale(1, 1.3)
839
840     # Color the header row
841     for j in range(len(table_header)):
842         table[(0, j)].set_facecolor('#4472C4')
843         table[(0, j)].set_text_props(color='white', fontweight='bold')
844
845     # Alternate row shading
846     for i in range(1, len(table_data) + 1):
847         color = '#D9E2F3' if i % 2 == 0 else '#FFFFFF'
848         for j in range(len(table_header)):
849             table[(i, j)].set_facecolor(color)
850
851     table_path = os.path.join(out_dir, "comparison_table.png")
852     plt.savefig(table_path, bbox_inches='tight', dpi=150)
853     plt.close()
854     print(f"Saved comparison table: {table_path}")
855
856
857 if __name__ == "__main__":
858     import os
859
860     problem = CoolingTowerProblem()
861
862     # Create output directory and clear old CSV to avoid stale data
863     out_dir = os.path.join(os.path.dirname(__file__), "output")
864     os.makedirs(out_dir, exist_ok=True)
865     csv_path = os.path.join(out_dir, "summary_metrics.csv")
866     if os.path.isfile(csv_path):
867         os.remove(csv_path)
868
869     print("Running all 8 cases across all 3 algorithms...")
870
871     # List of all available optimizer classes
872     optimizers = [SaOptimizer, BfgsOptimizer, PsoOptimizer]
873
874     # Scenarios
875     scenarios = [
876         (1, "Mandatory_Ref_Case"),
877         (2, "Variable_Heights"),
878         (3, "Full_Freedom"),
879         (4, "Tight_Waist"),
880         (5, "Cylindrical_Start"),
881         (6, "High_Volume"),
882         (7, "Restricted_Bounds"),
883         (8, "Hyperbolic_Fit")
884     ]
885
886     # Collect convergence histories per case for comparison plots
887     # Structure: {case_num: {algo_name: history}}
888     all_histories = {case_num: {} for case_num, _ in scenarios}
```

```
889
890 for opt_class in optimizers:
891     print(f"\n\n{' '*60}")
892     print(f"***_Starting_Runs_for_{opt_class.__name__}_***")
893     print(f"{' '*60}\n")
894
895     algo_name = opt_class.__name__.replace('Optimizer', '')
896     case_1_best_radaii = None
897
898     # Configure algorithm parameters for good convergence
899     kwargs = {}
900     if algo_name == 'Pso':
901         kwargs = {
902             'correction_mode': 'correct', # smooth landscape for swarm
903         }
904     elif algo_name == 'Sa':
905         kwargs = {'T_initial': 10000, 'max_iter': 15000}
906
907     for case_num, scenario_name in scenarios:
908         if case_num == 2:
909             if case_1_best_radaii is None:
910                 print("Skipping_Case_2:_Requires_radaii_from_Case_1.")
911                 continue
912             best_radaii, best_heights, history, metrics = run_scenario(
913                 opt_class, problem, case_num, scenario_name, fixed_radaii=
914                     case_1_best_radaii, **kwargs
915             )
916         else:
917             best_radaii, best_heights, history, metrics = run_scenario(
918                 opt_class, problem, case_num, scenario_name, **kwargs
919             )
920
921         if case_num == 1 and best_radaii is not None:
922             case_1_best_radaii = best_radaii
923
924         # Store history for comparison convergence plots
925         all_histories[case_num][algo_name] = history
926
927     # Generate comparison convergence plots (all 3 algos on one figure per case)
928     print("\n\nGenerating_comparison_convergence_plots...")
929     for case_num, _ in scenarios:
930         generate_comparison_convergence(case_num, all_histories[case_num], out_dir)
931
932     # Generate comparison table
933     print("\n\nGenerating_comparison_table...")
934     generate_comparison_table(out_dir)
935
936     print("\n\nAll_runs_complete!_Check_the_'output'_folder_for_all_artifacts.")
```


Appendix A.C BFGS Implementation — bfgs.py

The BFGS quasi-Newton implementation, including inverse Hessian approximation updates, Armijo line search, and numerical gradient computation via finite differences.

Listing 3: BFGS Implementation (bfgs.py)

```
1 import numpy as np
2 from typing import Callable, Tuple, List, Optional, Sequence
3
4 def shape_guidance_penalty(
5     objective_func, x, smooth_weight=0.0, slope_weight=0.0,
6     slope_min=0.20, slope_max=10.0, active_cases=None
7 ):
8     """Fallback dummy function if true shapes guidance is missing."""
9     return 0.0
10
11
12 def bfgs_optimize(
13     objective_func: Callable[[np.ndarray], float],
14     initial_guess: np.ndarray,
15     **kwargs
16 ) -> Tuple[np.ndarray, float, List[float], int]:
17     """
18     Martin's BFGS implementation.
19
20     Args:
21         objective_func: The function to minimize. Takes an array of variables and
22             returns the cost.
23         initial_guess: The starting point for the algorithm.
24
25     Returns:
26         best_position (np.ndarray): The best combination of parameters found.
27         best_cost (float): The cost evaluated at the best position.
28         history (List[float]): The history of the best cost over iterations.
29         n_fevals (int): Total number of times the objective function was evaluated.
30
31     """
32     max_iter = int(kwargs.get("max_iter", 300))
33     tol_grad = float(kwargs.get("tol_grad", kwargs.get("gtol", 1e-6)))
34     tol_step = float(kwargs.get("tol_step", 1e-10))
35     fd_eps = float(kwargs.get("fd_eps", kwargs.get("epsilon", 1e-6)))
36     c1 = float(kwargs.get("c1", 1e-4))
37     backtrack_beta = float(kwargs.get("backtrack_beta", 0.5))
38     min_step = float(kwargs.get("min_step", 1e-12))
39     verbose = bool(kwargs.get("verbose", False))
40     reset_hessian_on_bad_curvature = bool(kwargs.get("reset_hessian_on_bad_curvature",
41         True))
42     use_shape_guidance = bool(kwargs.get("use_shape_guidance", True))
43     # Stronger shape guidance defaults to discourage flat vertical stacks
44     # while keeping flexibility at the fixed-end transitions.
```

```
42 shape_smooth_weight = float(kwargs.get("shape_smooth_weight", 0.0))
43 shape_slope_weight = float(kwargs.get("shape_slope_weight", 5e4))
44 shape_slope_min = float(kwargs.get("shape_slope_min", 0.20))
45 shape_slope_max = float(kwargs.get("shape_slope_max", 10.0))
46 shape_active_cases = kwargs.get("shape_active_cases", (1, 2, 3, 4, 5, 6, 7))
47
48 x = np.asarray(initial_guess, dtype=float).reshape(-1)
49 n_vars = x.size
50
51 bounds = kwargs.get("bounds")
52 if bounds is None and kwargs.get("use_problem_bounds", True):
53     bounds = _infer_bounds_from_objective(objective_func)
54 lb, ub = _parse_bounds(bounds, n_vars)
55 has_bounds = lb is not None
56
57 if has_bounds:
58     x = _project_box(x, lb, ub)
59
60 n_fevals = 0
61 best_x = x.copy()
62 best_cost = np.inf
63
64 def evaluate(z: np.ndarray) -> float:
65     nonlocal n_fevals, best_cost, best_x
66     z = np.asarray(z, dtype=float)
67     if has_bounds:
68         z = _project_box(z, lb, ub)
69
70     cost = float(objective_func(z))
71     if use_shape_guidance:
72         cost += shape_guidance_penalty(
73             objective_func=objective_func,
74             x=z,
75             smooth_weight=shape_smooth_weight,
76             slope_weight=shape_slope_weight,
77             slope_min=shape_slope_min,
78             slope_max=shape_slope_max,
79             active_cases=shape_active_cases
80         )
81
82     if not np.isfinite(cost):
83         cost = np.finfo(float).max
84     n_fevals += 1
85     if cost < best_cost:
86         best_cost = cost
87         best_x = z.copy()
88     return cost
89
90 def numerical_gradient(z: np.ndarray) -> np.ndarray:
91     grad = np.zeros_like(z, dtype=float)
92     for i in range(n_vars):
```

```
93         step = fd_eps * max(1.0, abs(z[i]))
94         z_plus = z.copy()
95         z_minus = z.copy()
96         z_plus[i] += step
97         z_minus[i] -= step
98
99         if has_bounds:
100             z_plus = _project_box(z_plus, lb, ub)
101             z_minus = _project_box(z_minus, lb, ub)
102
103         denom = z_plus[i] - z_minus[i]
104         if abs(denom) < 1e-15:
105             grad[i] = 0.0
106             continue
107
108         f_plus = evaluate(z_plus)
109         f_minus = evaluate(z_minus)
110         grad[i] = (f_plus - f_minus) / denom
111
112     return grad
113
114 def armijo_backtracking(z: np.ndarray, p: np.ndarray, fz: float, g: np.ndarray):
115     alpha = 1.0
116     g_tp = float(np.dot(g, p))
117
118     # Ensure descent direction.
119     if g_tp >= 0.0:
120         p = -g
121         g_tp = -float(np.dot(g, g))
122
123     while alpha >= min_step:
124         z_new = z + alpha * p
125         if has_bounds:
126             z_new = _project_box(z_new, lb, ub)
127         f_new = evaluate(z_new)
128         if f_new <= fz + c1 * alpha * g_tp:
129             return alpha, z_new, f_new
130         alpha *= backtrack_beta
131
132     z_new = z + min_step * p
133     if has_bounds:
134         z_new = _project_box(z_new, lb, ub)
135     return min_step, z_new, evaluate(z_new)
136
137 f_x = evaluate(x)
138 g = numerical_gradient(x)
139 inv_hessian = np.eye(n_vars, dtype=float)
140
141 history: List[float] = [float(best_cost)]
142
143 for k in range(1, max_iter + 1):
```

```
144     grad_norm = float(np.linalg.norm(g))
145     if grad_norm < tol_grad:
146         break
147
148     if not np.all(np.isfinite(inv_hessian)) or np.linalg.norm(inv_hessian, ord=np.
149         inf) > 1e10:
150         inv_hessian = np.eye(n_vars, dtype=float)
151     direction = -inv_hessian @ g
152     alpha, x_new, f_new = armijo_backtracking(x, direction, f_x, g)
153     s = x_new - x
154
155     if float(np.linalg.norm(s)) < tol_step:
156         x, f_x = x_new, f_new
157         history.append(float(best_cost))
158         break
159
160     g_new = numerical_gradient(x_new)
161     y = g_new - g
162     y_s = float(np.dot(y, s))
163     curvature_scale = float(np.linalg.norm(y) * np.linalg.norm(s))
164     strong_curvature = y_s > max(1e-8, 1e-6 * curvature_scale)
165
166     if strong_curvature and np.isfinite(y_s):
167         rho = 1.0 / y_s
168         if not np.isfinite(rho) or rho > 1e10:
169             if reset_hessian_on_bad_curvature:
170                 inv_hessian = np.eye(n_vars, dtype=float)
171             else:
172                 eye = np.eye(n_vars, dtype=float)
173                 v = eye - rho * np.outer(s, y)
174                 try:
175                     with np.errstate(over="raise", divide="raise", invalid="raise"):
176                         h_candidate = v @ inv_hessian @ v.T + rho * np.outer(s, s)
177                         if np.all(np.isfinite(h_candidate)):
178                             inv_hessian = h_candidate
179                         elif reset_hessian_on_bad_curvature:
180                             inv_hessian = np.eye(n_vars, dtype=float)
181                     except FloatingPointError:
182                         if reset_hessian_on_bad_curvature:
183                             inv_hessian = np.eye(n_vars, dtype=float)
184                 elif reset_hessian_on_bad_curvature:
185                     inv_hessian = np.eye(n_vars, dtype=float)
186
187     if not np.all(np.isfinite(inv_hessian)) and reset_hessian_on_bad_curvature:
188         inv_hessian = np.eye(n_vars, dtype=float)
189
190     x, f_x, g = x_new, f_new, g_new
191     history.append(float(best_cost))
192
193     if verbose and (k == 1 or k % 25 == 0):
194         print(
```

```
194         f"[BFGS]_Iter_{k:4d}/{max_iter}_|_Best_cost:_{best_cost:.6f}|_"
195         f"||g||:_{grad_norm:.3e}|_alpha:_{alpha:.3e}"
196     )
197
198     return best_x.copy(), float(best_cost), history, int(n_fevals)
199
200
201 def _project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
202     return np.minimum(np.maximum(x, lb), ub)
203
204
205 def _parse_bounds(
206     bounds: Optional[Sequence[Tuple[Optional[float], Optional[float]]]],
207     n_vars: int
208 ) -> Tuple[Optional[np.ndarray], Optional[np.ndarray]]:
209     if bounds is None:
210         return None, None
211     if len(bounds) != n_vars:
212         raise ValueError(f"Expected_{n_vars}_bounds, got_{len(bounds)}")
213
214     lb = np.array(
215         [-np.inf if b[0] is None else float(b[0]) for b in bounds],
216         dtype=float
217     )
218     ub = np.array(
219         [np.inf if b[1] is None else float(b[1]) for b in bounds],
220         dtype=float
221     )
222     if np.any(lb > ub):
223         raise ValueError("Invalid_bounds:_lower_bound_greater_than_upper_bound.")
224     return lb, ub
225
226
227 def _infer_bounds_from_objective(
228     objective_func: Callable[[np.ndarray], float]
229 ) -> Optional[Sequence[Tuple[float, float]]]:
230     """
231     Try to infer bounds from solver wrapper:
232     objective_func -> BaseOptimizer._objective_func -> {problem, case_num}.
233     """
234     owner = getattr(objective_func, "__self__", None)
235     if owner is None:
236         return None
237
238     problem = getattr(owner, "problem", None)
239     case_num = getattr(owner, "case_num", None)
240     if problem is None or case_num is None or not hasattr(problem, "get_bounds"):
241         return None
242
243     try:
244         return problem.get_bounds(case_num)
```

```
245     except Exception:  
246         return None
```

Appendix A.D PSO Implementation — pso.py

The Particle Swarm Optimisation implementation, including swarm initialisation, velocity updates, boundary handling, and convergence tracking.

Listing 4: Particle Swarm Optimisation (pso.py)

```
1  """
2  Subject: PSO algorithm
3  Inheritance: Particle_Swarm_Optimization.py from Task 3
4  @author: Cl  mence-Philom  ne HINOT, John HOARAU, Copilot AI
5  @date: 10/02/2026
6  """
7
8  # ***** IMPORTATION *****
9
10 import numpy as np
11 from typing import Callable, Tuple, List
12
13 # pymoo engine -- mirrors Particle_Swarm_Optimization.py
14 from pymoo.algorithms.soo.nonconvex.pso import PSO
15 from pymoo.core.problem import Problem
16 from pymoo.core.callback import Callback
17 from pymoo.core.termination import Termination
18 from pymoo.operators.sampling.lhs import LHS
19 from pymoo.optimize import minimize
20
21 # ***** CONVERGENCE CALLBACK *****
22
23 class MyCallback(Callback):
24     """
25     Tracks the global best cost after each generation.
26     Mirrors MyCallback in Particle_Swarm_Optimization_multi_obj.py.
27
28     data["best_f1"]: global best cost per generation, returned as history
29     solver.py can plot the convergence curve.
30     data["F"]: all particle costs per generation.
31     data["n_nds"]: number of particles that matched the global best.
32     """
33
34     def __init__(self):
35         super().__init__()
36         self.data["best_f1"] = []
37         self.data["F"] = []
38         self.data["n_nds"] = []
39
40     def notify(self, algorithm):
41         F = algorithm.pop.get("F") # shape (pop_size, 1)
42         best = float(F.min())
43         self.data["F"].append(F.copy())
```

```
44         self.data["best_f1"].append(best)
45         self.data["n_nds"].append(int((F == best).sum()))
46
47     # ***** PYMOO PROBLEM WRAPPER *****
48
49     class _FuncProblem(Problem):
50         """
51         Wraps a plain callable f(x) -> float into a pymoo Problem so that
52         pymoo's PSO engine can be used without changing solver.py's interface.
53         """
54
55         def __init__(self, objective_func: Callable, bounds: list):
56             lb = np.array([b[0] for b in bounds], dtype=float)
57             ub = np.array([b[1] for b in bounds], dtype=float)
58             super().__init__(
59                 n_var=len(bounds),
60                 n_obj=1,
61                 xl=lb,
62                 xu=ub,
63             )
64             self._func = objective_func
65             self.n_fevals = 0
66
67         def _evaluate(self, X, out, *args, **kwargs):
68             """Evaluate all particles in the current generation."""
69             F = np.array([self._func(X[i]) for i in range(len(X))], dtype=float)
70             self.n_fevals += len(X)
71             out["F"] = F.reshape(-1, 1)
72
73     # ***** STAGNATION-AWARE TERMINATION *****
74
75     class StagnationTermination(Termination):
76         """
77         Stops when EITHER:
78         - max_gen generations are reached, OR
79         - the global best has not improved by more than 'tol' over the
80         - last 'stagnation_window' generations (early stopping).
81         """
82
83         def __init__(self, max_gen: int, stagnation_window: int = 50, tol: float = 1e-6):
84             super().__init__()
85             self.max_gen = max_gen
86             self.stagnation_window = stagnation_window
87             self.tol = tol
88             self._history = []
89
90         def _update(self, algorithm):
91             F = algorithm.pop.get("F")
92             self._history.append(float(F.min()))
93
94             if algorithm.n_gen >= self.max_gen:
```



```
95         return 1.0
96
97     if len(self._history) >= self.stagnation_window:
98         window = self._history[-self.stagnation_window:]
99         improvement = window[0] - window[-1]
100         if improvement < self.tol:
101             return 1.0
102
103     return algorithm.n_gen / self.max_gen
104
105 # ***** PARTICLE SWARM OPTIMISATION *****
106
107 """
108 Interface expected by solver.py (PsoOptimizer):
109
110 best_x, best_cost, history, n_fevals = particle_swarm(
111     objective_func=self._objective_func,
112     bounds=bounds,
113     num_particles=200,
114     max_iter=750,
115     ...
116 )
117
118 history is returned as an empty list, only the final result is needed.
119
120 Parameters
121 -----
122 objective_func: Callable[[x: np.ndarray] -> float], function to minimise.
123 bounds: list of (min, max) tuples, one tuple per decision variable.
124 num_particles: int, swarm size.
125 max_iter: int, number of generations.
126 w: float, inertia weight (pymoo adaptive=True decays this).
127 c1: float, cognitive coefficient.
128 c2: float, social coefficient.
129 max_velocity_rate: float, v_max = max_velocity_rate * (ub - lb).
130 seed: int | None, random seed for reproducibility.
131 verbose: bool, print pymoo progress each generation.
132 **kwargs: ignored, absorbs extra keys from solver.py.
133
134 Returns
135 -----
136 best_position: np.ndarray, best parameter vector found.
137 best_cost: float, objective value at best position.
138 history: [], empty list (history not tracked).
139 n_fevals: int, total objective-function evaluations.
140 """
141
142 def particle_swarm(
143     objective_func: Callable[[np.ndarray], float],
144     bounds: List[Tuple[float, float]],
145     num_particles: int = 200,
```

```
146     max_iter: int = 50000,
147     stagnation_window: int = 50,
148     tol: float = 1e-6,
149     w: float = 0.9,
150     c1: float = 2.0,
151     c2: float = 2.0,
152     max_velocity_rate: float = 0.3,
153     seed: int = 10,
154     verbose: bool = False,
155     **kwargs,
156 ) -> Tuple[np.ndarray, float, List[float], int]:
157
158     try:
159         # ----- Wrap objective into a pymoo Problem -----
160         problem = _FuncProblem(objective_func, bounds)
161
162         # ----- Configure PSO -- mirrors PSO_single_objective() -----
163         algorithm = PSO(
164             pop_size          = num_particles,
165             w                 = w,
166             c1                = c1,
167             c2                = c2,
168             max_velocity_rate = max_velocity_rate,
169             adaptive          = True,
170             initial_velocity  = "random",
171             pertube_best      = True,
172         )
173
174         # ----- Convergence callback -----
175         callback = MyCallback()
176
177         # ----- Stagnation-aware termination -----
178         termination = StagnationTermination(
179             max_gen          = max_iter,
180             stagnation_window = stagnation_window,
181             tol              = tol,
182         )
183
184         # ----- Run optimisation -----
185         result = minimize(
186             problem,
187             algorithm,
188             termination = termination,
189             seed        = seed,
190             verbose      = verbose,
191             callback     = callback,
192         )
193
194         # Store callback on result for external access
195         result.callback = callback
196
```

```
197         history = callback.data["best_f1"] # one float per generation
198
199         return result.X.flatten(), float(result.F.flatten()[0]), history, problem.
200             n_fevals
201
202     except Exception as e:
203         print(f"Error during PSO optimisation: {e}")
204         raise
```

Appendix A.E SA Implementation — sa.py

The Simulated Annealing implementation, including cooling schedule, neighbour generation, and Metropolis acceptance criterion.

Listing 5: Simulated Annealing (sa.py)

```
1 import numpy as np
2 from typing import Callable, Tuple, List
3
4
5
6 def simulated_annealing(
7     objective_func: Callable[[np.ndarray], float],
8     initial_guess: np.ndarray,
9     **kwargs
10 ) -> Tuple[np.ndarray, float, List[float], int]:
11     """
12     Pierre's Simulated Annealing implementation.
13
14     Args:
15         objective_func: The function to minimize. Takes an array of variables and
16             returns the cost.
17         initial_guess: The starting point for the algorithm.
18
19     Returns:
20         best_position (np.ndarray): The best combination of parameters found.
21         best_cost (float): The cost evaluated at the best position.
22         history (List[float]): The history of the best cost over iterations.
23         n_fevals (int): Total number of times the objective function was evaluated.
24     """
25
26     # ----- Extract parameters from kwargs -----
27     bounds = kwargs.get('bounds', None)
28     T_initial = kwargs.get('T_initial', 5000.0)
29     T_final = kwargs.get('T_final', 1e-8)
30     alpha = kwargs.get('alpha', 0.995)
31     max_iter = kwargs.get('max_iter', 5000)
32     n_neighbors = kwargs.get('n_neighbors', 3)
33     step_size = kwargs.get('step_size', 0.15)
34     adaptive_step = kwargs.get('adaptive_step', True)
35     seed = kwargs.get('seed', None)
36     verbose = kwargs.get('verbose', False)
37
38     if seed is not None:
39         np.random.seed(seed)
40
41     x = np.array(initial_guess, dtype=float)
42     n_vars = len(x)
```

```
43     # ----- Bounds handling -----
44     if bounds is not None:
45         lb = np.array([b[0] for b in bounds], dtype=float)
46         ub = np.array([b[1] for b in bounds], dtype=float)
47         rng = ub - lb # search-space width per dimension
48     else:
49         lb = None
50         ub = None
51         rng = np.abs(x) + 1.0 # fallback scaling
52
53     # ----- Initial evaluation -----
54     current_cost = objective_func(x)
55     n_fevals = 1
56
57     best_pos = x.copy()
58     best_cost = current_cost
59
60     history: List[float] = [best_cost]
61     log_every = max(1, max_iter // 10)
62
63     # ----- Temperature schedule -----
64     T = T_initial
65
66     # ----- Main optimisation loop -----
67     for iteration in range(max_iter):
68
69         # -- Adaptive step size: decreases with temperature --
70         # Allows large exploration at high T, fine-tuning at low T
71         if adaptive_step:
72             t_ratio = np.sqrt(T / T_initial)
73             t_ratio = np.clip(t_ratio, 0.01, 1.0)
74             step_current = step_size * t_ratio
75         else:
76             step_current = step_size
77
78         # -- Generate and evaluate neighbors --
79         # Sample n_neighbors candidates, keep the best for acceptance test
80         best_neighbor_x = None
81         best_neighbor_cost = np.inf
82
83         for _ in range(n_neighbors):
84             # Gaussian perturbation scaled by search range
85             perturbation = np.random.randn(n_vars) * step_current * rng
86             neighbor_x = x + perturbation
87
88             # Clipping boundary: project back onto bounds
89             if lb is not None and ub is not None:
90                 neighbor_x = np.clip(neighbor_x, lb, ub)
91
92             neighbor_cost = objective_func(neighbor_x)
93             n_fevals += 1
```

```
94
95     if neighbor_cost < best_neighbor_cost:
96         best_neighbor_cost = neighbor_cost
97         best_neighbor_x = neighbor_x.copy()
98
99     # -- Metropolis acceptance criterion --
100     delta_E = best_neighbor_cost - current_cost
101
102     if delta_E < 0:
103         # Better solution: always accept
104         accept = True
105     else:
106         # Worse solution: accept with probability exp(-ΔE/T)
107         acceptance_prob = np.exp(-delta_E / T)
108         accept = np.random.rand() < acceptance_prob
109
110     if accept and best_neighbor_x is not None:
111         x = best_neighbor_x
112         current_cost = best_neighbor_cost
113
114         # Update global best
115         if current_cost < best_cost:
116             best_cost = current_cost
117             best_pos = x.copy()
118
119     history.append(best_cost)
120
121     # -- Cooling: geometric decay --
122     T = alpha * T
123
124     # -- Early stopping --
125     if T < T_final:
126         if verbose:
127             print(f"[[SA]] Early stop at iter_{iteration+1}: T={T:.2e} < T_final={T_final:.2e}")
128         break
129
130     # -- Verbose output --
131     if verbose and (iteration + 1) % log_every == 0:
132         print(f"[[SA]] Iter_{iteration+1}>5d}/{max_iter} | "
133             f"T:{T:.2e} | Best cost:{best_cost:.6f}")
134
135     return best_pos, best_cost, history, n_fevals
```

Appendix B Detailed Results Data

Appendix B.A BFGS Results

Table 19: BFGS optimisation results across all eight study cases.

Case	Time (s)	Func Evals	Area (m ²)	Vol (m ³)	Vol Error (m ³)
1 — Ref Case	0.42	6 746	7 776.11	70 320.00	0.00
2 — Variable Heights	0.56	17 944	6 903.39	70 320.00	0.00
3 — Full Freedom	6.23	23 692	13 300.38	70 311.04	8.96
4 — Tight Waist	0.66	7 674	7 782.09	70 320.00	0.00
5 — Cylindrical Start	1.12	8 783	7 789.89	70 320.00	0.00
6 — High Volume	0.21	6 292	7 679.54	105 480.00	0.00
7 — Restricted Bounds	0.33	6 247	7 913.64	70 320.00	0.00
8 — Hyperbolic Fit	0.02	294	5 961.75	70 320.00	0.00

Appendix B.B PSO Results

Table 20: PSO optimisation results across all eight study cases.

Case	Time (s)	Func Evals	Area (m ²)	Vol (m ³)	Vol Error (m ³)
1 — Ref Case	50.55	237 600	8 053.97	70 320.00	0.00
2 — Variable Heights	3.52	23 800	7 879.19	70 320.18	0.18
3 — Full Freedom	33.97	122 400	11 467.28	70 321.09	1.09
4 — Tight Waist	101.35	359 200	8 054.57	70 320.10	0.10
5 — Cylindrical Start	66.16	237 600	8 053.97	70 320.00	0.00
6 — High Volume	2.17	15 600	15 400.98	105 478.87	1.13
7 — Restricted Bounds	9.90	48 000	8 055.59	70 319.95	0.05
8 — Hyperbolic Fit	2.83	26 200	5 695.72	70 320.00	0.00

Appendix B.C SA Results

Table 21: SA optimisation results across all eight study cases.

Case	Time (s)	Func Evals	Area (m ²)	Vol Error (m ³)
1 — Ref Case	1.28	16 540	7 781.65	0.05
2 — Variable Heights	0.51	16 540	7 855.09	0.01
3 — Full Freedom	1.79	16 540	8 775.38	0.10
4 — Tight Waist	1.38	16 540	7 794.58	0.21
5 — Cylindrical Start	1.71	16 540	7 844.38	0.74
6 — High Volume	0.51	16 540	39 172.04	0.02
7 — Restricted Bounds	1.50	16 540	7 876.03	0.10
8 — Hyperbolic Fit	0.59	16 540	5 732.76	0.02

Cooling Tower Optimisation

Task 7 (Group Report)



**Pierre BEDRUNE, Martin HARAMBAT, Clémence-Philomène HINOT,
John HOARAU**

Prepared for March 27, 2026

Abstract

Task 7 extends the optimisation study of Task 6 by introducing a structural verification stage under asymmetric wind loading. The objective is to assess the mechanical response of the optimised cooling-tower geometries when subjected to a one-directional wind pressure field. The selected geometries were imported into ABAQUS. Under a wind pressure field corresponding to a 130 km/h severe storm, the models were analysed using shell structural elements. Results show that Case 8, the most successful optimisation case from Task 6, presents the most convincing compromise between geometric efficiency and wind-load robustness, demonstrating low stress levels and limited lateral displacements. Ultimately, the results confirm that a regular, physically plausible profile is often preferable to a mechanically extreme optimum.

Contents

1	Objective	3
2	Methodology	4
2.1	Finite elements idealisation	4
2.2	Material model	4
2.3	Mesh strategy	5
2.4	Boundary conditions and loadings	6
2.5	Interpretation strategy	8
3	Results	9
3.1	Case 1 (Reference case)	9
3.2	Case 2 (Variable heights)	10
3.3	Case 3 (Full freedom)	11
3.4	Case 4 (Tight waist)	12
3.5	Case 5 (Cylindrical start)	12
3.6	Case 6 (High volume)	13
3.7	Case 7 (Restricted bounds)	14
3.8	Case 8 (Hyperbolic fit)	14
4	Discussion	16
4.1	Global remarks on the eight cases	16
	References	17

1 Objective

Task 7 extends the optimisation study of Task 6 [1] by introducing a structural verification stage under asymmetric wind loading. The objective is no longer to minimise surface area, but to assess the mechanical response of the optimised cooling-tower geometries when subjected to a one-directional wind pressure field. In accordance with the methodology developed in Task 6, the geometry retained for each case is the one associated with the lowest surface area among the optimisation methods previously benchmarked. For Cases 1, 2, 4, 5, and 6, the retained profile is the BFGS solution, whereas for Cases 3, 7, and 8 the retained profile is the SA solution.

The selected geometries were imported into ABAQUS using a Python script, ensuring a reproducible transfer from the optimisation stage to the structural simulation environment.

2 Methodology

2.1 Finite elements idealisation

The cooling towers were modelled using shell elements, which is appropriate for thin-walled civil structures whose load-bearing behaviour is governed primarily by membrane and bending actions rather than full three-dimensional stress states. The shell idealisation provides an efficient compromise between physical relevance and computational cost. It also remains fully consistent with the geometric modelling strategy adopted in Task 6, where the tower was represented as a stack of ten frustum-like sections defining a continuous hyperboloidal profile.

The shell thickness was not assumed uniform. Instead, the wall was divided into ten vertical sections, matching the geometric segmentation inherited from the optimisation problem. A stepped thickness law was introduced to reflect the expected structural logic of a natural-draught cooling tower: the three lowest sections were assigned a thickness of 500 mm, the four middle sections 300 mm, and the three upper sections 200 mm.

2.2 Material model

The material was represented as homogenised reinforced concrete. At this stage, the reinforcement was not modelled explicitly; instead, the shell was assigned equivalent isotropic elastic properties representative of a reinforced-concrete wall in the service range. The adopted values were:

- density: $\rho = 2400 \text{ kg/m}^3$ [2]
- Young's modulus: $E = 35 \text{ GPa}$ [3]
- Poisson's ratio: $\nu = 0.20$ [3]

The concrete tensile strength check and the elastic properties are defined in accordance with Eurocode 2 [3], while the density reflects the standard value from Eurocode 1 [2] (vs 2500 kg/m^3). In the ABAQUS unit system used here, these values correspond to a linear-elastic material definition with density and elastic stiffness only. No plasticity or damage law was introduced, since the present study addresses wind loading under a linear static assumption.

This modelling choice is appropriate for an initial engineering assessment of wind response, but its limitations must be stated clearly. A linear-elastic shell model in a Static, General step does not capture concrete cracking, nonlinear buckling, large deformations, dynamic wind effects, or behaviour close to ultimate failure.

2.3 Mesh strategy

The cooling tower was discretised with S4R elements, namely 4-node quadrilateral shell elements with reduced integration. This choice is appropriate because the structure behaves as a thin shell, with a response dominated by membrane action and moderate bending. The quadrilateral interpolation is well suited to the smooth curved geometry of the tower.

The use of shell elements rather than solid elements is also justified physically. The tower wall thickness remains small relative to the global height and diameter, so a shell formulation captures the dominant behaviour without the excessive computational burden associated with three-dimensional meshing.

Mesh dependence was assessed using the maximum displacement magnitude U as the principal convergence indicator, together with the total number of mesh nodes. The node count was retained as the descriptor of discretisation level, allowing a direct comparison between response stability and model size.

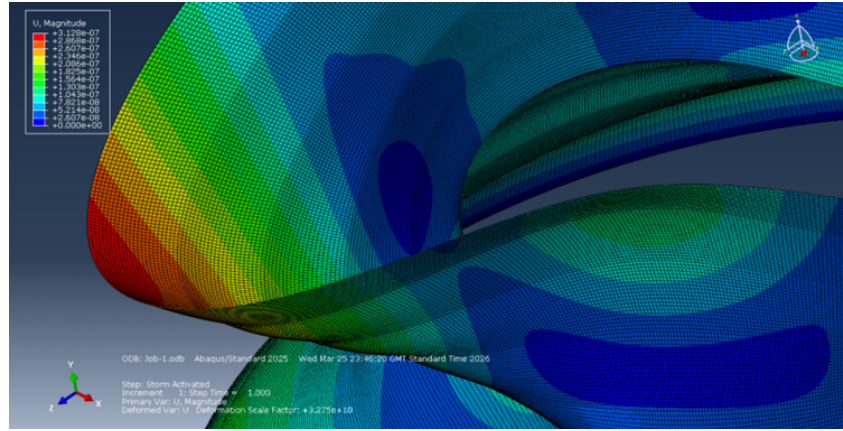


Figure 1: High resolution mesh

Mesh Size	Nodes	$U_{\max}$
12800	154	2.36E-07
6400	319	3.03E-07
3200	855	3.26E-07
1600	3680	3.11E-07
800	11532	3.12E-07
400	45133	3.13E-07
200	180290	3.13E-07

Table 1: Mesh dependence data

As the mesh was refined, the number of nodes increased rapidly, leading to a significant rise in computational cost. Beyond a mesh size of 1600, the additional refinement did not produce any meaningful improvement in the response quantities of interest.

2.4 Boundary conditions and loadings

The lower circular edge of the tower was fully fixed. The imposed boundary conditions were $U_1 = U_2 = U_3 = UR_1 = UR_2 = UR_3 = 0$. This corresponds to an encastré condition along the base ring, representing a rigid connection between the shell and the supporting foundation. Since the loading is not axisymmetric, no symmetry boundary condition was introduced.

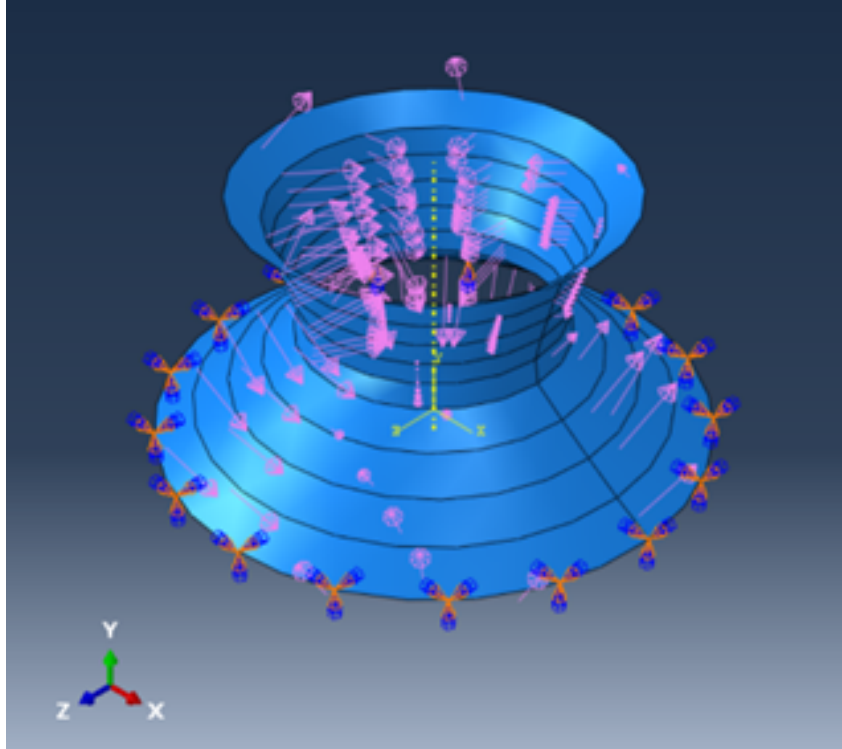


Figure 2: Boundaries and loadings

The wind action was modelled as an external pressure load applied in a Static, General step. A severe storm velocity of 130 km/h was adopted as the working hypothesis. Using an air density of $\rho_{\text{air}} = 1.225 \text{ kg/m}^3$, the corresponding dynamic pressure was taken as:

$$q = 8.0 \times 10^{-4} \text{ N/mm}^2 \quad (1)$$

Rather than applying this pressure uniformly, a continuous circumferential distribution was introduced through an ABAQUS Analytical Field, which governs the aerodynamic behaviour of hyperbolic shells [4]:

$$\text{WindField} = -0.25 + 0.75 \frac{X}{\sqrt{X^2 + Y^2 + 1.0}} + 0.30 \left(2 \left(\frac{X}{\sqrt{X^2 + Y^2 + 1.0}} \right)^2 - 1 \right) \quad (2)$$

The pressure load was then defined with magnitude $8.0 \times 10^{-4} \text{ N/mm}^2$, using the

above field as the spatial distribution. This approach is preferable to a sector-based approximation because it avoids artificial discontinuities between discrete wind regions and produces a smooth pressure variation around the shell circumference.

2.5 Interpretation strategy

Three main output quantities were considered for each case: von Mises stress S_{Mises} , displacement magnitude U , and reaction force magnitude RF.

The displacement field was used as the main global response indicator, both for mesh-sensitivity assessment and for comparing the lateral flexibility of the different tower geometries. It reflects the overall stiffness of the shell under wind loading and identifies the shapes that are most sensitive to lateral deformation.

The reaction-force field was examined at the base to evaluate how the wind-induced loads are transferred to the foundation. Although RF is not a direct design criterion, it is useful for showing whether the support reactions remain smoothly distributed or become concentrated in specific sectors, which would indicate more localised load paths.

Von Mises stress was retained as a compact scalar measure for comparing stress intensity between cases. Even though it is not the most physically representative criterion for reinforced concrete, it remains useful here as a comparative indicator of the most highly loaded regions.

For Case 1, S_{11} and S_{22} were examined because they distinguish the two principal shell directions. In a cooling-tower shell, one component is mainly associated with the meridional direction and the other with the circumferential direction. Their comparison is important because the response under asymmetric wind loading results from a combination of membrane action, circumferential bending, and meridional stress redistribution, which cannot be fully interpreted from von Mises stress alone.

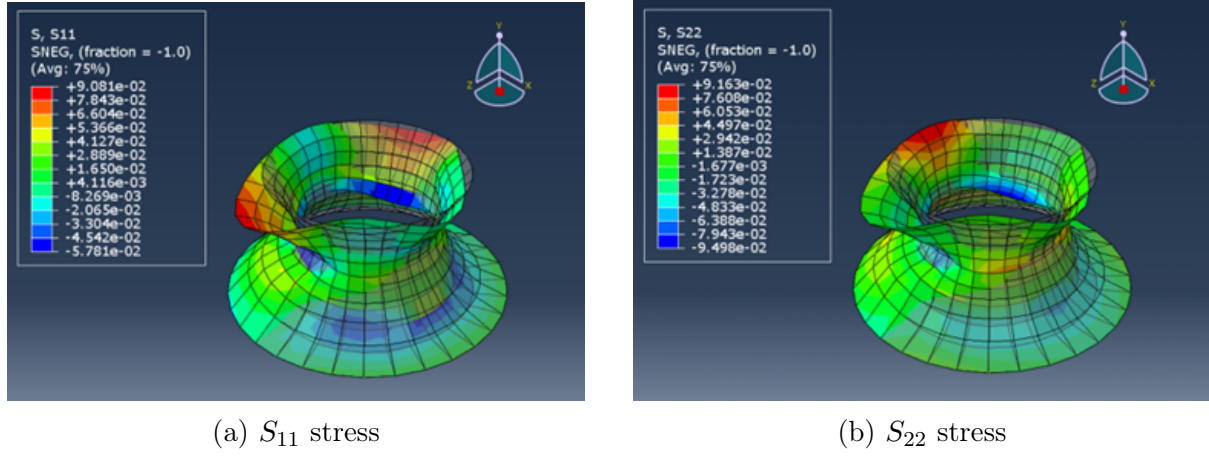


Figure 3: Directional stresses S_{11} and S_{22}

3 Results

This section presents the structural analysis results for the eight optimisation cases defined in Task 6. For each case, three primary fields are visualised: von Mises stress (S_{Mises}), displacement magnitude (U), and reaction forces at the base (RF).

3.1 Case 1 (Reference case)

The ABAQUS results show a moderate and coherent stress distribution, with a maximum von Mises stress of approximately $1.48 \times 10^{-1} \text{ N/mm}^2$. The highest stress levels appear in the upper part of the shell, near the throat-to-top transition, where curvature changes are strongest. This is consistent with the expected behaviour of a wind-loaded shell, since the throat and upper flare promote local bending amplification.

The maximum displacement magnitude is approximately 3.11×10^{-7} , indicating a very stiff global response under the applied pressure. The base reaction forces are concentrated mainly in the windward and leeward sectors, which is consistent with one-directional lateral loading.

The directional stresses provide additional insight. S_{11} ranges from about $-5.78 \times$

10^{-2} to $9.08 \times 10^{-2} \text{ N/mm}^2$, while S_{22} ranges from -9.50×10^{-2} to $9.16 \times 10^{-2} \text{ N/mm}^2$. Their comparable magnitude shows that the response is not dominated by a single stress direction. The more negative values of S_{22} suggest that circumferential compression is particularly significant in some parts of the shell, especially near the throat and lower flare.

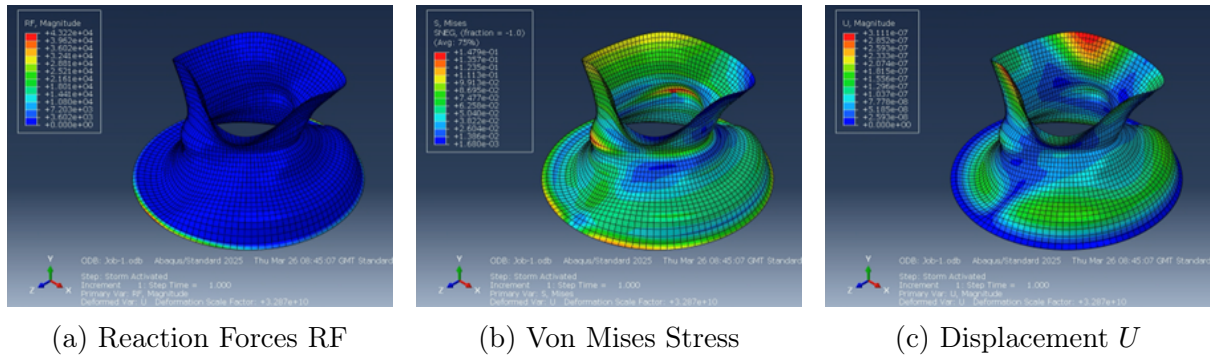


Figure 4: Structural response for Case 1

3.2 Case 2 (Variable heights)

This case shows a much stronger structural response than Case 1. The maximum von Mises stress reaches approximately $4.75 \times 10^{-1} \text{ N/mm}^2$, and the maximum displacement rises to about 1.20×10^{-6} . Both values are significantly higher than in the reference case, indicating that the geometry, while favourable in terms of area, is also more flexible and more stressed under lateral wind loading.

The stress concentrations remain mainly located near the upper shell and throat region, but they are more pronounced than in Case 1. This suggests that height redistribution increases the shell sensitivity to asymmetric pressure. The maximum base reaction also increases to approximately 4.75×10^4 , confirming a stronger global load transfer.

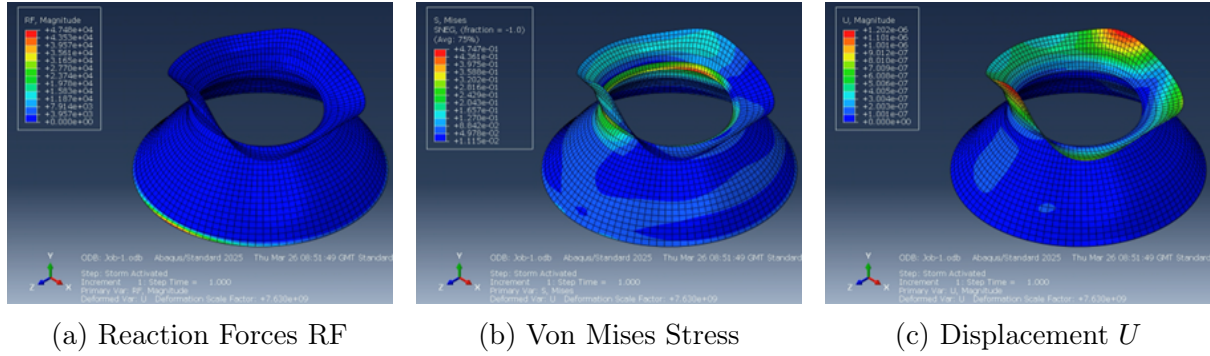


Figure 5: Structural response for Case 2

3.3 Case 3 (Full freedom)

Among the first four cases, Case 3 shows the most severe response. The maximum von Mises stress reaches approximately $5.98 \times 10^{-1} \text{ N/mm}^2$, the displacement magnitude rises to about 2.26×10^{-6} , and the maximum base reaction reaches approximately 7.52×10^4 . These values are clearly higher than those of Cases 1, 2, and 4.

This result is mechanically plausible. Because Case 3 corresponds to the least constrained optimisation problem, the resulting shell is more slender and more structurally extreme. The higher stresses and displacements indicate lower lateral stiffness and greater sensitivity to wind.

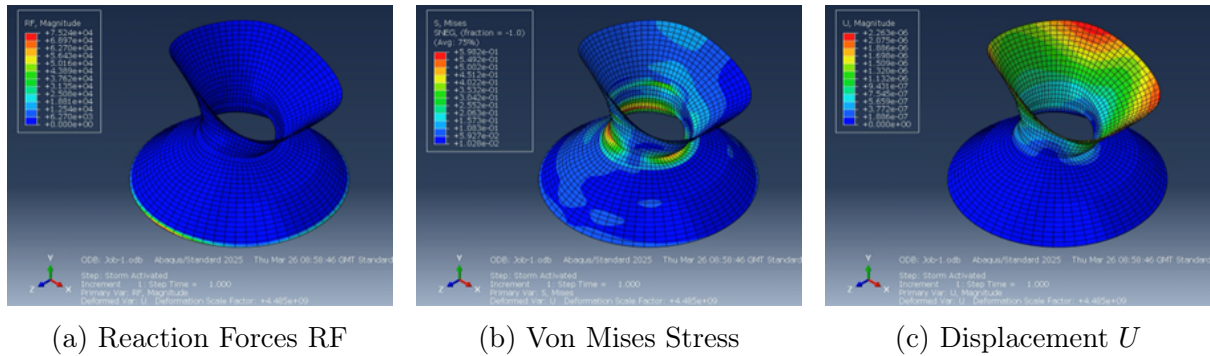


Figure 6: Structural response for Case 3

3.4 Case 4 (Tight waist)

Structurally, Case 4 behaves very similarly to Case 1. The maximum von Mises stress is approximately $1.50 \times 10^{-1} \text{ N/mm}^2$, and the maximum displacement is approximately 3.37×10^{-7} , both close to the reference case. The maximum base reaction, approximately 4.15×10^4 , is also slightly lower than in Case 1.

This similarity is expected. In Task 6, Case 4 differed from Case 1 mainly by a tighter waist constraint, while the final optimised shape remained in the same geometric family.

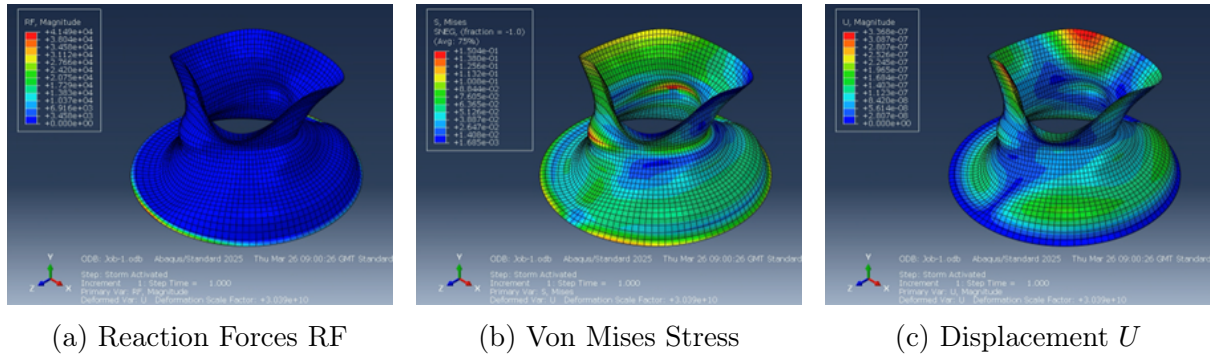


Figure 7: Structural response for Case 4

3.5 Case 5 (Cylindrical start)

The ABAQUS results confirm this. The maximum von Mises stress is approximately $1.52 \times 10^{-1} \text{ N/mm}^2$, the maximum displacement is approximately 2.19×10^{-7} , and the maximum base reaction is approximately 4.10×10^4 . These values place Case 5 in the same low-response group as Cases 1 and 4.

The stress distribution remains moderate, with the main concentrations still located near the throat and upper transition. The relatively low displacement confirms that an unfavourable optimisation starting point does not necessarily lead to a poor structural result if the final geometry is regular.

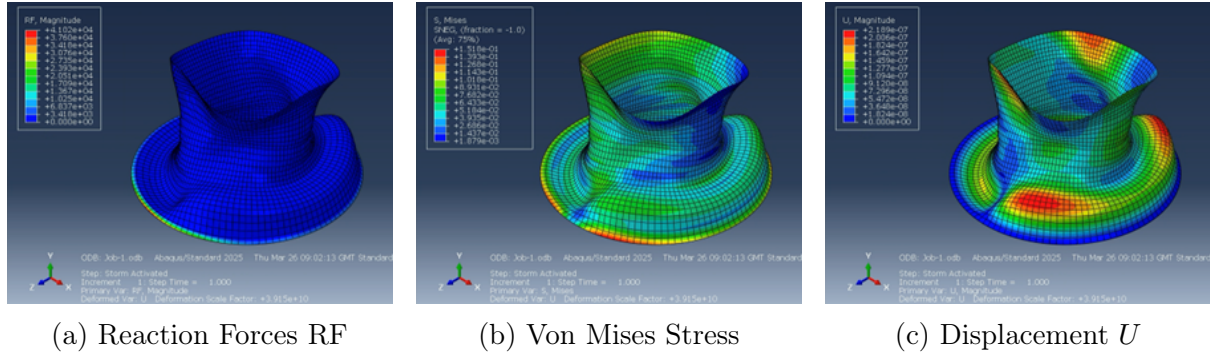


Figure 8: Structural response for Case 5

3.6 Case 6 (High volume)

The structural response remains moderate despite the increased volume. The maximum von Mises stress is approximately $1.30 \times 10^{-1} \text{ N/mm}^2$, the maximum displacement is approximately 1.42×10^{-7} , and the maximum base reaction is approximately 4.63×10^4 . Compared with Cases 1, 4, and 5, the displacement is even lower.

This behaviour is mechanically plausible. The larger enclosed volume leads to a broader and smoother shell with a less severe throat transition, which reduces local bending effects. Case 6 therefore shows that a larger tower does not necessarily lead to a worse structural response.

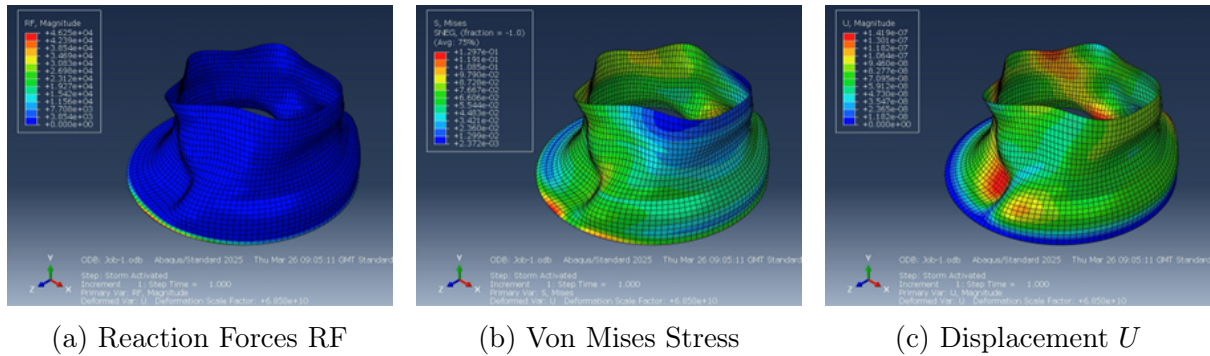
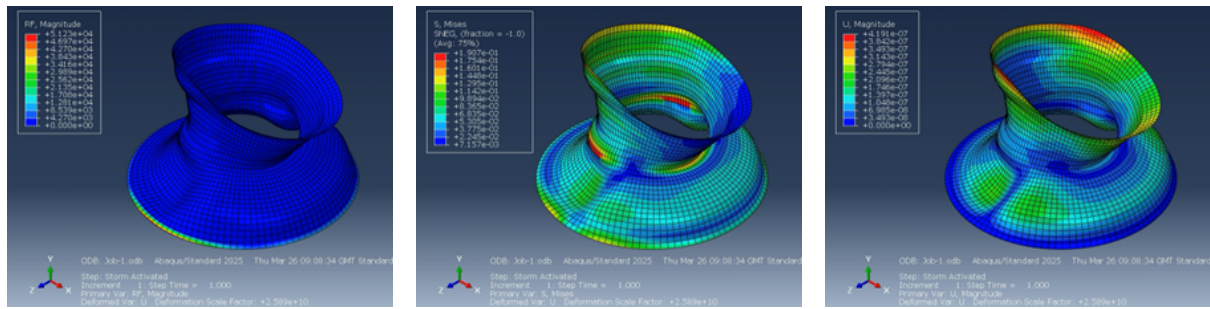


Figure 9: Structural response for Case 6

3.7 Case 7 (Restricted bounds)

Under wind loading, Case 7 shows a stronger response than Cases 1, 4, 5, and 6. The maximum von Mises stress reaches approximately $1.91 \times 10^{-1} \text{ N/mm}^2$, the maximum displacement rises to about 4.19×10^{-7} , and the maximum base reaction reaches approximately 5.12×10^4 . These values remain below those of Cases 2 and 3, but clearly place Case 7 in an intermediate category.

This is consistent with the optimisation framework. The restricted bounds reduce the freedom to obtain the most favourable curvature distribution, so the final shell remains smooth but is less efficient structurally than the best regular hyperboloidal cases.



(a) Reaction Forces RF

(b) Von Mises Stress

(c) Displacement U

Figure 10: Structural response for Case 7

3.8 Case 8 (Hyperbolic fit)

Structurally, Case 8 combines low stress levels with a regular deformation pattern. The maximum von Mises stress is approximately $1.25 \times 10^{-1} \text{ N/mm}^2$, the maximum displacement magnitude is approximately 2.48×10^{-7} , and the maximum base reaction is approximately 5.51×10^4 . The stress field remains relatively smooth, without the strong local amplification observed in Cases 2, 3, and 7.

This result is especially important because it confirms that the most successful opti-

misation case also performs well structurally. Case 8 therefore appears to be the most convincing compromise between geometric efficiency and wind-load robustness.

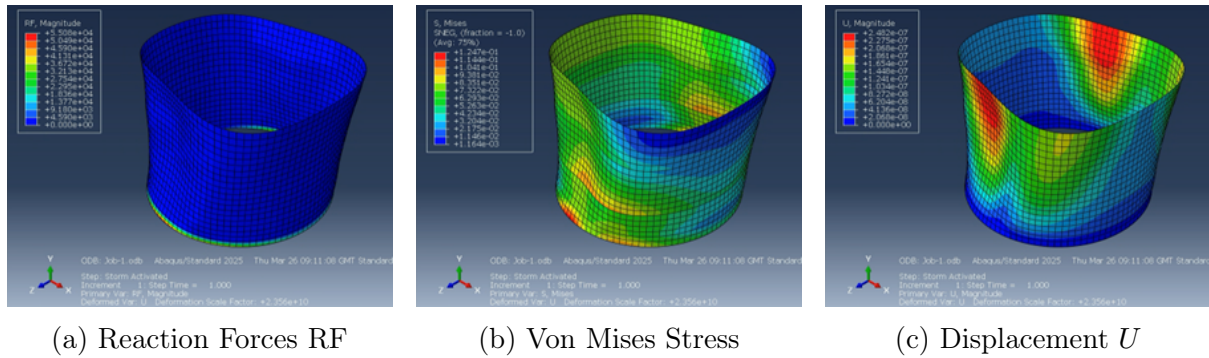


Figure 11: Structural response for Case 8

4 Discussion

4.1 Global remarks on the eight cases

A clear structural hierarchy emerges from the eight cases. Cases 1, 4, 5, 6, and 8 form the most regular and mechanically efficient group, with low von Mises stresses and limited displacements. Among them, Cases 5 and 6 are particularly stiff in terms of displacement, while Case 8 combines a low stress level with the strongest overall optimisation performance inherited from Task 6. Cases 2 and 7 occupy an intermediate position, showing noticeably higher stress and displacement levels. Case 3 remains the most critical case, with the highest displacement and one of the highest stress levels.

These results reinforce the main conclusion already suggested by Task 6: the geometry that minimises area is not always the most structurally favourable, and a regular, physically plausible profile is often preferable to a mechanically extreme optimum.

References

- [1] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 6: Structural Optimisation – Group Chenonceau*. Tech. rep. Cranfield University, 2026.
- [2] European Committee for Standardization. *EN 1991-1-1: Eurocode 1: Actions on structures - Part 1-1: General actions - Densities, self-weight, imposed loads for buildings*. 2002.
- [3] European Committee for Standardization. *EN 1992-1-1: Eurocode 2: Design of concrete structures - Part 1-1: General rules and rules for buildings*. 2004.
- [4] VGB PowerTech. *VGB-R 610e: Structural Design of Cooling Towers*. 2010.